



POLITÉCNICA

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INDUSTRIALES
UNIVERSIDAD POLITÉCNICA DE MADRID

José Gutiérrez Abascal, 2. 28006 Madrid
Tel.: 91 336 3060
info.industriales@upm.es

www.industriales.upm.es



Lucía Gorostidi García

05 TRABAJO FIN DE MASTER

INDUSTRIALES

TRABAJO FIN DE MASTER

EDGE-AI EN UAS PARA MANTENIMIENTO PREDICTIVO DE PLANTAS FOTOVOLTAICAS

SEPTIEMBRE 2026

Lucía Gorostidi García

DIRECTOR DEL TRABAJO FIN DE MASTER:
Eduardo Caro Huertas

TRABAJO FIN DE MASTER
PARA LA OBTENCIÓN DEL
TÍTULO DE MASTER EN
INGENIERÍA INDUSTRIAL



POLITÉCNICA



UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros
Industriales



Máster en Ingeniería Industrial

EDGE-AI EN UAS PARA MANTENIMIENTO PREDICTIVO DE PLANTAS FOTOVOLTAICAS

TRABAJO FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Lucía Gorostidi García

Tutor:

Eduardo Caro Huertas

El profesor-tutor de este trabajo forma parte del equipo investigador del proyecto “*Strategic Adaptation of Load-Forecasting Tools in Response to Recent Transformations in the Spanish Electricity Sector*” que ha recibido la ayuda PID2023-151007OA-I00, financiada por el Ministerio de Ciencia, Innovación y Universidades, la Agencia Estatal de Investigación y el Fondo Europeo de Desarrollo Regional MCIU/AEI/10.13039/501100011033/FEDER, UE.

«Soy un gran creyente en la suerte, y he descubierto que cuanto más trabajo, más suerte tengo.»

AGRADECIMIENTOS

A mis padres, por ser un apoyo incondicional a lo largo de estos años. Se que siempre estaréis ahí.

A mis hermanos, por mirarme con orgullo y admirarme, dándome fuerzas cuando más las necesito. Yo os admiro más, sois el mejor regalo.

A Fer, por estar siempre a mi lado, este logro es tan tuyo como mío. Y a su familia, que también es la mía, nadie mejor que vosotros para aguantar mis tardes de estudio, y nadie mejor que las grandes alegrías de esa casa, mis compañeros de cuatro patas, Rumba y Mate, que me animan con su amor incondicional.

A mis amigos (y mis primas, que por supuesto además de familia sois las mejores amigas que podría tener), gracias infinitas por animarme siempre. Ana y Marta, este ciclo no habría sido lo mismo sin vosotras.

Y, por último, a mi tutor, por estar siempre a mi disposición y sus mensajes de ánimo. Pero, sobre todo, por la gran confianza depositada en mí.

RESUMEN

Este Trabajo Fin de Máster aborda el diseño de un sistema de inspección fotovoltaica automatizada mediante una plataforma UAS equipada con visión artificial procesada en Edge-AI, como alternativa a las inspecciones manuales convencionales, costosas, lentas y con riesgos laborales en instalaciones de difícil acceso.

Se ha definido la arquitectura completa de una plataforma UAS (propulsión, control de vuelo, alimentación y carga útil de procesamiento), y se ha construido un conjunto de datos de defectos fotovoltaicos (soiling, nieve y grietas) combinando un dataset público depurado con 165 imágenes propias. Sobre este conjunto se entrenaron y compararon cuatro configuraciones de un modelo YOLO11n, seleccionando como definitiva la versión E4 por su mejor equilibrio entre precisión y sensibilidad ($F1 \approx 0,623$).

El modelo se desplegó sobre una plataforma Edge-AI (Raspberry Pi 5 + Hailo-8), logrando reducir la latencia un 94,4 % y multiplicar por 17,8 la velocidad de procesamiento frente a la CPU. El sistema se validó con vídeo real de un UAS comercial sobre una instalación fotovoltaica, confirmando su capacidad de detección en condiciones distintas a las de entrenamiento.

Los resultados confirman la viabilidad del concepto como prueba de concepto de inspección autónoma, aunque la plataforma completa no llegó a construirse ni validarse en vuelo. Como líneas futuras se plantean su construcción física, la ampliación del dataset y la incorporación de sensores térmicos.

PALABRAS CLAVE

Sistemas de Aeronaves No Tripuladas (UAS); Edge-AI; Visión por computador; Detección de objetos (YOLO); Mantenimiento predictivo fotovoltaico; Inspección aérea automatizada

CÓDIGOS UNESCO

120304 Inteligencia Artificial

330104 Aeronaves

330406 Arquitectura de Ordenadores

330790 Microelectrónica

332205 Fuentes no Convencionales de Energía

ABSTRACT

This Master's Thesis addresses the design of an automated photovoltaic inspection system based on a UAS platform equipped with computer vision running on Edge-AI, as an alternative to conventional manual inspections, which are costly, slow and pose safety risks in hard-to-access installations.

A complete UAS architecture was defined (propulsion, flight control, power and onboard-processing payload), and a dataset of photovoltaic defects (soiling, snow and cracks) was built by combining a curated public dataset with 165 in-house images. Four configurations of a YOLO11n model were trained and compared on this dataset, selecting the final version (E4) for its best balance between precision and recall ($F1 \approx 0.623$).

The model was deployed on an Edge-AI platform (Raspberry Pi 5 + Hailo-8), reducing latency by 94.4% and increasing processing speed 17.8-fold compared to CPU execution. The system was validated with real footage from a commercial UAS over a photovoltaic installation, confirming its detection capability under conditions different from those used for training.

The results confirm the viability of the concept as a proof of concept for autonomous inspection, although the complete platform was not physically built or flight-validated. Future work includes its physical construction, dataset expansion and the incorporation of thermal sensors.

KEYWORDS

Unmanned Aircraft Systems (UAS); Edge-AI; Computer vision; Object detection (YOLO); Photovoltaic predictive maintenance; Automated aerial inspection.

GLOSARIO Y LISTA DE ABREVIATURAS

[AESA] - Agencia Estatal de Seguridad Aérea: autoridad española competente en materia de seguridad aérea y regulación de operaciones con UAS.

[BEC] - Battery Eliminator Circuit: circuito regulador que suministra tensión continua estable a la electrónica del dron sin necesidad de fuentes de alimentación auxiliares.

[C2] - Command and Control Link (Enlace de Mando y Control): canal de datos bidireccional que transmite las instrucciones hacia la aeronave y recibe telemetría e información a bordo.

[CNN] - Convolutional Neural Network (Red Neuronal Convolucional): arquitectura de red neuronal especializada en el procesamiento de imágenes, empleada como base de los modelos de detección de objetos como YOLO.

[CPU] - Central Processing Unit (Unidad Central de Procesamiento): procesador de propósito general de la Raspberry Pi 5, utilizado como referencia de comparación frente a la ejecución acelerada con Hailo-8.

[CRSF] - Crossfire: protocolo digital de comunicación entre el receptor de radiocontrol y la controladora de vuelo.

[CSI] - Camera Serial Interface: interfaz de vídeo de alta velocidad (bus MIPI CSI) empleada para conectar el módulo de cámara directamente a la Raspberry Pi 5.

[DRI] - Direct Remote Identification (Identificación Remota Directa): módulo obligatorio que emite en tiempo real la identidad, posición y datos de vuelo del UAS conforme a la normativa europea.

[EASA] - European Union Aviation Safety Agency (Agencia de la Unión Europea para la Seguridad Aérea): responsable de la normativa europea aplicable a los UAS.

[ELRS]- ExpressLRS: protocolo de radiofrecuencia de baja latencia empleado por la emisora de radiocontrol para enlazar directamente con el receptor sin necesidad de módulos externos.

[ESC] - Electronic Speed Controller (Variador Electrónico de Velocidad): regula la velocidad de los motores a partir de las señales de la controladora de vuelo.

[FCC] - Federal Communications Commission: variante regulatoria estadounidense del espectro de radiofrecuencia, alternativa a la normativa LBT.

[FP32] - Floating Point 32-bit: formato de representación numérica en coma flotante de 32 bits, correspondiente a la precisión original del modelo antes de su cuantización a INT8.

[FPS] - Frames Per Second (Fotogramas por Segundo): métrica de velocidad de procesamiento de vídeo, empleada para comparar el rendimiento del detector en CPU frente a Hailo-8.

[FV] - Fotovoltaico/a: relativo a la conversión directa de la radiación solar en electricidad mediante módulos fotovoltaicos.

[GCS] - Ground Control Station (Estación de Control en Tierra): interfaz desde la que el operador planifica la misión, monitoriza la telemetría y controla la aeronave.

[GNSS] - Global Navigation Satellite System: sistema de navegación por satélite empleado para el posicionamiento y la navegación de la aeronave.

[GND] - Ground (masa): referencia de tensión cero de un circuito eléctrico.

[GPIO] - General Purpose Input/Output: pines de entrada/salida de propósito general de la Raspberry Pi, empleados para la conexión con otros componentes del sistema.

[GPS] - Global Positioning System (Sistema de Posicionamiento Global): sistema de navegación por satélite estadounidense; en este TFM se emplea como fuente de posición dentro del módulo GNSS y del sistema de identificación remota (DRI).

[GPU] - Graphics Processing Unit (Unidad de Procesamiento Gráfico): procesador empleado para acelerar el entrenamiento de los modelos de detección.

[HAT] - Hardware Attached on Top: estándar de placas de expansión para la Raspberry Pi; en este TFM, el acelerador Hailo-8 se integra mediante un módulo Hailo-8 HAT conectado directamente a la Raspberry Pi 5.

[HEF] - Hailo Executable Format: formato final del modelo ('.hef'), compatible y optimizado para su ejecución sobre el acelerador Hailo-8.

[IA] - Inteligencia Artificial.

[IEA] - International Energy Agency (Agencia Internacional de la Energía): organismo que publica indicadores globales del mercado fotovoltaico, como el informe "Snapshot of Global PV Markets".

[INT8] - Integer 8-bit: formato de cuantización que representa los parámetros del modelo con enteros de 8 bits, reduciendo el tamaño y acelerando la inferencia respecto a la precisión original.

[IoU] - Intersection over Union: métrica que mide el grado de solapamiento entre la caja delimitadora predicha y la real, utilizada para evaluar la precisión de las detecciones.

[IRT] - Infrared Thermography (Termografía Infrarroja): técnica basada en el espectro infrarrojo empleada para localizar gradientes térmicos anómalos en los módulos.

[LBT] - Listen Before Talk: variante regulatoria europea del espectro de radiofrecuencia utilizada por la emisora de radiocontrol.

[MTOW] - Maximum Take-Off Weight (Masa Máxima al Despegue): masa total admisible de la plataforma en orden de vuelo.

[NPU] - Neural Processing Unit (Unidad de Procesamiento Neuronal): chip acelerador especializado en la ejecución de modelos de inteligencia artificial, como el Hailo-8.

[OACI] - Organización de Aviación Civil Internacional: organismo de referencia en la normativa aeronáutica internacional.

[ODS] - Objetivos de Desarrollo Sostenible: conjunto de 17 objetivos definidos por Naciones Unidas en la Agenda 2030, con los que se relaciona el sistema desarrollado en este trabajo.

[ONNX] - Open Neural Network Exchange: formato abierto e intermedio para representar modelos de redes neuronales, utilizado como paso previo a la conversión del modelo YOLO al formato compatible con el acelerador Hailo-8.

[PDB] - Power Distribution Board (Placa de Distribución de Potencia): distribuye la energía de la batería entre los distintos subsistemas del UAS.

[PV] - Photovoltaic: acrónimo inglés equivalente a FV, referido a los sistemas de generación fotovoltaica.

[PYMEs] - Pequeñas y Medianas Empresas.

[RAM] - Random Access Memory (Memoria de Acceso Aleatorio): memoria de trabajo del sistema embarcado, cuyo consumo se redujo notablemente al ejecutar el modelo mediante el acelerador Hailo-8.

[RGB] - Red-Green-Blue: espectro visible captado por la cámara óptica convencional, empleado para la inspección visual de los módulos.

[RPAS] - Remotely Piloted Aircraft System: sistema de aeronave pilotada a distancia por un operador desde una estación remota.

[RX] - Recepción/receptor: canal o elemento encargado de recibir la señal en un enlace de comunicación.

[SBC] - Single Board Computer (Ordenador de Placa Única): equipo de bajo consumo y tamaño reducido, como la Raspberry Pi 5, empleado para el procesamiento embarcado.

[SCADA] - Supervisory Control and Data Acquisition: sistema de supervisión, control y adquisición de datos empleado en grandes plantas fotovoltaicas para monitorizar su rendimiento.

[SITL] - Software In The Loop: modo de simulación de Ardupilot que permite probar el comportamiento de la controladora de vuelo sin necesidad de disponer físicamente de la Pixhawk ni del UAS.

[TX] - Transmisión/transmisor: canal o elemento encargado de enviar la señal en un enlace de comunicación.

[UART] - Universal Asynchronous Receiver-Transmitter: protocolo de comunicación serie empleado para conectar la Raspberry Pi con la controladora de vuelo y otros módulos del sistema.

[UAS] - Unmanned Aircraft System (Sistema de Aeronave No Tripulada): denominación general que engloba tanto a los RPAS como a los UAV autónomos.

[UAV] - Unmanned Aerial Vehicle: plataforma o vehículo aéreo en sí, es decir, el dron o aeronave física que vuela sin tripulante a bordo.

[UBEC] - Universal Battery Eliminator Circuit: circuito regulador (Matek UBEC Duo) que convierte la tensión de la batería a los niveles estables necesarios para alimentar la electrónica embarcada, sin necesidad de baterías auxiliares.

[VTOL] - Vertical Take-Off and Landing: capacidad de despegue y aterrizaje vertical.

[YOLO] - You Only Look Once: familia de modelos de detección de objetos de una sola etapa, capaz de localizar y clasificar elementos en una imagen en una única pasada; es el modelo utilizado en este TFM para la detección de defectos fotovoltaicos.

ÍNDICE

AGRADECIMIENTOS	iii
RESUMEN.....	v
ABSTRACT	vi
GLOSARIO Y LISTA DE ABREVIATURAS	vii
1. INTRODUCCIÓN Y JUSTIFICACIÓN	1
1.1. El mantenimiento predictivo en el sector fotovoltaico	1
1.2. Clasificación y fundamentos de los UAS	2
1.2.1. Definición, acrónimos, concepto del sistema	2
1.2.2. Tipologías de vectores aéreos y arquitecturas.....	2
1.2.3. Aplicación de UAS en la industria: la necesidad de <i>Edge-AI</i>	4
1.2.4. Marco regulatorio y limitaciones operativas en el espacio aéreo	4
1.3. Defectología en instalaciones fotovoltaicas	4
1.3.1. Tipologías de anomalías y degradación en paneles solares.....	4
1.3.2. Manifestación óptica y térmica de los fallos (espectro visible vs. infrarrojo)	5
1.4. Visión por Computador e Inteligencia Artificial en la inspección fotovoltaica ..	6
1.4.1. Fundamentos de <i>Deep Learning</i> para detección de objetos: YOLO.....	6
1.4.2. Limitaciones del procesamiento offline y la transición hacia el Edge-AI ...	6
2. OBJETIVOS Y ENFOQUE	7
2.1. Objetivo General	7
2.2. Objetivos Específicos	7
3. METODOLOGÍA Y ARQUITECTURA	8
3.1. Enfoque metodológico y fases del proyecto.....	8
3.2. Estructura de la memoria	8
4. DISEÑO E INTEGRACIÓN DEL HARDWARE.....	9
4.1. Estructura física y grupo propulsor.....	9
4.1.1. Soporte mecánico: El chasis	9
4.1.2. Tren de aterrizaje (<i>Landing Gear</i>).....	10
4.1.3. Grupo motopropulsor: motores, hélices y ESCs.....	10
4.1.4. Placa de Distribución de Potencia (<i>PDB</i>)	14

4.1.5.	Esquema final estructura y propulsión	15
4.2.	Subsistema de control de vuelo y alimentación	16
4.2.1.	Controladora de vuelo	16
4.2.2.	Módulo de Navegación GNSS	17
4.2.3.	Emisora de Radiocontrol (Estación Manual de Tierra)	18
4.2.4.	Receptor de Radiocontrol	19
4.2.5.	Módulo de Telemetría de Radio	19
4.2.6.	Esquema final control de vuelo	21
4.3.	Arquitectura de alimentación y acondicionamiento eléctrico	22
4.4.	Carga útil (<i>payload</i>)	24
4.4.1.	Unidad de procesamiento embarcada y acelerador NPU	25
4.4.2.	Sensor óptico de adquisición de datos: Cámara RGB	27
4.4.3.	Viabilidad de sensor termográfico IR	28
4.4.4.	Esquema final de carga útil y alimentación	29
4.5.	Integración y Balance Global	30
4.5.1.	Esquemas maestros de integración	30
4.5.2.	Inventario de hardware	32
4.5.3.	Análisis de sustentación y perfil energético	33
5.	CONSTRUCCIÓN Y PREPARACIÓN DEL DATASET	35
5.1.	Definición del problema y estrategia de datos	36
5.2.	Selección y preparación del dataset inicial	37
5.2.1.	Revisión y depuración inicial	38
5.2.2.	Homogeneización de las clases	41
5.2.3.	Construcción del conjunto de referencia	41
5.3.	Ampliación del dataset: datos propios	42
5.3.1.	Obtención de los datos	42
5.3.2.	Selección de datos externos	47
5.3.3.	Preparación del material	48
6.	DESARROLLO DEL MODELO DE DETECCIÓN	51
6.1.	Flujo de entrenamiento	51
6.2.	Familia YOLO	53

6.3.	Entorno de entrenamiento	55
6.3.1.	Prueba preliminar	56
6.3.2.	Entorno de ejecución: Google Colab	59
6.3.3.	Configuración experimental	62
6.4.	E1: <i>baseline</i>	63
6.5.	E2: <i>extended_dataset</i>	69
6.6.	E3: <i>revised_classes</i>	74
6.7.	E4: <i>augmented_dataset</i>	79
6.8.	Modelo final	83
7.	INTEGRACIÓN DEL SISTEMA EDGE – AI	85
7.1.	Adaptación del modelo a la plataforma <i>Edge-AI</i>	85
7.2.	Flujo de inferencia e integración con el UAS.....	91
7.2.1.	Secuencia operativa de inferencia a bordo	91
7.2.2.	Software embarcado y adquisición de imágenes	92
7.2.3.	Integración con el sistema de vuelo y supervisión en tierra	94
7.3.	Validación funcional con material real de UAS	98
7.4.	Rendimiento y consideraciones de despliegue	101
8.	RESULTADOS Y DISCUSIÓN	103
8.1.	Resultado del diseño del UAS.....	103
8.2.	Resultados del sistema de visión artificial.	104
8.3.	Integración, validación y rendimiento Edge-AI.....	106
9.	MARCO NORMATIVO.....	109
9.1.	Normativa UAS	109
9.2.	Protección de datos y captación de imágenes	112
9.3.	Normativa y estándares relacionados con las instalaciones fotovoltaicas ...	113
9.4.	Marco normativo relacionado con la IA	113
10.	CONCLUSIONES Y LÍNEAS FUTURAS	114
11.	BIBLIOGRAFÍA	117
12.	CONTRIBUCIÓN A LOS ODS	121

13. ANÁLISIS DE IMPACTOS	122
14. PLANIFICACIÓN TEMPORAL Y PRESUPUESTO	123
14.1. Planificación temporal	123
14.2. Presupuesto	124

1. INTRODUCCIÓN Y JUSTIFICACIÓN

1.1. El mantenimiento predictivo en el sector fotovoltaico

En los últimos años, la energía solar fotovoltaica se ha consolidado gracias al proceso de transición energética hacia modelos neutros en carbono, como una de las tecnologías de generación renovable con mayor tasa de crecimiento a nivel internacional y a mayor alcance del ser humano, derivando en un masivo despliegue de plantas fotovoltaicas a pequeñas y grandes escalas.

La IEA publica anualmente el informe “*Snapshot of Global PV Markets*”, en el que se puede observar de manera gráfica la enorme evolución de instalaciones fotovoltaicas en los últimos años:

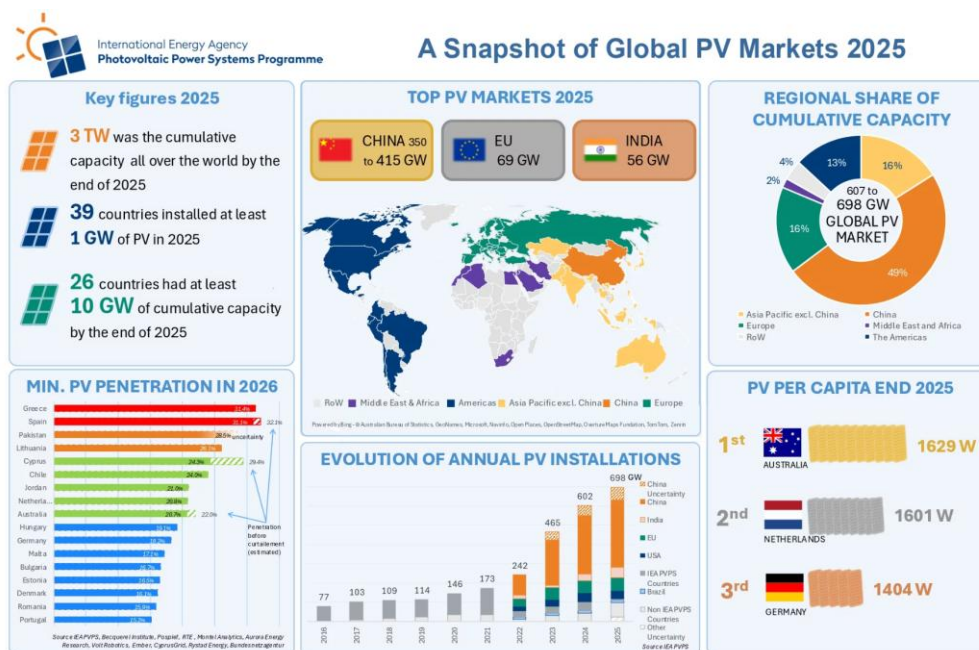


Figura 1.1. Crecimiento global de la capacidad FV. Fuente: IEA-PVPS [1].

Es este crecimiento el que obliga a preocuparse por la viabilidad económica de estas infraestructuras, que están diseñadas para operar en ciclos de vida superiores a los 25 años, viabilidad que depende de manera crítica de la optimización de los costes de Operación y Mantenimiento y de la maximización del rendimiento energético. La exposición continuada de estas plantas a condiciones ambientales adversas, ciclos térmicos, impactos mecánicos... induce la aparición de anomalías y defectos físicos en las placas, lo que podría derivar en una disminución de la eficiencia de esa planta.

Tradicionalmente, las estrategias de mantenimiento en instalaciones solares han tenido un carácter puramente reactivo o correctivo, interviniendo únicamente tras la detección de pérdidas de rendimiento significativas mediante los sistemas SCADA¹ de la planta (sistema que solo tienen las plantas de gran escala). Progresivamente, el sector ha evolucionado hacia esquemas de mantenimiento predictivo, con el objetivo de identificar y clasificar anomalías en fases tempranas, para evitar fallos catastróficos, degradaciones y pérdidas económicas.

¹ Sistema industrial de *software* y *hardware* diseñado para monitorizar y controlar procesos a distancia en tiempo real.

Este cambio de paradigma no se circunscribe solo a grandes plantas de generación centralizada, sino que cobra especial relevancia en el creciente mercado del autoconsumo, sobre cubiertas residenciales, comerciales e industriales en pequeñas y medianas empresas (PYMEs). En estas instalaciones de menor escala, la aparición de fallos desatendidos, sombreados parciales o acumulación de suciedad (*soiling*) representa una penalización económica elevada.

Sin embargo, las metodologías de inspección técnica convencional (basadas en revisiones manuales) presentan ciertas limitaciones operativas. Por un lado, conllevan elevados tiempos de ejecución y costes de horas de personal cuando se trata de una gran planta fotovoltaica; por otro, en pequeñas cubiertas e instalaciones de difícil acceso, incrementan drásticamente los riesgos de seguridad laboral.

Ante esta problemática, la integración de Sistemas de Aeronaves No Tripuladas (UAS) con algoritmos de Inteligencia Artificial se erige como la solución idónea para automatizar, homogeneizar y escalar el proceso de inspección y diagnóstico en cualquier tipo de instalación fotovoltaica.

1.2. Clasificación y fundamentos de los UAS

1.2.1. Definición, acrónimos, concepto del sistema

En el ámbito de la ingeniería y la normativa aeronáutica oficial (OACI, EASA, AESA), son habituales diversos términos para referirse a la robótica aérea, comúnmente conocida en el lenguaje coloquial y divulgativo bajo la denominación de “dron” (*drone*).

- **UAV (*Unmanned Aerial Vehicle*):** Hace referencia a la plataforma o vehículo aéreo en sí, es decir, al dron o aeronave física que vuela sin tripulante a bordo.
- **RPAS (*Remotely Piloted Aircraft System*):** Término específico para los sistemas de aeronaves pilotadas a distancia por una persona desde una estación remota. El sistema completo se compone del vehículo (RPA), la estación de control en tierra, y el enlace de comunicaciones.
- **UAS (*Unmanned Aircraft System*):** Representa la denominación marco más completa, formal y rigurosa. Engloba tanto a los RPAS como a los UAV autónomos.

Un UAS se compone de tres subsistemas fundamentales interconectados:

- La plataforma aérea (*Unmanned Aircraft* o UAV). Es la estructura física que integra la planta motopropulsora, el sistema de control de vuelo, los sensores de navegación inercial y posicionamiento y la carga útil (*payload*).
- La Estación de Control en Tierra (GCS, *Ground Control Station*). Es la interfaz desde la cual el operador planifica las misiones, monitoriza la telemetría en tiempo real o ejerce el control de la aeronave.
- El Enlace de Mando y Control (C2, *Command and Control Link*). Canal de datos bidireccional encargado de transmitir las instrucciones hacia la aeronave y recibir la telemetría y el flujo de información a bordo.

1.2.2. Tipologías de vectores aéreos y arquitecturas

Los vectores aéreos, o UAV, se clasifican técnicamente atendiendo al principio físico mediante el cual generan sustentación aerodinámica y a la configuración de su arquitectura estructural:

- Aeronaves de ala fija (*Fixed-Wing*):
-

La sustentación se genera de forma dinámica, mediante el paso del flujo de aire a través del perfil alar, impulsado por motores de empuje horizontal.

Sus ventajas son su eficiencia aerodinámica y sus grandes autonomías de vuelo (superan los 60 – 90 minutos).

Sus limitaciones son la imposibilidad de realizar vuelo estacionario (*hover*), que requiere áreas despejadas o mecanismos auxiliares para maniobras de despegue y aterrizaje y que la maniobrabilidad a baja altura es más complicada.



Figura 1.2. Plataforma de Ala Fija (Delair UX11). Fuente: Adaptado de El Vuelo del Drone [2].

- Aeronaves multi-rotores (*Rotary-Wing*) o de ala rotativa:

La sustentación se obtiene mediante el empuje vertical variable generado por el giro independiente de múltiples hélices accionadas por motores eléctricos.

Sus ventajas son su destacable capacidad de despegue y aterrizaje vertical (VTOL, *Vertical Take-Off and Landing*), su maniobrabilidad y la estabilidad en punto fijo (*hover*).

Su mayor limitación es que, al requerir un consumo energético continuo y elevado para sostener la masa de la aeronave, su autonomía operativa se ve limitada a rangos de entre 20 – 35 minutos.



Figura 1.3. Dron Multi-rotor o Plataforma de Ala Rotativa. Fuente: Adaptado de UAVforDrone [3].

Se clasifican a su vez en función del número de motores. Los más comunes son los tricópteros, cuadricópteros, hexacópteros y octocópteros. Su nombre indica el número de brazos y, por consiguiente, la cantidad de hélices y motores de su configuración.

- Plataformas Híbridas (VTOL de Ala Fija):

Combinan motores verticales para el despegue/aterrizaje con alas para vuelo de crucero. Son más complejas mecánicamente y tienen un coste mucho mayor.

1.2.3. Aplicación de UAS en la industria: la necesidad de *Edge-AI*

En los últimos años, los UAS han experimentado una rápida transición, pasando de ser herramientas de ámbitos principalmente militar o recreativo, a consolidarse como un vector tecnológico fundamental en multitud de sectores industriales y de servicios.

Gracias a la capacidad de desplazar sensores de forma tridimensional, alcanzando zonas de difícil acceso de manera rápida, económica y segura, lo que reduce drásticamente los riesgos laborales, elimina la necesidad de medios auxiliares (por ejemplo, andamios o escaleras) y evita la parada de las instalaciones, se ha impulsado la adopción masiva de los UAS en actividades donde los métodos tradicionales resultaban muy costosos o complejos. En sectores como la topografía, la agricultura, la supervisión de líneas de alta tensión... ya se trabaja con plataformas aéreas, eliminando los riesgos de trabajo en altura y reduciendo drásticamente los costes operativos (OPEX).

Sin embargo, la inmensa mayoría de las soluciones comerciales actuales se limitan a un esquema de inspección pasiva o diferida (*offline*). En este paradigma, el vector aéreo actúa exclusivamente como una plataforma de adquisición y almacenamiento de datos masivos, posponiendo la fase de procesado y diagnóstico al volcado posterior en estaciones de trabajo en tierra (y normalmente manuales).

Esta limitación establece la necesidad de evolucionar hacia un nuevo salto tecnológico: el procesamiento de inteligencia artificial a bordo (*Edge-AI*).

1.2.4. Marco regulatorio y limitaciones operativas en el espacio aéreo

El despliegue de los UAS en entornos industriales y de generación energética está condicionado por el marco normativo vigente, definido por la Agencia Estatal de Seguridad Aérea, AESA [4], y la normativa europea EASA [5].

El cumplimiento de los requisitos operacionales (como la clasificación en categorías de vuelo, las restricciones en espacio aéreo controlado y la gestión de permisos) representa un factor determinante en la viabilidad de las misiones de inspección.

Asimismo, condicionantes físicos, como la autonomía de las baterías, las limitaciones de carga útil (*payload*) y las condiciones meteorológicas establecen el marco operativo dentro del cual deben planificarse los vuelos de adquisición de datos.

1.3. Defectología en instalaciones fotovoltaicas

La operación continuada de los módulos fotovoltaicos en emplazamientos al aire libre expone a sus componentes a un estrés ambiental y electromecánico constante. La detección temprana de las degradaciones es fundamental para garantizar la eficiencia global de la planta y prevenir fallos catastróficos o pérdidas económicas.

En este apartado se analizan las principales tipologías de anomalías físicas y eléctricas que afectan a los paneles solares, evaluando sus mecanismos de degradación y sus manifestaciones en los espectros visible e infrarrojo.

1.3.1. Tipologías de anomalías y degradación en paneles solares

El rendimiento energético de un módulo fotovoltaico y su vida útil operativa dependen de la integridad física y eléctrica de sus componentes. Los defectos en los paneles se originan por

fallos en el proceso de fabricación, manipulación inadecuada durante el transporte e instalación, o por el envejecimiento acelerado resultante de la exposición ambiental continuada.

Para su análisis y posterior diagnóstico automatizado, las anomalías se clasifican en las siguientes categorías:

- **Acumulación de suciedad y contaminantes (*Soiling*):** Consiste en el depósito de partículas de polvo, polen, cenizas o deyecciones de aves sobre la superficie del vidrio protector. Aunque no representa una avería interna del semiconductor, reduce la transmitancia de la radiación solar, y genera sombreados heterogéneos que provocan desequilibrios eléctricos y desaprovechamiento.
- **Daños estructurales y roturas mecánicas:** Fracturas o grietas en la cubierta de vidrio originadas por esfuerzos mecánicos, vibraciones durante el transporte o impactos de granizo. Su presencia altera la geometría del panel y puede evolucionar hacia el aislamiento de regiones completas de la célula.
- **Puntos calientes (*Hotspots*):** Cuando una o varias celdas sufren un sombreado parcial permanente o cortocircuito interno, pasando a trabajar en polarización inversa. Al no permitir el paso fluido de la corriente, la celda actúa como un obstáculo resistivo que consume la energía del resto del módulo y la disipa en forma de calor intenso por el efecto Joule. Este sobrecalentamiento localizado puede alcanzar temperaturas críticas que queman los materiales de protección o incluso inutilizan el panel por completo.
- **Degradación de componentes eléctricos y diodos de *bypass*:** Fallos en las cajas de conexiones, corrosión en las líneas de interconexión o averías en los diodos de protección, los cuales deshabilitan subcadenas completas del módulo.

1.3.2. Manifestación óptica y térmica de los fallos (espectro visible vs. infrarrojo)

La caracterización y diagnóstico de las anomalías descritas se aborda mediante el análisis en dos dominios principales:

- **Inspección en el espectro visible (RGB):** Permite evaluar directamente la morfología de la superficie del módulo, detectando así las anomalías mecánicas y los depósitos de suciedad (*soiling*, deyecciones de aves, sombras, roturas de cristal...).
- **Inspección en el espectro infrarrojo (IRT):** Constituye el estándar para la localización de gradientes térmicos anómalos derivados de disipaciones por el efecto Joule (puntos calientes, diodos en cortocircuito, desequilibrios...)

Aunque la termografía infrarroja se mantiene como la técnica de referencia para la caracterización de fallos térmicos severos, el análisis en el espectro visible ofrece ventajas determinantes en términos de resolución espacial, menor coste y reducción de la carga útil (*payload*) embarcable.

Estas ventajas permiten utilizar el RGB como una primera línea de criba de alto rendimiento. Identificar visualmente la presencia de defectos puede prevenir la degradación térmica antes de que esta cause daños irreversibles o active innecesariamente los mecanismos de protección del módulo fotovoltaico.

1.4. Visión por Computador e Inteligencia Artificial en la inspección fotovoltaica

La automatización del mantenimiento predictivo en el sector fotovoltaico encuentra en la Inteligencia Artificial y concretamente en la Visión por Computador, un pilar fundamental para el procesamiento masivo de datos.

La integración de estos algoritmos en UAS permite sustituir la inspección visual humana por sistemas capaces de identificar patrones de degradación de forma rápida, homogénea y objetiva.

1.4.1. Fundamentos de *Deep Learning* para detección de objetos: YOLO

El diagnóstico automatizado a partir de imágenes se apoya en algoritmos de aprendizaje profundo (*Deep Learning*). En el ámbito de las instalaciones fotovoltaicas no es suficiente con saber si una imagen tiene o no un fallo (clasificación); es necesario localizar los problemas, marcarlos con *bounding boxes* y etiquetarlos según su categoría.

Las arquitecturas de detección de objetos se dividen fundamentalmente en dos categorías:

- Detectores en dos etapas (*Two-Stage Detectors*): Modelos como *Faster R-CNN* que primero buscan zonas sospechosas de la imagen y luego las analizan una a una. Son muy precisos, pero son lentos y requieren mucha capacidad de cálculo, por lo que no serían ideales para el procesamiento de vídeos en tiempo real a bordo de un UAS.
- Detectores de una sola etapa (*Single-Stage Detectors*): La familia YOLO (*You Only Look Once*) analiza la imagen completa de una sola pasada, y al procesarlo todo a la vez, predice al mismo tiempo qué fallos hay y dónde están ubicados.

Por su gran velocidad y precisión, las redes YOLO se han convertido en la opción estándar para el trabajo con drones. Permiten analizar las imágenes en tiempo real y reconocer patrones muy variados (acumulaciones de polvo, excrementos de aves, sombras, grietas...)

1.4.2. Limitaciones del procesamiento offline y la transición hacia el Edge-AI

Tradicionalmente, las inspecciones con drones han funcionado bajo un esquema *offline*. La aeronave se limita a grabar el vídeo o fotos, guardándolo todo en una tarjeta de memoria. Una vez en tierra, los datos se vuelcan en un ordenador o servidor para que los algoritmos analicen las imágenes.

Aunque este método permite usar ordenadores potentes sin límite de peso o consumo, presenta dos principales inconvenientes:

- Falta de respuesta en tiempo real: Al no procesar las imágenes durante el vuelo, no es posible detectar un fallo crítico en el momento ni corregir la ruta del dron para tomar una foto más detallada en un panel sospechoso, además aumenta el tiempo de trabajo.
- Manejo de volúmenes masivos de datos: Se almacenan y transfieren horas de vídeo donde la gran mayoría de los paneles están en buen estado, generando un cuello de botella en el análisis posterior.

Para resolver estos problemas surge la Inteligencia Artificial en el Borde (*Edge-AI*). Este enfoque consiste en integrar un pequeño módulo de cómputo dentro del propio dron para que la red neuronal analice el flujo de vídeo en tiempo real durante el vuelo.

2. OBJETIVOS Y ENFOQUE

El mantenimiento predictivo en plantas fotovoltaicas mediante aeronaves no tripuladas requiere evolucionar desde la mera captación de imágenes hacia la toma de decisiones a bordo en tiempo real.

El presente trabajo tiene como finalidad desarrollar una solución para la inspección automatizada de instalaciones fotovoltaicas mediante UAS, combinando visión artificial y procesamiento *Edge-AI*. A partir de este planteamiento se define un objetivo general y una serie de objetivos específicos que guían las diferentes etapas del proyecto.

2.1. Objetivo General

OG. Diseñar y validar un sistema de inspección fotovoltaica basado en una plataforma UAS y un sistema de visión artificial con procesamiento *Edge-AI*, capaz de detectar automáticamente anomalías visibles en los módulos fotovoltaicos. Ejecutar el modelo sobre una plataforma de procesamiento embarcable.

2.2. Objetivos Específicos

Para alcanzar el objetivo general, el trabajo se desglosa en los siguientes objetivos específicos.

OE1. Diseñar la arquitectura de una plataforma UAS adecuada para tareas de inspección fotovoltaica, seleccionando los principales elementos de vuelo, adquisición de imágenes, procesamiento y comunicación. Comprender su funcionamiento individual y en conjunto.

OE2. Construir y preparar un conjunto de datos representativo de defectos visibles en módulos fotovoltaicos, combinando diferentes fuentes de imágenes y aplicando procesos de selección, etiquetado y aumento de datos.

OE3. Desarrollar y evaluar distintas configuraciones de un modelo de detección basado en YOLO, hasta seleccionar una solución final capaz de identificar los defectos considerados para el trabajo.

OE4. Adaptar el modelo seleccionado para su ejecución sobre una plataforma *Edge-AI* formada por una Raspberry Pi 5 y, si aplicase, un acelerador.

OE5. Integrar el sistema de visión artificial dentro de la arquitectura propuesta del UAS, contemplando su relación con el sistema de vuelo y las funciones de supervisión desde tierra.

OE6. Validar funcionalmente la solución mediante material obtenido en vuelos reales con UAS. Evaluar el rendimiento del despliegue embarcado mediante métricas.

OE7. Adquirir conocimientos prácticos sobre el funcionamiento y operación de sistemas UAS, incluyendo su arquitectura, los principales componentes de vuelo, herramientas de configuración y supervisión y requisitos básicos asociados a su utilización.

OE8. Adquirir experiencia práctica en el despliegue y ejecución de modelos de visión artificial sobre hardware embarcado, incluyendo la configuración y programación de la Raspberry Pi 5, el estudio de la integración sobre esta de un acelerador y la integración de las herramientas necesarias para la validación del sistema.

OE9. Conocer y aplicar el marco normativo básico vigente para la operación de UAS en España, incluyendo categorías operacionales, requisitos de formación del piloto, registro de operador e identificación de restricciones aplicables al entorno de vuelo

3. METODOLOGÍA Y ARQUITECTURA

El desarrollo del trabajo se planteó de forma progresiva, combinando los tres bloques principales: diseño de la plataforma de vuelo, desarrollo experimental del sistema de visión artificial e integración de estos dos últimos sobre una plataforma de vuelo con *Edge-AI*.

3.1. Enfoque metodológico y fases del proyecto

La metodología seguida en el trabajo se basa en un enfoque **iterativo y experimental**, en el que cada parte del sistema se desarrolla y valida progresivamente antes de avanzar hacia su integración. En lugar de plantear desde el inicio una solución cerrada, se parte de una arquitectura general que se va concretando a partir de las pruebas realizadas y sus resultados.

A partir de este enfoque, el desarrollo se organizó en cinco fases principales:

Fase 1. Estudio inicial y definición del sistema. Se analizó el problema de la inspección fotovoltaica mediante UAS, se revisaron las tecnologías disponibles y se definieron las necesidades generales del sistema. Esta fase incluyó también una primera aproximación al funcionamiento de los UAS y a la normativa asociada a su utilización.

Fase 2. Diseño de la plataforma UAS. Se definió la arquitectura del dron y se realizó un estudio y posterior selección de los principales componentes necesarios para el vuelo, adquisición de imágenes, procesamiento embarcado y comunicación con tierra.

Fase 3. Preparación de datos, entrenamiento y optimización del modelo IA. Se recopilaron, organizaron y etiquetaron imágenes de módulos fotovoltaicos, incorporando distintas fuentes de datos. A partir de ellas se realizaron diferentes experimentos de entrenamiento y evaluación del detector de defectos.

Fase 4. Adaptación e integración sobre la plataforma *Edge-AI*. Una vez seleccionado el modelo de IA, se adaptó para su ejecución sobre la Raspberry Pi y el acelerador, incorporándolo al flujo de procesamiento previsto para el sistema embarcado.

Fase 5. Validación del sistema. Finalmente, se realizaron pruebas con material capturado mediante UAS y se evaluó tanto el funcionamiento del sistema integrado como el rendimiento de la plataforma de procesamiento empleada.

3.2. Estructura de la memoria

La memoria se estructura siguiendo las principales etapas del desarrollo realizado.

Tras los capítulos iniciales de introducción, objetivos y metodología, se presenta el diseño de la plataforma UAS y la selección de los elementos necesarios para la inspección.

A continuación, se aborda la preparación del conjunto de datos y el desarrollo experimental del sistema de IA.

Posteriormente, se describe la adaptación e integración del modelo sobre la plataforma *Edge-AI*, así como las pruebas de validación y rendimiento realizadas.

Finalmente se recogen y discuten los principales resultados obtenidos, junto con las conclusiones del trabajo y los aspectos que se dejan para líneas futuras para después centrarse en el marco normativo que aplica al trabajo, la planificación y presupuesto, impacto y sostenibilidad.

4. DISEÑO E INTEGRACIÓN DEL HARDWARE

En este capítulo se desarrolla el diseño hardware de la plataforma UAS propuesta para las inspecciones, basada en una configuración multirrotor frente a una solución de ala fija, de acuerdo con las ventajas expuestas previamente en “1.INTRODUCCIÓN Y JUSTIFICACIÓN”. Se describen y justifican los principales subsistemas que intervienen en su funcionamiento, desde la estructura física y el sistema de propulsión hasta el control de vuelo, la navegación, las comunicaciones, la alimentación eléctrica y la *payload*.

Finalmente, se aborda la integración conjunta de todos estos elementos, se recoge el inventario de hardware seleccionado y se analiza el comportamiento previsto de la plataforma en términos de sustentación, consumo energético y autonomía.

Conviene aclarar que el presente trabajo no busca ensamblar o construir físicamente el dron, sino definir de forma conceptual su diseño y comprobar en simulación cómo funcionaría todo el programa a bordo. Se considera importante detallar esta arquitectura para entender cómo se integran la unidad de control, la tarjeta de procesamiento y los sensores para analizar las imágenes durante el vuelo.

A continuación, se describen los componentes teóricos que se han elegido para conformar los subsistemas del dron.

4.1. Estructura física y grupo propulsor

4.1.1. Soporte mecánico: El chasis

El soporte mecánico de la aeronave (el “cuerpo” del dron), se basa en un chasis tipo F450, una estructura estándar y muy utilizada en robótica aérea compuesta por cuatro brazos y una distancia diagonal entre ejes de 450mm.



Figura 4.1. Chasis F450 para plataforma multirrotor de cuatro motores. Fuente: Adaptado de AliExpress [6].

Esta elección responde a dos motivos principales de diseño:

- **Configuración en cuadricóptero (4 motores):** Se tiende a pensar que cuantos más motores tenga el dron, mejor funcionará o mayor capacidad tendrá. Sin embargo, en la práctica de la ingeniería esto no es así, y para esta misión una configuración de cuatro motores es más que suficiente.

Frente a opciones de hexacópteros (6 motores), el cuadricóptero ofrece el mejor equilibrio entre peso, consumo de batería y sencillez. Al llevar menos motores, el dron consume menos energía y puede volar durante más tiempo con la misma batería. Además, esta configuración proporciona la estabilidad necesaria para realizar recorridos rectilíneos a velocidad constante sobre las filas de paneles.

- **El chasis (F450):** Es un chasis estándar, resistente y económico, que absorbe bien las vibraciones de los motores. Lo más importante es que su plataforma central dispone del espacio suficiente para colocar los bloques electrónicos principales sin que molesten.

Como detalle funcional de diseño, la estructura utiliza brazos de dos colores diferenciados (en *¡Error! No se encuentra el origen de la referencia.* rojos y negros, pero se pueden encontrar de otros colores). Esto facilita al piloto identificar visualmente la orientación del dron durante los ensayos de vuelo manual y comprobaciones en campo.

4.1.2. Tren de aterrizaje (*Landing Gear*)

Además, es una buena práctica añadir un tren de aterrizaje por motivos de seguridad:

Se contempla para el chasis elegido un tren de patas elevadoras en la base de este, para elevar el cuerpo del dron unos 10-15 cm sobre el suelo. Esto protege distintos elementos que se encontrarán en la parte inferior del dron, aislando la cámara de vídeo y las tarjetas de procesamiento del contacto directo con el terreno evitando suciedades y absorbiendo las vibraciones al aterrizar o despegar y posarse sobre superficies irregulares.



Figura 4. 2. Tren de aterrizaje para F450. Fuente: Adaptado de Amazon [7].

4.1.3. Grupo motopropulsor: motores, hélices y ESCs

- **Motores sin escobillas (*brushless*):**

Cuatro motores eléctricos, que se encargan de transformar la energía eléctrica de la batería en energía cinética (de movimiento) para hacer girar las hélices. A diferencia de los motores tradicionales, con escobillas, la tecnología *brushless* elimina el rozamiento interno, lo que reduce el calentamiento, aumentando la eficiencia y la vida útil del motor.

Es importante destacar el criterio de sentido de giro de los motores para el entendimiento del funcionamiento del dron. Para garantizar su estabilidad, los motores no giran todos hacia el mismo sentido. Para anular el par de rotación que cada hélice ejerce sobre el chasis, dos motores situados en diagonal giran en sentido horario (*CW*) y los otros dos en sentido antihorario (*CCW*).

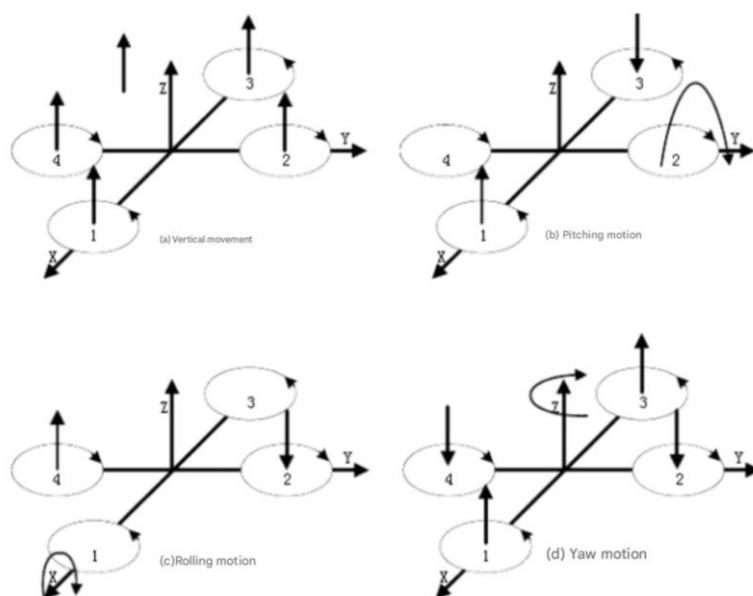


Figura 4.3. Ejemplo sentido motores cuadricóptero. M1 y M3 en CCW y M2 y M4 en CW. Fuente: Adaptado de Veishida [8].

La combinación de estas fuerzas opuestas permite que la aeronave se mantenga estable en el aire, o que gire sobre su propio eje según las órdenes del piloto.

En cuanto al criterio de selección para la elección de los motores, se realiza en función de la constante KV, que se refiere a las rpm (revoluciones por minuto) teóricas que gira un motor por cada voltio que le suministra la batería, sin carga. (Por ejemplo, con un motor de 1000 KV y una batería de 10 voltios, el motor girará a 10000 rpm en vacío).

Los motores con un bajo KV (800-1200) giran más despacio, pero tienen mucho torque. Se usan para mover hélices grandes que empujan mucho aire, priorizan la estabilidad y la capacidad de carga (*payload*). Esto es muy atractivo para el proyecto ya que el dron tendrá bastante carga por los elementos añadidos para el *Edge-AI*.

Por otro lado, los motores con un KV más alto (2000-3000) giran extremadamente rápido con poca fuerza, lo que resulta ideal para mini drones de carreras.

En un rango de bajo KV se han encontrado los siguientes motores:

Modelo	KV	Peso [g]		Empuje Máx. [g]		Corriente Máx. [A]	Alimentación [V]	Precio aprox. [€]	
		ud.	x4	ud.	x4			ud.	x4
Racerstar BR2212	920	52	208	780	3120	15	11,1	8	32
EMAX MT2213	935	55	220	850	3400	18	11,1 – 14,8	13	52
Sunnysky X2212 III	980	58	232	960	3840	22	11,1 – 14,8	16	64
T-Motor AIR 2213	920	54	216	950	3800	20	11,1 – 14,8	24	96

Tabla 4.1. Comparación motores brushless comerciales. Fuente: Elaboración propia.

La elección del grupo motopropulsor busca equilibrar la capacidad de elevación de la aeronave, el peso total del conjunto y la eficiencia eléctrica, garantizando una respuesta ágil durante la misión².

Analizando los datos de la *¡Error! No se encuentra el origen de la referencia.*, se descarta el modelo Racerstar BR2212 por ofrecer un empuje máximo combinado (3120 g) más ajustado que los demás frente a posibles rachas de viento en el campo.

Por otro lado, se descarta la opción T-Motor AIR 2213 por su elevado coste total (96€) sin aumentar proporcionalmente el rendimiento.

Finalmente, se ha seleccionado el motor **SunnySky X2212 III** (980 KV) por ofrecer las siguientes ventajas técnicas:

- Mayor empuje conjunto de la comparativa (3840 g), lo que aporta mayor holgura en el trabajo para contar con una buena relación empuje – peso y no tener tantas limitaciones y tener menos probabilidades de tener que realizar ajustes conforme se vayan eligiendo los demás componentes.
- Flexibilidad de alimentación, soportando tensiones nominales desde 11,1 V hasta 14,8 V, lo que permite trabajar con baterías que soporten voltajes más altos, reduciendo la corriente demandada para un mismo nivel de potencia y mejorando el rendimiento global, lo que extendería la autonomía de vuelo.
- Margen de corriente máxima (22 A), permitiendo soportar picos de potencia con solvencia durante la misión.

Además, aunque la selección de los motores responda a un diseño de prototipo en fase experimental, la evaluación del coste sigue constituyendo un factor de diseño relevante. El presupuesto total del conjunto (64€) se sitúa en un rango estándar y competitivo dentro del mercado, evitando sobrecostes de alternativas de gama superior sin renunciar a sus prestaciones.



Figura 4.4. Motor SunnySky X2212 KV: 980. Fuente: Adaptado de Aliexpress [9].

² Entiéndase “misión” como la inspección fotovoltáica.

- **Hélices:**

Se ha seleccionado el modelo de hélices estándar, 4 **hélices 1045** de nylon, este código indica un diámetro de 10 pulgadas (25,4 cm) y un paso de 4,5 pulgadas (11,43 cm). Esta combinación se ajusta de forma óptima a la constante de 980 KV de los motores seleccionados. Además, es necesario tener en cuenta los sentidos de los motores, por lo que un par de hélices serán *CW* y un par *CCW*.

Esta configuración permite generar la sustentación necesaria, operando a un régimen de revoluciones moderado, lo que maximiza la eficiencia energética del vuelo estacionario y reduce las vibraciones mecánicas transmitidas a la cámara (aspecto crítico para garantizar la calidad de la captura de vídeo que procesará la unidad *Edge-AI*).



Figura 4.5. Juego de hélices 1045. Fuente: Adaptado de Aliexpress [10].

- **Variadores Electrónicos de Velocidad (ESC – *Electronic Speed Controller*):**

Son los módulos electrónicos encargados de transformar las órdenes de control de bajo voltaje procedentes de la controladora de vuelo en señales de corriente trifásica de potencia, para regular la velocidad de giro de los motores.

Se seleccionan variadores (x4, uno por motor) con una capacidad de corriente continua de 30 A. Optando por unidades con firmware tipo BLHeli, para asegurar una respuesta de aceleración lineal e inmediata.



Figura 4.6. Variador Electrónico de Velocidad ESC BLHeli 30A. Fuente: Adaptado de Aliexpress [11].

Teniendo en cuenta que los motores elegidos presentan un consumo máximo de 22 A, esta elección proporciona un margen de seguridad que le permite prevenir sobrecalentamientos por picos de demanda durante maniobras más complicadas en la misión.

$$\text{Margen de seguridad} = \left(\frac{I_{ESC} - I_{motor_max}}{I_{motor_max}} \right) \times 100 = \left(\frac{30 - 22}{22} \right) \times 100 \approx 36,36 \%$$

$$\text{Margen de seguridad} = \left(\frac{30 - 22}{22} \right) \times 100 \approx 36,36 \%$$

Un margen de seguridad mayor al 30% es suficiente para garantizar que los ESC no trabajen al límite ni sufran estrés térmico.

4.1.4. Placa de Distribución de Potencia (*PDB*)

La placa de distribución de potencia consta de circuito impreso de cobre de alto grosor integrada en el centro del F450, diseñada para la gestión y acondicionamiento del flujo de energía principal de la plataforma.

- Recibe la alimentación directamente de la batería principal de litio, y distribuye la corriente de alta intensidad hacia las líneas de entrada de los cuatro variadores *ESC*. Además, su disposición en forma radial minimiza las caídas de tensión por resistencia óhmica y desacopla el ruido magnético.
- Regulación y acondicionamiento secundario (*BEC*): Incorpora dos circuitos eliminadores de batería (*BEC – Battery Eliminator Circuit*), basados en reguladores que son capaces de suministrar dos buses de tensión continua estabilizada.

Línea de 5V: Destinada a la alimentación de los subsistemas de control de bajo voltaje y receptores de radiofrecuencia.

Línea de 12 V: Reservada para la alimentación de la carga útil de captura de imagen.

La integración de estos reguladores en la *PDB* elimina la necesidad de fuentes de alimentación auxiliares, optimizando la masa total del dron y preservando el margen de masa máxima al despegue (*MTOW*).

Con todo esto, se ha seleccionado la Placa de Distribución de Potencia Maket-XT60 con doble regulador BEC (5V/12V), ya que:

- Cuenta con doble BEC regulado, soporta las baterías 3-4S, (que son justo las que se necesitan, admitiendo tensiones de 7 V a 18 V, para usar sin riesgo de sobrecalentamiento de los reguladores).
- Tiene una capacidad de corriente elevada, soportando un flujo continuo de 4 x 25 A con picos de hasta 4 x 30 A, lo que cubre la demanda de los ESCs y los motores.
- Cuenta con un conector XT60 directamente soldado para la toma de la batería principal.
- Consta de las dimensiones estándar para el patrón de taladros, que encaja directo en las placas centrales del chasis F450 (36 x 36mm).

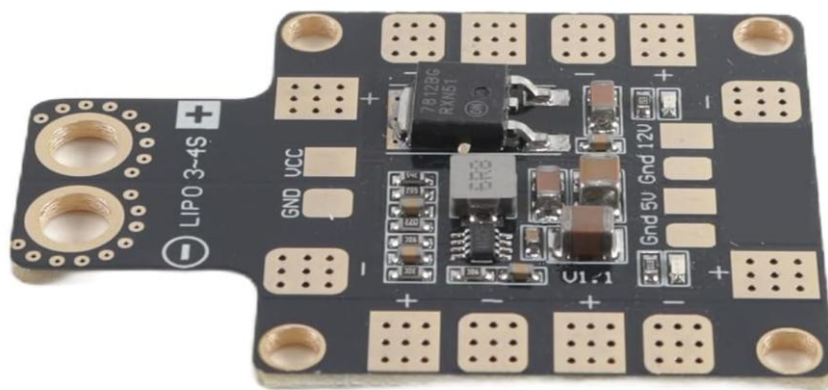


Figura 4.7. Placa de distribución de potencia PDB Maket XT60. Fuente: Adaptado de Amazon [12].

Por último, toda esta arquitectura se alimenta gracias a la fuente de alimentación principal (batería), definida en “4. DISEÑO E INTEGRACIÓN DEL HARDWARE”

4.1.5. Esquema final estructura y propulsión

Por tanto y por recapitular, el funcionamiento de la propulsión de plataforma aérea quedaría definido de la siguiente manera:

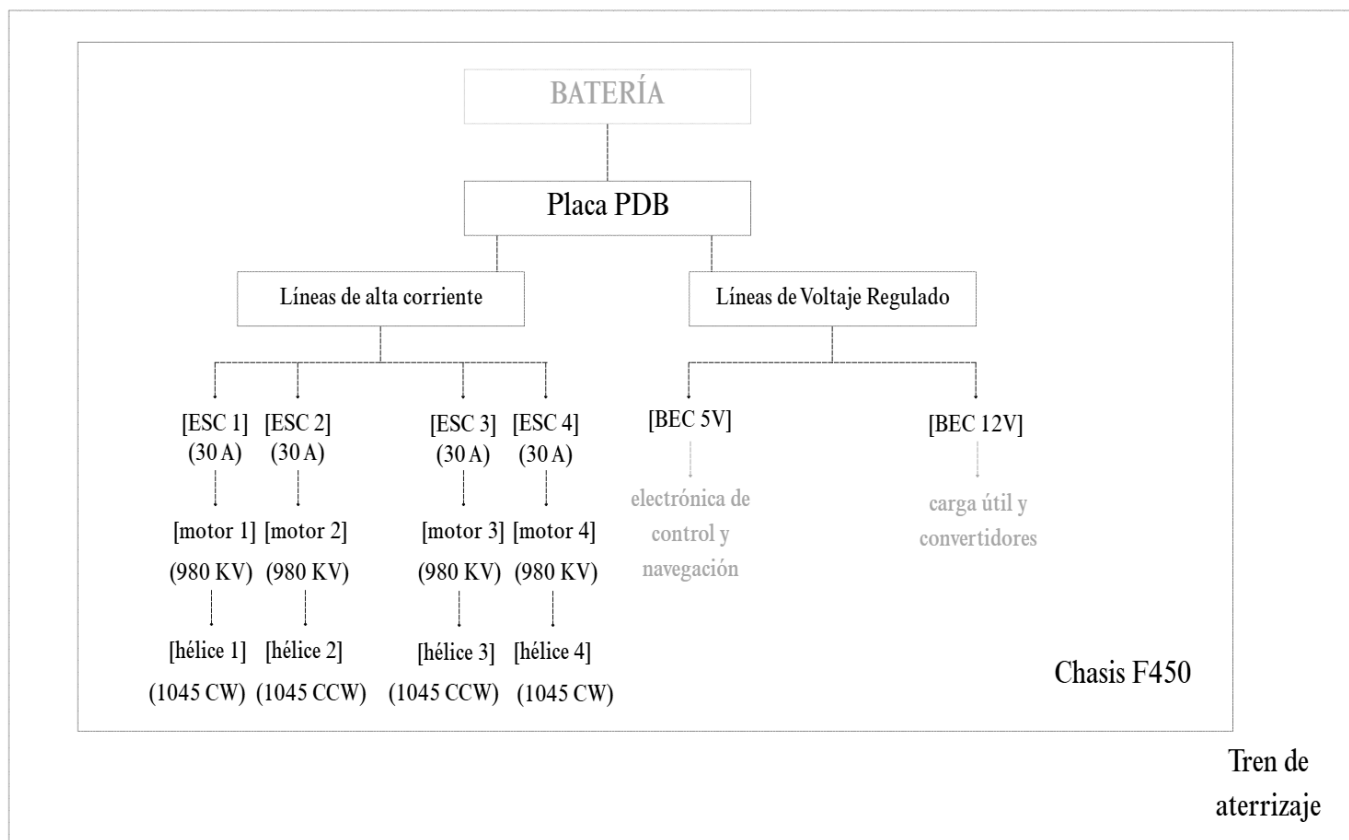


Figura 4.8. Diagrama de bloques de la arquitectura de distribución de potencia y grupo motopropulsor del UAS.

Fuente: Elaboración propia

4.2. Subsistema de control de vuelo y alimentación

Una vez caracterizada la plataforma física, el grupo motopropulsor y la red de distribución de energía, resulta necesario definir la arquitectura electrónica encargada de la navegación autónoma, la estabilización dinámica y el enlace telemétrico del UAS.

Este subsistema actúa como la unidad lógica central de la aeronave, procesando la información para ejecutar las maniobras de pilotaje. Asimismo, establece los canales de comunicación bidireccional, tanto con la estación de control en tierra (*GCS*) para la supervisión de la misión, como con el subsistema de procesamiento encargado (*Edge-AI*) para la transmisión de coordenadas *GPS* y datos de telemetría necesarios para georreferenciar las anomalías y defectos detectados sobre la superficie de las placas.

4.2.1. Controladora de vuelo

La controladora de vuelo (*Flight Controller*) es la unidad electrónica central encargada de mantener el dron estable en el aire y dirigir todos sus movimientos. Es como el “cerebro” del pilotaje de la misión: sus sensores internos (giroscopios, acelerómetros, barómetro) miden constantemente inclinación y altura del dron. Con estos datos, la placa calcula en cuestión de milisegundos las correcciones necesarias y envía órdenes eléctricas a los variadores *ESC* para acelerar o frenar cada motor.

Para el prototipo se ha seleccionado la placa **Pixhawk 6C**, por ser el estándar de hardware abierto en investigación. Su elección viene justificada por:

- Compatibilidad con software libre. Permite utilizar programas de navegación abiertos y totalmente configurables (*ArduPilot / PX4*), evitando las limitaciones de los sistemas comerciales cerrados.
- Conexión con la Raspberry Pi. Aunque se desarrollará más adelante, es importante destacar que incluye puertos serie (*UART*) para comunicarse por cable con la Raspberry Pi que se va a utilizar. A través del protocolo *MAVLink*, la Pixhawk le envía a la Raspberry Pi la posición *GPS* y la altitud en tiempo real. Esto es imprescindible para que cuando la IA detecte un fallo en una de las placas, se sepan las coordenadas exactas.
- Cuestiones de seguridad. Permite programar rutas automáticas por puntos de paso (*waypoints*) y activar rutinas de emergencia, como el retorno automático al punto de despegue (*RTL*) si se pierde la señal de radio o baja la batería (con riesgo de que se acabe y lo que ello conlleva).



Figura 4.9. Controladora de vuelo Holybro Pixhawk 6C. Fuente: Adaptado de RC Innovations [13]

Una vez introducida la Pixhawk, es importante pincelar el protocolo *MAVLink* (*Micro Air Vehicle Link*): Es un protocolo de software (un lenguaje digital) serie de alta eficiencia y reducida sobrecarga, diseñado específicamente para la transmisión de telemetría y comandos en microaeronaves no tripuladas.

Cabe destacar que la selección de una unidad de control de vuelo de semejantes prestaciones profesionales constituye una de las partidas de mayor peso en el presupuesto de ejecución del hardware del UAS (aprox. 192€). No obstante, el incremento en el coste directo del componente queda justificado en el marco de la ingeniería del proyecto, ya que garantiza la máxima precisión inercial, cuenta con redundancia en sensores y tiene disponibles puertos serie de alta velocidad para la comunicación fluida por protocolo *MAVLink* con la unidad de procesamiento *Edge-AI*.

Este incremento sustancial en el presupuesto de ejecución directa es una de las justificaciones de que, en el alcance del presente proyecto, la plataforma se desarrolle únicamente como un diseño conceptual y modelo de integración de arquitectura física y lógica.

4.2.2. Módulo de Navegación GNSS

Es la unidad que se encarga de proporcionar la posición geográfica exacta (definida por la latitud, longitud y altitud) y el rumbo magnético de la aeronave en tiempo real. Combina un receptor satelital multi constelación (también conocido como *GPS*) junto con una brújula digital de tres ejes (magnetómetro). Para evitar cualquier error de medición, se monta sobre un mástil elevado en el F450, alejando la electrónica sensible de los campos electromagnéticos que genera la batería y los motores.



Figura 4.10. Módulo de GPS Holybro M9N GPS. Fuente: Adaptado de RC Innovations [14].

Se selecciona el modelo adoptado **Holybro M9N GPS**, justificado bajo los siguientes puntos:

- Compatibilidad y conexión directa con la Pixhawk 6C. Al pertenecer al mismo ecosistema del fabricante (*Figura 4.9*) incorpora un conector estandarizado que permite un enlace de comunicación directo sin necesidad de realizar soldaduras o adaptar cableado.
- Recepción multiconstelación. Permite el sintonizado simultáneo de hasta cuatro redes satelitales (*GPS*, *GLONASS*, *Galileo* y *BeiDou*), lo que garantiza una fijación de posición rápida y métrica.
- Aislamiento de interferencias magnéticas. Incluye la brújula (*IST8310*) dentro del propio módulo elevado sobre el mástil, evitando desviaciones de rumbo provocadas por la corriente continua que circula por la *PDB*.

- Navegación autónoma y estabilidad de vuelo. Proporciona los datos posicionales requeridos para que la Pixhawk mantenga la posición fija de forma automática y ejecute misiones de inspección programadas.
- Georreferenciación para la unidad de *Edge-AI*. Suministra de forma continua las coordenadas de vuelo a la Raspberry Pi a través del protocolo *MAVLink*. Con ello se logra que en el instante en el que se detecta un fallo óptico sobre una de las placas, esta quede etiquetada automáticamente con su ubicación exacta.

4.2.3. Emisora de Radiocontrol (Estación Manual de Tierra)

Es el dispositivo portátil de tierra equipado con *joysticks* (palancas de mando), conmutadores y un transmisor de radiofrecuencia. Su función es codificar de forma continua las acciones manuales del operador y emitir las órdenes hacia el vehículo. La presencia de este componente es un requisito de seguridad obligatorio exigido por la normativa de aviación civil de AESA/EASA [4]/[5] para la operación de UAS.

Para la estación de tierra se ha seleccionado la emisora **RadioMaster Boxer** en su versión **2.4GHz ELRS**, que constituye la solución más extendida y estandarizada en el ámbito de los UAS.



Figura 4.11. Emisora RadioMaster Boxer 2.4GHz LBT. Fuente: Adaptado de Aliexpress [15].

Cabe destacar que se disponen de dos versiones de la RadioMaster Boxer, una de ellas es la FCC y la otra es la LBT. La diferencia radica en la normativa regional de telecomunicaciones y la regulación sobre el uso del espectro de radiofrecuencia.

La LBT (*Listen Before Talk*) es la versión desarrollada para cumplir obligatoriamente con la normativa europea de telecomunicaciones (normas ETSI de la Unión Europea) [16]. Incorpora un protocolo de seguridad que obliga a la emisora a “escuchar” el canal durante unos microsegundos antes de emitir datos (“hablar”). Si detecta que otra señal está usando esa frecuencia, salta a otro canal para no provocar interferencias.

Su elección además se fundamenta en:

- Integra el protocolo de transmisión *ExpressLRS (ELRS)* a 2,4 GHz, que enlaza de forma directa con el receptor sin requerir módulos externos.
- Ofrece un tiempo de respuesta óptimo e inmunidad frente a interferencias electromagnéticas en entornos industriales.

4.2.4. Receptor de Radiocontrol

Es el módulo electrónico de tamaño reducido instalado a bordo del F450. Su función es captar las señales de radiofrecuencia de 2,4GHz emitidas por la estación de tierra y transferir las órdenes de pilotaje a la Pixhawk 6C en tiempo real.

Se selecciona la **Radiomaster RP1 ELRS 2.4GHz** (versión LBT para Europa) por tres motivos:

- Conexión directa y rápida. Enlaza de forma nativa con la emisora *RadioMaster Boxer* con una respuesta ≤ 5 ms.
- Peso mínimo. Pesa 2,2 gramos, por lo que prácticamente no añade peso al dron y deja todo el margen para la Raspberry y la cámara.
- Conexión sencilla con la Pixhawk 6C. Se conecta a la controladora mediante un cable de 4 hilos (alimentación 5V, masa *GND*, transmisión *TX* y recepción *RX*) usando el protocolo digital *CRSF*, la Pixhawk está ya “preparada de fábrica” con estos conectores.



Figura 4.12. *RadioMaster RP1 2.4GHz ELRS Nano Receiver (LBT Specification)*. Fuente: Adaptado de AliExpress [17].

La interconexión física se realiza mediante la soldadura de cuatro conductores de silicona a la alimentación de 5 V, masa, TX y RX sobre los Pads de la placa, los cuales se ensamblan en el extremo opuesto a un conector JST-GH para su acoplamiento directo al puerto dedicado de radiocontrol (*SBUS* de la Pixhawk), garantizando la transmisión de órdenes.

4.2.5. Módulo de Telemetría de Radio

Este subsistema de comunicaciones de la plataforma se divide en dos canales independientes:

- **Enlace telemétrico de operación para el control en tierra.**

Es un transceptor de radiofrecuencia bidireccional que establece un enlace de datos serie entre la controladora Pixhawk 6C y la Estación de Control en Tierra ejecutada en un equipo portátil.

Se ha seleccionado el módulo comercial **Holybro SIK Telemetry Radio** de 433MHz, que constituye la solución estándar y más extendida en el ámbito de prototipado con controladoras Pixhawk. Además, consta de 3 ventajas clave:

- Dispone de conectores nativos compatibles con los puertos de telemetría de la controladora, lo que facilita la integración física inmediata.
- Opera en la frecuencia ISM de 433 MHz, autorizada para el uso libre en la Unión Europea, evitando frecuencias reservadas.

- Permite enviar directamente las rutas de vuelo sobre los paneles fotovoltaicos directamente desde el ordenador de tierra, sin necesidad de conectar cableado extra al dron.



Figura 4.13. Sistema de telemetría Holybro SiK 433MHz. Fuente: Adaptado de Holybro [18].

Este sistema consta de dos transceptores; uno de ellos instalado a bordo de la aeronave, conectado al puerto de telemetría de la Pixhawk (*TELEM 1*), el otro conectado mediante USB a la Estación de Control en Tierra (GCS) en el equipo portátil, permitiendo así la comunicación bidireccional en tiempo real. Y gracias a él, se puede ver el mapa en tiempo real, la batería distante, el modo de vuelo, enviarle rutas de puntos (*waypoints*) e incluso la orden de volver a casa (*Return to Home*).

- **Módulo de Identificación Remota Directa (*Direct Remote-ID*):**

En estricto cumplimiento del marco regulatorio de la Agencia Estatal de Seguridad Aérea [4] y la normativa europea (Reglamentos delegado UE 2019/945 y de Ejecución UE 2019/947 [5]), la plataforma debe incorporar un módulo físico de identificación remota (*DRI*).

Este dispositivo se conecta a la Pixhawk a través de un puerto serie de telemetría secundario (*TELEM 2*) para la lectura continua del flujo de datos *MAVLink* procedentes del sensor *GNSS*. El módulo emite de manera permanente e inalámbrica (vía 2,4 GHz / *Bluetooth*) los parámetros exigidos por la autoridad aeronáutica:

- Número de registro oficial del operador de UAS.
- Número de serie único del módulo de identificación.
- Posición geográfica exacta (*GPS*), altitud. Velocidad y rumbo en tiempo real.
- Ubicación del piloto o punto de despegue (*Home Position HP*).

Esta emisión en abierto garantiza la trazabilidad operativa y la capacidad de supervisión en tiempo real por parte de los cuerpos de seguridad y autoridades del espacio aéreo durante las maniobras de inspección en placas solares.



Figura 4.14. Identificador remoto de UAS Holybro. Fuente: Adaptado de Holybro [19].

Cabe señalar que la interconexión física de ambos subsistemas (el transceptor de telemetría de radio y el módulo de identificación remota) con la controladora de vuelo se efectúa mediante latiguillos estandarizados provistos de conectores JST-GH de 6 pines en sus respectivos puertos dedicados, garantizando tanto la alimentación eléctrica de los dispositivos como la integridad en la transmisión del bus de datos *MAVLink*.

4.2.6. Esquema final control de vuelo

Con todos estos elementos, se define la arquitectura integral de control de vuelo, navegación y comunicaciones de la plataforma F450.

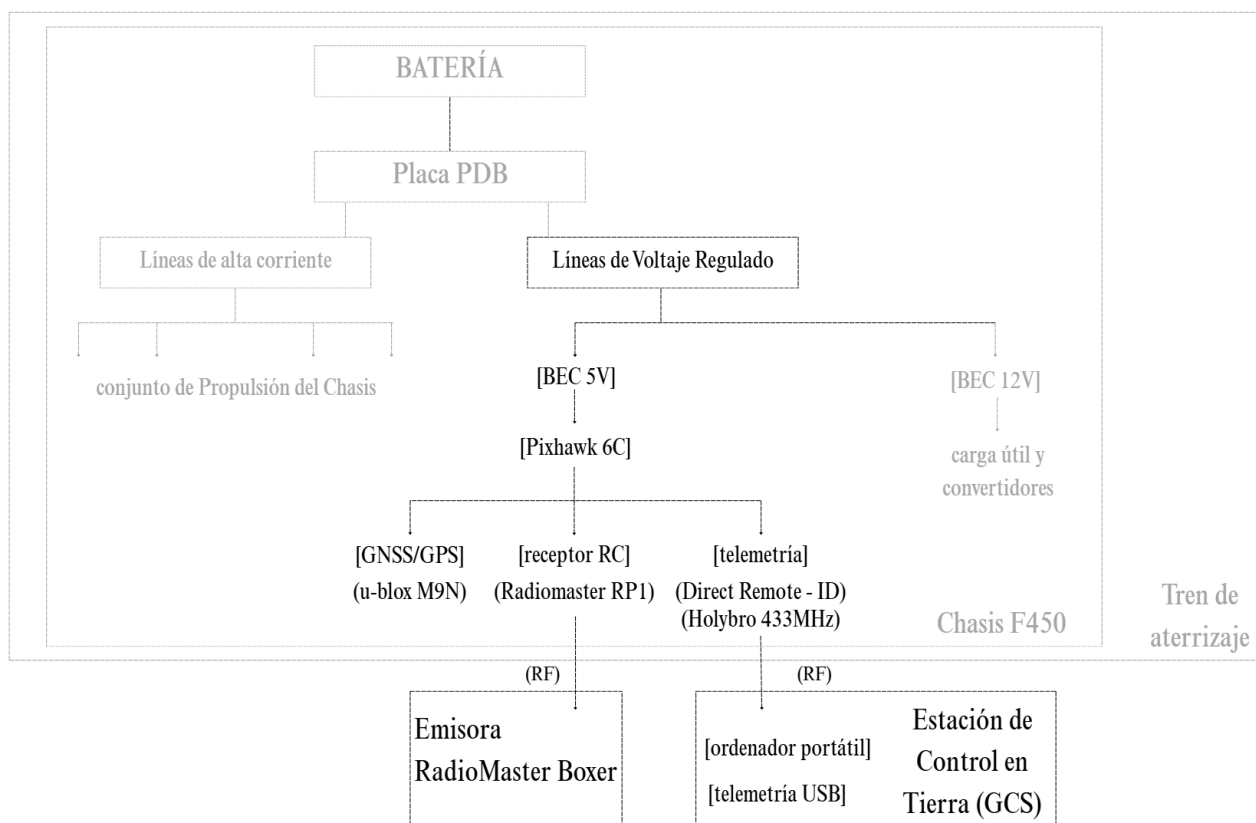


Figura 4.15. Diagrama de bloques de la arquitectura integral de control de vuelo, navegación y comunicaciones del UAS. Fuente: Elaboración propia.

Como se sintetiza en “Figura 4.15” el sistema articula dos canales por radiofrecuencia independientes y diferenciados: el primer enlace al mando a distancia mediante el receptor Radiomaster RP1, para la intervención manual del piloto en vuelo desde la emisora RadioMaster Boxer, y un segundo canal de telemetría bidireccional mediante el ya definido kit Holybro SiK, que conecta la Estación de Control en Tierra (GCS) en el equipo portátil, garantizando la supervisión de parámetros críticos y la carga inalámbrica de rutas autónomas para las misiones si se quisiera.

Además, a fin de cumplir con la normativa, el sistema integra el módulo de Identificación Remota Directa, garantizando de forma continua en abierto la posición del dron y el registro oficial del operador

4.3. Arquitectura de alimentación y acondicionamiento eléctrico

Para que todo el UAS funcione, la fuente de energía principal es una batería tipo *LiPo* (o acumulador electroquímico de Polímero de Litio). Su función es almacenar y suministrar de forma continuada la energía requerida tanto por el subsistema de propulsión de alta potencia como por la electrónica de navegación y cómputo *Edge-AI*.

Como se ha definido anteriormente, en el apartado “4.3. Arquitectura de alimentación y acondicionamiento eléctrico” la conexión principal entre la batería y la Placa de Distribución de Potencia (PDB) se realiza mediante un conector estándar *XT60* que se conecta fácilmente con la placa elegida.

Se ha seleccionado una **batería con arquitectura de 4 celdas en serie (LiPo 4S)**, con una capacidad nominal de 5200mAh y una tasa de descarga continua de 35C. Esta elección se adapta de forma óptima a las especificaciones de los elementos previamente definidos:

- Adaptación al sistema de propulsión: Proporciona una tensión nominal de 14,8V (llega a alcanzar los 16,8V a plena carga). Este rango de voltaje se sitúa dentro de la ventana operativa ideal de los ESC de 30 A y permite que los motores de 980 KV giren a las revoluciones necesarias con las hélices 1045, para generar el empuje requerido por el F450.
- Margen de seguridad en entrega de corriente:

$$I_{m\acute{a}x} = C \cdot C_{descarga} = 5,2 \text{ Ah} \times 35 \text{ h}^{-1} = 182 \text{ A}$$

$$I_{pico,ESC} = N_{motores} \cdot I_{ESC,m\acute{a}x} = 4 \times 30 \text{ A} = 120 \text{ A}$$

$$M_{seguridad} = \left(\frac{I_{m\acute{a}x} - I_{pico,ESC}}{I_{pico,ESC}} \right) \times 100 = \left(\frac{182 \text{ A} - 120 \text{ A}}{120 \text{ A}} \right) \times 100 = 51,67 \%$$

Donde:

$I_{m\acute{a}x}$:	Corriente máxima continua, medida en Amperios (A).
C :	Capacidad nominal, expresada en Amperios-hora (Ah).
$C_{descarga}$:	Tasa de descarga (<i>C-Rating</i>), expresado en h^{-1} .
$I_{pico,ESC}$:	Consumo pico total, medido en Amperios (A).
$N_{motores}$:	Número de motores de la plataforma.
$I_{ESC,m\acute{a}x}$:	Corriente máxima continua que soporta cada ESC, medida en Amperios (A).
$M_{seguridad}$:	Margen de seguridad relativo, en porcentaje (%).

Con una tasa de descarga continua de 35C, el paquete ofrece una capacidad de entrega máxima de hasta 182 A continuos. Dado que el consumo pico combinado de los 4 ESCs es

de 120 A, se dispone de un margen del 51,67%, esto previene caídas bruscas de tensión durante maniobras exigentes y evita el sobrecalentamiento de la celda.



Figura 4. 16. Batería ReadyEDI LiPo 4S 5200mAh 35C. Fuente: Adaptado de [20].

Como apoyo a la operación se contempla disponer de una segunda batería de iguales características, lo que permitiría, tras cada vuelo, sustituir la batería utilizada y continuar con la inspección sin esperar a completar un nuevo ciclo de carga.

Para su mantenimiento y recarga se selecciona un **cargador balanceador SkyRC e680**, compatible con baterías LiPo de hasta 6 celdas y con una potencia máxima de carga de 80W.



Figura 4. 17. Cargador de baterías SkyRC e680 + cable cargador XT60 incluido. Fuente: Adaptado de RCInnovations.

- **Convertidor CC-CC:**

Pero hay que tener en cuenta que la unidad de procesamiento (la *Raspberry Pi*, no definida todavía) exige una fuente de alimentación continua de 5V con capacidad de entregar hasta 5A durante la ejecución de los algoritmos de IA.

Por tanto, para evitar caídas de tensión en la Pixhawk 6C o ruidos en los sensores, no se utiliza la salida de la controladora de vuelo.

Para solucionar esta interfaz de potencia se integra un convertidor conmutado reductor que se conecta a la salida auxiliar de la PDB. Se selecciona el **Matek UBEC Duo 4A/5A** porque entrega una tensión regulada de 5,2 V y 5 A continuos con una eficiencia superior al 90%. La salida de este convertidor cumple con la especificación oficial recomendada para compensar la caída de tensión en el cableado de la unidad de procesamiento (*USB-C*) y evitar alertas de subvoltaje en el sistema operativo embarcado.



Figura 4.18. Sistema Matek for UBEC 4A 5V (Convertidor Buck). Fuente: Adaptado de Amazon [21].

Recapitulando, el camino recorrido por la corriente es el siguiente:

1. Batería → PDB: La batería de 4 celdas da 14,8 V directos a la Placa de Distribución de Potencia.
2. PDB → UBEC / Convertidor Buck: Desde la PDB salen dos cables (“+” y “-”) hacia la entrada del convertidor Matek UBEC, que se encarga de rebajar esos 14,8 V a 5,2 V y 5 A.
3. UBEC → Cable USB-C → Unidad de procesamiento: A las patillas de salida de 5,2 V se sueldan los cables de alimentación de un USB-C. El extremo con la clavija se enchufa directamente a la unidad de procesamiento.

Y, por último, a modo de síntesis y para evitar ambigüedades en el esquema eléctrico del prototipo, se resumen los dos convertidores reductor (*BEC/UBEC*) integrados en el circuito y la función operacional que desempeña cada uno de ellos:

- BEC/Módulo de potencia de aviación: Alimentado a 14,8 V desde la PDB, regula y entrega una tensión continua de 5 V para garantizar el funcionamiento estable y libre de interferencias de la controladora de vuelo y sus periféricos de navegación.
- UBEC dedicado (Matek UBEC Duo): Conectado igualmente en paralelo al bus de 14,8 V de la batería, suministra una línea independiente de 5,2 V y 5 A mediante interfaz USB-C para abastecer la demanda de la unidad de procesamiento de *Edge-AI*.

La disposición en paralelo de ambos convertidores asegura que cada uno reciba la tensión nominal completa de la batería de litio (14,8 V) sin generar caídas de voltaje en el bus principal. Dado que la *LiPo 4S* seleccionada ofrece una capacidad de descarga de hasta 182 A continuos y la demanda combinada de la electrónica es inferior a 3 A, la coexistencia de ambos reguladores no sobrecarga el circuito ni provoca caídas de tensión, garantizando un aislamiento total entre la plataforma de vuelo y la carga útil.

4.4. Carga útil (*payload*)

La misión principal del prototipo exige la integración de una carga útil (*payload*). En este caso, la función principal de la *payload* del dron es capturar imágenes de la planta fotovoltaica y procesar algoritmos de Inteligencia Artificial en tiempo real sobre la propia plataforma aérea.

Este subsistema es independiente de la aviónica de navegación, y está formado por la unidad de procesamiento y la unidad de procesamiento de imágenes, ambos alimentados por la línea de 5,2 V/ 5 A desde el convertidor *UBEC* descrito en el apartado anterior.

4.4.1. Unidad de procesamiento embarcada y acelerador NPU

La inspección fotovoltaica en tiempo real requiere que el dron sea capaz de analizar las imágenes en el propio instante en el que las captura. A esta capacidad de ejecutar modelos de Inteligencia Artificial directamente en el dispositivo embarcado, sin depender de servidores externos, se le denomina procesamiento en el borde (*Edge-AI*).

Selección de la unidad de procesamiento:

Para la elección del procesador embarcado se han evaluado tres familias tecnológicas:

- Microcontroladores tradicionales (Arduino / ESP32). Son dispositivos diseñados para tareas sencillas de control y lectura de sensores básicos. Se descartan, ya que ni su memoria ni su capacidad de cálculo son suficientes para manejar el flujo de vídeo de una cámara y ejecutar algoritmos de *Edge-AI*.
- Módulos con procesador gráfico dedicado (tipo NVIDIA Jetson). Ofrecen gran potencia de cálculo (ideal para la IA), pero se descartan por su elevado coste económico, mayor peso (factor muy importante en la carga útil) y consumo eléctrico superior (lo que reduciría drásticamente la autonomía de vuelo del dron).
- Ordenadores mono-placa (*Single Board Computers - SBC*). La Raspberry Pi 5 (version de 8 GB de RAM) representa un punto de equilibrio óptimo para este prototipo. Proporciona capacidad de cálculo necesaria para ejecutar algoritmos de detección de fallos con un peso reducido, un consumo contenido y coste asequible. Como se ha ido adelantando a lo largo del capítulo, se ha optado por esta unidad de procesamiento.



Figura 3.1. Raspberry Pi 5 8 GB. Fuente: Adaptado de Kubii [23].



Figura 3.2. Raspberry Pi Active Cooler. Fuente: Adaptado de RaspberryPi [24].

Control térmico y refrigeración (*Active Cooler*):

Al ejecutar tareas de procesamiento continuo y gestión de comunicaciones, la Raspberry genera un calentamiento importante en su procesador central. Para asegurar un funcionamiento estable, se instala el sistema de refrigeración oficial *Active Cooler* (“Figura 3.2”). Este módulo combina un disipador de aluminio acoplado sobre los chips principales con un ventilador con control de velocidad por temperatura, garantizando la disipación del calor dentro de la carcasa sin comprometer el espacio del chasis.

Aceleración de la IA (Módulo *Hailo-8 HAT*):

Aunque la Raspberry Pi 5 ya dispone de este sistema de refrigeración activa, la ejecución directa y continuada de modelos de visión artificial sobre su procesador principal generaría un consumo energético elevado y una velocidad de procesamiento reducida.

Para solucionar esto, se añade una pequeña tarjeta de expansión llamada Hailo-8 HAT, que se conecta directamente a la Raspberry Pi 5 mediante una cinta plana de datos. Este módulo incorpora un chip especializado (NPU) diseñado exclusivamente para resolver los cálculos de la IA. Al delegar esta tarea en el acelerador Hailo, el sistema logra analizar el vídeo en tiempo real a más de 30 FPS (imágenes por segundo) con un consumo muy bajo (2,5 W), dejando el procesador de la Raspberry libre para gestionar la telemetría y el envío de datos.

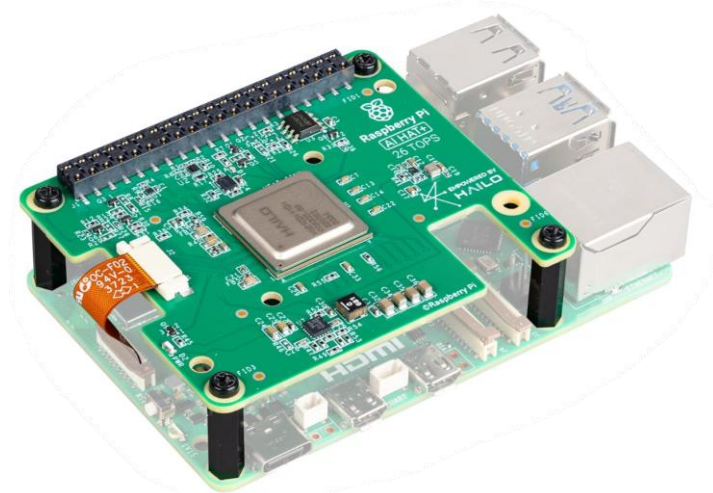


Figura 4.19. Raspberry AI Hailo-8 HAT 26 TOPS. Fuente: Adaptado de Amazon [24].

Alimentación eléctrica y cables de conexión:

Para evitar caídas de tensión o apagones durante la misión, la Raspberry Pi 5 recibe una corriente estable de 5,2 V procedente del módulo regulador *Matek UBEC Duo*. La conexión se realiza mediante un cable adaptado: conector USB-C macho cuyos cables corrientes y masa (5V y GND) se sueldan a las salidas del módulo UBEC.

Lectura de datos GPS (MAVLink) y envío de alertas por Wi-Fi:

Para que el dron sepa en qué lugar exacto se encuentra cada fallo, la Raspberry se conecta a la controladora de vuelo. Esta unión se realiza mediante un cable serie (*UART*) que va desde el puerto (*TELEM 3*) de la Pixhawk hasta los pines de entrada (*GPIO*) de la Raspberry Pi, permitiendo leer de forma continua las coordenadas y la altitud bajo el protocolo *MAVLink*.

Finalmente, cuando la red neuronal detecta un defecto en un panel fotovoltaico, la Raspberry envía una alerta en tiempo real con la foto y las coordenadas GPS hacia el ordenador de tierra utilizando su tarjeta Wi-Fi integrada. Esta elección se justifica para evitar añadir el peso y el gasto de batería que supondría un módulo de telefonía 4G/5G.

Con el fin de evitar la saturación de la CPU principal y eliminar el riesgo de estrangulamiento térmico, la Raspberry Pi 5 se complementa con el módulo de aceleración *hardware* Hailo-8 HAT.

Además, cuenta con *CSI* (Conexión directa de vídeo), lo que permite conectar el módulo de cámara por cable, asegurando que las imágenes lleguen al procesador de forma instantánea y sin saturar los puertos de la placa.

Por otro lado, se le puede añadir un sistema de ventilación que evita que el equipo se sobrecaliente y pierda rendimiento cuando se trabaja de forma continuada en la misión.

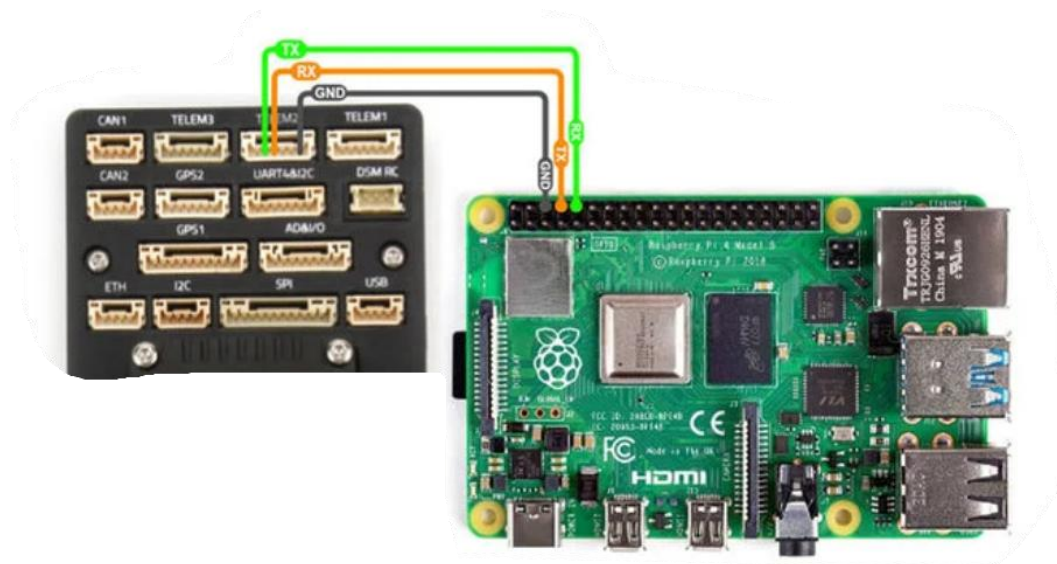


Figura 4.20. Enlace UART del cable de telemetría. Fuente: Adaptado de UnmannedTech [26].

4.4.2. Sensor óptico de adquisición de datos: Cámara RGB

Su función es imprescindible ya que es el sensor encargado de la captación de imágenes en el espectro visible de la planta fotovoltaica durante el vuelo. Estas imágenes sirven como entrada directa para que el módulo con Inteligencia Artificial, que se encuentra en la Raspberry Pi 5, detecte y clasifique en tiempo real los fallos mecánicos y estructurales en las placas (roturas de vidrio, suciedad, sombreados...).

Como se ha introducido anteriormente, este módulo se conecta de manera directa a la Raspberry Pi 5 mediante un cable plano de cinta (*FPC*) a través del bus nativo MIPI CSI. Esta interfaz de alta velocidad es imprescindible para transmitir el flujo de vídeo no comprimido directamente a la memoria de la unidad de procesamiento con un retardo (latencia) inferior a 15 ms.

Esta drástica reducción del tiempo de transmisión evita los cuellos de botella y retardos habituales de las cámaras conectadas por puerto USB (los cuales suelen superar los 200 ms). Gracias a esta baja latencia, en el preciso instante en que la red neuronal detecte un defecto sobre las fotos, el programa lee de forma simultánea las coordenadas geográficas (mediante el *GPS*) que la controladora de vuelo envía continuamente a la Raspberry Pi mediante el protocolo MAVLink. Esto garantiza que la posición geográfica registrada coincida exactamente con el panel afectado, evitando que el desplazamiento del dron genere desfases.

Por tanto, y de acuerdo con el requisito de disponer de una interfaz nativa *CSI*, se ha seleccionado la cámara **Raspberry Pi High Precision Camera** (basada en el sensor óptico **Sony IMX477** de 12,3 Megapíxeles) equipada con lente de enfoque natural.

La elección se justifica en:

- Resolución óptica. Sus 12,3 MP garantizan la resolución espacial necesaria para identificar los defectos buscados.

- Conexión CSI de baja latencia. Dispone de salida nativa para asegurar la transmisión instantánea por CSI.
- Control de nitidez mediante obturación rápida. Permite configurar altas velocidades de obturación, eliminando el desenfoque por movimiento derivado del desplazamiento del dron.
- Masa reducida y amortiguación pasiva. Su peso (<30 g) permite fijarla con soportes de silicona (*dampers*), lo que aísla al sensor de las vibraciones de los motores sin tener que recurrir a un estabilizador motorizado (*gimbal*), lo que optimiza la masa total y el precio final.

Por último, la selección garantiza la compatibilidad nativa con la procesadora por pertenecer al mismo ecosistema de desarrollo.



Figura 4.21. Módulo de cámara con sensor IMX477 conectado a Raspberry Pi 5. Fuente: Adaptado de AliExpress [27].

4.4.3. Viabilidad de sensor termográfico IR

Asimismo, en el diseño de la carga útil del UAS se ha evaluado y denegado la inclusión de sensorización termográfica infrarroja *IR*, que constituye la herramienta tradicional para la detección de puntos calientes en instalaciones solares. La decisión de prescindir de una cámara térmica y focalizar la misión en un sensor óptico *RGB* responde a tres criterios principales de viabilidad del prototipo:

En primer lugar, las cámaras termográficas suponen una restricción severa en cuanto a masa y presupuesto de adquisición, con costes que habitualmente superan los 1500€, lo que comprometería tanto la autonomía de vuelo como la viabilidad económica de la solución propuesta.

En segundo lugar, mientras que la termografía identifica la manifestación térmica del fallo, la captura de imágenes *RGB* de alta resolución permite a los algoritmos de visión artificial determinar la causa raíz física, como la presencia de grietas, la acumulación de suciedad...

Finalmente, la integración de un flujo de vídeo *RGB* directo por la interfaz *CSI* evita la sobrecarga de realizar calibraciones térmicas complejas en tiempo real, optimizando el rendimiento de la Raspberry Pi 5 para ejecutar el modelo con la menor latencia posible.

4.4.4. Esquema final de carga útil y alimentación

Con los elementos de *payload* y batería seleccionados, el esquema de los últimos apartados, con la arquitectura de la carga útil para el procesamiento *Edge-AI* queda sintetizado en “Figura 4.22”. En él se destaca el aislamiento eléctrico de la Raspberry Pi mediante la salida dedicada con el UBEC y la captura de vídeo vía *MIPI CSI*.

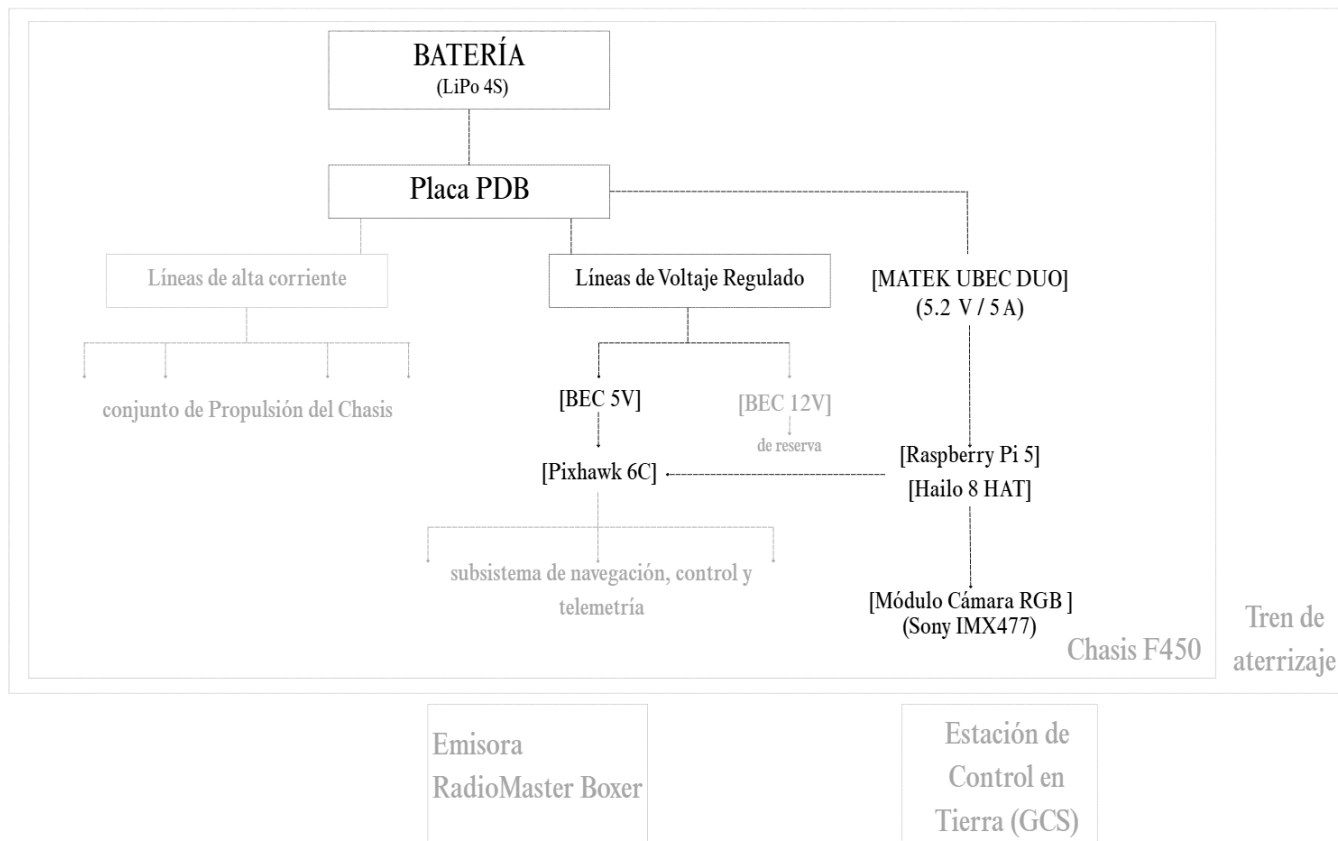


Figura 4.22. Esquema de la carga útil y la alimentación en el UAS. Fuente: Elaboración propia.

4.5. Integración y Balance Global

4.5.1. Esquemas maestros de integración

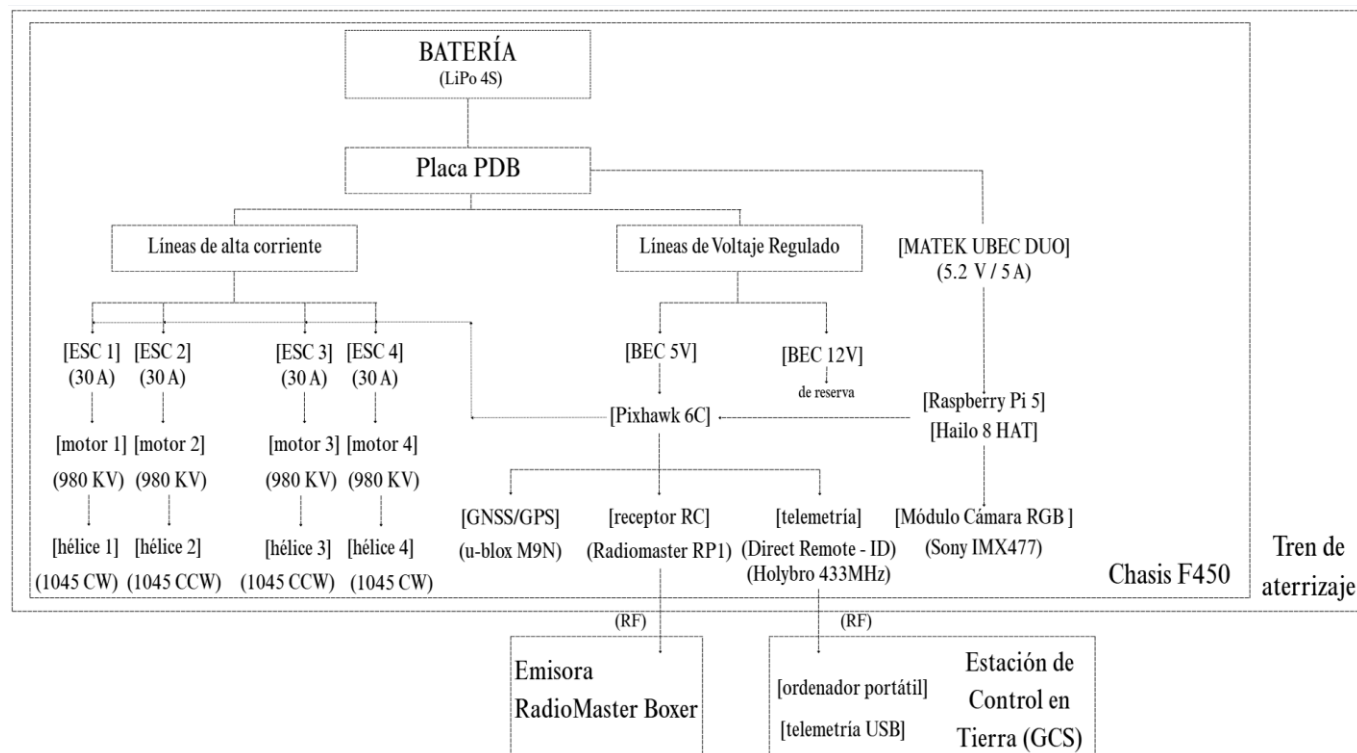


Figura 4.23. Esquema maestro de integración de componentes del UAS. Fuente: Elaboración Propia.

Origen (1)	Destino (2)	Cable/Bus	Conex. (1)	Conex. (2)	Montaje
Batería LiPo 4S	Placa PDB	Manguera 14,8 V	XT60 M	XT60 H	Enchufe directo
Placa PDB	ESC (x4)	Alimentación	Pads alta corriente	Extremo libre ESC	Soldadura estaño
BEC 5V (PDB)	Pixhawk 6C	Alimentación 5 V	BEC5V	<i>POWER 1</i>	Soldadura → JST-GH 6p
PDB V _{bat}	Matek UBEC Duo	Entrada 14,8 V	Pads V _{bat}	Inputs UBEC	Soldadura estaño
Matek UBEC Duo	Raspberry Pi 5	Cable adaptado	Output 5 A	USB C M	Soldadura → Enchufe USB-C
Raspberry Pi 5	Hailo-8 HAT	Bus <i>PCIe</i>	Conector PCIe	Conector PCIe HAT	Cinta FPC + tornillería
RadioMaster RP1	Pixhawk 6C	Protocolo <i>CRSF</i>	Pads RP1	<i>PPM/SBUS RC</i>	Soldadura → JST-GH 4p
Radio Holybro SiK	Pixhawk 6C	Bus <i>MAVLink</i> 5V	Puerto SiK	<i>TELEM1</i>	Latiguillo JST-GH 6p a JST-GH 6p
Remote-ID	Pixhawk 6C	Bus <i>MAVLink</i> 5V	Puerto DRI	<i>TELEM2</i>	Latiguillo JST-GH 6p
Pixhawk 6C	Raspberry Pi 5	<i>UART</i>	<i>TELEM3</i>	Pines GPIO (14/15/GND)	JST-GH → Pines DuPont
Sony IMX477	Pi 5 / Hailo-8	Bus <i>MIPI CSI</i>	Conector CSI	Puerto CSI Pi 5	Cinta FPC 15 pines

Tabla 4. 2. Matriz de interconexión física y montaje de componentes del UAS. Fuente: Elaboración propia.³

³ Los términos en cursiva y tono gris corresponden a la etiqueta oficial sobre los puertos físicos de conexión.

El esquema maestro representa la interconexión global del prototipo, tomando como punto de partida la batería LiPo 4S (14,8 V nominales), la cual se conecta a la Placa de Distribución de Potencia (PDB) mediante un terminal de alta corriente XT60. Desde la PDB se bifurcan tres líneas principales de alimentación y datos:

1. Líneas de Alta Corriente (Grupo Motopropulsor):

Conducen la tensión directa desde la batería (14,8 V) hacia los variadores electrónicos de velocidad (ESC de 30 A), encargados de operar los cuatro motores *brushless* de 980 KV distribuidos en los brazos del chasis F450, respetando los sentidos de giro alternados para las hélices 1045 (*CW* y *CCW*).

Los ESC se conectan mediante soldadura directa de sus cables de alimentación positiva y masa a los Pads de cobre de alta corriente de la PDB. Luego, el servocable de 3 pines de cada ESC se conecta a las salidas PWM de la Pixhawk para recibir las órdenes de velocidad.

2. Línea de Control de Vuelo (BEC Integrado):

La PDB integra dos circuitos reguladores BEC (*Battery Eliminator Circuit*), uno de 5 V y otro de 12 V, alimentados ambos a los 14,8 V desde la batería primaria. El regulador de 12 V permanece en “*stand by*” como línea auxiliar para líneas futuras de cargas útiles de mayor requerimiento numérico (como sensores termográficos). Por otro lado, el BEC de 5 V suministra energía a la controladora Pixhawk a través de su puerto dedicado de alimentación (*POWER 1*) (cable adaptado, soldadura de extremos a Pads de salida del BEC e insertar a presión conector JST-GH en la Pixhawk).

A la controladora se interconectan los subsistemas de navegación, radiocontrol y cumplimiento normativo: el módulo GPS, el receptor RC RP1 (vinculado por radiofrecuencia a 2,4 GHz con la emisora del piloto RadioMaster Boxer), transceptor de telemetría, que se comunica por radiofrecuencia con el receptor USB acoplado a la Estación de Control en Tierra (GCS) y el módulo Direct Remote – ID (*DRI*) acoplado al puerto *TELEM2*, el cual emite la posición y el registro del operador para el cumplimiento estricto de la normativa AESA.

3. Línea de Alimentación *Edge-AI* y Bus Inter-Procesador:

En paralelo, la tensión directa de 14,8 V se deriva desde la PDB hacia el convertidor regulado Maket UBEC Duo, que incluye cables de entrada que se sueldan directamente a los Pads positivos y de masa de entrada de batería en la PDB. De su salida regulada parten los cables de 5,2 V / 5 A, que se adaptan mediante un conector USB – C a la entrada de alimentación de la Raspberry Pi.

Sobre la propia Raspberry se integra el acelerador Hailo-8 HAT, conectado a través de la interfaz de alta velocidad PCIe mediante una cinta plana de datos, actuando como Unidad de Procesamiento Neuronal (NPU) para asumir la ejecución pesada del modelo de visión artificial sin sobrecargar la CPU principal.

La Raspberry Pi recibe el flujo de vídeo de la cámara RGB mediante el bus de alta velocidad *MIPI CSI*. Asimismo, la unidad *Edge* se interconecta con la Pixhawk mediante un enlace serie *UART* (latiguillo con extremo JST-GH en el puerto *TELEM3* de la Pixhawk y extremos pines conectados GPIO de la Raspberry Pi) para el intercambio de telemetría bajo el protocolo *MAVLink*, sincronizando en tiempo real la posición de la aeronave con cada imagen procesada por la red neuronal.

4.5.2. Inventario de hardware

En “*¡Error! No se encuentra el origen de la referencia..3*” se puede observar el inventario físico detallado del prototipo desarrollado, integrando los componentes del sistema motopropulsor, la unidad de control de vuelo, la carga útil y el equipamiento asociado de estación de tierra. Se desglosan las unidades necesarias por componente, y la masa y precio totales de componente, sirviendo de base cuantitativa para el análisis de balance de masas y posterior evaluación presupuestaria del proyecto.

Elemento	Ud.	Peso Total [g]	Precio Total [€]
Chasis F450	x1	370	8,00 €
Tren de aterrizaje	x1	200	6,30 €
Motor SunnySky X2212 III 980 KV	x4	228	54,80 €
Hélice 1045	x4	26	4,26 €
ESC BLHeli 30A	x4	112	25,96 €
PDB XT60	x1	20	8,99 €
Pixhawk 6C Holybro	x1	35	192,00 €
Módulo Holybro M9N GPS	x1	32	70,00 €
RadioMaster Boxer LBT	x1	-	168,43 €
RadioMaster RP1 ELRS Nano Receiver	x1	2	20,99 €
Holybro Remote ID	x1	16,5	35,55 €
Módulo telemetría Holybro SiK 433 MHz	x1	21	52,00 €
Batería LiPo 4S 5200 mAh ReadyEDI	x2	475	197,26 €
Cargador SkyRc e680	x1	-	49,90 €
Matek UBEC 4A 5V	x1	25	60,29 €
Raspberry Pi 5	x1	60	204,00 €
Raspberry Pi 5 Active Cooler	x1	28	6,00 €
Hailo-8 HAT	x1	5	125,95 €
Raspberry Pi 5 IMX477 Camera	x1	40	106,00 €
Cableado, conectores, tornillería, estaño...	-	45	15,00 €
TOTAL	-	1740,5	1411,68 €

Tabla 4. 3. Inventario de elementos, peso y precio del UAS.

La masa total de la plataforma en orden de vuelo (*Maximum Take-Off Weight, MTOW*) se fija en 1,74 kg; incluyendo la estimación global de 45g destinada a todas las líneas de cableado, conectores y elementos de fijación. Cabe destacar que el “mando del piloto”, el RadioMaster Boxer LBT, no se ha tenido en cuenta en los cálculos de peso ya que no forma parte de la aeronave, del mismo modo que solo se ha tenido en cuenta uno de los receptores RP1 (el otro va conectado al ordenador de tierra), y del mismo modo que no se ha tenido en cuenta la estación de tierra ni los elementos de apoyo al vuelo (la segunda batería y el cargador de estas).

En cuanto al balance económico, la inversión directa acumulada en componentes para el UAS, añadiendo al cálculo esta vez el segundo receptor RP1, el RadioMaster Boxer, la segunda batería, y el cargador de esta, asciende a 1411,68 €.

4.5.3. Análisis de sustentación y perfil energético

1. Relación empuje-peso (T/W):

A partir del desglose de componentes de “*¡Error! No se encuentra el origen de la referencia..3*” la masa total de la plataforma ($MTOW$) queda fijada en 1,74 kg.

El grupo motopropulsor se compone de cuatro motores SunnySky X2212 III de 980 KV y, como se puede observar en “*Tabla 4.1*”, cada motor proporciona un empuje máximo ($T_{m\acute{a}x}$) unitario de 960g al 100% de aceleración. Por consiguiente, el empuje máximo total de la aeronave es:

$$T_{m\acute{a}x} = 4 \times 960 \text{ g} = 3.840 \text{ g}$$

La relación empuje-peso del prototipo se calcula como:

$$Relaci\acute{o}n \text{ } T/W = \frac{T_{m\acute{a}x}}{MTOW} = \frac{3.840 \text{ g}}{1.750,5 \text{ g}} = 2,21$$

Una relación $T/W \geq 2$ garantiza la capacidad de mantener el vuelo estacionario (*hovering*) utilizando aproximadamente un 45 – 50% del acelerador. El valor obtenido (2,21) asegura un margen de potencia del 50% para compensar posibles perturbaciones por ráfagas de viento en las misiones y ejecutar maniobras de corrección.

2. Consumo eléctrico total:

El consumo continuo de intensidad demandado a la batería en vuelo estacionario (I_{total}) se desglosa en la corriente requerida por el grupo motopropulsor para la sustentación y la corriente absorbida por la carga útil, la electrónica de control y la aviónica del UAS.

a. Consumo del grupo motopropulsor:

Para mantener en vuelo estacionario los 1.740,5 g de la plataforma, cada uno de los cuatro motores debe generar un empuje unitario de:

$$T_{estacionario} = \frac{1.740,5 \text{ g}}{4 \text{ motores}} \approx 435 \text{ g/motor}$$

Y, consultando las tablas de caracterización del motor SunnySky X2212 III 980 KV a 14,8V, la entrega de un empuje de 435g requiere una corriente aproximada de 4,8 A por rama. Por tanto:

$$I_{motores} = 4 \times 4,8 \text{ A} = 19,2 \text{ A (a 14,8 V)}$$

b. Consumo de la carga útil ($I_{payload}$):

Los componentes electrónicos operan a una tensión regulada de 5 V. Se desglosan las especificaciones de consumo a 5 V de cada elemento, fundamentadas en sus manuales de especificaciones según sus fabricantes:

- Raspberry Pi 5 (8 GB): 1,5 A.
- Acelerador NPU Hailo-8 HAT: 0,5 A.
- Cámara Sony IMX477 y Active Cooler: 0,5 A.

$$I_{payload} = 1,5 \text{ A} + 0,5 \text{ A} + 0,5 \text{ A} = 2,5 \text{ A (a 5 V)}$$

c. Consumo de aviónica y control de vuelo ($I_{aviónica}$):

Según el manual de especificaciones eléctricas de Holybro:

- Pixhawk 6C: 0,15 A
- GPS M9N: 0,10 A
- Telemetría SiK: 0,10 A
- RP1: 0,05 A
- DIR: 0,10 A

$$I_{\text{aviónica}} = 0,15 A + 0,10 A + 0,10 A + 0,05 A + 0,10 A = 0,5 A \text{ (a } 5 V)$$

d. Transformación a la línea principal de la batería (14,8 V):

$$P_{\text{electrónica}} = 5 V \times (2,5 A + 0,5 A) = 15 W$$

Y, considerando una eficiencia de conversión del UBEC del 85%, además de aplicando al valor final un margen de seguridad (frente a picos transitorios en la NPU), la corriente equivalente que esta potencia le exige a la batería de 14,8 V es:

$$I_{\text{electrónica (14,8 V)}} = \frac{15 W}{0,85 \times 14,8 V} \approx 1,2 A \rightarrow \text{seg} \rightarrow I_{\text{electrónica (14,8 V)}} = 2,5 A$$

e. Consumo total:

$$I_{\text{total}} = I_{\text{motores}} + I_{\text{electrónica (14,8 V)}} = 19,2 A + 2,5 A \approx 22 A$$

3. Estimación teórica de la autonomía de vuelo:

Para la alimentación del prototipo se dispone de una batería LiPo 4S de 5200mAh (5,2Ah). Con el objetivo de prevenir la degradación prematura de las celdas de litio, en ingeniería aeronáutica se aplica un límite de profundidad de descarga de seguridad del 80%, reservando un margen para maniobras de aproximación y aterrizaje de emergencia.

La capacidad útil utilizable se calcula:

$$C_{\text{útil}} = C \times 0,80 = 5,2 Ah \times 0,80 = 4,16 Ah$$

Relacionando esta capacidad con el consumo continuo total de la plataforma, el tiempo teórico de vuelo se obtiene mediante:

$$t_{\text{vuelo}} = \frac{C_{\text{útil}}}{I_{\text{total}}} \times 60 \text{ min} = \frac{4,16 Ah}{22 A} \times 60 \text{ min} \approx 11,34 \text{ minutos}$$

Por tanto, el UAS dispone de una autonomía operativa estimada de 11 minutos. Este tiempo resulta suficiente para completar pasadas de inspección sobre las hileras de paneles, permitiendo procesar el flujo de vídeo en tiempo real antes del retorno a la base.

Para una operación real de inspección se contempla, además, disponer de dos baterías. De este modo, una vez finalizado el vuelo, sería posible sustituir rápidamente la batería utilizada y continuar la misión con una segunda unidad, reduciendo los tiempos de espera entre pasadas.

Esta solución no incrementa la autonomía de cada vuelo individual, pero sí mejora la continuidad operativa del sistema durante inspecciones de mayor extensión.

5. CONSTRUCCIÓN Y PREPARACIÓN DEL DATASET

El desarrollo de un sistema de detección basado en técnicas de visión artificial depende, en gran medida, de la calidad de los datos empleados durante el entrenamiento. La arquitectura del modelo influye, y aunque constituye una parte relevante del sistema, su capacidad de generalización está directamente condicionada por aspectos como la diversidad de las imágenes, la calidad de las anotaciones, el equilibrio entre clases o la similitud entre los datos de entrenamiento y las condiciones reales de operación.

En las misiones del prototipo (inspección fotovoltaica), esta problemática adquiere especial relevancia debido a la variabilidad que pueden presentar las imágenes. Factores como las condiciones de iluminación, la orientación y distancia de la cámara del dron, los reflejos sobre la superficie de las placas o las características visuales de cada anomalía pueden modificar considerablemente el aspecto de una misma instalación.

Por este motivo, antes de abordar el entrenamiento del modelo de detección, se ha llevado a cabo un proceso de construcción y preparación del conjunto de datos específicamente orientado a la inspección de paneles solares. El objetivo no es solamente conseguir un número elevado de imágenes, sino construir progresivamente un dataset coherente, representativo y adaptado a la misión planteada.

El proceso parte de un conjunto de datos público previamente anotado, seleccionado entre diferentes alternativas disponibles y posteriormente sometido a una fase de revisión y depuración. Sobre esta primera base se realiza una limpieza de clases y anotaciones, dando lugar a una versión inicial del dataset, que servirá como referencia para evaluar posteriormente el efecto de las diferentes estrategias de ampliación.

Con el objetivo de complementar las imágenes disponibles públicamente, se han llevado a cabo campañas propias de adquisición de imágenes. Para estas visitas se ha utilizado una plataforma comercial de captura de imágenes y vídeo RGB: DJI Neo 2.

Es importante diferenciar este equipo del prototipo UAS desarrollado en el capítulo anterior; dicho prototipo constituye el diseño de referencia del sistema propuesto, pero su fabricación completa no se aborda en el presente trabajo debido al coste asociado a la construcción e integración de sus componentes. Por este motivo, para la adquisición experimental de datos se ha optado por emplear un dron comercial que ya se encontraba disponible, manteniendo el prototipo diseñado como propuesta tecnológica del sistema final.

Las imágenes obtenidas con estas campañas permitirán ampliar progresivamente el conjunto de datos inicial y aumentar su proximidad a las condiciones reales de operación; posteriormente se estudiará también la aplicación de técnicas de aumento de datos (*data augmentation*) para incrementar la variabilidad de muestras usadas en el entrenamiento.

De esta forma, la construcción del dataset se plantea como un proceso progresivo que combina datos procedentes de fuentes públicas, un proceso de curación y homogeneización de las anotaciones y datos propios adquiridos mediante UAS, estableciendo así la base de desarrollo y evaluación del sistema de visión artificial.

5.1. Definición del problema y estrategia de datos

Para abordar el problema desde el punto de vista de la visión artificial, el primer aspecto a determinar es la forma en la que se representan las anomalías dentro del conjunto de datos. Para ello pueden considerarse principalmente dos enfoques: la clasificación de imágenes y la detección de objetos.

En un problema de clasificación, cada imagen recibe una etiqueta global que describe su contenido: aplicado a la inspección fotovoltaica, un planteamiento de este tipo permitiría, por ejemplo, determinar si una imagen corresponde a un panel aparentemente correcto o si presenta algún tipo de anomalía.

Sin embargo, este enfoque resulta limitado cuando se pretende conocer también la posición de la anomalía o cuando en un mismo panel aparecen varios defectos de forma simultánea. Por este motivo, se ha considerado más adecuado abordar inicialmente el problema mediante la detección de objetos. En este caso, el modelo no trata solo de identificar qué imagen aparece, sino de localizar el elemento mediante una caja limitadora (*bounding box*). De esta forma, una misma fotografía puede contener varias detecciones independientes.

Cada una de las categorías que un modelo debe aprender a diferenciar es una clase. No obstante, en esta fase no se establece todavía un conjunto cerrado de clases, ya que su definición dependerá de los datos disponibles, de su calidad y de la representatividad de las imágenes encontradas. En su lugar, se identifican inicialmente diferentes tipos de anomalías y situaciones deseables de reconocer en las misiones:

- Acumulaciones de polvo o suciedad sobre la superficie.
- Excrementos de aves.
- Grietas, fisuras, roturas.
- Nieve.
- Alteraciones visibles (cambios de color en las células).
- Sombra.

Esta relación constituye por tanto un punto de partida para orientar la búsqueda y selección de los datos, y no una definición definitiva de las categorías que formarán parte del modelo final. La disponibilidad de un número suficiente de ejemplos, coherencia de anotaciones y resultados obtenidos del entrenamiento serán los factores que determinarán posteriormente qué anomalías resulta viable mantener como clases independientes.

Además de las imágenes convencionales RGB, se ha considerado de interés valorar la disponibilidad de imágenes termográficas. Aunque el prototipo UAS desarrollado no incorpora la cámara térmica, esta tecnología resulta interesante porque permite visualizar diferencias de temperatura localizando puntos calientes que pueden no ser apreciables mediante una cámara convencional.

Por este motivo, durante la búsqueda de datos ha sido un punto favorable que los conjuntos encontrados incluyeran también información termográfica. Su incorporación permitiría estudiar de forma experimental las posibilidades que ofrece este tipo de imagen y, al mismo tiempo, dejar planteada una posible evolución del sistema mediante la integración de una cámara térmica en líneas futuras.

La estrategia seguida para la construcción del dataset se plantea, por tanto, de forma progresiva y experimental. En una primera fase se parte de imágenes disponibles públicamente que permitan representar parte de las anomalías. Estos datos serán posteriormente revisados y depurados antes de establecer un primer conjunto al que entrenar. A continuación, se incorporarán imágenes RGB obtenidas específicamente en las campañas de adquisición realizadas y, por último, se estudiará la aplicación de técnicas de *data augmentation* para incrementar la diversidad del conjunto del entrenamiento.

Este planteamiento permite que la composición del dataset evolucione a medida que avanza el desarrollo, evitando establecer de antemano categorías o configuraciones que posteriormente puedan resultar poco representativas.

5.2. Selección y preparación del dataset inicial

Partiendo de las anomalías consideradas, se realizó una búsqueda de conjuntos de datos públicos relacionados con la inspección de paneles fotovoltaicos. El objetivo es disponer de una base inicial lo suficientemente amplia que permita realizar un primer entrenamiento.

Durante la búsqueda se valoraron especialmente aquellos conjuntos que estuvieran orientados a detección de objetos y que, por tanto, incluyeran las anomalías mediante *bounding boxes*. También se tuvo en cuenta la variedad de defectos representados, el número de imágenes disponibles y, como característica adicional y como se ha introducido anteriormente, la presencia de imágenes termográficas.

Para localizar conjuntos de datos relacionados con la inspección fotovoltaica se realizaron búsquedas usando distintos términos asociados tanto al tipo de instalación como a los defectos de interés, entre las palabras clave se incluyeron expresiones como: *solar panel defects*, *solar panel damage*, *photovoltaic defects*, *solar panel crack* y *solar panel dust*.

Entre las diferentes alternativas analizadas finalmente se seleccionó un conjunto de datos disponible a través de **Roboflow Universe**, plataforma que permite acceder a datasets públicos previamente anotados y trabajar sobre ellos, versionándolos dentro de proyectos propios de Roboflow.

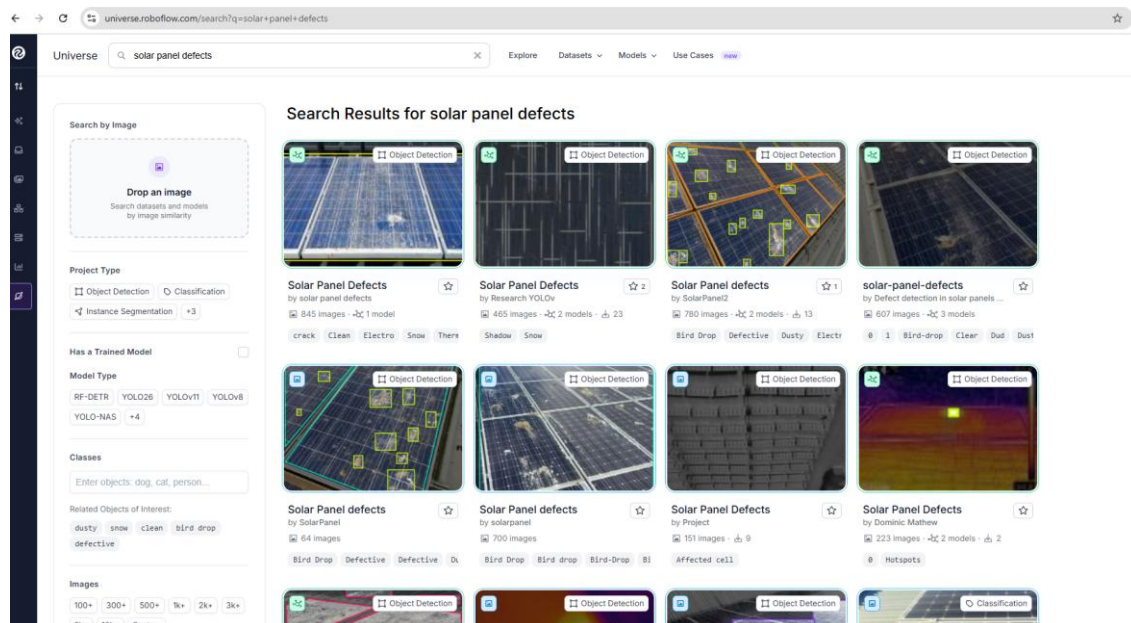


Figura 5.1. Resultados de la búsqueda de datasets relacionados con defectos fotovoltaicos en Roboflow.

El dataset seleccionado contenía inicialmente 845 imágenes, y estaba configurado como un problema de detección de objetos y no de categorización (esto es importante porque ya venía con defectos definidos en *bounding boxes*). Una de las principales razones para seleccionarlo fue la variedad de contenido disponible, combinando fotografías RGB y termográficas de placas, y contemplando diferentes tipos de anomalía próximos a los considerados inicialmente para el proyecto.

Una vez seleccionado, el proyecto se duplicó dentro de un espacio de trabajo propio en Roboflow, obteniendo así una copia independiente sobre la que realizar las modificaciones necesarias sin alterar el conjunto publicado originalmente. Esta copia se utilizó como base para todo el proceso posterior de limpieza, reorganización de categorías y versionado.

Las categorías del conjunto de datos, manteniendo la nomenclatura utilizada originalmente, y el número de anotaciones asociadas a cada una de ellas eran las siguientes:

‘crack’	→	91
‘a’	→	1
‘bird drops’	→	172
‘Clean’	→	372
‘dust’	→	110
‘Electro’	→	77
‘Snow’	→	87
‘Thermo’	→	523

El número total de anotaciones es superior al número de imágenes, lo que indica que una misma fotografía puede contener varias *bounding boxes*, pertenecientes a la misma categoría o incluso a categorías diferentes.

5.2.1. Revisión y depuración inicial

Antes de utilizar el conjunto de datos para realizar el primer entrenamiento, se llevó a cabo una revisión manual de las imágenes y sus anotaciones. Durante esa fase se detectaron diferentes inconsistencias que hacían necesario limpiar el dataset antes de considerarlo adecuado como conjunto de referencia.

Se empezó la revisión por la categoría ‘a’. Esta clase aparecía solo en una imagen, asociada a una *bounding box* que ocupaba prácticamente el total de la imagen. La misma fotografía disponía, además, de una anotación correcta (‘Snow’). Al no encontrarse una interpretación coherente para la categoría ‘a’, se consideró una clase y anotación accidentales, eliminándose esa *bounding box*, pero manteniendo la imagen con la anotación válida. Por tanto, se eliminó también la clase.

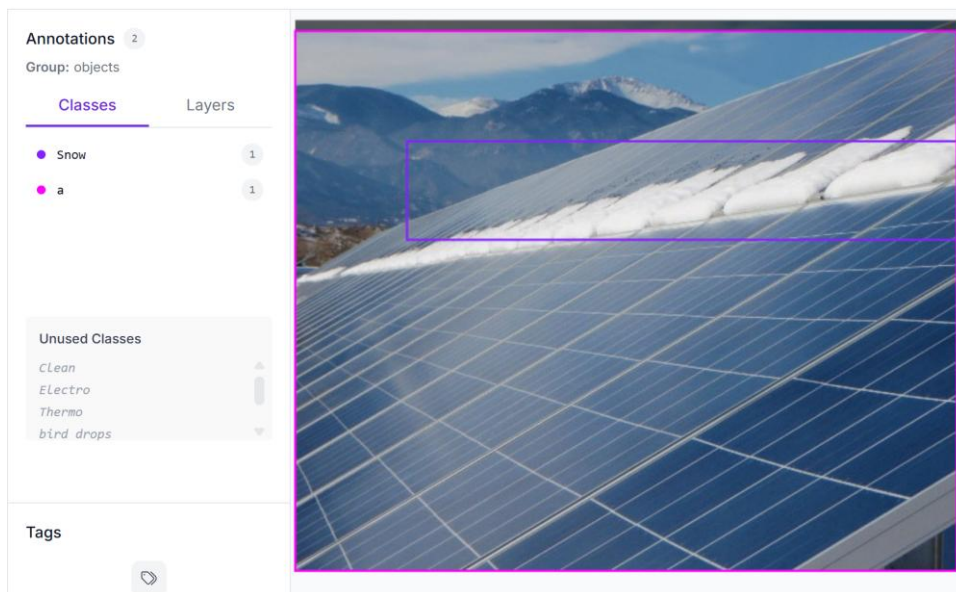


Figura 5.2. Revisión de anotación asociada a la categoría 'a'.

Después se revisó la categoría ‘Clean’. En el dataset original, esta se utilizaba como una clase más del detector y, en numerosos casos, se representaba mediante cajas que delimitaban gran parte del panel o incluso toda la imagen. Sin embargo, dentro del enfoque adoptado para este trabajo, la falta de defectos (o el estado “limpio”), no constituye un objeto que sea necesario localizar.

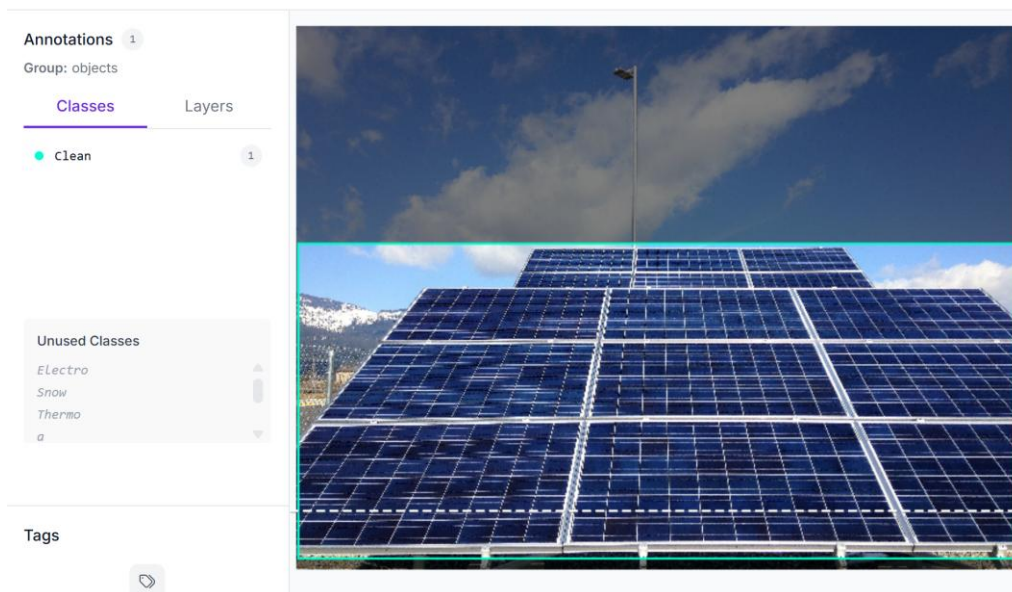


Figura 5.3. Ejemplo de anotación correspondiente a 'Clean'.

Por este motivo, se acabó eliminando ‘Clean’ como clase, conservando las imágenes correspondientes. Las fotografías que no contuvieran ninguna otra anomalía quedaron entonces sin *bounding boxes* y pasaron a actuar como muestras negativas del dataset, lo que proporciona al modelo ejemplos en los que no debería producir ninguna detección.

Además, Roboflow mostraba algunas imágenes bajo la denominación ‘null’. Este término no corresponde a ninguna clase definida en el proyecto, sino a imágenes sin ninguna anotación. No se creó ninguna categoría específica para representar los paneles sin anomalías,

manteniendo estas imágenes simplemente sin etiquetas (o como se adelanta en el párrafo anterior, dejándolas como muestras negativas).

La categoría ‘Thermo’ requirió una revisión específica debido a la heterogeneidad de las imágenes asociadas. En gran parte de los casos, las *bounding boxes* delimitaban regiones térmicamente diferenciadas o puntos calientes sobre las placas, por lo que estas imágenes resultaban de interés para el estudio.

Sin embargo, también se localizaron ejemplos en los que aparecían regiones luminosas aisladas sobre fondos oscuros, sin poder reconocer con claridad la placa o determinar la relación entre la región marcada y una anomalía real del módulo. Al considerar que estas muestras podían introducir patrones ambiguos durante el aprendizaje, se eliminaron las imágenes cuya interpretación no resultaba suficientemente clara:

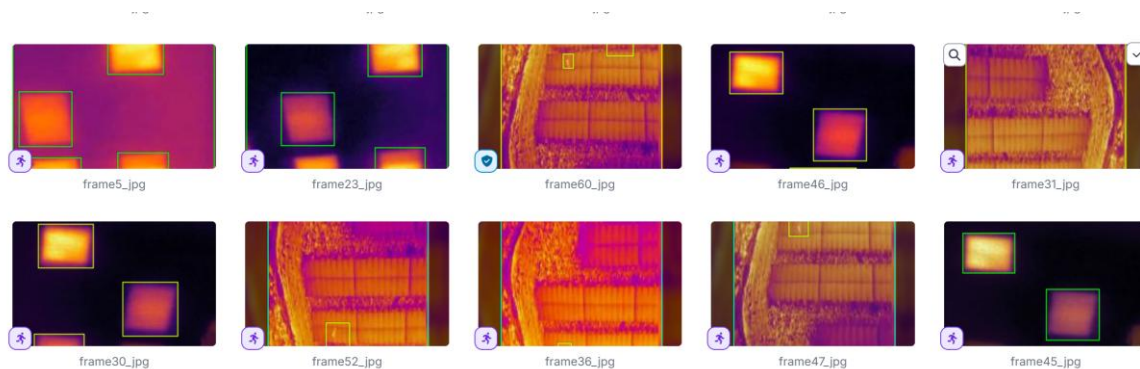


Figura 5.4. Ejemplos de muestras de 'Thermo'.

También se analizaron las muestras de ‘Electro’ con el objetivo de comprobar el significado exacto de esta clase. En ellas se observaron principalmente alteraciones visibles sobre determinadas células, como cambios de coloración o regiones marrones y anaranjadas.

Dado que una imagen RGB no permite determinar de forma concluyente que estas alteraciones tengan un origen eléctrico, se decidió conservar esta clase para evaluar en los entrenamientos su comportamiento, evitando interpretarla como un diagnóstico eléctrico directo.

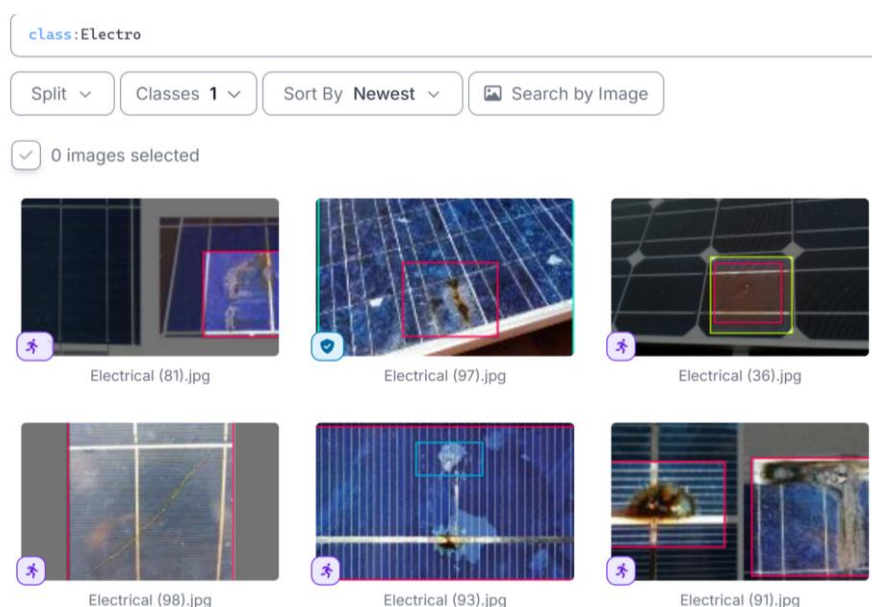


Figura 5.5. Ejemplos de anotaciones correspondientes a 'Electro'.

El resto de las clases también se revisó rigurosamente con el objetivo de encontrar imágenes poco representativas, anotaciones incorrectas o *bounding boxes* cuya posición o tamaño no correspondieran con la anomalía representada.

5.2.2. Homogeneización de las clases

Una vez completada la revisión del contenido, se homogeneizó la nomenclatura de las categorías que se mantendrían para el primer experimento (llamado E1 - baseline). El objetivo fue utilizar nombres consistentes y suficientemente descriptivos, evitando diferencias de mayúsculas, espacios o denominaciones poco precisas.

La correspondencia entre las clases del dataset original y el versionado para el E1 – Baseline es:

Categoría original	Categoría versionada
bird drops	bird_droppings
crack	crack
dust	dust
Electro	electro
Snow	snow
Thermo	thermal_anomaly

Tabla 5.1. Distribución de imágenes por clase en el dataset inicial.

Las categorías ‘a’ y ‘Clean’, como se ha mencionado anteriormente, no se mantuvieron como clase de detección.

Esta primera definición se ha utilizado como punto de partida para el experimento Baseline, pero podrá ser revisada posteriormente en función de la calidad de los datos y del comportamiento observado durante los entrenamientos.

5.2.3. Construcción del conjunto de referencia

Tras finalizar el proceso de revisión y depuración, el conjunto quedó formado por 724 imágenes, frente a las 845 iniciales.

A partir de este conjunto se generó en Roboflow una versión destinada específicamente para el primer experimento. En esta etapa no se aplicaron técnicas adicionales de *data augmentation* durante la generación del dataset, de forma que pudiera establecerse una referencia inicial antes de incorporar imágenes propias o realizar nuevas modificaciones sobre los datos.

Para el entrenamiento de un modelo de visión artificial es necesario separar las imágenes en diferentes subconjuntos, de manera que no todas participen de la misma forma en el proceso de aprendizaje. En este caso se han utilizado tres particiones:

- Entrenamiento (*train*): contiene las imágenes utilizadas directamente por el modelo para ajustar sus parámetros durante el entrenamiento.
- Validación (*validation*): se emplea durante el entrenamiento para comprobar cómo se comporta el modelo sobre las imágenes que no está utilizando directamente para ajustar sus parámetros y permite seguir la evolución de su capacidad de generalización.
- Prueba (*test*): se mantiene separado del proceso de entrenamiento y se utiliza posteriormente para realizar una evaluación independiente del modelo obtenido.

Se adoptó una distribución aproximada de 70 – 20 – 10. Esto implica utilizar un 70% de las imágenes en *train*, un 20% en *validation* y un 10% en *test*.

Este reparto es habitual en problemas de aprendizaje supervisado y resulta adecuado para el tamaño del conjunto disponible, ya que permite dedicar la mayor parte de las imágenes al aprendizaje y, al mismo tiempo, conservar una cantidad suficiente de muestras para validar y evaluar posteriormente el modelo de forma independiente.

La distribución resultante por número de imágenes queda:

Entrenamiento (<i>train</i>)	→	507
Validación (<i>valid</i>)	→	145
Prueba (<i>test</i>)	→	72
Total	→	724

Esta versión constituye el conjunto E1 – Baseline, que servirá como referencia para estudiar posteriormente el efecto producido por la incorporación de nuevos datos y por las diferentes estrategias de ampliación del dataset.

5.3. Ampliación del dataset: datos propios

Aunque el conjunto de datos público seleccionado permitió establecer una base inicial para el desarrollo del sistema, se consideró de interés complementarlo con imágenes obtenidas específicamente durante el desarrollo del trabajo. La realización de campañas propias no solo permitía ampliar el conjunto de datos, sino también conocer de forma práctica cómo se lleva a cabo la adquisición de imágenes de paneles solares mediante UAS, y qué características presentan las grabaciones obtenidas en condiciones reales.

Este aspecto resulta especialmente relevante en el contexto del proyecto, ya que el sistema de visión artificial se plantea para su utilización conjunta con la plataforma aérea. Por ello, se consideró necesario analizar directamente factores como la distancia al panel, el ángulo de visión, el movimiento de la cámara, la iluminación, reflejos... todos los aspectos capaces de influir posteriormente en la capacidad de detección del modelo.

Para ello se llevaron a cabo diferentes campañas de adquisición de imágenes RGB en instalaciones fotovoltaicas accesibles, combinando fotografías y grabaciones de vídeo. El material recopilado fue posteriormente procesado para obtener fotogramas (“frames”) representativos, eliminar muestras redundantes o de baja calidad y preparar las imágenes seleccionadas para su posterior anotación e incorporación al proceso experimental

5.3.1. Obtención de los datos

Para la obtención de material propio fue necesario localizar tejados con paneles solares o instalaciones fotovoltaicas en las que fuera posible realizar capturas mediante el dron comercial, o conseguir las imágenes proporcionadas directamente por sus propietarios.

Esta tarea presentaba además una dificultad añadida. Gran parte de las instalaciones accesibles se encuentran en viviendas particulares, y la realización de grabaciones aéreas (sobre todo por parte de un desconocido), podía generar dudas o desconfianza entre los posibles colaboradores.

Por este motivo, se consideró conveniente disponer de un medio que permitiera presentar:

- Presentar el proyecto de forma clara.
 - Explicar su finalidad académica.
 - Facilitar diferentes vías de colaboración.
-

Con este objetivo se desarrolló la página web uas.luciagorostidi.com, utilizada como herramienta de apoyo para contactar con propietarios de instalaciones fotovoltaicas, personas vinculadas al sector, y otros posibles colaboradores.

La plataforma permitió, además, ampliar el alcance de la búsqueda más allá de los contactos directos. En lugar de depender únicamente de explicar personalmente el proyecto a cada posible colaborador, el enlace podía compartirse entre conocidos y llegar a personas que, de otra forma, no habría sido tan sencillo llegar. De esta forma, cualquier interesado podía conocer de manera autónoma el objetivo del proyecto y las distintas formas de colaboración.

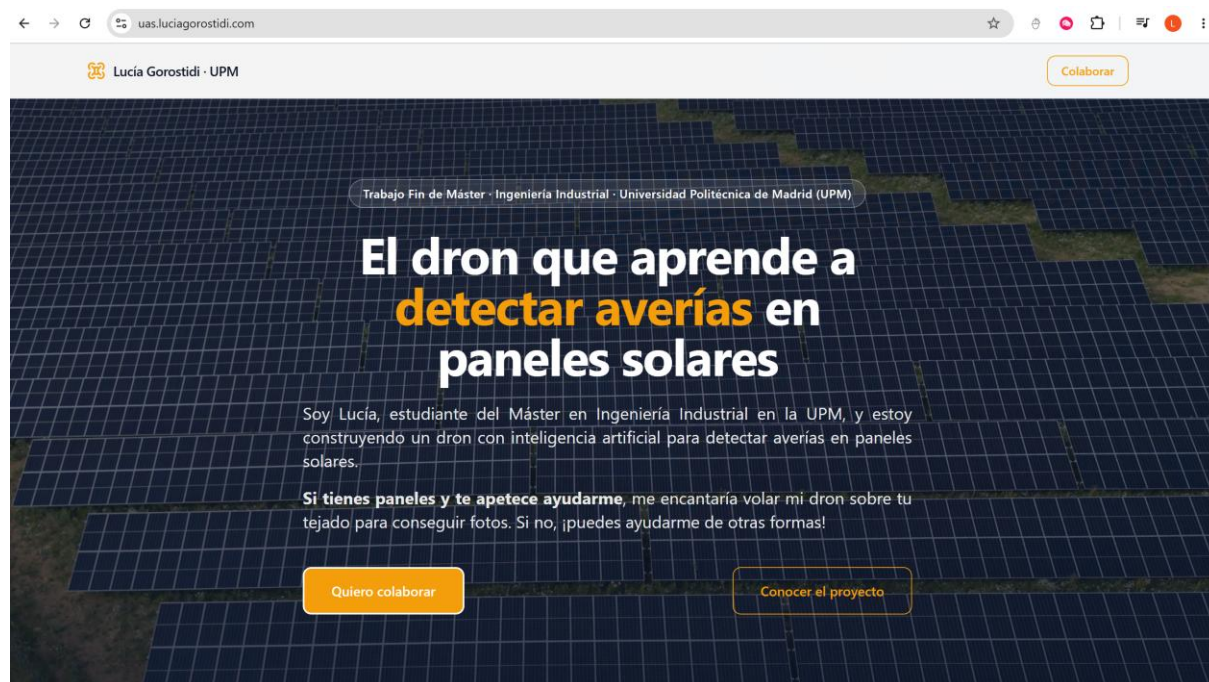


Figura 5.6. Página web desarrollada.

Para el desarrollo de la página, se utilizó un enfoque de programación asistida por inteligencia artificial, trabajando con Claude Code desde un entorno local. Antes de empezar con la implementación se definieron los objetivos principales de la página, la información que debía mostrarse, las formas de colaboración que se querían ofrecer y el funcionamiento general de la plataforma.

A partir de esta definición, se elaboró un *prompt* inicial bastante detallado, que sirvió como punto de partida para generar una primera versión de la web. Esta versión se probó en local, y se fue revisando de forma iterativa, modificando la estructura, los contenidos, el diseño y el funcionamiento de los distintos elementos hasta alcanzar una versión estable y lista para publicar.

Como parte de la preparación del entorno fue necesario instalar Node.js, utilizado para ejecutar Claude Code y gestionar las dependencias necesarias para el desarrollo de la web.

Una vez obtenida una versión estable de la plataforma, se procedió a su publicación mediante Vercel, que permitió disponer de una versión accesible desde internet, obteniendo inicialmente una dirección con el dominio vercel.app.

Aunque esta URL permitía acceder correctamente a la web, se decidió adquirir un dominio propio para reforzar la identidad del proyecto y transmitir una mayor confianza a los posibles colaboradores.

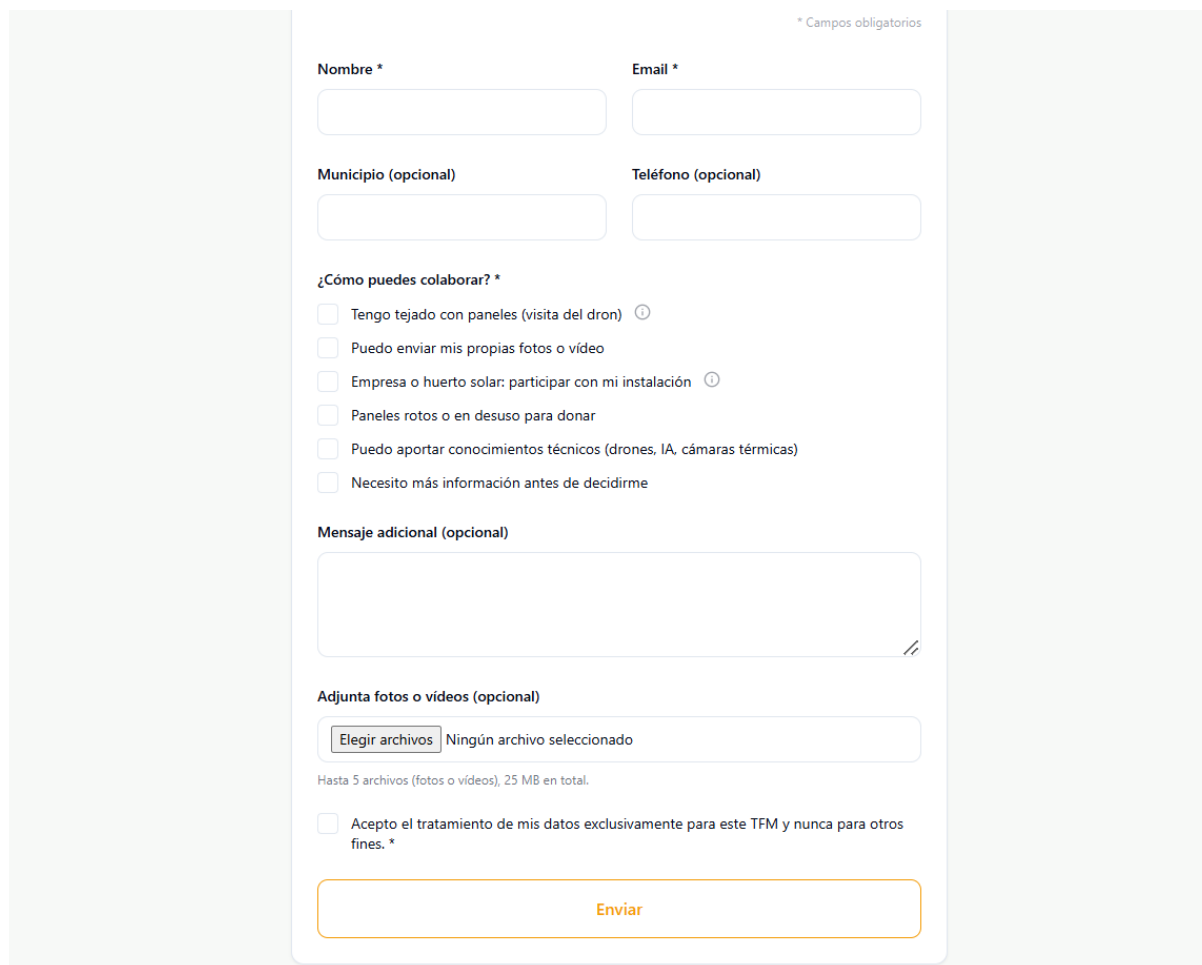
Publicación inicial: `tfm-solar-lucia.vercel.app`

Dominio adquirido: `luciagorostidi.com`

Subdominio del proyecto: `uas.luciagorostidi.com`

Registrador del dominio: Cloudflare

Una de las partes principales y más importantes de la plataforma fue el formulario de colaboración, diseñado para facilitar distintas formas de participación.



The image shows a web form for collaboration. At the top right, it says "* Campos obligatorios". The form has several sections: 1. "Nombre *" and "Email *" with text input fields. 2. "Municipio (opcional)" and "Teléfono (opcional)" with text input fields. 3. "¿Cómo puedes colaborar? *" with a list of checkboxes: "Tengo tejado con paneles (visita del dron)", "Puedo enviar mis propias fotos o vídeo", "Empresa o huerto solar: participar con mi instalación", "Paneles rotos o en desuso para donar", "Puedo aportar conocimientos técnicos (drones, IA, cámaras térmicas)", and "Necesito más información antes de decidirme". 4. "Mensaje adicional (opcional)" with a large text area. 5. "Adjunta fotos o vídeos (opcional)" with a file upload button "Elegir archivos" and a status "Ningún archivo seleccionado". Below this, it says "Hasta 5 archivos (fotos o vídeos), 25 MB en total." 6. A checkbox for "Acepto el tratamiento de mis datos exclusivamente para este TFM y nunca para otros fines. *". 7. An "Enviar" button at the bottom.

Figura 5. 7. Formulario de colaboración desarrollado para la recopilación del material.

Como se puede observar en “Figura 5. 7”, el formulario permite adaptar la colaboración a diferentes situaciones, desde autorizar una visita con el dron hasta enviar directamente fotografías o vídeos, aportar paneles dañados o facilitar conocimientos técnicos relacionados con el proyecto.

Además, se consideró de suma importancia incluir una casilla obligatoria de consentimiento para el tratamiento de los datos exclusivamente en el marco del TFM, dejando explícito que la información proporcionada no se emplearía para otros fines ni se compartiría con terceros.

La iniciativa permitió recibir un total de 38 respuestas, de las cuales una parte importante se tradujo en visitas, aportación directa de imágenes o propuestas de acceso a instalaciones que finalmente no pudieron hacerse.

En la siguiente tabla, se recogen de forma muy resumida las interacciones y formas de aportación de cada colaborador.

Resultado interacción	Nº de colaboradores	Aportación final
Visitas realizadas con dron	6	Grabación de instalaciones con UAS propio; una de ellas permitió además realizar una captura antes y después de añadir arena de forma controlada.
Visitas descartadas por restricciones de ENAIRE	3	Las instalaciones fueron ofrecidas para realizar vuelos, pero la operación no resultó viable en la zona. Una de ellas incluía más visitas dentro de la misma urbanización.
Visitas no realizadas por coordinación o logística	3	No fue posible compatibilizar disponibilidad, desplazamientos o gestión de la visita. En uno de estos casos se recibieron fotografías y un vídeo como alternativa.
Envío de material inicialmente válido para el dataset	6	Imágenes de instalaciones fotovoltaicas y ejemplos de paneles dañados o rotos, algunos procedentes de personas vinculadas al sector.
Envío de material directamente descartado	13	Imágenes recibidas que fueron revisadas, pero descartadas por no resultar adecuadas para su incorporación al dataset (aprox. 35)
Recomendaciones de recursos externos	2	Se facilitaron fuentes para localizar material adicional, como Kaggle y Pexels.
Colaboración técnica	1	Miembro de un club de aeromodelismo que se ofreció a compartir conocimientos y facilitar visitas gratuitas al club.
Mensajes de apoyo al proyecto	4	Participantes que contactaron únicamente para mostrar interés y animar a continuar con el trabajo.
Total	38	

Tabla 5. 2. Resumen de las interacciones y aportaciones recibidas durante la captación de colaboradores.

Entre las colaboraciones recibidas, resultó especialmente útil el contacto con un miembro de un club de aeromodelismo (Club de Aeromodelismo Alas de Galapagar [34]), que se ofreció a compartir conocimientos relacionados con el manejo de drones y facilitar visitas al club. Este apoyo permitió reforzar la experiencia práctica y teórica necesaria para realizar posteriormente las campañas de adquisición con mayor seguridad y confianza.

Cabe destacar que, además del material finalmente seleccionado, se recibieron aportaciones que incluían un número elevado de imágenes que se descartaron antes de su incorporación a Roboflow al no resultar adecuadas para el dataset porque no presentaban la calidad suficiente.

En cuanto a las visitas de vuelo, la planificación de estas debía tener en cuenta, no solo la disponibilidad de la instalación y logística, sino (y más importante) las condiciones legales y

operativas necesarias para realizar el vuelo. Por este motivo, antes de organizar cualquier desplazamiento se comprobaba previamente la viabilidad de la zona y las posibles restricciones aplicables [4]/[5]. El marco normativo asociado a las operaciones UAS se desarrolla con mayor detalle en “9. MARCO NORMATIVO”.

Para realizar estas comprobaciones se utilizó ENAIRE Drones [29], herramienta que permite consultar las zonas geográficas UAS y sus limitaciones existentes para ubicaciones concretas.

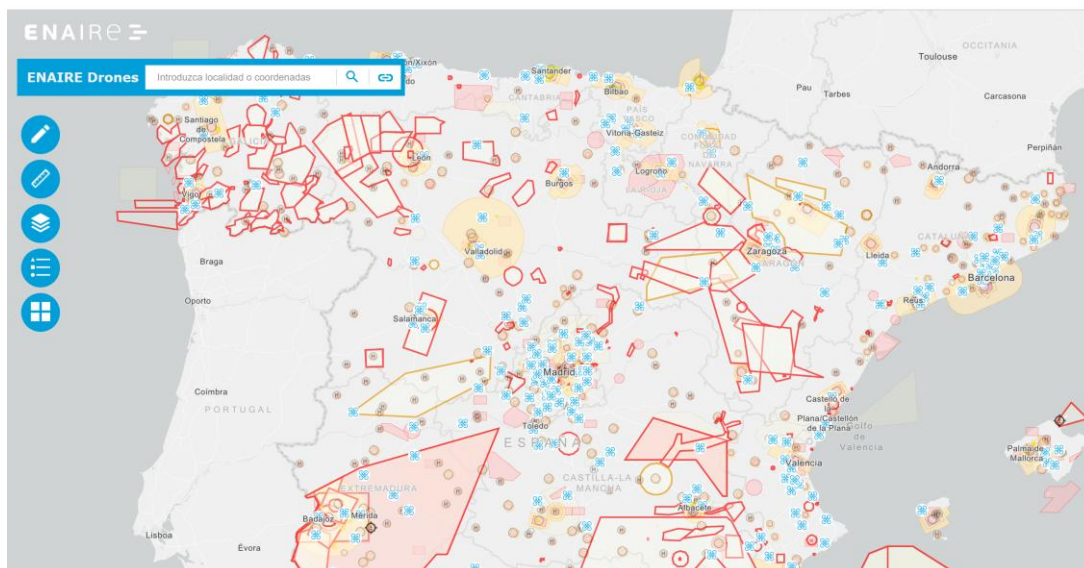


Figura 5.8. Vista general de las zonas geográficas UAS mostradas en ENAIRE Drones.

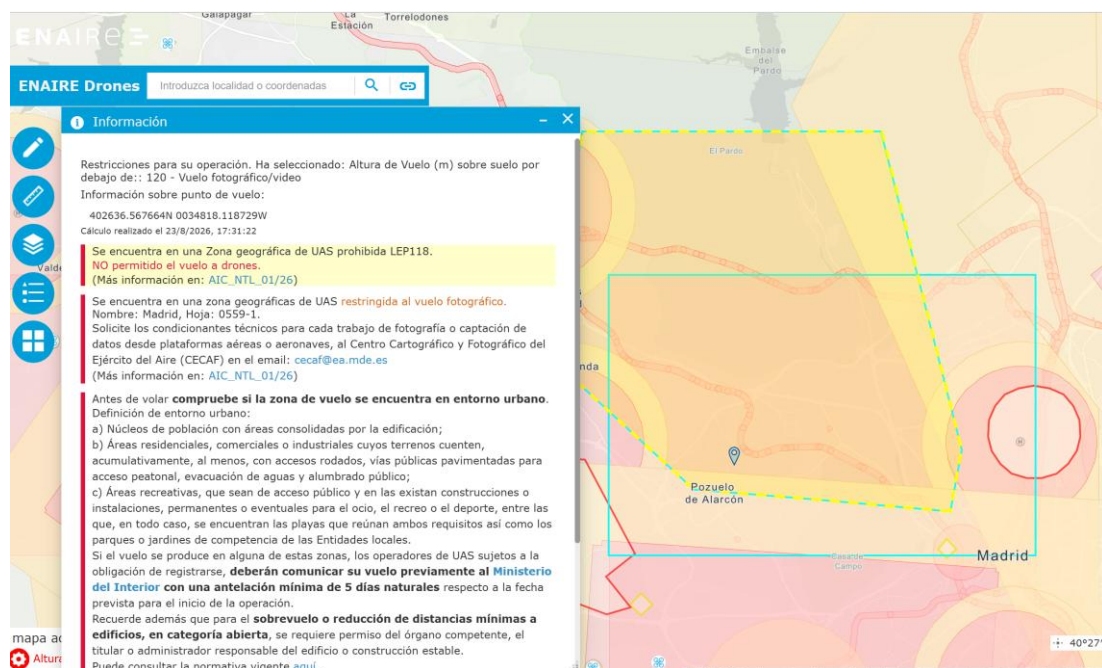


Figura 5.9. Ejemplo de zona con prohibición expresa de vuelo UAS.

Las restricciones mostradas por ENAIRE no siempre implicaban una prohibición total del vuelo. En determinadas ubicaciones la operación podía ser posible mediante el cumplimiento de condiciones adicionales, como la coordinación con aeródromos o helipuertos próximos, o la realización de comunicaciones previas al ministerio.

Sin embargo, este tipo de procedimientos aumentaba la complejidad de organizar una visita, pudiendo generar cierta incertidumbre entre los propietarios que ofrecieron sus instalaciones para colaborar. Por eso se decidió priorizar aquellas ubicaciones en las que el vuelo pudiera realizarse sin requerir coordinaciones adicionales complejas.

Las visitas en zonas con prohibición expresa, como la de “Figura 5.9”, se descartaban directamente. En algunos casos, los propios colaboradores optaron por enviar sus propias fotografías como alternativa.

Una vez confirmada la viabilidad de la ubicación y establecidas con los propietarios la hora y fecha de visita, las capturas se realizaron con un dron propio comercial, el DJI Neo 2:



Figura 5.10. Ejemplo de pilotaje de dron en uno de los vuelos de las visitas.

Durante los vuelos se optó principalmente por grabar vídeo, recorriendo los paneles desde diferentes ángulos y distancias. Esto permitía revisar a posteriori el material, seleccionando los fotogramas (*frames*) más representativos de cada visita, y evitando así depender solamente de fotografías tomadas en momentos concretos durante el vuelo.

5.3.2. Selección de datos externos

Además del material obtenido durante las campañas y de las aportaciones directas recibidas, algunos colaboradores recomendaron recursos externos que podían resultar útiles para ampliar el conjunto de datos. Entre las propuestas recibidas se encontraban Kaggle [35] y Pexels [36], ambas orientadas a la consulta de datasets o material audiovisual disponible públicamente.

Tras revisar ambas plataformas, se observó que Pexels ofrecía una cantidad interesante de vídeos de instalaciones fotovoltaicas grabadas desde diferentes perspectivas, incluyendo tomas aéreas realizadas con drones.

Por este motivo, se decidió realizar una búsqueda más exhaustiva y seleccionar manualmente aquellos videos que pudieran aportar variedad respecto al material ya disponible.

Durante esta búsqueda también se intentó localizar material termográfico relacionado con paneles solares. Sin embargo, los recursos encontrados correspondían únicamente a imágenes y vídeos RGB, por lo que la ampliación realizada en esta fase se centró en este tipo de datos.

Finalmente, se seleccionaron 5 vídeos para añadir al dataset inicial.

5.3.3. Preparación del material

Una vez recopilado el material procedente de las campañas de adquisición, las aportaciones de los colaboradores y las fuentes externas seleccionadas, se empezó con la preparación para incorporarlo más adelante en la segunda versión del dataset (dataset ampliado obtenido a partir del Baseline).

Para ello se utilizó de nuevo Roboflow, que permite añadir nuevas imágenes al proyecto e importar archivos de vídeos para extraer de ellos fotogramas (*frames*) individuales.

En el caso de los vídeos, la plataforma permite revisar las grabaciones de los archivos a subir, estableciendo la frecuencia con la que se extraerán los fotogramas. Este valor se ajustó de forma independiente para cada vídeo, ya que ni la duración, ni el movimiento de la cámara ni la velocidad de vuelo fueron iguales en todas las grabaciones.

De esta forma, no se aplicó una frecuencia de extracción (FPS) fija a todo el material. En los vídeos con movimientos más lentos podía utilizarse una separación mayor entre *frames*, mientras que en los vuelos que se hicieron con mayor rapidez y con más cambios de perspectiva, resultaba conveniente conservar un número mayor de imágenes. El objetivo en esta primera fase era obtener suficientes *frames* representativos sin extraer innecesariamente todas las imágenes de cada vídeo.

A partir de este proceso, el inventario inicial de imágenes recogidas queda mostrado en la siguiente tabla:

Procedencia	Fuente	Frames
Datos de campo + imágenes de colaboradores (campañas propias)	Visita 1	10
	Visita 2	16
	Visita 3 – vídeo 1	21
	Visita 3 – vídeo 2	23
	Visita 4	17
	Visita 5	32
	Visita 6	11
	Fotografías colaboradores	18
	Vídeo colaboradora	14
	Vídeo instalación fotovoltaica (no fue posible la visita)	18
Material externo seleccionado	Pexels 01	9
	Pexels 02	27
	Pexels 03	12
	Pexels 04	8
	Pexels 05	7
TOTAL		243

Tabla 5.3. Inventario inicial de imágenes candidatas obtenidas para la ampliación del dataset.

En total, este proceso permitió obtener 243 imágenes candidatas para la ampliación del dataset. Esta cifra corresponde únicamente al material que superó la selección inicial (es decir, no contabilizan las 35 imágenes que se descartaron directamente).

Y, aunque se ajustó la frecuencia de *frames* en función del vídeo, todavía fue necesaria una revisión más detallada. Por este motivo, el siguiente paso fue la depuración conjunta de las imágenes, sin establecer un número concreto que debiera conservarse. Tras completar la revisión, se conservaron 165 imágenes, descartando un total de 78. Esta depuración permitió reducir de forma considerable la redundancia en el material obtenido, consiguiendo un conjunto más representativo para la siguiente fase.

Los principales motivos de descarte fueron la presencia de *frames* repetidos por una baja velocidad de vuelo, imágenes desenfocadas y tomas con poco detalle. También se eliminaron algunas vistas aéreas demasiado alejadas, en las que no era posible apreciar con claridad posibles anomalías o si el panel estaba realmente limpio.

Por último, el conjunto final de 165 imágenes añadidas quedó preparado para iniciar el proceso de anotación manual (también desde Roboflow) de las anomalías presentes. Estas anomalías se marcaron siguiendo el mismo proceso que el dataset inicial: mediante *bounding boxes*, manteniendo además las mismas clases, para conservar la coherencia entre ambas versiones. Y, como anteriormente, las imágenes sin ninguna anomalía se mantuvieron como ejemplos negativos.

Durante este proceso se observó cierta dificultad para diferenciar algunos casos de las clases ‘bird_droppings’ y ‘dust’, especialmente cuando las manchas eran pequeñas o poco definidas. Se decidió mantener ambas categorías separadas en esta fase y comprobar posteriormente, a partir de los resultados de los entrenamientos, si esta ambigüedad afecta también al comportamiento del modelo, estudiando si resultaría conveniente agruparlas en una clase común.

Una vez incorporado el nuevo material, se generó en Roboflow la versión E2_expanded_dataset, formada por un total de 889 imágenes. Las 165 imágenes añadidas se incorporaron al conjunto de entrenamiento (*train*), que pasó de 507 a 672 imágenes, mientras que los conjuntos de *valid* y *test* se mantuvieron en 145 y 72 imágenes respectivamente.

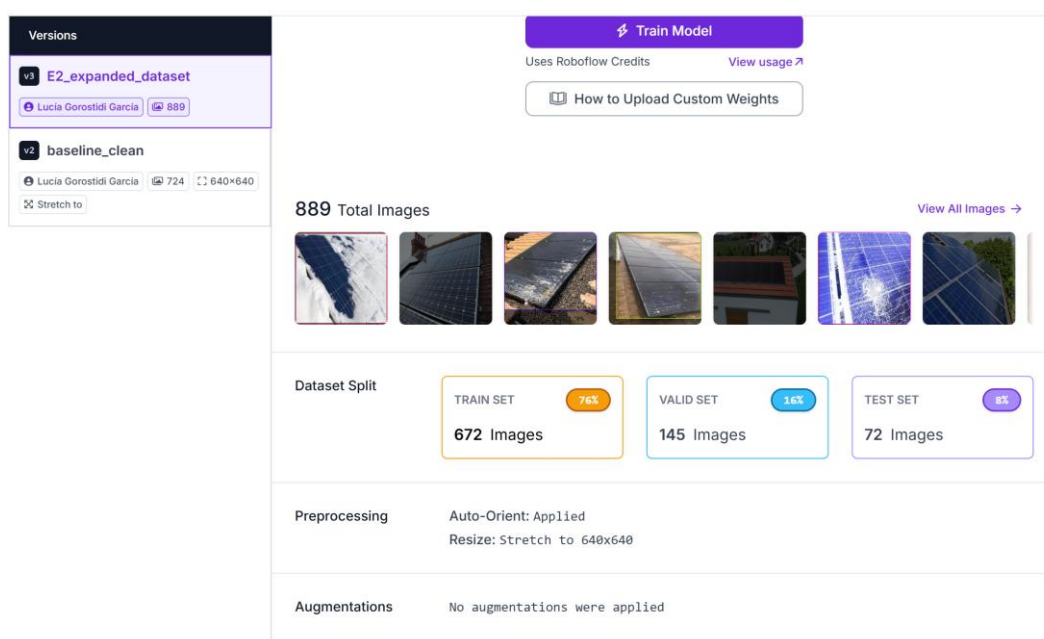


Figura 5.11. Versión E2_expanded_dataset generada en Roboflow tras la ampliación de train.

Esta configuración resulta especialmente interesante para los experimentos, permite plantear una comparación directa con el conjunto de referencia (baseline), se plantea de la siguiente forma:

“¿Qué pasa si se entrena el mismo modelo, bajo las mismas condiciones y se evalúa sobre las mismas imágenes, pero disponiendo de un conjunto de entrenamiento más amplio?”

Y es una manera de relacionar las diferencias observadas entre los experimentos con ambos datasets con la incorporación de datos añadidos.

Así, queda preparada la versión ampliada del dataset para su respectivo experimento. El siguiente paso será comprobar si la incorporación de las nuevas imágenes permite mejorar el comportamiento del modelo respecto al conjunto de referencia.

6. DESARROLLO DEL MODELO DE DETECCIÓN

Una vez preparado el conjunto de datos, el siguiente paso consiste en desarrollar y evaluar un modelo encargado de detectar esas anomalías presentes en las placas solares. Para ello se plantea un proceso experimental basado en diferentes entrenamientos, analizando los resultados obtenidos y utilizando cada experimento como punto de partida para introducir posibles mejoras en el siguiente, hasta dar con el modelo ideal.

6.1. Flujo de entrenamiento

Un modelo de visión artificial es un sistema capaz de analizar una imagen y extraer información útil de ella. En este caso, busca aprender a reconocer y localizar visualmente las anomalías previamente definidas en las placas solares.

Para poder realizar esta tarea, el modelo necesita aprender a partir de ejemplos, este proceso se denomina entrenamiento. Durante el entrenamiento se le proporcionan imágenes junto con las anotaciones que indican qué anomalías aparecen y dónde se encuentran (por eso es importante toda la organización previa sobre el dataset). A partir de estos ejemplos, el modelo intenta realizar sus propias predicciones y las compara con la información correcta disponible en el dataset.

Cuando una predicción no coincide con la anotación real, se calcula el error cometido. El modelo utiliza esta información para modificar progresivamente sus parámetros internos, de forma que en las iteraciones pueda realizar predicciones más precisas. El entrenamiento consiste, por tanto, en repetir este proceso muchas veces hasta conseguir que el modelo aprenda patrones que le permitan reconocer situaciones similares en imágenes nuevas.

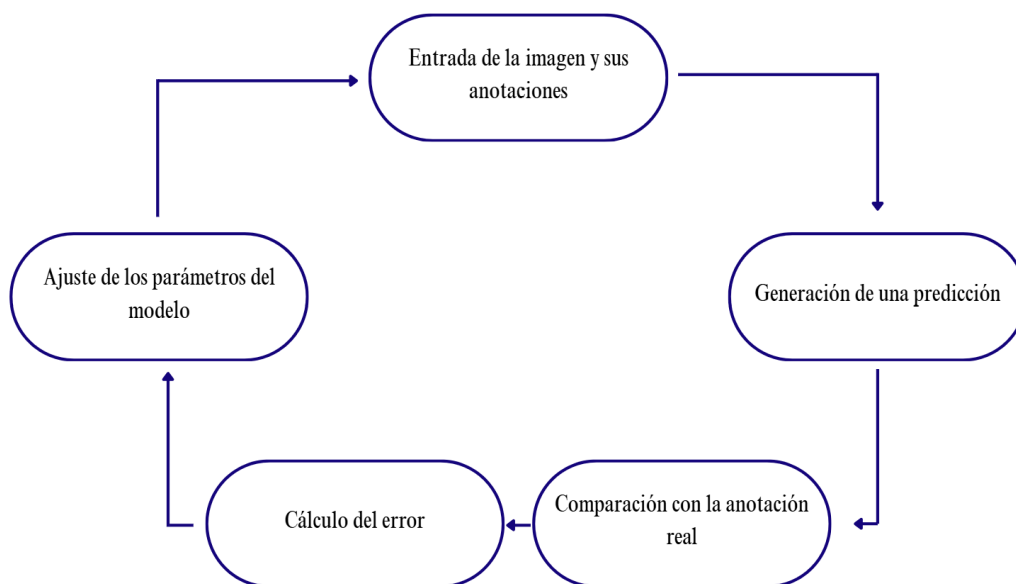


Figura 6. 1. Flujo simplificado del proceso de entrenamiento del modelo. Fuente: Elaboración propia.

En resumen, el flujo de un entrenamiento quedaría así:

1. Entrada de la imagen y sus anotaciones.

El modelo recibe las imágenes y sus anotaciones que indican qué anomalías aparecen (lo recibe con información numérica, en forma de píxeles).

2. Generación de una predicción.

A partir de esa información, el modelo intenta localizar las anomalías presentes y determinar a qué clase pertenece cada una.

3. Comparación con la anotación real.

Las predicciones obtenidas se comparan con las anotaciones utilizadas como referencia para comprobar qué aciertos y errores se han producido.

4. Cálculo del error.

Las diferencias entre la predicción y la anotación permiten calcular una función de pérdida (*loss*), que cuantifica el error cometido.

5. Ajuste de los parámetros internos.

A partir de ese error, el modelo modifica progresivamente sus parámetros para intentar mejorar las predicciones posteriores. Y vuelve al punto 1 con otro grupo de imágenes del conjunto de entrenamiento.

Las clases que el modelo debe aprender a reconocer se encuentran definidas previamente en el conjunto de datos. Sin embargo, el modelo no interpreta el significado de nombres como ‘dust’, ‘crack’ o ‘bird_droppings’ de la misma forma que una persona. Cada clase actúa simplemente como una etiqueta asociada a determinados ejemplos, y a partir de las imágenes anotadas el modelo va aprendiendo qué patrones visuales se relacionan con cada una de ellas.

En resumen, el modelo sabe que tiene que aprender a distinguir X clases porque son las categorías que se dejan definidas en el dataset, pero el modelo no sabe qué aspecto tiene cada una, para ello tiene que aprender (entrenar).

Este aprendizaje se organiza en épocas (*epochs*). Una época corresponde a un recorrido completo por todas las imágenes destinadas al entrenamiento. Una vez finalizado este recorrido, comienza una nueva época y el proceso se repite tantas veces como se haya definido en la configuración del entrenamiento. Al finalizar cada una de ellas puede evaluarse como está evolucionando el modelo y comprobar si realmente está mejorando.

Una vez entendido cómo aprende el modelo, es necesario distinguir qué papel desempeñan los distintos subconjuntos durante este proceso. Aunque su función se presentó en el capítulo anterior, en este punto sí que interesa conocer cuándo interviene cada uno durante el entrenamiento.

Conjunto	¿Cuándo interviene?	¿Modifica el modelo?	Función
<i>Train</i>	En cada época	Sí	Permite aprender y ajustar los parámetros
<i>Validation</i>	Durante el entrenamiento, después de evaluar cada época	No	Comprueba cómo evoluciona el modelo sobre imágenes distintas
<i>Test</i>	Al finalizar el entrenamiento	No	Evalúa de forma independiente el modelo seleccionado

Tabla 6. 1. Función de los subconjuntos de train, validate y test durante el desarrollo del modelo.

Por tanto:

- El uso de *validation* permite comprobar si la mejora observada durante el entrenamiento también se mantiene sobre imágenes distintas. De esta forma, es posible detectar situaciones en las que el modelo aprende cada vez mejor las imágenes con las que está entrenando, pero sin mejorar realmente su capacidad para trabajar con datos nuevos.

En un conjunto de N épocas:

Época 1: *train* → *validation*

Época 2: *train* → *validation*

...

Época N: *train* → *validation*

- Una vez finalizado el entrenamiento, se seleccionan los pesos correspondientes a la época que mejores resultados sobre el conjunto *validation* haya tenido. Esta versión del modelo se evalúa posteriormente sobre el conjunto de *test*, que se ha mantenido independiente durante todo el proceso, para comprobar cómo responde ante datos no utilizados durante el entrenamiento.

El objetivo del entrenamiento no es que el modelo memorice las imágenes utilizadas para aprender, sino que sea capaz de reconocer patrones similares en imágenes nuevas. Esta capacidad se conoce como generalización. La comparación entre los resultados de entrenamiento y validación permite comprobar si el aprendizaje está siendo útil más allá de los ejemplos utilizados para ajustar el modelo.

Este funcionamiento constituye la base del desarrollo experimental que se presenta a continuación. Sobre este esquema se plantearán los distintos entrenamientos, analizando sus resultados y utilizando cada experimento como punto de partida para introducir en el siguiente las mejoras necesarias.

6.2. Familia YOLO

Una red neuronal artificial es un modelo computacional formado por capas de unidades interconectadas que procesan información numérica. En visión artificial, estas redes reciben como entrada los valores de los píxeles de una imagen y, a través de sucesivas transformaciones, aprenden a identificar patrones visuales cada vez más complejos.

Cuando una red neuronal se diseña específicamente para detección de objetos, su objetivo no es solo el de reconocer que aparece en la imagen, sino localizarlo dentro de ella. Para realizar esta tarea existen las arquitecturas de redes neuronales.

A lo largo de los años se han desarrollado distintas arquitecturas y familias de detección, como Faster R-CNN, SSD, EfficientDet, DETR o YOLO. Aunque todas persiguen un objetivo similar, no realizan este proceso de la misma forma y pueden presentar diferencias importantes en precisión, velocidad y coste computacional.

- **Faster R-CNN:** realiza la detección en varias etapas, primero localiza posibles regiones de interés y después clasifica los objetos encontrados. Suele ofrecer buenos resultados, pero el precio computacional es muy elevado.
- **SSD:** realiza la localización y clasificación en una sola etapa, lo que permite reducir el tiempo de procesamiento.

- **EfficientDet**: también usa una sola etapa, pero está diseñado para obtener buenos resultados utilizando menos capacidad de cálculo y memoria, lo que permite adaptarlo a dispositivos más limitados.
- **DETR**: utiliza mecanismos de atención para relacionar distintas zonas de la imagen y predecir conjuntos de objetos, evita varias etapas.
- **YOLO**: realiza la detección en una única pasada por la red, buscando localizar y clasificar los objetos de forma rápida. Destaca por el equilibrio entre velocidad de procesamiento y capacidad de detección.

De entre estas alternativas se decidió trabajar con la familia YOLO, principalmente por el equilibrio que ofrece entre precisión y velocidad de procesamiento. Este aspecto resulta especialmente importante en un sistema pensado para analizar imágenes de forma ágil y que, posteriormente, deberá ejecutarse sobre hardware embarcado con capacidad de cálculo y memorias limitadas.

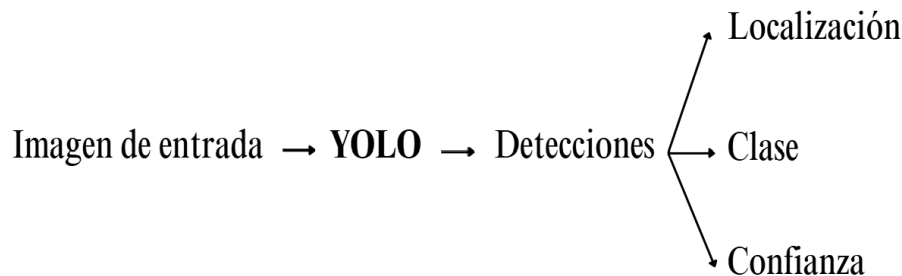


Figura 6.2. Esquema simplificado del proceso de detección realizado por YOLO.

A partir de una imagen de entrada, YOLO genera directamente las detecciones encontradas, indicando mediante *bounding boxes* su posición, la clase predicha y un nivel de confianza asociado. El nombre (*You Only Look Once*) hace referencia precisamente a este planteamiento, basado en realizar la detección mediante una única pasada por la red.

Pero YOLO no hace referencia a un único modelo, sino a una familia de modelos de detección que ha ido evolucionando mediante distintas versiones. Cada una introduce cambios en la arquitectura y en el proceso de entrenamiento, manteniendo como objetivo común la detección rápida de objetos dentro de imágenes.

Dentro de esta evolución existen además modelos de distintos tamaños, pensados para adaptarse a diferentes necesidades de precisión y capacidad de cálculo. Estas variantes se identifican con las letras:

- n (*nano*)
- s (*small*)
- m (*medium*)
- l (*large*)
- x (*extra-large*)

A medida que aumenta el tamaño del modelo, también lo hace su complejidad y, generalmente, la capacidad necesaria para ejecutarlo.

Entre las distintas versiones disponibles de la familia YOLO se escogió YOLO11 como base para los primeros experimentos. Esta versión ofrece un buen equilibrio entre precisión, velocidad y complejidad, además de que dispone de las variantes ligeras, un aspecto muy importante para su compatibilidad con el acelerador Hailo-8 previsto para la etapa de integración.

Y por este mismo motivo, dentro de YOLO11 se ha elegido la variante YOLO11n (tamaño *nano*). Al ser la opción más ligera de esta versión, requiere una menor capacidad de cálculo y memoria, lo que la convierte en una alternativa adecuada para los primeros entrenamientos y facilita su posterior ejecución sobre hardware embarcado.

Esta elección además introduce otro concepto importante: su proceso de entrenamiento se basa en el *transfer learning*. En lugar de partir de una red completamente vacía, se utiliza un modelo previamente entrenado que ya ha aprendido características visuales generales a partir de un conjunto amplio de imágenes. Ese conocimiento inicial se adapta después al problema concreto mediante las imágenes y anotaciones de los datasets preparados para los experimentos.

En este caso, se parte de un modelo YOLO11n preentrenado, proporcionado por Ultralytics [38], empresa que desarrolla y mantiene esta implementación de YOLO. Este modelo ha sido entrenado previamente sobre COCO (*Common Objects in Context*), un dataset con imágenes de objetos cotidianos pertenecientes a 80 categorías. Gracias a ello, ya ha aprendido patrones visuales generales que pueden entrenarse para detectar las anomalías de las placas solares.

Comprender la relación entre YOLO, Ultralytics y COCO resulta especialmente importante para interpretar correctamente el proceso de entrenamiento utilizado en los siguientes apartados.

6.3. Entorno de entrenamiento

Antes de comenzar con los experimentos para el modelo de detección de defectos en las placas, la siguiente decisión clave fue la de dónde realizarlos. Se puede entrenar al modelo desde el propio equipo, trabajando en local, o recurriendo a un entorno de computación en la nube con mayor capacidad de cálculo.

- Entrenamiento en local: el procesamiento se realiza directamente en el propio ordenador. Permite trabajar sin depender de una conexión externa y mantener todos los datos y archivos en el equipo, pero el tiempo de entrenamiento queda limitado por la capacidad de cálculo y memoria disponibles.
- Entrenamiento en la nube: el entrenamiento se ejecuta en servidores externos a través de internet. Esto permite acceder a hardware más potente, especialmente GPU, sin necesidad de disponer físicamente de ese equipo, aunque depende de la disponibilidad del servicio y de la conexión.

Esta decisión era especialmente relevante, ya que el entorno seleccionado condicionaría el desarrollo de los experimentos posteriores. Mantener una misma forma de trabajo permitiría repetir los entrenamientos de manera más ordenada y evitar cambios innecesarios entre pruebas.

Para valorar las posibilidades y tomar una decisión se realizó una primera prueba muy sencilla en local, lo que permitió comprobar los límites del equipo disponible.

Este modelo preliminar fue un modelo de detección de frutas.

6.3.1. Prueba preliminar

Se eligió este tipo de objetos porque permitía trabajar con un dataset ya disponible en Roboflow y, al mismo tiempo, familiarizarse con el uso de la plataforma (antes de comenzar con los datasets de paneles).

Además, se disponía fácilmente de distintas frutas para generar nuevas imágenes propias, lo que permitía realizar una prueba rápida sin necesidad de organizar todavía campañas de adquisición específicas. De esta forma, se aprovechó también para aprender a incorporar y anotar nuevas imágenes.

Sobre el dataset seleccionado, de 100 imágenes, se añadieron 31 imágenes propias distribuidas en 12 clases.

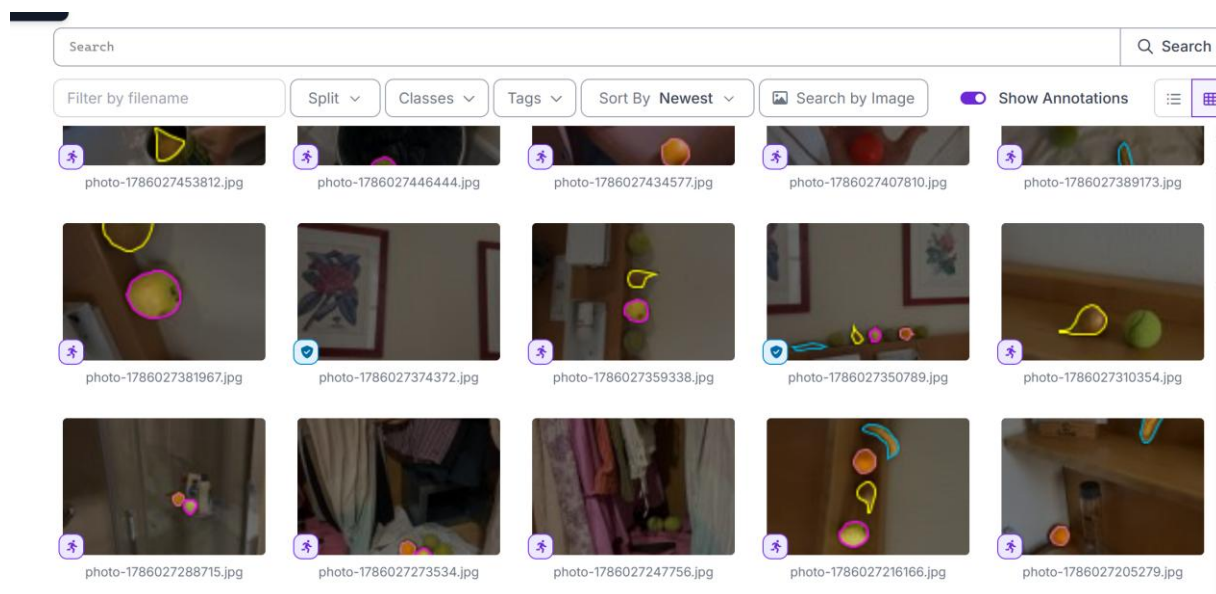


Figura 6.3. Ejemplos de imágenes propias anotadas e incorporadas al dataset en la prueba preliminar.

Durante esta primera prueba también se aprovechó para experimentar con las herramientas de *data augmentation* disponibles en Roboflow.

Se aplicaron distintas transformaciones sobre las imágenes de entrenamiento:

- *Outputs per training example: 3*
- *Flip: horizontal*
- *90° rotate*
- *Crop (0% - 20% zoom)*
- *Rotation (Between -15° and +15°)*
- *Saturation (Between -25% and +25%)*
- *Brightness (Between -15% and +15%)*
- *Blur: Up to 2.5 px*

Esta primera toma de contacto permitió conocer su funcionamiento antes de valorar su utilización en experimentos posteriores.

El *data augmentation* se aplicó solo sobre las imágenes de *train*, ya que su objetivo es proporcionar al modelo más variedad de ejemplos con los que aprender.

Las imágenes de *validation* y *test* se mantuvieron sin modificar. Estos conjuntos se utilizan para comprobar cómo responde el modelo ante imágenes reales que no han sido generadas artificialmente, por lo que aplicarles esta técnica podría hacer la evaluación menos representativa.

Como resultado, la versión generada quedó formada por 315 imágenes, distribuidas en 276 para *train*, 26 para *validation* y 13 para *test*, y como consecuencia, esta distribución ya no mantiene la proporción aproximada de 70/20/10 mencionada anteriormente. No obstante, sigue siendo una configuración válida (88/8/4).

Con esta versión del dataset preparada, se realizó el entrenamiento en entorno local mediante Visual Studio Code, utilizando la siguiente configuración:

Modelo base: YOLOv8n preentrenado
Epochs: 40
Tamaño de imagen: 640x640 px
Batch: 8
Entorno: Visual Studio Code – local
Tiempo total de entrenamiento: 1 h 6 min 47 s

Como se puede comprobar, para esta prueba se utilizó YOLOv8n preentrenado, una variante *nano* de la familia YOLO. La elección respondía al carácter exploratorio del ensayo: no se buscaba seleccionar aún el modelo definitivo sino familiarizarse con el proceso de entrenamiento y comprobar la capacidad del equipo local.

El entrenamiento se configuró con 40 épocas, es decir, 40 recorridos completos sobre el conjunto *train*.

Las imágenes se procesaron con el tamaño de 640x640 píxeles porque es una resolución muy habitual en modelos YOLO, permitiendo un equilibrio entre nivel de detalle y tiempo de procesamiento.

Se utilizó un *batch*: 8, lo que significa que el modelo procesa grupos de 8 imágenes de entrenamiento antes de calcular el error y continuar con el ajuste de parámetros. Este valor influye en el uso de memoria y en la velocidad de entrenamiento.

Para conocer la duración completa de la ejecución se añadió al *script* una medición del tiempo mediante la librería *'time'* de Python, registrando el instante anterior al inicio del entrenamiento y el momento de su finalización: la prueba necesitó 1 h 6 min y 47 s para completarse.

Además del tiempo empleado, durante la ejecución se observaron algunos aspectos que permitieron valorar mejor la viabilidad (o falta de) del entrenamiento en local:

- Respuesta del equipo: el ordenador alcanzó una temperatura muy elevada, y mantener el entrenamiento activo dificultaba utilizarlo con normalidad para realizar mientras tanto otras tareas.
- Escalabilidad de los experimentos: si una prueba relativamente pequeña ya requería más de una hora, realizar numerosos entrenamientos con datasets de mayor tamaño, o

planteándose usar más *epochs*, podría ralentizar considerablemente el desarrollo experimental.

Estas observaciones fueron suficientes para considerar que los experimentos principales se realizarían posteriormente en un **entorno de computación en la nube**.

De esta primera prueba no solo se obtuvo la conclusión de que convenía trasladar los entrenamientos principales a la nube. También permitió comprobar que el planteamiento seguido iba en la dirección correcta, ya que el flujo completo de preparación de datos, entrenamiento y uso posterior del modelo funcionaba correctamente.

Al finalizar el entrenamiento se obtuvo el archivo ‘best.pt’, correspondiente a los pesos con mejor comportamiento durante la validación y preparado para realizar predicciones sobre nuevos datos.

Para realizar esta comprobación, se grabó un pequeño vídeo con el dron utilizando algunas de las frutas empleadas durante la prueba. El archivo se incorporó en el proyecto en Visual Studio Code y se ejecutó el modelo ‘best.pt’ sobre la secuencia obtenida.

De esta forma fue posible observar directamente las detecciones realizadas sobre el vídeo y comprobar que el modelo era capaz de localizar algunas de las frutas fuera del conjunto utilizado durante el entrenamiento.

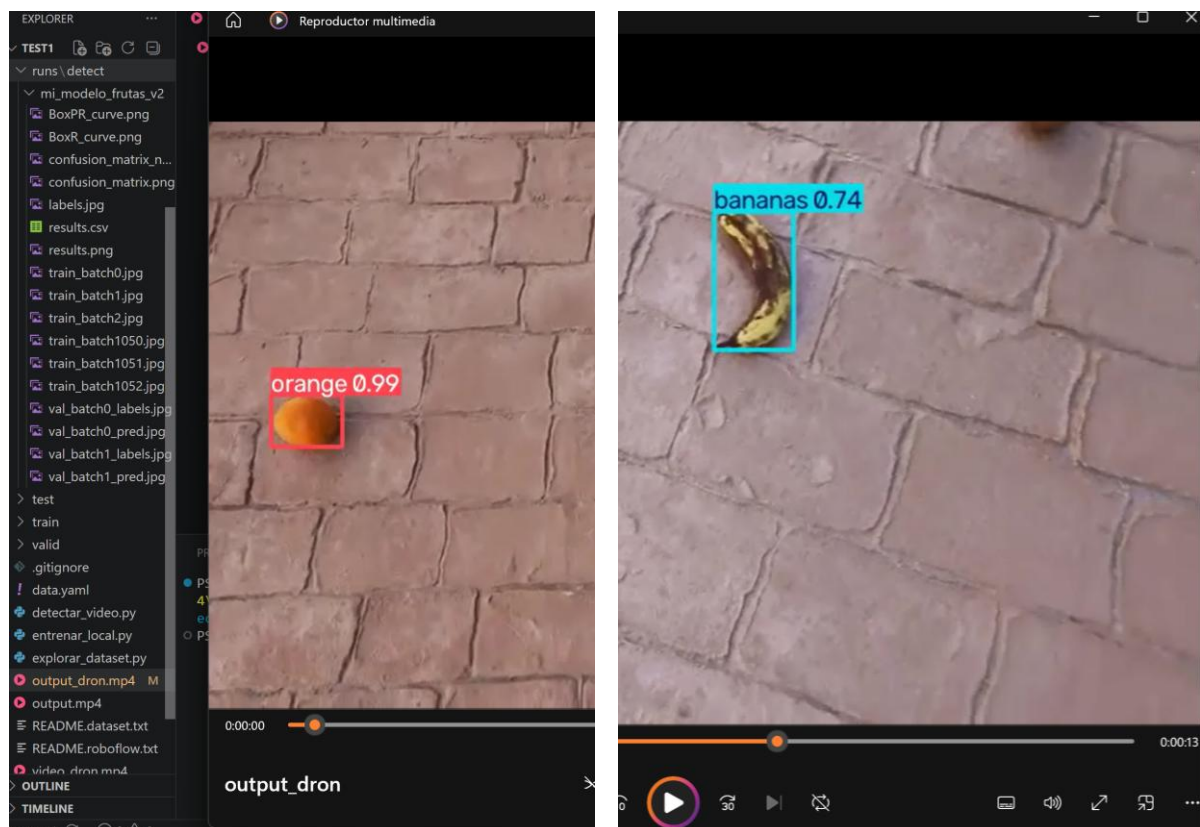


Figura 6.4. Ejemplos de detección de frutas sobre una secuencia de vídeo utilizada en la prueba preliminar.

Aunque los resultados no eran todavía lo suficientemente precisos como para considerar el detector definitivo, la prueba permitió comprobar que el flujo completo funcionaba y confirmó la decisión de utilizar un entorno en la nube para los experimentos posteriores.

6.3.2. Entorno de ejecución: Google Colab

A partir de esta experiencia, los siguientes entrenamientos se plantearon directamente en un entorno de computación en la nube. El objetivo: disponer de una mayor capacidad de cálculo sin depender de las limitaciones del equipo local y poder repetir los experimentos de forma más ágil.

Para ello, el propio Roboflow ofrece la posibilidad de realizar entrenamientos directamente desde la plataforma. Sin embargo, se decidió utilizar Google Colab, principalmente por la familiaridad con este entorno y por el mayor control que permite mantener sobre la ejecución, los parámetros de entrenamiento y los archivos generados.

De esta forma, Roboflow se mantuvo como herramienta para la gestión, preparación y versionado de los datasets, mientras que Colab se utilizó como entorno principal para ejecutar los entrenamientos en la nube a partir de esos datasets.

Google Colab *¡Error! No se encuentra el origen de la referencia.* es un entorno de programación en la nube que permite ejecutar código Python directamente desde el navegador. Los cálculos no se realizan en el ordenador propio, sino en una máquina remota proporcionada por Google, lo que permite utilizar recursos de procesamiento superiores a los disponibles en muchos equipos personales.

Una de sus principales ventajas para este proyecto es la posibilidad de utilizar una GPU (*Graphics Processing Unit*) durante los entrenamientos. Aunque este tipo de procesadores nació para el tratamiento de gráficos, su capacidad para realizar numerosos cálculos de forma paralela los hace especialmente interesantes para el entrenamiento de redes neuronales.

Para entender por qué disponer de una GPU es tan relevante, conviene compararla con la CPU (*Central Processing Unit*), el procesador de propósito general encargado de ejecutar la mayoría de las tareas habituales de un ordenador.

En “*Tabla 6.2*” se resumen las principales diferencias entre ambos tipo de procesador en relación con los entrenamientos en redes neuronales:

	CPU	GPU
Diseño	Pocas unidades de cálculo muy potentes	Muchas unidades de cálculo trabajando en paralelo
Tipo de tareas	Operaciones generales, lógica y control	Grandes cantidades de operaciones matemáticas similares
Papel durante un entrenamiento	Gestiona tareas generales y preparación del proceso	Realiza gran parte de los cálculos necesarios para entrenar la red
Entrenamiento en redes neuronales	Puede realizarlo, con mucho tiempo de ejecución	Especialmente adecuada para reducir los tiempos de entrenamiento
Memoria utilizada principalmente	RAM del sistema	Memoria de la GPU (VRAM)

Tabla 6.2. Comparación entre CPU y GPU en el contexto del entrenamiento de redes neuronales.

Una vez activado el entorno GPU en Google Colab, se comprobó el dispositivo asignado mediante el comando `'nvidia-smi'`. En las sesiones utilizadas para los entrenamientos se dispuso de una NVIDIA Tesla T4, que sería la GPU encargada de realizar la mayor parte de los cálculos asociados al entrenamiento de los modelos. NVIDIA Tesla T4 está disponible directamente de forma remota en determinados entornos de Colab.

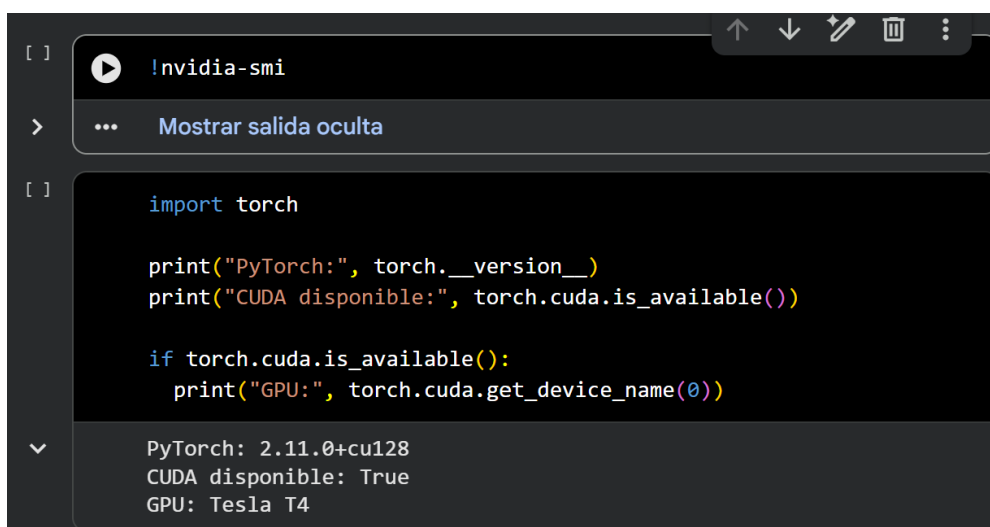
Por último, para que las librerías utilizadas durante el entrenamiento puedan aprovechar la GPU, se utiliza CUDA (*Compute Unified Device Architecture*), una plataforma desarrollada por NVIDIA que permite ejecutar cálculos sobre sus tarjetas gráficas. Así, herramientas como PyTorch pueden trasladar a la GPU gran parte de las operaciones necesarias durante el entrenamiento.

Por tanto, se tiene:

- **YOLO11n**: el modelo que se quiere entrenar.
- **Ultralytics**: proporciona la implementación de YOLO y las funciones utilizadas para entrenarlo.
- **PyTorch**: librería de Python que gestiona las operaciones necesarias para construir y entrenar la red neuronal.
- **CUDA**: permite que PyTorch ejecute esas operaciones sobre la GPU.
- **NVIDIA Tesla T4**: hardware que finalmente ejecuta gran parte de esos cálculos.

Una vez definido Google Colab como entorno de ejecución, se preparó un cuaderno de trabajo desde el que se realizarían los entrenamientos. El primer paso consistió en activar el entorno GPU y comprobar que esta se encontraba correctamente disponible antes de comenzar a entrenar a los modelos:

1. Activación del entorno GPU. Se configuró Colab para utilizar aceleración por hardware.
2. Comprobación del dispositivo. Mediante `'nvidia-smi'` se verificó la asignación de la Tesla T4.
3. Comprobación desde Python. Se utilizó `'torch.cuda.is_available()'` para confirmar que PyTorch podía acceder correctamente a la GPU.



```
[ ] !nvidia-smi
> ... Mostrar salida oculta
[ ]
import torch

print("PyTorch:", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

PyTorch: 2.11.0+cu128
CUDA disponible: True
GPU: Tesla T4
```

Figura 6. 5. Verificación del entorno GPU y disponibilidad de la NVIDIA Tesla T4 en Google Colab.

Tras comprobar que la GPU estaba disponible, el siguiente paso consistió en preparar el entorno de Colab para trabajar con los modelos YOLO.

Para ello se instaló la librería Ultralytics (previamente mencionada), que proporciona las herramientas necesarias para cargar, entrenar y evaluar estos modelos:

```
!pip install -U Ultralytics
```

Una vez instalada, la clase YOLO se importó en Python para poder cargar posteriormente el modelo que se utilizaría en los entrenamientos:

```
from Ultralytics import YOLO
```

Una vez preparado el entorno, fue necesario establecer la conexión con Roboflow, la plataforma de versionado de datasets. Esta conexión permitiría descargar directamente desde Colab una versión concreta del conjunto de datos, evitando tener que realizar transferencias manuales de archivos.

Para realizar esta conexión se utilizó la API de Roboflow, un mecanismo que permite que una aplicación externa acceda de forma controlada a los recursos almacenados en la plataforma. Este acceso se realiza mediante una clave personal asociada a la cuenta de usuario.

Por motivos de seguridad, la clave de acceso no se escribió directamente en el cuaderno, sino que se almacenó mediante la función ‘Secrets’ de Colab, desde donde puede recuperarse durante la ejecución sin necesidad de que su valor quede visible en las celdas del notebook.

```
from google.colab import userdata
api_key = userdata.get("ROBOFLOW_API_KEY")
```

De esta forma, la clave almacenada en Colab podía recuperarse durante la ejecución y se guardaba temporalmente en la variable ‘api_key’, que posteriormente se utiliza para establecer la conexión final con Roboflow:

```
!pip install -q roboflow
import roboflow
rf = roboflow.Roboflow(api_key=api_key)
```

De esta forma, el entorno de Colab quedó conectado con Roboflow y preparado para acceder posteriormente a las distintas versiones del dataset utilizadas en los experimentos. La selección y descarga de cada versión concreta se realizará en su respectivo entrenamiento.

Finalmente, aunque Google Colab permite trabajar con recursos de cálculo en la nube, las sesiones no son permanentes. Al finalizar una sesión, reiniciar el entorno o producirse una desconexión imprevista, pueden perderse las variables, las librerías instaladas y los archivos almacenados temporalmente en el entorno de ejecución.

Para evitar la pérdida de los resultados, se utilizó Google Drive como almacenamiento persistente. De esta forma, los archivos de cada experimento podían conservarse independientemente de la sesión de Colab.

```
from google.colab import drive  
drive.mount('/content/drive')
```

Una vez montada la unidad, Google Drive queda accesible desde Colab, permitiendo almacenar de forma permanente los pesos entrenados, métricas, gráficas y demás resultados de interés.

6.3.3. Configuración experimental

Una vez preparado el entorno de ejecución, se definió una configuración común para los experimentos principales con el objetivo de mantener constantes los parámetros de entrenamiento que no formaban parte de la comparación, para así, poder relacionar en los resultados, las modificaciones introducidas en los datos.

Modelo base:	YOLO11n preentrenado
<i>max. epochs</i> :	100
<i>patience</i> :	20
Tamaño de imagen (<i>imgsz</i>):	640x640 px
<i>batch</i> :	16
GPU:	NVIDIA Tesla T4
<i>seed</i> :	42
Ejecución determinista (<i>deterministic</i>):	Sí (<i>True</i>)

Se estableció un máximo de 100 épocas para proporcionar al modelo un margen suficiente de aprendizaje sin fijar de antemano una duración excesivamente restrictiva. Sin embargo, esto no implica necesariamente que el entrenamiento deba completar las 100 épocas.

Para evitar continuar el entrenamiento cuando el modelo ya no mejora, se utilizó el mecanismo de *early stopping*, incorporado en Ultralytics. Mediante el parámetro '*patience* = 20', el entrenamiento se detiene automáticamente si durante 20 épocas consecutivas no se observa una mejora en el rendimiento de validación.

De esta forma, las 100 épocas actúan como un límite máximo, mientras que '*patience*' permite finalizar antes el entrenamiento cuando el modelo ha alcanzado una situación estable.

El tamaño de entrada se fijó en 640x640 píxeles, de forma que las imágenes utilizadas durante el entrenamiento se adaptaron a una dimensión común. Este valor permite conservar un nivel de detalle adecuado sin incrementar excesivamente el tiempo computacional y la memoria de la GPU:

A mayor tamaño → más detalle de imagen → más cálculos tiene que realizar la GPU → más VRAM necesita → más tarda en entrenar

El *batch* se estableció en 16, lo que significa que durante el entrenamiento las imágenes se procesan en grupos de 16 antes de calcular el error correspondiente y continuar con el ajuste del modelo. Este valor permite aprovechar el procesamiento paralelo de la GPU sin exigir una cantidad excesiva de memoria.

Para facilitar una comparación controlada entre los experimentos se fijó ‘*seed* = 42’. Esta semilla no trata de eliminar los procesos aleatorios del entrenamiento, sino que permite reproducirlos de forma similar entre ejecuciones, reduciendo así la posibilidad de que las diferencias observadas se deban únicamente al azar.

Además, se activó ‘*deterministic* = *True*’, lo que favorece que las operaciones del entrenamiento se ejecuten de la forma más reproducible posible. Junto con ‘*seed* = 42’, esta configuración ayuda a reducir variaciones entre ejecuciones y facilita la comparación controlada que se busca entre los distintos experimentos.

Por último, aunque algunas de las versiones de los datasets utilizados para los primeros experimentos no utilicen *augmentation* durante su generación en Roboflow, Ultralytics aplica determinadas transformaciones de forma automática durante el entrenamiento. Estas modificaciones se realizan de manera dinámica sobre las imágenes, lo que puede incluir cambios como giros, variaciones geométricas o técnicas de mosaicos, sin crear nuevas imágenes dentro del dataset.

Con esta configuración se establecieron las condiciones comunes de los experimentos principales. Mantener constantes estos parámetros permitirá comparar los resultados de forma controlada, atribuyendo las diferencias observadas principalmente a los cambios introducidos en los datos de entrenamiento.

6.4. E1: *baseline*

El primer experimento se planteó como punto de referencia para evaluar el comportamiento inicial del modelo sobre el dataset preparado en la fase anterior. Para ello se utilizó la versión “baseline”, que estaba compuesta por 724 imágenes: 507 para *train*, 145 para *validation* y 72 para *test*.

El modelo empleado, como ya se ha introducido, fue YOLO11n preentrenado, utilizando la configuración experimental definida en el apartado anterior. El entrenamiento se estableció con máximo de 100 *epochs* y *patience* = 20, de forma que pudiera finalizar anticipadamente si el rendimiento dejaba de mejorar.

El tiempo total de entrenamiento fue de 14 minutos y 21,9 segundos.

Este resultado refuerza la decisión de trasladar los experimentos principales a Google Colab, ya que la prueba preliminar realizada en local necesitó aproximadamente 1 hora y 7 minutos, a pesar de utilizar un conjunto de datos menor y limitarse a 40 *epochs*. En este caso, el entrenamiento alcanzó muchas más épocas sobre un dataset considerablemente mayor en un tiempo muy inferior, evidenciando la ventaja práctica de utilizar la GPU Tesla T4 para desarrollar y repetir los experimentos.

Durante el entrenamiento, el rendimiento del modelo fue variando a lo largo de las épocas, alcanzándose el mejor resultado en la época 61. A partir de ese punto, el modelo continuó entrenándose, pero no consiguió superar dicho resultado durante las 20 épocas siguientes.

Como consecuencia, el mecanismo de *early stopping* detuvo automáticamente el entrenamiento en la época 81, evitando continuar hasta las 100 épocas máximas inicialmente establecidas y reduciendo así cálculos innecesarios.

Los pesos correspondientes al mejor estado del modelo quedaron almacenados en el archivo ‘*best.pt*’, utilizado posteriormente para la evaluación sobre el conjunto de *test*.

Antes de analizar los valores obtenidos, conviene introducir brevemente las principales métricas utilizadas para evaluar el comportamiento del modelo:

- **Precision:** indica qué proporción de las detecciones realizadas por el modelo son correctas. Una precisión elevada significa que, cuando el modelo afirma haber detectado una anomalía, suele acertar y genera pocos falsos positivos.
- **Recall:** mide qué proporción de los objetos realmente presentes en las imágenes consigue detectar el modelo. Un valor elevado indica que se dejan pocas anomalías sin detectar, es decir, que existen pocos falsos negativos.
- **mAP50:** resume el rendimiento de detección considerando tanto la clase predicha como la localización del objeto. Para considerar correcta una detección se utiliza un nivel mínimo de solapamiento del 50% entre la caja predicha y la anotación real. Este solapamiento lo mide el parámetro **IoU (Intersection over Union)**.
- **mAP50-95:** aplica una evaluación más exigente, calculando el rendimiento para distintos niveles de IoU comprendidos entre 0,50 y 0,95. Por esta razón, no solo exige detectar correctamente el objeto, sino también localizarlo con mayor precisión, es muy habitual que su valor sea inferior al de *mAP50*.

Como se ha introducido anteriormente, la época 61 fue la que alcanzó el mejor rendimiento sobre el conjunto de validación. Los valores obtenidos de esta época se recogen en “*Tabla 6.3*”:

Métrica	Valor
<i>Precision</i>	0,3728
<i>Recall</i>	0,4514
<i>mAP50</i>	0,3958
<i>mAP50-95</i>	0,2363

Tabla 6.3. Métricas obtenidas en valid para la mejor época de E1.

Una vez finalizado el entrenamiento y seleccionado el modelo correspondiente a la mejor época, se realizó una evaluación independiente utilizando las 72 imágenes reservadas para el test. Estas imágenes no habían intervenido ni en *train* ni en *valid*, por lo que permiten comprobar cómo se comporta el modelo ante datos no utilizados durante su aprendizaje.

Sobre *test*, el modelo obtuvo los resultados globales mostrados en “*Tabla 6.4*”:

Métrica	Valor
<i>Precision</i>	0,366
<i>Recall</i>	0,457
<i>mAP50</i>	0,429
<i>mAP50-95</i>	0,315

Tabla 6.4. Métricas globales obtenidas sobre test en E1.

Los resultados obtenidos sobre este conjunto muestran un comportamiento ligeramente superior al observado durante *validation*, alcanzando un *mAP50* de 0,429 y un *mAP50-95* de 0,315. Esto indica que el modelo es capaz de mantener su capacidad de detección sobre imágenes que no participaron en ninguna fase del entrenamiento.

Sin embargo, estos valores globales no permiten conocer si el comportamiento es homogéneo entre todas las clases. Para identificar con mayor precisión dónde se encontraban las principales fortalezas y limitaciones del modelo, se analizaron las métricas obtenidas individualmente para cada una de las clases. Los resultados se muestran en “Tabla 6.5”:

Clase	<i>Precision</i>	<i>Recall</i>	<i>mAP50</i>	<i>mAP50-95</i>
<i>bird droppings</i>	0,376	0,600	0,417	0,334
<i>crack</i>	0,198	0,167	0,191	0,174
<i>dust</i>	0,380	0,636	0,669	0,560
<i>electro</i>	0,113	0,111	0,036	0,008
<i>snow</i>	0,728	0,894	0,875	0,680
<i>thermal anomaly</i>	0,403	0,333	0,385	0,135

Tabla 6.5. Métricas obtenidas por clase sobre test en E1.

Como se puede observar, las diferencias entre clases son considerables. La clase ‘*snow*’ presenta el mejor comportamiento, con un *mAP50* de 0,875 y un *Recall* de 0,894, lo que indica que el modelo consigue detectar correctamente la mayor parte de los casos presentes en el conjunto de test.

Por su parte, ‘*dust*’ también obtiene resultados elevados, alcanzando un *mAP50* de 0,669 y un *mAP50-95* de 0,560.

La clase ‘*bird droppings*’ presenta un comportamiento intermedio, con un *mAP50* de 0,417 y un *Recall* de 0,600. El modelo consigue detectar una parte importante de los casos presentes, aunque la *Precision* de 0,376 indica que todavía se producen detecciones incorrectas.

En ‘*thermal anomaly*’ el comportamiento es más irregular. Aunque el *mAP50* alcanza 0,385, el *mAP50-95* desciende hasta 0,135, lo que indica que el modelo consigue localizar algunas anomalías, pero presenta mayores dificultades cuando se exige una delimitación más precisa de la zona afectada.

Las mayores limitaciones aparecen en las clases ‘*crack*’ y ‘*electro*’. En el caso de ‘*crack*’, el *mAP50* se sitúa en 0,191, mientras que ‘*electro*’ presenta el peor resultado del conjunto, con un *mAP50* de apenas 0,036 y un *Recall* de 0,111. Estos valores muestran que el dataset baseline no proporciona todavía al modelo información suficiente para aprender de forma sólida con estas categorías.

Para analizar con mayor detalle el origen de estos resultados se utilizó la matriz de confusión normalizada (Figura 6.6). Esta representación permite observar qué proporción de los casos de cada clase se identifica correctamente y cuáles terminan confundiendo con otras categorías o con el fondo (*background*).

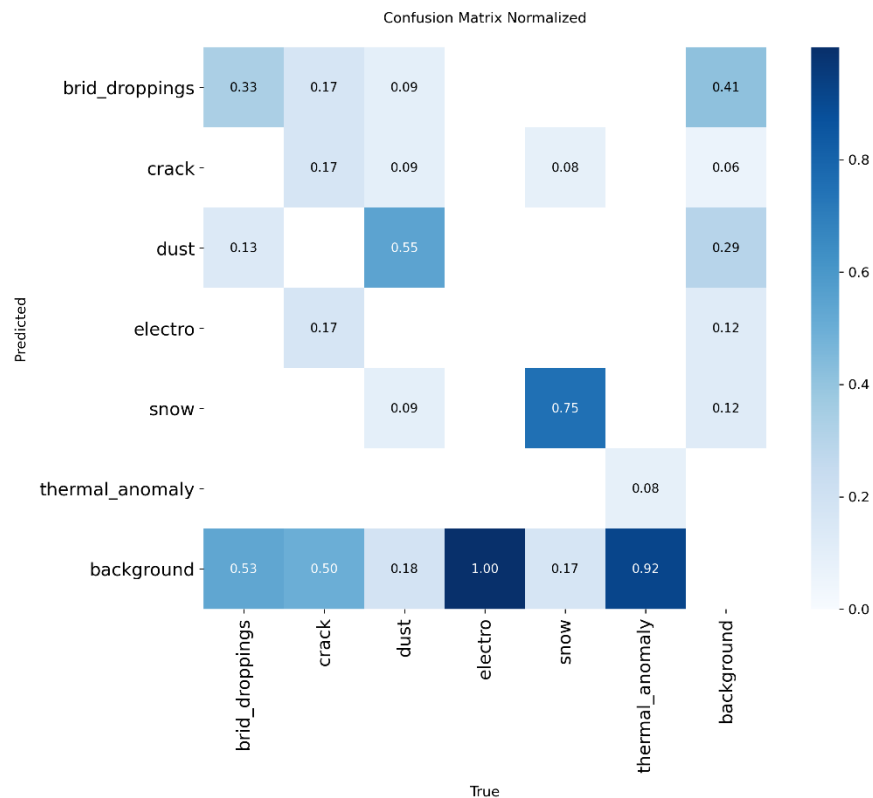


Figura 6.6. Matriz de confusión normalizada de EI sobre el conjunto test. Elaboración propia a partir de los resultados generados por Ultralytics.

Para comprender bien la matriz:

- Los valores situados en la diagonal representan las detecciones correctas.
- La fila correspondiente a *background* muestra aquellos objetos reales que el modelo no consiguió detectar.
- La columna correspondiente a *background* recoge detecciones realizadas sobre zonas en las que no existía ninguna anomalía.

La matriz confirma el buen comportamiento observado en ‘*snow*’, que alcanza aproximadamente un 0,75 de detecciones correctas, seguida de ‘*dust*’ con 0,55. En el caso de ‘*bird_droppings*’, el valor de la diagonal se sitúa en 0,33, observándose detecciones confundidas con ‘*dust*’ y una proporción importante de casos asociados a ‘*background*’.

Las principales dificultades aparecen de nuevo en ‘*crack*’, ‘*electro*’ y ‘*thermal_anomaly*’.

En ‘*crack*’, únicamente alrededor de 0,17 de los casos aparecen correctamente clasificados, mientras que aproximadamente 0,50 terminan asociados a *background*.

La situación es todavía más acusada para ‘*electro*’ y ‘*thermal_anomaly*’. En el caso de ‘*electro*’, la matriz muestra que las instancias reales terminan siempre asociadas a *background*, lo que indica que el modelo no consigue detectarlas bajo las condiciones utilizadas para generar la matriz. Para ‘*thermal_anomaly*’, aproximadamente un 0,92 de los casos también se asocia a *background*, mientras que solamente un 0,08 aparece correctamente identificado.

Además del comportamiento final del modelo, resulta útil analizar cómo evolucionó el entrenamiento a lo largo de las distintas épocas. Para ello se revisaron las curvas generadas por

Ultralytics, en las que se representan tanto las pérdidas de *train* y *validation* como la evolución de las principales métricas.

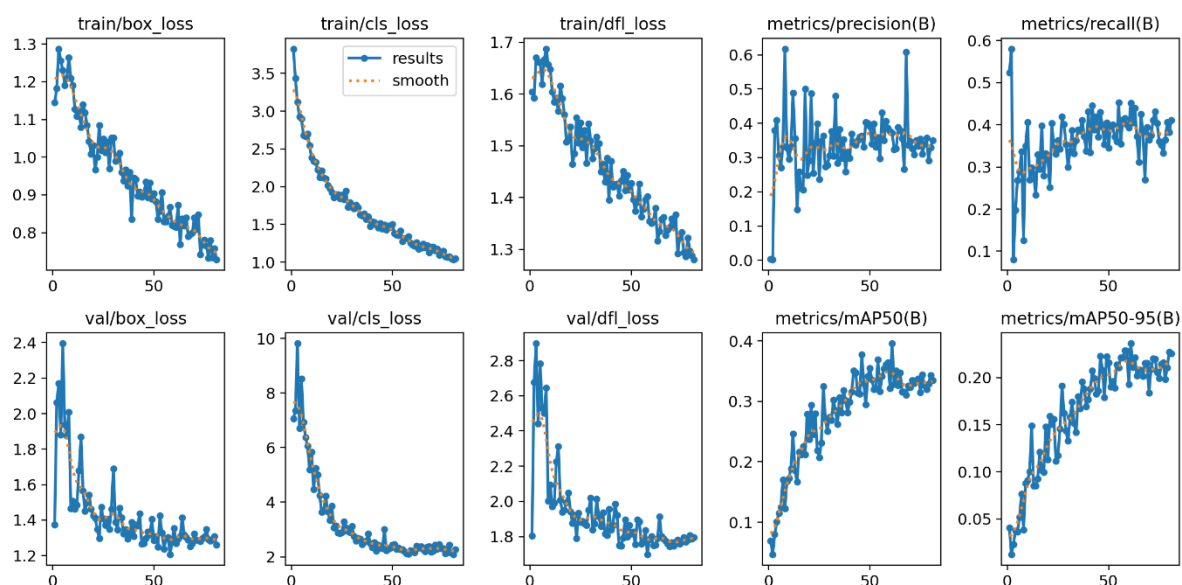


Figura 6.7. Evolución de las pérdidas y métricas durante train en E1. Elaboración propia a partir de los resultados generados por Ultralytics.

En la figura se representan las principales pérdidas (*loss*) y métricas obtenidas durante las 81 épocas completadas. Las pérdidas ‘*box_loss*’, ‘*cls_loss*’ y ‘*dfl_loss*’ están relacionadas, respectivamente, con la localización de los objetos, su clasificación y el ajuste de sus *bounding boxes*. En general, cuanto menores sean estos valores, mejor se está ajustando el modelo al propuesto.

Las tres pérdidas asociadas a *train* presentan una tendencia descendente prácticamente durante todo el proceso, lo que indica que el modelo continúa aprendiendo a partir de las imágenes disponibles.

Las pérdidas de *validation* también disminuyen con claridad durante las primeras épocas, aunque posteriormente comienzan a estabilizarse y muestran mayores oscilaciones.

Esta evolución se refleja también en las métricas de *validation*. Tanto el *Recall* como el *mAP50* y el *mAP50-95* mejoran progresivamente durante las primeras fases del entrenamiento, pero a partir de aproximadamente las épocas 50 – 60, las mejoras son mucho menores y predominan las fluctuaciones, alcanzándose el mejor resultado en la *epoch* 61, sin conseguir superarse durante las 20 posteriores.

Por tanto, las curvas son coherentes con la actuación del mecanismo de *early stopping*: el entrenamiento había entrado en una fase de estabilización en la que continuar hasta las 100 épocas máximas no estaba produciendo una mejora sostenida del modelo.

No se aprecia un sobreajuste severo, aunque la estabilización de las pérdidas de validación frente al descenso continuado de las pérdidas de entrenamiento indica que el margen de mejora comenzaba a reducirse.

Más allá de las métricas numéricas, se revisaron también las predicciones realizadas por el modelo sobre imágenes del conjunto de *test*. Ultralytics genera automáticamente mosaicos correspondientes a algunos *batches* de imágenes procesados, disponiéndose para cada uno las anotaciones reales por un lado y, por el otro, las predicciones realizadas por el modelo.

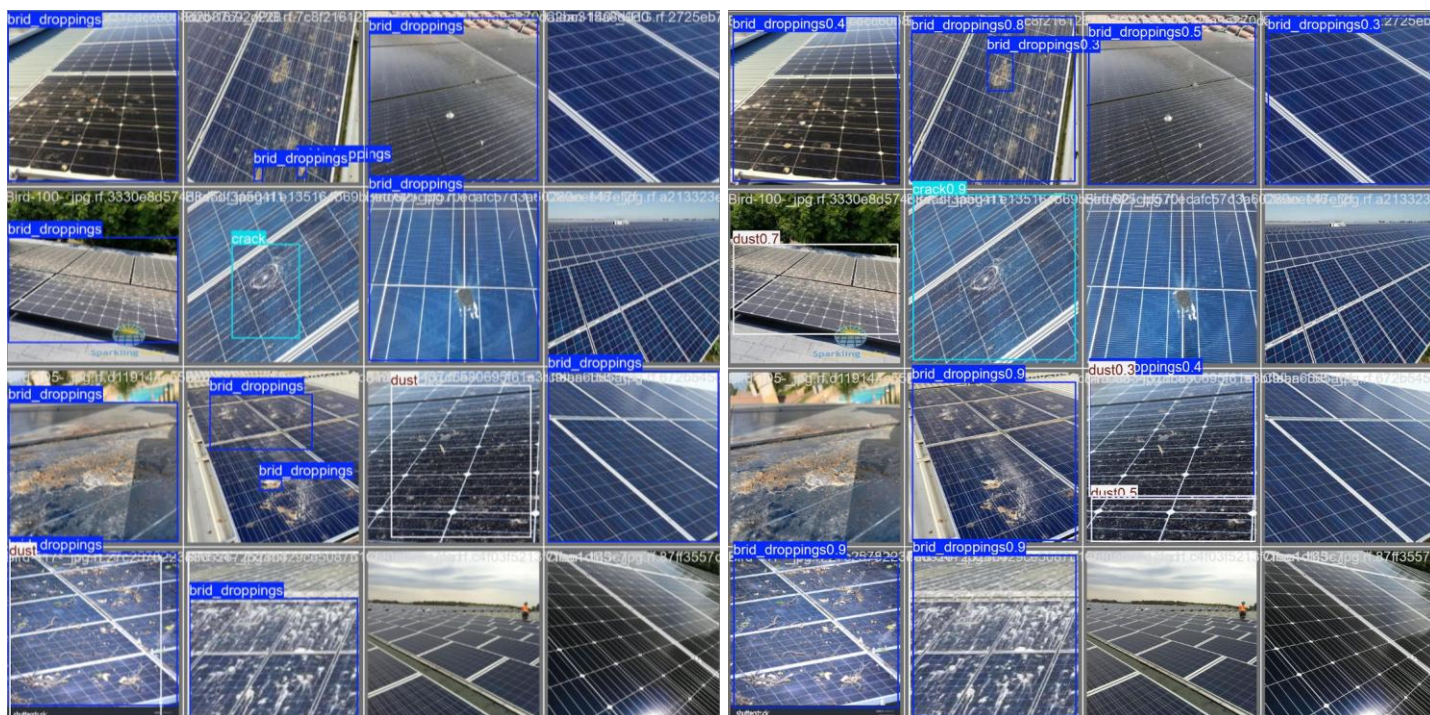


Figura 6.8. Comparación entre anotaciones reales y predicciones E1 sobre test.

La comparación entre ambas representaciones permite observar visualmente los aciertos, las detecciones omitidas y las posibles confusiones entre clases.

En un segundo *batch* se observan situaciones más complejas, especialmente en clases que ya habían mostrado un comportamiento más débil en el análisis de métricas.

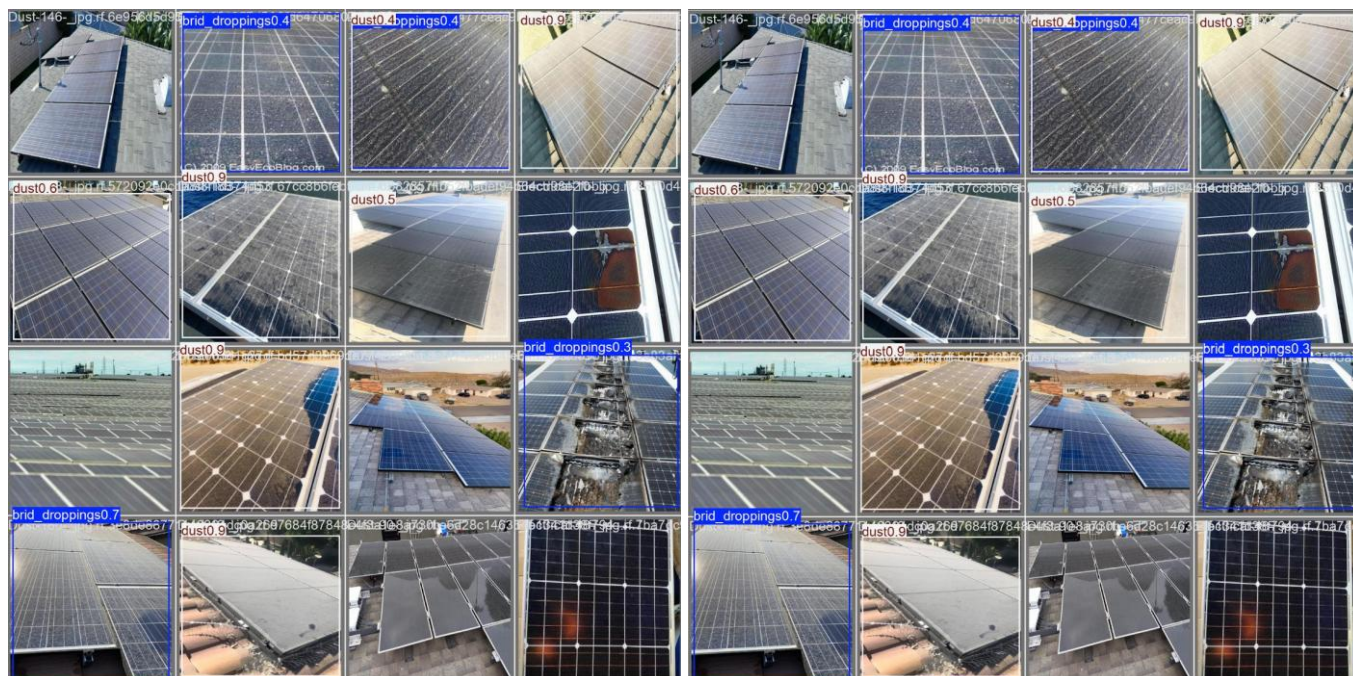


Figura 6.9. Ejemplos de situaciones “problemáticas” observadas en predicciones de E1 sobre test.

En este segundo grupo se aprecia con mayor claridad que el modelo todavía presenta dificultades ante determinadas anomalías. Algunos casos de ‘*electro*’ no llegan a ser detectados, mientras que otras zonas son asignadas a clases como ‘*dust*’ o ‘*bird_droppings*’.

Estas situaciones son coherentes con los bajos valores obtenidos anteriormente para las clases más problemáticas y muestran que el rendimiento global del modelo no es homogéneo para todas las categorías.

En conjunto, el experimento E1 demuestra que YOLO11n es capaz de aprender patrones asociados a las anomalías presentes en el dataset baseline, aunque con diferencias importantes entre clases.

‘snow’ y ‘dust’ presentan los resultados más sólidos, mientras que ‘crack’, ‘electro’ y ‘thermal_anomaly’ muestran un margen de mejora considerable.

A pesar de estas limitaciones, en este punto se decidió no modificar todavía la definición de las clases ni introducir nuevas estrategias de aumento de datos. Antes de alterar el planteamiento del problema, se consideró necesario evaluar qué efecto tendría únicamente ampliar el conjunto de entrenamiento con nuevas imágenes, manteniendo constantes el modelo, los parámetros de entrenamiento y los conjuntos de *valid* y *test*.

Esta será la finalidad del experimento E2, permitiendo analizar de forma aislada la aportación del nuevo material incorporado al dataset (E2_expanded_dataset).

6.5. E2: *extended_dataset*

Una vez establecido E1 como punto de referencia, el segundo experimento se planteó con el objetivo de comprobar qué efecto tenía ampliar el conjunto de entrenamiento sin modificar el resto de las condiciones experimentales. Para ello se utilizó la versión ‘*E2_extended_dataset*’ compuesta por 889 imágenes.

En esta nueva versión, el conjunto de *train* pasó de 507 a 672 imágenes, mientras que los conjuntos de *validation* y *test* se mantuvieron sin cambios, con 145 y 72 imágenes respectivamente. De esta forma, E1 y E2 podían evaluarse sobre exactamente las mismas imágenes, siendo la ampliación del conjunto de *train* la principal diferencia entre ambos experimentos.

Conjunto	E1 baseline	E2 extended
<i>train</i>	507	672
<i>valid</i>	145	145
<i>test</i>	72	72
Total	724	889

Tabla 6.6. Comparación de la distribución de los subconjuntos de datos entre E1 y E2.

El entrenamiento de E2 se realizó manteniendo exactamente la misma configuración utilizada en E1 modelo YOLO11n preentrenado, tamaño de entrada de 640x640 píxeles, ‘*batch*’ = 16, máximo de 100 *epochs*, *patience* = 20, *seed* = 42 y ejecución determinista sobre GPU Tesla T4. De esta forma, la ampliación del conjunto de entrenamiento se mantuvo como principal modificación entre ambos experimentos.

En este caso, el mejor comportamiento sobre el conjunto *valid* se alcanzó en la época 79. A partir de ese momento, no se consiguió superar dicho resultado durante las 20 épocas siguientes, por lo que, mediante el mecanismo *early stopping*, el entrenamiento finalizó en la época 99.

El tiempo total de entrenamiento fue de 22 minutos y 21,5 segundos, superior al registrado en E1 debido al mayor número de imágenes procesadas y al mayor número de épocas ejecutadas antes de producirse la parada anticipada.

Las principales métricas correspondientes a la mejor época de E2 se muestran en Tabla 6.7.

Métrica <i>validation</i>	E2 extended	E1 baseline
<i>Precision</i>	0,3755	0,3728
<i>Recall</i>	0,4736	0,4514
<i>mAP50</i>	0,4147	0,3958
<i>mAP50-95</i>	0,2802	0,2363

Tabla 6.7. Comparación de métricas de *validation* obtenidas en la mejor época de E2 vs E1.

Frente a E1, los resultados de *validation* muestran una mejora en las cuatro métricas principales. El incremento resulta especialmente visible en el *Recall*, que pasa de 0,4514 a 0,4736 y en el *mAP50-95*, que aumenta de 0,2363 a 0,2802. Esto sugiere que la ampliación del conjunto de entrenamiento aporta una mejora inicial en la capacidad de generalización del modelo.

Una vez analizado el comportamiento en *validation*, se evaluó el modelo ‘*best.pt*’ de E2 sobre el mismo conjunto de 72 imágenes de *test* utilizado en E1. Esta comparación permite comprobar si la mejora observada durante el entrenamiento se mantiene también sobre imágenes que no han intervenido ni en el aprendizaje ni en la selección del mejor modelo.

Métrica <i>test</i>	E2 <i>extended</i>	E1 <i>baseline</i>
<i>Precision</i>	0,390	0,366
<i>Recall</i>	0,481	0,457
<i>mAP50</i>	0,431	0,429
<i>mAP50-95</i>	0,319	0,315

Tabla 6.8. Comparación de los resultados globales de E1 y E2 sobre el conjunto de *test*.

Los resultados muestran una mejora global moderada en E2. La *Precision* aumenta de 0,366 a 0,390 y el *Recall* de 0,457 a 0,481, lo que indica que el modelo realiza algo más de detecciones correctas y consigue localizar una mayor proporción de las anomalías presentes.

Sin embargo, el incremento de las métricas *mAP* es reducido: el *mAP50* pasa de 0,429 a 0,431 y el *mAP50-95* de 0,315 a 0,319. Por tanto, aunque la ampliación del conjunto de entrenamiento aporta una mejora general, el aumento de datos por sí solo no parece suficiente para resolver las principales limitaciones del modelo.

Para determinar si la mejora global de E2 se distribuía de forma homogénea entre las distintas categorías, se compararon los resultados obtenidos individualmente por cada clase sobre el conjunto de *test*.

Clase	<i>mAP50</i> E2	<i>mAP50</i> E1	Evolución
<i>bird droppings</i>	0,456	0,417	Mejora
<i>crack</i>	0,138	0,191	Empeora
<i>dust</i>	0,640	0,669	Ligero descenso
<i>electro</i>	0,108	0,036	Mejora, pero sigue bajo
<i>snow</i>	0,915	0,875	Mejora
<i>thermal anomaly</i>	0,326	0,385	Empeora

Tabla 6.9. Comparación del *mAP50* por clase E2 vs E1.

La ampliación del conjunto de entrenamiento no afecta de la misma forma a todas las categorías. ‘*snow*’ continúa siendo la clase con mejor comportamiento y mejora hasta alcanzar un *mAP50* de 0,915. También se observa una mejora en ‘*bird_droppings*’, que pasa de 0,417 a 0,456.

El caso de ‘*electro*’ resulta especialmente interesante. Su *mAP50* aumenta de 0,036 a 0,108, lo que indica que la incorporación de nuevas imágenes aporta información útil para esta categoría. Sin embargo, su rendimiento continúa siendo preocupantemente inferior al del resto de clases, por lo que el aumento de datos no parece haber resuelto completamente el problema.

En ‘*crack*’ se observa la evolución contraria. El *mAP50* desciende de 0,191 en E1 a 0,138 en E2, por lo que la ampliación del dataset no ha permitido mejorar su comportamiento. Esto sugiere que el problema no depende únicamente de la cantidad de datos, sino también de la dificultad visual de la clase o de la variabilidad de sus ejemplos.

En ‘*dust*’, el *mAP50* disminuye ligeramente, pasando de 0,669 a 0,640. Sin embargo: *Precision* aumenta de 0,380 a 0,430 y *Recall* de 0,636 a 0,727. Por tanto, de forma general no puede considerarse que la clase haya empeorado, sino que presenta un comportamiento algo diferente según la métrica utilizada.

Finalmente, ‘*thermal_anomaly*’ reduce su *mAP50* de 0,385 a 0,326, a pesar de que su *Recall* aumenta. Este comportamiento confirma que sigue siendo una categoría difícil para el modelo y plantea además una cuestión adicional sobre su adecuación dentro de un sistema basado principalmente en imágenes RGB.

Para completar el análisis por clases, se revisó la matriz de confusión normalizada obtenida sobre *test*.

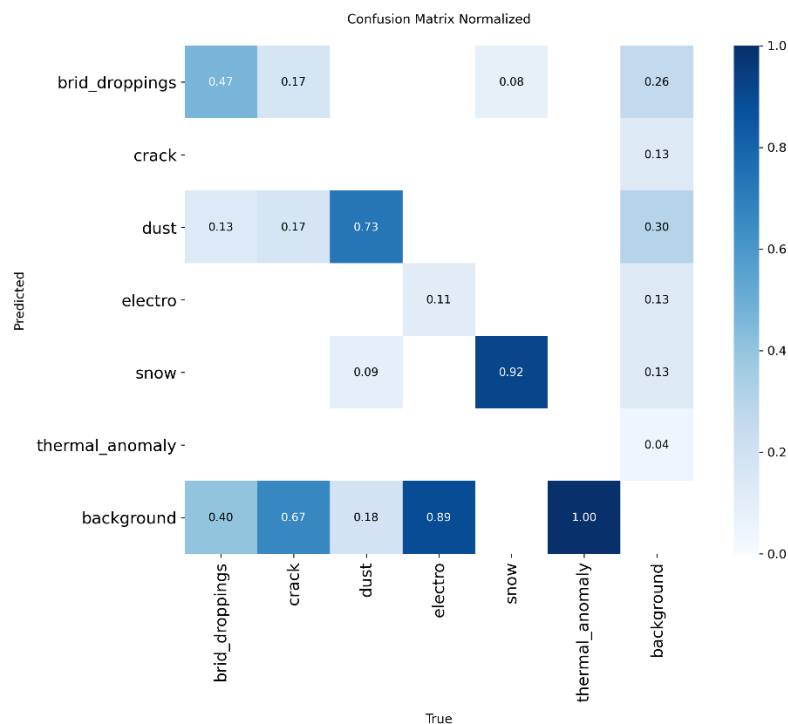


Figura 6.10. Matriz de confusión normalizada obtenida sobre el conjunto de test en E2.

La matriz de confusión de E2 confirma que la ampliación del conjunto de entrenamiento ha beneficiado especialmente a algunas de las clases que ya mostraban un comportamiento favorable. ‘*snow*’ alcanza aproximadamente un 0,92 de detecciones correctas, frente al 0,75 obtenido en E1, mientras que ‘*dust*’ aumenta de 0,55 a 0,73. También mejora ‘*bird_droppings*’, cuya diagonal pasa de 0,33 a 0,47.

Sin embargo, las dificultades observadas en determinadas categorías continúan presentes. En ‘*electro*’ aparece por primera vez una proporción de detecciones correctas de aproximadamente 0,11, aunque un 0,89 de las instancias reales sigue terminando asociado a *background*.

Por tanto, el aumento de datos ha producido cierta mejora, pero no la suficiente como para considerar ‘*electro*’ una clase correctamente aprendida.

El comportamiento de ‘*crack*’ y ‘*thermal_anomaly*’ resulta todavía más problemático. En ‘*crack*’, aproximadamente un 0,67 de las instancias termina en *background*, sin observarse detecciones correctas en la diagonal para el umbral utilizado en la matriz.

En ‘*thermal_anomaly*’, la totalidad de las instancias aparece asociada a *background*, lo que evidencia que la ampliación del dataset no ha permitido consolidar esta categoría.

También se mantienen algunas confusiones entre categorías visualmente próximas. Este resultado es coherente con lo señalado previamente durante la preparación del dataset, donde ya se había identificado la posible dificultad de diferenciar determinadas anomalías con apariencia similar.

En particular, la matriz muestra que una parte de los casos reales de ‘*bird_droppings*’ se clasifica como ‘*dust*’. También aparecen algunos ejemplos de ‘*crack*’ asignados a otras categorías.

Estas confusiones sugieren que podría ser necesario revisar la definición de algunas clases antes de continuar optimizando el modelo.

Para completar el análisis de E2 se revisó también la evolución del entrenamiento a lo largo de las épocas. En este caso interesa comprobar si la ampliación del dataset modifica la estabilidad del aprendizaje y en qué momento comienzan a estabilizarse las métricas de *validation*.

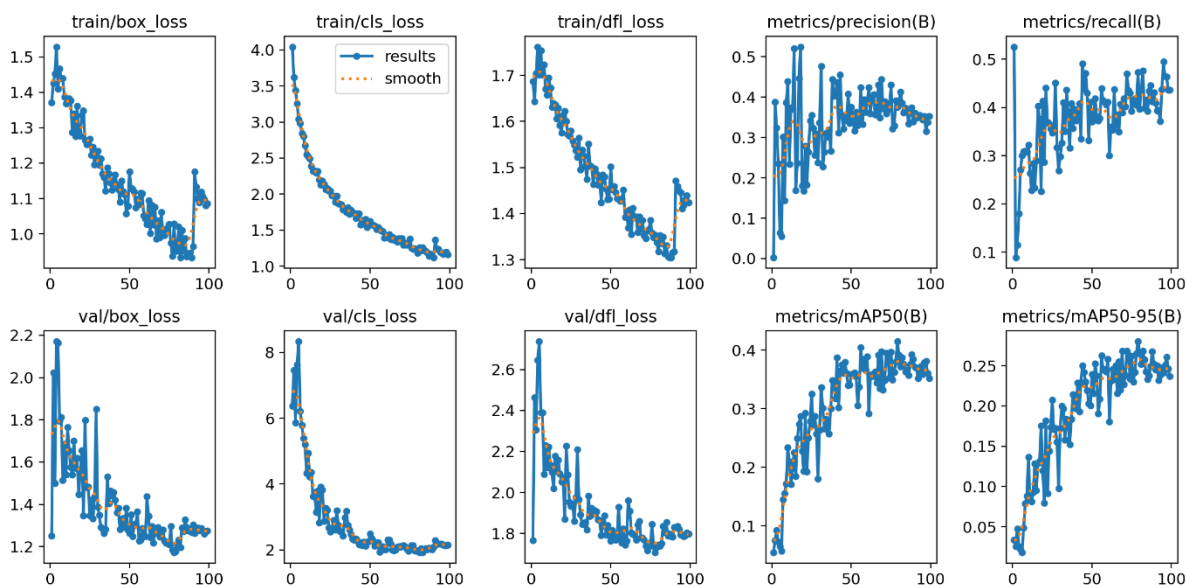


Figura 6.11. Evolución de las pérdidas y métricas durante el entrenamiento del experimento E2. Elaboración propia a partir de los resultados generados por Ultralytics.

Las *loss* de *train* y *validation* presentan, en términos generales, una evolución descendente durante gran parte del proceso. Al mismo tiempo, el *Recall*, el *mAP50* y el *mAP50-95* aumentan progresivamente, lo que indica que el modelo continúa aprovechando la información incorporada con la ampliación del dataset.

A partir de, aproximadamente, las épocas 60-70, las métricas comienzan a estabilizarse y aparecen mayores fluctuaciones. El mejor comportamiento se alcanzó en la época 79, tras la cual no se consiguió una mejora sostenida durante las 20 épocas siguientes, finalizando el entrenamiento en la época 99.

En las últimas épocas se observa además un cambio en algunas pérdidas de entrenamiento. Este comportamiento coincide con la fase final en la que Ultralytics desactiva la técnica de aumento *mosaic*, permitiendo que el modelo termine el ajuste utilizando imágenes con una composición más próxima a la original.

En conjunto, las curvas no muestran un sobreajuste severo, aunque sí una clara estabilización del aprendizaje. Esto es coherente con los resultados obtenidos anteriormente: la ampliación del dataset mejora el comportamiento general del modelo, pero el margen de mejora comienza a reducirse sin resolver por completo las dificultades asociadas a determinadas clases.

Por último, para completar el análisis de E2, se revisaron también algunas predicciones realizadas sobre el conjunto de *test*. En este caso, el objetivo no era volver a describir el funcionamiento de los *batches*, sino comprobar visualmente si la ampliación del dataset se reflejaba también en una mejora de las detecciones.



Figura 6.12. Comparación entre las anotaciones reales y las predicciones realizadas por E2 sobre un batch del conjunto de test.

El análisis visual confirma las tendencias observadas en las métricas. Algunas categorías, especialmente ‘dust’, presentan detecciones más consistentes, mientras que en clases como ‘electro’ y ‘crack’ continúan apareciendo detecciones omitidas, confusiones y niveles de confianza reducidos.

Estos resultados muestran que la ampliación del dataset ha permitido mejorar el comportamiento general del modelo, aunque determinadas limitaciones no parecen depender únicamente de disponer de un mayor número de imágenes.

En conclusión, E2 confirma que ampliar el número de imágenes de entrenamiento mejora el comportamiento global del modelo. La mejora es visible tanto en *validation* como en *test*, especialmente en *Precision*, *Recall* y en determinadas clases como ‘snow’.

Sin embargo, el incremento de datos no resuelve por sí solo todas las limitaciones observadas. Clases como ‘crack’, ‘electro’ y ‘thermal_anomaly’ continúan mostrando un comportamiento débil, mientras que persisten algunas confusiones entre categorías visualmente próximas.

Estos resultados indican que el siguiente paso no debe centrarse en aumentar el número de imágenes, sino en revisar principalmente la propia definición del problema.

A partir de las evidencias obtenidas en E1 y E2, se plantea, por tanto, un tercer experimento orientado a **optimizar la estructura de clases del dataset**.

6.6. E3: *revised_classes*

Los resultados obtenidos en E1 y E2 mostraron que el aumento del número de imágenes mejoraba moderadamente el comportamiento global del modelo, pero no resolvía las dificultades de algunas categorías. Por este motivo, E3 se centró en revisar la estructura de clases.

Para aislar el efecto de esta modificación, se mantendrán el modelo YOLO11n y la misma configuración de entrenamiento utilizada anteriormente. Tampoco se introducirán todavía técnicas adicionales de *data augmentation*.

A partir de los resultados obtenidos en E1 y E2, se revisó de forma individual el comportamiento de cada una de las clases con el objetivo de identificar cuáles estaban siendo aprendidas de forma consistente y cuáles seguían presentando problemas incluso después de ampliar el conjunto de entrenamiento.

Las decisiones adoptadas basadas en los experimentos anteriores se resumen en:

Clase	Situación E1/E2	Decisión
<i>snow</i>	Buen rendimiento y comportamiento estable	Mantener
<i>dust</i>	Buen rendimiento, pero confusión con <i>bird droppings</i>	Fusionar
<i>bird droppings</i>	Confusión visual con <i>dust</i>	Fusionar
<i>crack</i>	Rendimiento bajo	Pendiente
<i>electro</i>	Rendimiento muy bajo	Pendiente
<i>thermal_anomaly</i>	Rendimiento débil además de las dudas iniciales sobre su encaje en RGB	Pendiente

Tabla 6.10. Revisión de clases y decisiones adoptadas tras E1 y E2.

La clase ‘*snow*’, tras mostrar el comportamiento más estable en los dos experimentos anteriores, se mantiene sin ninguna modificación. En E2 alcanzó un *mAP50* de 0,915, confirmando que el modelo dispone de ejemplos suficientes para aprender sus características visuales de forma consistente.

En cambio, las clases ‘*dust*’ y ‘*bird droppings*’ requieren una revisión conjunta. Durante la propia preparación del dataset ya se observó que, en determinadas imágenes, la separación entre ambas categorías no resultaba evidente.

Los resultados de E1 y E2 han confirmado esta dificultad, mostrando confusiones entre ambas clases. Por este motivo, en E3 se propone fusionarlas en una única categoría: ‘*soiling*’, representativa de la presencia de suciedad sobre la superficie del panel independientemente de su origen concreto.

La clase ‘*crack*’ presenta un comportamiento débil en ambos experimentos y no mejora tras ampliar el conjunto de entrenamiento. Sin embargo, se decide mantenerla en E3, ya que representa un defecto físicamente relevante y visualmente identificable mediante imágenes RGB.

Además, su comportamiento podrá volver a evaluarse en E4, donde se introducirán técnicas de *data augmentation* con el objetivo de comprobar si una mayor variabilidad de ejemplos permite mejorar su aprendizaje.

La clase ‘*electro*’ también fue considerada para E3. Aunque su rendimiento mejoró tras ampliar el dataset, pasando de un *mAP50* de 0,036 en E1 a 0,108 en E2, continuó siendo una de las categorías con peor comportamiento.

En este caso, la decisión de eliminarla no se basa únicamente en sus métricas. Las alteraciones visuales incluidas bajo esta categoría no permiten determinar de forma inequívoca, a partir de una imagen RGB, que su origen corresponda realmente a un defecto eléctrico. Realizar esta clasificación requeriría un diagnóstico adicional que queda fuera del alcance del trabajo.

La inclusión de *'thermal_anomaly'* fue una de las decisiones que más dudas generó durante el desarrollo del trabajo. Inicialmente se valoró prescindir de la información térmica para mantener una arquitectura basada únicamente en imágenes RGB. Sin embargo, durante la búsqueda de datos se encontró un conjunto que incluía imágenes térmicas, por lo que se decidió mantener la clase temporalmente, dándole una oportunidad dentro de los primeros experimentos.

Los resultados obtenidos en E1 y E2 no han permitido consolidar esta opción. La clase continúa presentando un comportamiento débil y, además, su mantenimiento obligaría a incorporar una modalidad de adquisición distinta a la utilizada por el resto del sistema. Por este motivo, en E3 se retoma el planteamiento inicial y se decide trabajar exclusivamente con anomalías identificables mediante imágenes RGB.

Los principales motivos que justifican esta decisión son:

- **Rendimiento insuficiente:** el *mAP50* descendió de 0,385 en E1 a 0,326 en E2, sin observarse una mejora tras ampliar el conjunto de entrenamiento (entendible porque tampoco entraron las imágenes térmicas dentro de la campaña de adquisición).
- **Dependencia de información térmica:** una anomalía térmica se identifica adecuadamente mediante información proporcionada por una cámara térmica, mientras que el sistema desarrollado se basa principalmente en imágenes RGB.
- **Coherencia del modelo:** mantener esta categoría implicaría entrenar un mismo detector con modalidades de imagen diferentes, introduciendo una complejidad adicional que queda fuera del alcance.
- **Posibilidad de desarrollo futuro:** la eliminación de la clase no descarta el uso de termografía. Su incorporación podría abordarse posteriormente mediante una cámara térmica y un modelo específico, o mediante una arquitectura que combine información RGB y térmica.

Como resultado de esta revisión, el experimento E3 pasa de las seis categorías utilizadas inicialmente a una estructura más reducida de tres clases: *'soiling'*, *'crack'* y *'snow'*. La nueva clase *'soiling'* agrupa *'dust'* y *'bird_droppings'*, mientras que *'electro'* y *'thermal_anomaly'* dejan de formar parte del problema de detección planteado.

Esta simplificación pretende reducir ambigüedades entre categorías y concentrar el aprendizaje en anomalías que puedan identificarse de forma coherente mediante imágenes RGB. El resto de las condiciones experimentales se mantendrán iguales a E2, de forma que E3 permita evaluar específicamente el efecto de la revisión de clases.

Por tanto, se generó en Roboflow una nueva versión del conjunto de datos, denominada *'E3 revised classes'*. Sobre la versión utilizada en E2 se fusionaron las clases *'dust'* y *'bird_droppings'* en *'soiling'*, se eliminaron *'electro'* y *'thermal_anomaly'* y se depuraron las imágenes que dejaron de ser coherentes con la nueva estructura de clases.

La depuración redujo el conjunto de datos hasta 757 imágenes. La distribución resultante quedó formada por 573 imágenes de *train*, 122 de *validation* y 62 de *test*, manteniendo aproximadamente la proporción 76/16/8 utilizada en E2.

Conjunto	E1_baseline	E2_extended	E3_revised
<i>train</i>	507	672	573
<i>valid</i>	145	145	122
<i>test</i>	72	72	62
Total	724	889	757

Tabla 6.11. Evolución de la distribución de imágenes entre los experimentos E1, E2 y E3.

Se decidió conservar esta distribución en lugar de realizar un nuevo reparto 70/20/10 de las imágenes para que las muestras que permanecían en el dataset continuaran, en la medida de lo posible, dentro del mismo subconjunto en el que habían sido utilizadas anteriormente, evitando introducir una modificación adicional al experimento.

Una vez versionado el nuevo dataset ‘E3_revised_classes’ se procede a realizar el experimento en Colab.

El entrenamiento de E3 mantuvo la misma configuración utilizada en los experimentos anteriores, con un máximo de 100 épocas y ‘patience’ = 20. El proceso finalizó en la época 81, siendo la 61 la que presentó el mejor comportamiento sobre el conjunto de *validation*.

El tiempo total de entrenamiento fue de 15 minutos y 31,4 segundos. Los resultados obtenidos en la mejor época comparados con los experimentos anteriores se muestran en Tabla 6.12.

Métrica <i>validation</i>	E3_revised
<i>Precision</i>	0,5148
<i>Recall</i>	0,4298
<i>mAP50</i>	0,4272
<i>mAP50-95</i>	0,3137

Tabla 6.12. Métricas de validación obtenidas en la mejor época de E3.

Respecto a E2, destaca especialmente el aumento de *Precision*, que pasa de 0,3755 a 0,5148, así como la mejora del *mAP50-95*, de 0,2802 a 0,3137. El *Recall* disminuye ligeramente, lo que indica que el modelo realiza detecciones más fiables, aunque todavía deja algunas anomalías sin identificar.

Es en la evaluación sobre el conjunto de *test* donde el modelo muestra una mejora clara del comportamiento tras la revisión de clases. E3 alcanza una *Precision* de 0,693, un *mAP50* de 0,585 y un *mAP50-95* de 0,449, valores considerablemente superiores a los obtenidos en los experimentos anteriores.

Métrica <i>test</i>	E3_revised
<i>Precision</i>	0,693
<i>Recall</i>	0,466
<i>mAP50</i>	0,585
<i>mAP50-95</i>	0,449

Tabla 6.13. Métricas de validación obtenidas en la mejor época de E3.

Esta comparación debe interpretarse teniendo en cuenta que E3 ya no utiliza el mismo problema de detección que E1 y E2. Por tanto, la mejora no puede atribuirse únicamente a un incremento directo del rendimiento sobre el mismo conjunto de evaluación sino a una reformulación más coherente del modelo.

Para analizar con mayor detalle el comportamiento de E3, se revisaron también los resultados obtenidos de forma individual para cada una de las clases. Esta comparación permite identificar qué categorías se benefician más de la nueva estructura del dataset y cuáles continúan presentando dificultades.

Clase	<i>Precision</i>	<i>Recall</i>	<i>mAP50</i>	<i>mAP50-95</i>
<i>crack</i>	0,411	0,143	0,244	0,165
<i>snow</i>	0,882	0,833	0,924	0,701
<i>soiling</i>	0,786	0,423	0,586	0,480

Tabla 6.14. Métricas obtenidas por clase sobre el conjunto de test en E3.

La clase ‘*snow*’ vuelve a presentar el mejor comportamiento del conjunto. En E3 alcanza un *mAP50* de 0,924 y un *mAP50-95* de 0,701, manteniendo el rendimiento elevado observado en los experimentos anteriores.

La nueva clase ‘*soiling*’ obtiene también resultados positivos, con una *Precision* de 0,786 y un *mAP50* de 0,586. La fusión de ‘*dust*’ y ‘*bird droppings*’ parece haber reducido parte de la ambigüedad existente entre ambas categorías, aunque el *Recall* de 0,423 indica que todavía hay casos de suciedad que el modelo no es capaz de detectar.

‘*crack*’ continúa siendo la clase más problemática. Aunque su *mAP50* aumenta hasta 0,244, el *Recall* se mantiene en 0,143, lo que significa que una parte importante de las grietas sigue pasando inadvertida. Este comportamiento convierte a ‘*crack*’ en la clase principal sobre la que intentar mejorar en el siguiente experimento.

Para completar el análisis por clases, se revisó la matriz de confusión normalizada para el conjunto en *test*.

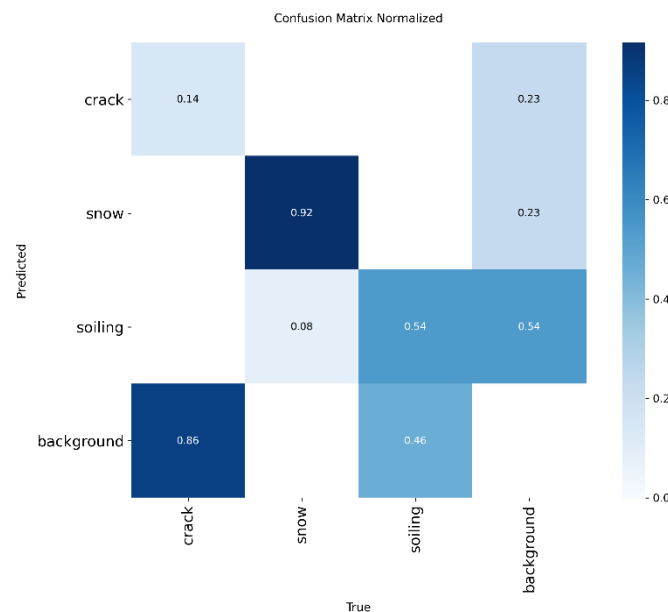


Figura 6.13. Matriz de confusión normalizada obtenida sobre el conjunto de test en E3.

‘*snow*’ mantiene el comportamiento más sólido, con aproximadamente un 0,92 de detecciones correctas. Solamente una pequeña proporción, cercana a 0,08 se confunde con ‘*soiling*’.

En ‘*soiling*’, alrededor de un 0,54 de las instancias se identifica correctamente, el 0,46 restante termina asociado a *background*. Aunque la nueva clase presenta un comportamiento más razonable, todavía existe margen de mejora para el siguiente experimento.

La principal dificultad, una vez más, queda demostrado que se centra en ‘*crack*’. Solo aproximadamente un 0,14 de las grietas se detecta correctamente, mientras que un 0,86 pasa a *background*. El problema principal de esta clase sigue siendo la elevada cantidad de grietas que el modelo no consigue detectar.

También aparecen falsos positivos sobre imágenes sin anomalías, principalmente asociados a *soiling*, lo que muestra que todavía es necesario mejorar la capacidad de generalización del modelo.

Después, se analizó la evolución del entrenamiento de E3 para comprobar cómo respondió el modelo a la nueva estructura de clases y en qué momento comenzaron a estabilizarse las métricas.

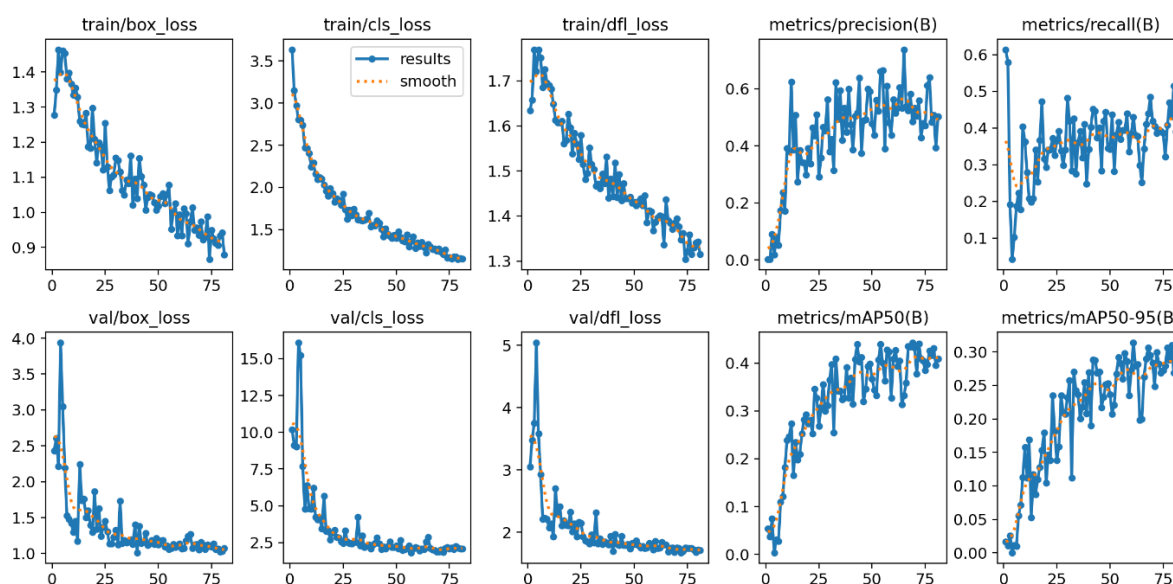


Figura 6.14. Evolución de las pérdidas y métricas durante el entrenamiento de E3. Elaboración propia a partir de los resultados generados por Ultralytics.

Las curvas de E3 muestran una evolución estable del entrenamiento. Las pérdidas de *train* y *validation* disminuyen progresivamente, mientras que el *mAP50* y el *mAP50-95* aumentan durante las primeras fases, estabilizándose a medida que avanza el proceso.

A partir de aproximadamente las épocas 50-60, las mejoras son cada vez menores y las métricas oscilan alrededor de valores similares. Este comportamiento es coherente con la selección de la época 61 como mejor estado del modelo y con la posterior actuación del mecanismo de *early stopping*.

En conclusión, E3 confirma que la revisión de clases ha permitido obtener un modelo más coherente con el problema planteado. La evaluación sobre *test* alcanza una *Precision* de 0,693 y un *mAP50* de 0,585 y un *mAP50-95* de 0,449, mejorando claramente el comportamiento global observado en los experimentos anteriores.

6.7. E4: *augmented_dataset*

Tras la mejora obtenida en E3, el último experimento se orientó a comprobar si era posible aumentar todavía más la capacidad de generalización del modelo mediante técnicas de *data augmentation*.

El objetivo no es generar nuevas anomalías, sino presentar al modelo distintas variaciones de las imágenes disponibles para reducir su dependencia de condiciones concretas de iluminación, orientación o encuadre y comprobar si esto permite mejorar especialmente la detección de las clases que todavía presentan dificultades.

Para ello se mantuvo la misma estructura de clases definida en E3 (*'crack'*, *'soiling'*, *'snow'*) y se conservaron sin cambios los conjuntos de *validation* y *test*. La modificación introducida en E4 se aplicó únicamente al conjunto *train*, que fue el ampliado con las técnicas de *data augmentation*.

El *data augmentation*, por tanto, consiste en generar variaciones artificiales de las imágenes originales mediante transformaciones controladas, como cambios de orientación, iluminación o escala. En E4 se seleccionarán transformaciones que puedan reproducir variaciones razonables en imágenes captadas desde un UAS, evitando modificaciones excesivas que puedan alterar la apariencia real de las anomalías.

A partir de estos criterios se seleccionó un conjunto de transformaciones moderadas, orientadas a reproducir variaciones razonables durante la adquisición de imágenes mediante UAS.

<i>Flip</i> :	Horizontal y vertical
<i>Crop</i> :	0 – 15%
<i>Rotation</i> :	$\pm 15^\circ$
<i>Saturation</i> :	$\pm 15\%$
<i>Brightness</i> :	$\pm 15\%$
<i>Noise</i> :	Hasta 0,93% de los píxeles
<i>Motion Blur</i> :	30 px, 1 frame

Tabla 6.15. Técnicas de *data augmentation* para E4.

Las transformaciones de orientación y recorte permiten presentar al modelo una misma anomalía desde posiciones y encuadre diferentes, reproduciendo parte de la variabilidad que puede aparecer durante la captura con el UAS.

Los cambios de brillo y saturación se introdujeron para simular variaciones moderadas en las condiciones de iluminación. Del mismo modo, se añadió un pequeño nivel de ruido para aumentar la tolerancia frente a alteraciones propias de la adquisición de imagen.

También se incorporó un *motion blur* moderado, con una longitud de 30 píxeles, considerando que parte de las imágenes pueden proceder de vídeo capturado durante el movimiento del dron. Su intensidad se mantuvo limitada para evitar una degradación excesiva de detalles finos especialmente relevantes para la detección de roturas (*'crack'*).

En Roboflow se configuró un multiplicador de *augmentation* de 3x. De esta forma el conjunto de entrenamiento pasó de 573 imágenes a 1719, mientras que *validation* y *test* se mantuvieron sin cambios, con las 122 y 62 imágenes respectivamente de los anteriores experimentos.

La versión final *E4_data_augmentation* quedó así formada por 1903 imágenes.

El entrenamiento se realizó manteniendo la misma configuración utilizada en E3: YOLO11n preentrenado, un máximo de 100 épocas, *patience* = 20, tamaño de imágenes 640x640 y *batch* = 16. De esta forma la principal diferencia entre ambos experimentos se mantiene centrada en la ampliación del conjunto de *train* mediante *data augmentation*.

En este caso se completaron las 100 épocas, alcanzándose el mejor comportamiento en la *epoch* 83. Al encontrarse este punto relativamente próximo al final del entrenamiento, no llegaron a transcurrir las 20 épocas necesarias para que actuase el mecanismo de *early stopping*.

Métrica <i>validation</i>	E4 <i>augmented</i>
<i>Precision</i>	0,5857
<i>Recall</i>	0,6011
<i>mAP50</i>	0,5374
<i>mAP50-95</i>	0,3601

Tabla 6.16. Métricas de validación obtenidas en la mejor época de E4.

La comparación muestra que E4 alcanza los mejores resultados de *validation* del conjunto de experimentos, destacando especialmente el incremento de *Recall* y *mAP50* respecto al anterior experimento.

Después de seleccionar el modelo correspondiente a la mejor época, se evaluó ‘*best.pt*’ sobre el conjunto de *test*. Los resultados obtenidos en comparación con los experimentos anteriores son:

Métrica <i>test</i>	E4 <i>augmented</i>
<i>Precision</i>	0,615
<i>Recall</i>	0,631
<i>mAP50</i>	0,558
<i>mAP50-95</i>	0,417

Tabla 6.17. Métricas de validación obtenidas en la mejor época de E4.

En el conjunto de *test*, E4 alcanza un *Recall* de 0,631, el valor más alto obtenido entre los experimentos realizados. Sin embargo, la *Precision* y las métricas *mAP* quedan ligeramente por debajo de los valores alcanzados por E3.

Por tanto, el efecto del *data augmentation* no se traduce en una mejora uniforme de todas las métricas. El modelo aumenta su capacidad para localizar anomalías, aunque esta mayor sensibilidad viene acompañada de un incremento de detecciones incorrectas.

Clase	<i>Precision</i>	<i>Recall</i>	<i>mAP50</i>	<i>mAP50-95</i>
<i>crack</i>	0,420	0,429	0,192	0,141
<i>snow</i>	0,794	0,917	0,912	0,627
<i>soiling</i>	0,632	0,549	0,571	0,485

Tabla 6.18. Métricas obtenidas por clase sobre el conjunto de *test* en E4.

El análisis por clases mantiene diferencias importantes entre las tres categorías: ‘*snow*’ continúa siendo la clase con mejor comportamiento, alcanzando un *Recall* de 0,917 y un *mAP50* de 0,912. ‘*soiling*’ presenta también un comportamiento más estable, con un *Recall* de 0,549 y un *mAP* de 0,571.

‘*crack*’ sigue siendo la categoría más difícil para el modelo. Aunque su *Recall* aumenta con respecto a E3, alcanzando un 0,429 (frente a 0,143), los valores de *mAP* permanecen claramente por debajo de los obtenidos para las otras clases.

Para analizar la evolución del entrenamiento se revisaron las curvas generadas durante las 100 épocas del experimento 4:

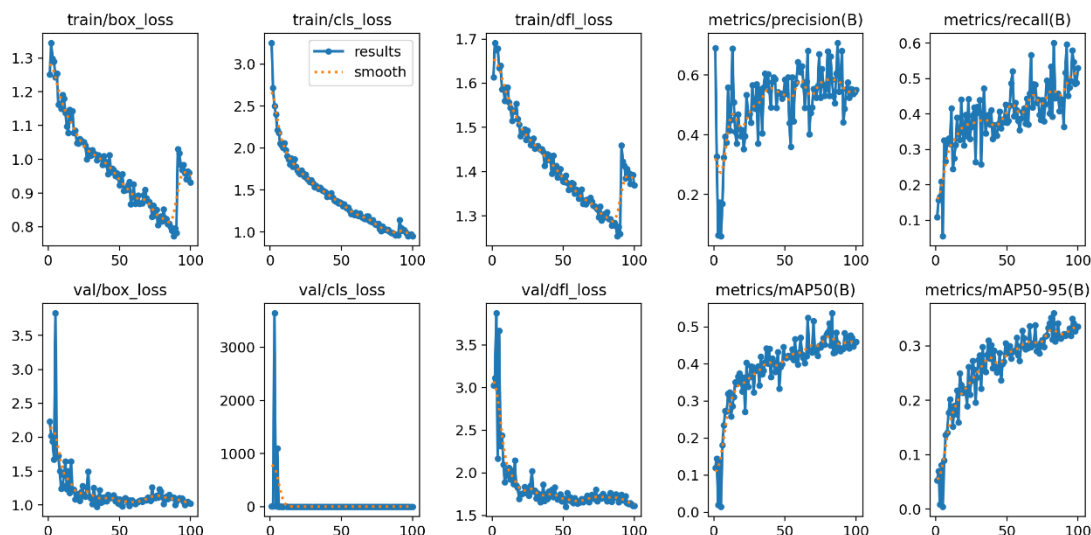


Figura 6.15. Evolución de las pérdidas y métricas durante el entrenamiento de E4.

Las curvas muestran una evolución favorable durante el entrenamiento. Las *loss* en *train* y *validation* disminuyen progresivamente, mientras que las métricas de *Precision*, *Recall*, y *mAP* presentan una tendencia general ascendente, aunque con las oscilaciones habituales entre épocas.

En conjunto, la evolución de las curvas indica que el entrenamiento alcanzó una zona relativamente estable antes de finalizar las 100 épocas.

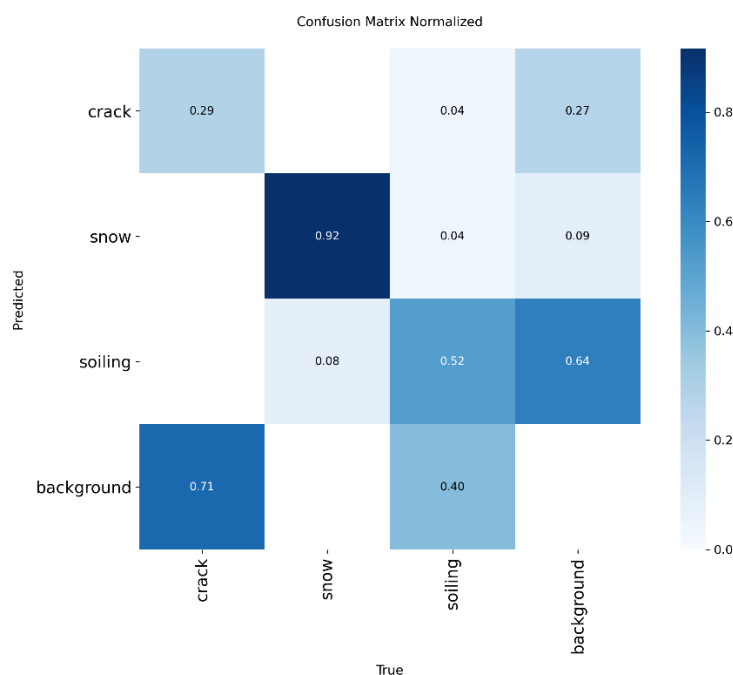


Figura 6.16. Matriz de confusión normalizada obtenida sobre el conjunto de test en E4.

Para completar el análisis del comportamiento del modelo se revisó también la matriz de confusión normalizada obtenida sobre el conjunto *test*.

Esta matriz confirma las diferencias observadas entre las tres clases. ‘*snow*’ presenta el comportamiento más sólido, con un 0,92 de detecciones correctas. ‘*soiling*’ alcanza un valor de 0,52, mientras que ‘*crack*’ continúa mostrando mayores dificultades, con un 0,29 de detecciones correctas en la matriz.

En el caso de ‘*crack*’, una parte importante de las instancias, 0,71, no llega a ser detectada y se asigna a *background*. En ‘*soiling*’ este valor se reduce a 0,40, mientras que ‘*snow*’ apenas presenta pérdidas hacia el fondo. También se observan falsos positivos sobre *background*, principalmente asociados a ‘*soiling*’ y ‘*crack*’.

En general, los errores se producen principalmente por anomalías que no llegan a detectarse o por detecciones sobre el fondo, mientras que la confusión directa entre las tres clases es relativamente reducida.



Figura 6.17. Comparación entre anotaciones reales y predicciones del modelo E4 en un lote de imágenes de *test*.

La comparación visual entre anotaciones reales y predicciones en un lote del conjunto de *test* confirma que el modelo detecta correctamente buena parte de los casos de ‘*soiling*’ y algunos ejemplos de ‘*crack*’, aunque esta última clase sigue presentando mayor dificultad. En general, el comportamiento observado en las imágenes es coherente con las métricas obtenidas para E4.

En conjunto, E4 demuestra que la incorporación de *data augmentation* modifica de forma relevante el comportamiento del detector. El experimento alcanza los mejores resultados de *validation* de toda la serie y, sobre *test*, obtiene el mayor *Recall* global, con la ligera reducción de *Precision* y *mAP* con respecto a E3.

Los resultados muestran, por tanto, una mejora clara en la capacidad del modelo para recuperar anomalías, pero también confirman que ‘*crack*’ continúa siendo la clase más difícil de detectar de forma consistente. Este comportamiento será tenido en cuenta en el siguiente apartado para comparar los experimentos realizados y seleccionar el modelo más adecuado para su integración en el sistema *Edge-AI*.

6.8. Modelo final

Una vez completados los cuatro experimentos, se realizó una comparación conjunta con el objetivo de seleccionar el modelo más adecuado para su integración en el sistema *Edge-AI*. En primer lugar, se analizaron los resultados obtenidos sobre *validation*, utilizado durante el desarrollo para evaluar la capacidad de generalización de cada configuración.

Métrica <i>validation</i>	<i>E4 augmented</i>	<i>E3 revised</i>	<i>E2 extended</i>	<i>E1 baseline</i>
<i>Precision</i>	0,5857	0,5148	0,3755	0,3728
<i>Recall</i>	0,6011	0,4298	0,4736	0,4514
<i>mAP50</i>	0,5374	0,4272	0,4147	0,3958
<i>mAP50-95</i>	0,3601	0,3137	0,2802	0,2363

Tabla 6. 19. Comparación de las métricas de *validation* obtenidas en los experimentos E1-E4.

Los resultados muestran una evolución especialmente favorable en E4, que alcanza los valores más altos de *Precision*, *Recall*, *mAP50* y *mAP50-95*. La incorporación de *data augmentation* permitió, por tanto, mejorar el comportamiento del modelo sobre imágenes no utilizadas directamente durante el entrenamiento.

Métrica <i>test</i>	<i>E4 augmented</i>	<i>E3 revised</i>	<i>E2 extended</i>	<i>E1 baseline</i>
<i>Precision</i>	0,615	0,693	0,390	0,366
<i>Recall</i>	0,631	0,466	0,481	0,457
<i>mAP50</i>	0,558	0,585	0,431	0,429
<i>mAP50-95</i>	0,417	0,449	0,319	0,315

Tabla 6. 20. Comparación de las métricas obtenidas sobre el conjunto de *test* en los experimentos E1-E4.

Sin embargo, la evaluación final sobre el conjunto de *test* introduce un matiz importante. En este caso no existe un modelo claramente superior en todas las métricas: E3 obtiene mejores valores de *Precision* y *mAP*, mientras que E4 alcanza un *Recall* considerablemente mayor.

Los resultados de *test* muestran que la decisión final no puede realizarse atendiendo únicamente al experimento con el valor más alto de *mAP*. E3 y E4 presentan comportamientos diferentes y, por tanto, la elección depende también de qué tipo de error se considera más crítico para la aplicación del modelo.

- E3 se comporta como un modelo más conservador. Presenta una *Precision* mayor y los mejores valores de *mAP*, por lo que cuando genera una detección existe una mayor probabilidad de que sea correcta.
- E4 se comporta como un modelo más sensible. Su *Recall* aumenta hasta 0,631, frente al 0,466 de E3, por lo que consigue recuperar una mayor proporción de las anomalías presentes en las imágenes.

En el contexto de este trabajo se considera más adecuado el comportamiento de E4. Durante la inspección de paneles fotovoltaicos, un falso negativo implica que existe una anomalía real pero el sistema no genera ninguna detección. Ese panel podría, por tanto, pasar inadvertido durante el recorrido del UAS.

Un falso positivo tiene una consecuencia diferente: el sistema genera una alerta sobre una zona que posteriormente puede comprobarse mediante la imagen capturada. Esto supone una revisión adicional, pero no implica perder una anomalía real.

Por este motivo, para una herramienta concebida como apoyo a la inspección, se ha considerado preferible asumir un cierto aumento de falsos positivos antes que incrementar el número de defectos que pasan desapercibidos.

Como comprobación adicional, se calculó el valor F1 de los dos modelos finalistas. Esta métrica combina *Precision* y *Recall* en un único indicador, por lo que resulta útil cuando se quiere valorar el equilibrio entre la capacidad de detectar anomalías y la fiabilidad de las detecciones realizadas.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$
$$F1_{E3} = 2 \cdot \frac{0,693 \cdot 0,466}{0,693 + 0,466} \approx 0,557$$
$$F1_{E4} = 2 \cdot \frac{0,615 \cdot 0,631}{0,615 + 0,631} \approx 0,623$$

El resultado vuelve a favorecer a E4, que presenta un mejor equilibrio entre *Precision* y *Recall*. Aunque E3 es más preciso y alcanza mayores valores de *mAP*, E4 combina una *Precision* todavía elevada con una capacidad considerablemente superior para recuperar defectos reales.

Este resultado refuerza la decisión tomada, ya que confirma que la elección de E4 no se apoya únicamente en “un mayor *Recall*”, sino en un mejor equilibrio global entre detección y precisión.

A partir de estos resultados, **E4 se selecciona como modelo final del sistema**. Su comportamiento responde mejor al objetivo planteado para la inspección fotovoltaica, donde se considera prioritario reducir el número de defectos que pasan inadvertidos aun a costa de asumir falsas detecciones. El modelo mantiene la arquitectura ligera YOLO11n, por lo que resulta totalmente válido para abordar su posterior ejecución sobre la plataforma *Edge-AI*.

7. INTEGRACIÓN DEL SISTEMA EDGE – AI

Una vez seleccionado E4 como modelo final, el siguiente paso consiste en integrarlo dentro de la arquitectura UAS planteada anteriormente. Hasta ese punto, el diseño del prototipo y el desarrollo del sistema de visión artificial se han abordado como dos bloques diferenciados, aunque ambos forman parte de un mismo objetivo: automatizar la inspección de instalaciones fotovoltaicas.

La integración *Edge-AI* permite acercar el procesamiento de las imágenes al propio sistema de adquisición. En lugar de depender necesariamente de una infraestructura externa para analizar cada captura, el objetivo es disponer de una plataforma capaz de recibir las imágenes obtenidas durante el vuelo y ejecutar localmente el modelo de detección.

Para ello se plantea el uso de una Raspberry Pi 5 junto con un acelerador Hailo-8, sobre los que deberá adaptarse el modelo YOLO11n desarrollado en el capítulo anterior. Esto implica no solo preparar el modelo para su ejecución sobre hardware sino también definir el flujo de inferencia completo: desde la captura de una imagen hasta la generación de una detección.

El objetivo de este capítulo es, por tanto, cerrar la integración entre el UAS prototipado y el sistema de inteligencia artificial modelado, definiendo una arquitectura de ejecución, el flujo de procesamiento y la comunicación entre sus distintos componentes. Finalmente, se realizará una prueba funcional con el propósito de comprobar que el modelo puede integrarse en un flujo de vídeo generado por una cámara embarcada y producir detecciones durante una misión de inspección.

7.1. Adaptación del modelo a la plataforma *Edge-AI*

Como se definió en el capítulo dedicado a la arquitectura UAS, el procesamiento embarcado se apoya en una Raspberry Pi 5 junto con un acelerador Hailo-8. La primera actúa como unidad principal de gestión y coordinación del sistema, mientras que el segundo se encarga de acelerar las operaciones de inferencia asociadas al modelo de visión artificial.

Antes de abordar la adaptación del modelo, conviene situar de forma general el flujo que seguirá la información durante la inferencia: cada imagen o *frame* capturado por la cámara durante el vuelo, será recibido por la unidad de procesamiento, preparado para su análisis y enviado al acelerador, donde se ejecutará el E4. El resultado de la inferencia estará formado por las detecciones realizadas, incluyendo la clase identificada, su nivel de confianza y la localización mediante *bounding boxes*.

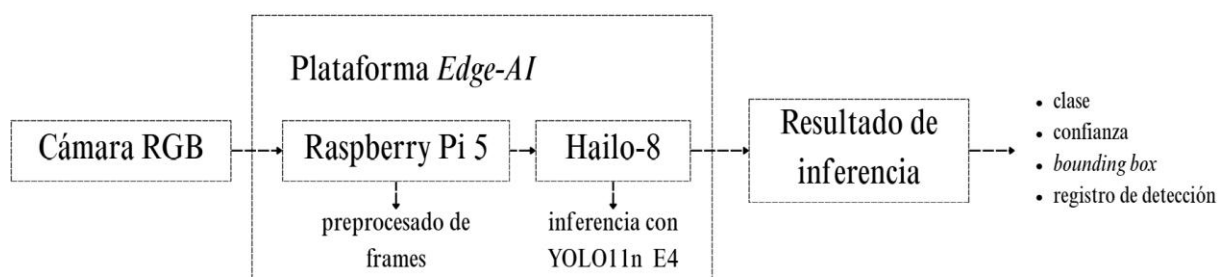


Figura 7.1. Flujo general de inferencia sobre la plataforma Edge-AI propuesta. Fuente: Elaboración propia.

A partir de este flujo general, la integración requiere adaptar el modelo obtenido durante el entrenamiento al entorno de ejecución del acelerador. Como se explicó en el capítulo anterior,

los experimentos se realizaron en Google Colab y los resultados se almacenaron de forma persistente en Google Drive, ya que las sesiones de Colab son temporales y los archivos almacenados únicamente en el entorno de ejecución pueden perderse al finalizar la sesión.

Durante el entrenamiento, Ultralytics generó automáticamente los pesos correspondientes a la mejor época del modelo, almacenados en un archivo denominado *'best.pt'*. Este archivo, conservado en Google Drive, contiene los pesos del modelo E4 seleccionado finalmente y constituye el punto de partida para su posterior despliegue sobre la plataforma *Edge-AI*.

Sin embargo, *'best.pt'* corresponde al formato utilizado en el entorno Ultralytics/PyTorch y no puede ejecutarse directamente sobre el Hailo-8. Es necesario, por tanto, realizar un proceso de conversión y optimización hasta obtener un formato compatible con el acelerador.

Para explicar el proceso de adaptación del modelo de una forma ordenada, se seguirá el mismo razonamiento que surge al pasar del entorno de entrenamiento al despliegue real. Partiendo del archivo generado en Colab, se analizará por qué no puede ejecutarse directamente sobre Hailo-8, qué formato intermedio es necesario, qué papel juega ONNX, por qué debe realizarse una cuantización y qué modelo se obtiene finalmente para la plataforma *Edge-AI*.

'best.pt' → ¿por qué no sirve? → ¿qué se necesita? → ¿qué es ONNX? → ¿por qué se cuantiza? → ¿qué se obtiene? → ¿sigue funcionando igual?

¿Qué es realmente *'best.pt'*?

Es el punto de partida, el archivo generado durante el entrenamiento. Durante cada *epoch*, Ultralytics evalúa la evolución del modelo y conserva diferentes estados del entrenamiento. Este archivo *'best.pt'* corresponde al estado que presentó el mejor comportamiento de acuerdo con los criterios de validación utilizados.

Este archivo contiene los parámetros aprendidos por la red neuronal y la información necesaria para volver a cargar el modelo dentro del entorno Ultralytics/PyTorch. Es, por tanto, el resultado que permite conservar todo lo aprendido durante el entrenamiento y utilizar posteriormente el detector sin necesidad de volver a entrenarlo desde cero.

dataset → entrenamiento → E4 → *'best.pt'*

¿Por qué *'best.pt'* no puede utilizarse directamente en Hailo-8?

El hecho de disponer de un modelo entrenado no significa que este pueda ejecutarse directamente sobre cualquier dispositivo. *'best.pt'* pertenece al ecosistema de PyTorch utilizado durante el entrenamiento, mientras que Hailo-8 es un acelerador especializado que dispone de su propia arquitectura y de un entorno específico de ejecución.

Por este motivo, el acelerador no interpreta directamente el archivo *'pt'*. Antes de poder utilizar E4 sobre Hailo-8 es necesario transformar la red neuronal en una representación que pueda ser analizada, optimizada y finalmente compilada para dicho hardware.

En el flujo soportado actualmente por Ultralytics, este proceso parte del modelo YOLO en formato *'pt'*, genera una representación intermedia ONNX, optimiza y cuantiza la red a INT8 y, finalmente, produce un archivo HEF ejecutable por Hailo.

Esta diferencia entre el entorno utilizado durante el desarrollo y el previsto para el despliegue puede resumirse de forma sencilla comparando ambos escenarios:

Durante el desarrollo	Durante el despliegue
Google Colab	Plataforma <i>Edge-AI</i>
PyTorch/Ultralytics	HailoRT
<i>best.pt</i>	<i>.hef</i>
GPU NVIDIA T4	Hailo-8
Entrenamiento y evaluación	Inferencia

Tabla 7.1. Comparación entre el entorno de desarrollo y el entorno de despliegue del modelo.

Por tanto, no se vuelve a entrenar el modelo durante esta etapa. Lo que cambia es la forma en la que se representa y ejecuta la red para adaptarla al hardware de destino.

¿Qué se necesita?

Se necesita una representación intermedia, que permita sacar el modelo del entorno específico de PyTorch y describir su estructura de una forma que otras herramientas lo puedan interpretar. Para ello se utiliza ONNX (*Open Neural Network Exchange*).

¿Qué es ONNX y por qué se necesita?

ONNX (*Open Neural Network Exchange*) es un formato abierto diseñado para representar modelos de aprendizaje automático de forma independiente del *framework* con el que fueron desarrollados. En lugar de almacenar únicamente sus pesos aprendidos, describe la estructura computacional de la red: sus capas, operaciones, conexiones, entradas y salidas.

En el presente proyecto, ONNX funciona como un formato intermedio o “traductor” entre el modelo E4 entrenado con PyTorch y las herramientas utilizadas posteriormente para preparar su ejecución sobre Hailo-8.

De esta forma, el modelo deja de estar ligado exclusivamente al entorno utilizado durante el entrenamiento y puede ser interpretado por el proceso de compilación del acelerador.

‘best.pt’ → exportación → *‘best.onnx’*

La exportación puede realizarse desde el mismo entorno de Colab utilizado durante el entrenamiento. Una vez montado Google Drive, se carga el archivo *‘best.pt’* correspondiente al modelo E4 y se utiliza la función de exportación proporcionada por Ultralytics para generar su representación en formato ONNX:

```
from ultralytics import YOLO

model = YOLO(
    "/content/drive/MyDrive/TFM/Experimentos/"
    "E4_augmentation_100ep_pat20_yolo11n/weights/best.pt"
)

model.export(
    format="onnx",
    imgsz=640
)
```


Al ejecutarse este código, Ultralytics carga la arquitectura YOLO11n junto con los pesos aprendidos durante E4 y genera el correspondiente ‘.onnx’. Se mantiene una resolución de entrada de 640px, coincidente con la utilizada durante el entrenamiento de los experimentos.

```
best.pt → model.export(format="onnx") → best.onnx
```

Esta conversión no implica un nuevo entrenamiento ni modifica las clases aprendidas por el detector. El objetivo es representar la misma red neuronal en un formato diferente, adecuado para continuar con el proceso de adaptación al hardware.

¿Qué ocurre después de obtener ‘best.onnx’?

El archivo ‘best.onnx’ ya contiene una representación independiente de PyTorch, pero todavía no está preparado específicamente para ejecutarse sobre el Hailo-8. El siguiente paso consiste en introducir este modelo en las herramientas de Hailo para que puedan interpretar su arquitectura y adaptarla al acelerador seleccionado.

Para ello, interviene el Hailo Dataflow Compiler (DFC). Su primera tarea es analizar el modelo ONNX y comprobar cómo está construida la red: qué capas tiene, cómo se conectan entre ellas y cuáles son sus entradas y salidas. Esta etapa se conoce como *parsing* y puede entenderse como la forma en que Hailo “lee” e interpreta el modelo antes de poder optimizarlo.

Como resultado, se obtiene una representación intermedia dentro del ecosistema Hailo denominada HAR (Hailo *Archive*). Este archivo sigue sin ser el modelo que se ejecutará a bordo, sino una versión preparada para continuar con las siguientes etapas (optimización, cuantización y compilación).

‘best.onnx’ → *parsing* con Hailo DFC → modelo HAR

¿Optimizar y cuantizar el modelo?

Una vez obtenido el modelo HAR, la red ya puede ser interpretada por las herramientas de Hailo, pero todavía conserva una representación numérica que es más adecuada para el entorno de desarrollo que para un acelerador *Edge*. Para ejecutar el modelo de forma eficiente sobre Hailo-8 es necesario reducir el coste de cálculo y memoria asociado a sus operaciones.

Para ello se realiza un proceso de optimización y cuantización. La optimización adapta la red a las características del acelerador, mientras que la cuantización reduce la precisión numérica con la que se representan determinados valores del modelo.

Antes de la cuantización	Después de la cuantización
Mayor precisión numérica	Representación INT8
Mayor consumo de memoria	Menor consumo de memoria
Mayor coste computacional	Operaciones más eficientes
Modelo orientado al entorno de desarrollo	Modelo preparado para inferencia <i>Edge</i>

Tabla 7.2. Comparación de las características del modelo antes y después de la cuantización a INT8.

¿Qué significa cuantizar a INT8?

Durante el entrenamiento, los parámetros de una red neuronal suelen representarse mediante números con una precisión relativamente elevada.

La cuantización permite sustituir estas representaciones por valores enteros de 8 bits, un formato conocido como INT8. En lugar de trabajar con representaciones numéricas de mayor precisión, se utiliza un conjunto mucho más reducido de valores posibles, lo que disminuye la

memoria necesaria y simplifica las operaciones que debe realizar el acelerador durante cada inferencia.

modelo original → cuantización → modelo INT8

En términos simples, INT8 no cambia lo que el modelo ha aprendido, sino la forma numérica en la que se almacenan y procesan sus parámetros. El objetivo no es modificar el aprendizaje de E4, sino representar los parámetros de una forma más eficiente.

Sin embargo, al reducir la precisión numérica, podrían aparecer pequeñas diferencias respecto al comportamiento del modelo original, lo que lleva a la siguiente pregunta:

¿Cómo se limita la pérdida de precisión en el modelo?

Para realizar correctamente la cuantización se utiliza un conjunto de imágenes de calibración. Estas imágenes permiten observar los rangos de valores que aparecen en distintas capas de la red cuando el modelo procesa ejemplos representativos del problema.

En este caso, las imágenes de calibración deben proceder del mismo dominio que el modelo encontrará durante la inspección fotovoltaica. De este modo, la cuantización puede ajustarse utilizando imágenes representativas de paneles solares y de las condiciones visuales presentes en el dataset desarrollado.

Para esta calibración se utilizarían imágenes representativas procedentes del conjunto de entrenamiento, evitando emplear el conjunto de *test*, que se reserva para la evaluación final del comportamiento del modelo.

¿Qué ocurre después de la cuantización y la calibración?

Una vez utilizadas las imágenes de calibración, las herramientas de Hailo pueden determinar cómo representar de forma eficiente los valores que circulan por la red utilizando INT8. Como resultado se obtiene una versión optimizada y cuantizada del modelo preparada para la arquitectura del acelerador.

¿Por qué todavía es necesario compilar el modelo?

Aunque la red ya se encuentra optimizada y cuantizada, todavía es necesario indicar cómo se ejecutarán físicamente sus operaciones sobre los recursos disponibles de Hailo-8. Esta última adaptación se realiza mediante el proceso de compilación.

Durante la compilación, las herramientas de Hailo distribuyen las operaciones de la red sobre los recursos internos del acelerador y generan una representación específicamente preparada para dicho hardware. Por tanto, ya no se trata únicamente de describir el modelo sino de crear una versión directamente ejecutable sobre Hailo-8.

¿Qué es el archivo HEF?

El resultado final de la compilación es un archivo HEF (*Hailo Executable Format*). Este archivo contiene la versión del modelo preparada específicamente para su ejecución sobre el acelerador Hailo-8 y será el que posteriormente se cargue en la plataforma *Edge-AI* del UAS.

‘best.pt’ → ‘best.onnx’ → HAR → optimización → cuantización INT8 → calibración →
compilación → ‘E4.hef’

En otras palabras: ‘best.pt’ representa el modelo obtenido al finalizar el entrenamiento, mientras que ‘E4.hef’ representa el mismo detector, pero adaptado para ser ejecutado de forma eficiente sobre el sistema de *Edge-AI* del dron.

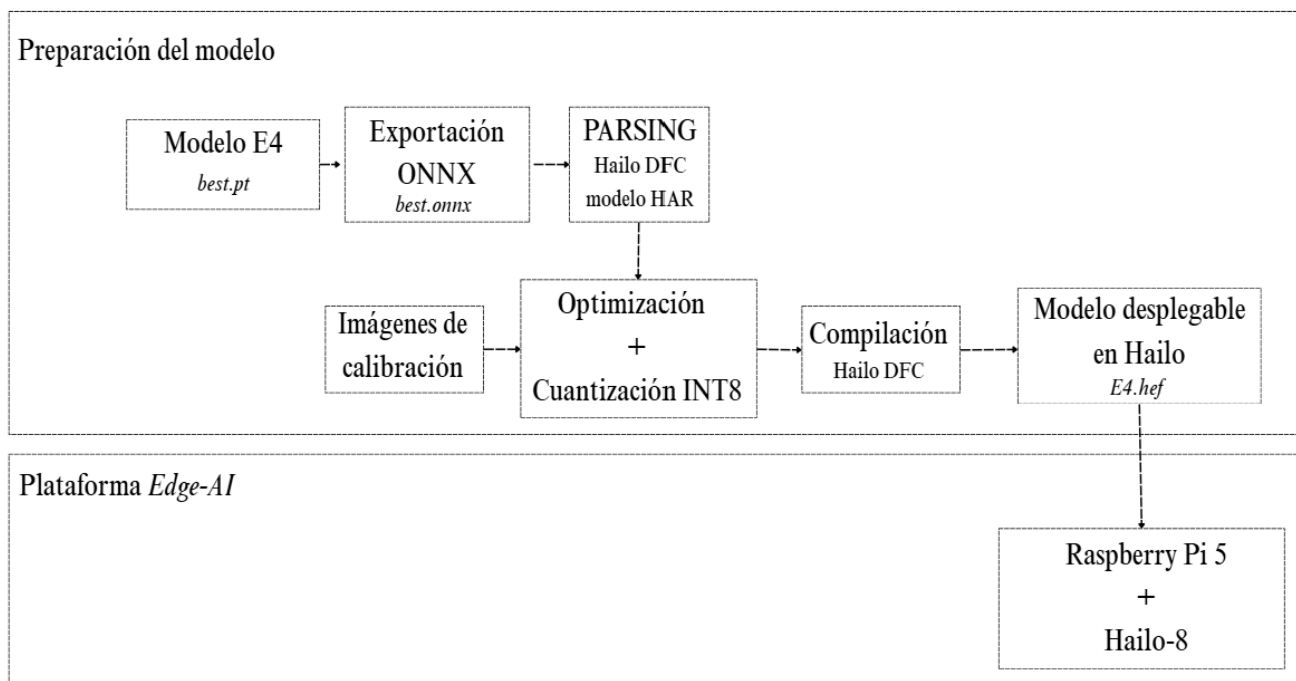


Figura 7.2. Proceso de adaptación del modelo E4 para su despliegue sobre la plataforma Edge-AI. Fuente: Elaboración propia.

La figura resume el proceso completo de adaptación, desde el modelo seleccionado en formato ‘*best.pt*’ hasta la generación del archivo ‘*E4.hef*’, preparado para su ejecución sobre la plataforma formada por la Raspberry Pi 5 y el Hailo-8.

La última pregunta para entender el proceso completo:

¿Cómo se realiza este proceso en la práctica?

Aunque las etapas anteriores pueden entenderse de forma separada, en la práctica gran parte del flujo puede automatizarse mediante Ultralytics y las herramientas de Hailo (el ecosistema). A partir del modelo ‘*best.pt*’, el proceso de exportación puede generar la representación ONNX, realizar la optimización y cuantización y compilar finalmente el modelo en formato HEF.

La preparación del modelo se realiza en una máquina de desarrollo con Linux x86_64, donde se instala el Hailo Dataflow Compiler. A partir del modelo YOLO entrenado, Ultralytics puede coordinar automáticamente las etapas descritas anteriormente (exportación a *.onnx*, interpretación del modelo, calibración, cuantización INT8, compilación para Hailo-8).

Para un modelo personalizado como E4, además, se proporciona el archivo ‘*data.yaml*’ del dataset (también guardado en Google Drive durante el proceso de entrenamiento) de manera que la calibración utilice imágenes representativas con una resolución de entrada de 640x640 píxeles, como la utilizada durante el entrenamiento.

Una vez finalizada la compilación se obtiene el archivo ‘*E4.hef*’, acompañado de los metadatos necesarios para interpretar las salidas del detector. Este directorio es el que se transfiere a la Raspberry Pi 5, donde Hailo se encarga de ejecutar la inferencia.

El proceso de exportación podría lanzarse mediante:

```
from ultralytics import YOLO  
  
model = YOLO ("best.pt")  
  
output = model.export(  
    format="hailo",  
    name="hailo8",  
    imgsz=640,  
    data="data.yaml"  
)  
  
print(output)
```

Este comando permite coordinar las etapas descritas previamente. Ultralytics gestiona la exportación del modelo y utiliza las herramientas del ecosistema Hailo para realizar su adaptación, optimización, cuantización y compilación hasta obtener el archivo HEF preparado para el Hailo-8.

Conviene diferenciar las dos herramientas de Hailo que intervienen en esta etapa. Hailo DFC se utiliza en la máquina de desarrollo para adaptar y compilar el modelo hasta obtener el archivo ‘*E4.hef*’, mientras que HailoRT se utiliza posteriormente en la Raspberry Pi 5 para cargar dicho archivo y gestionar su ejecución sobre el acelerador Hailo-8.

De este modo, el proceso de adaptación finaliza con la obtención de ‘*E4.hef*’, versión del modelo E4 preparada específicamente para la arquitectura del Hailo-8. El archivo se transfiere posteriormente a la Raspberry Pi 5, donde puede ser cargado mediante HailoRT sin necesidad de realizar nuevas conversiones o compilaciones. Como resultado, el modelo queda preparado para integrarse en el flujo de inspección embarcado.

7.2. Flujo de inferencia e integración con el UAS

Una vez preparado el modelo para poder ejecutarse en la plataforma *Edge-AI*, el siguiente paso es ver cómo funciona realmente durante la misión.

La Raspberry Pi 5 coordina el proceso de inferencia: recibe las imágenes de la cámara, las prepara para el modelo, las envía al Hailo-8 y recoge el resultado. A partir de ahí, el sistema gestiona las posibles detecciones y repite el proceso durante toda la inspección.

7.2.1. Secuencia operativa de inferencia a bordo

La secuencia operativa de la inferencia a bordo podría definirse como:

1. Inicio del sistema. La Raspberry Pi 5 inicia el programa de inspección y carga el modelo ‘*E4.hef*’ mediante HailoRT, para que pueda ejecutarse sobre el acelerador Hailo-8.
 2. Captura de la imagen. La Raspberry recibe los *frames* procedentes de la cámara embarcada en el dron.
 3. Preparación del *frame*. Cada imagen se adapta al formato que espera el modelo mediante su redimensionamiento a 640 x 640 píxeles.
 4. Inferencia. La Raspberry envía el *frame* preparado al Hailo-8, que es el encargado de ejecutar el modelo E4 y realizar la detección.
-

5. Obtención y postprocesado del resultado. Una vez terminada la inferencia, el resultado vuelve a la Raspberry Pi para ser interpretado, obteniendo clase, confianza y posición del *bounding box*.
6. Gestión de la detección. Si se identifica una anomalía que supera el umbral de confianza definido, el sistema puede guardar el resultado, almacenar una captura o generar un aviso.
7. Siguiendo *frame*. El proceso vuelve a comenzar con la siguiente imagen a modo de bucle.

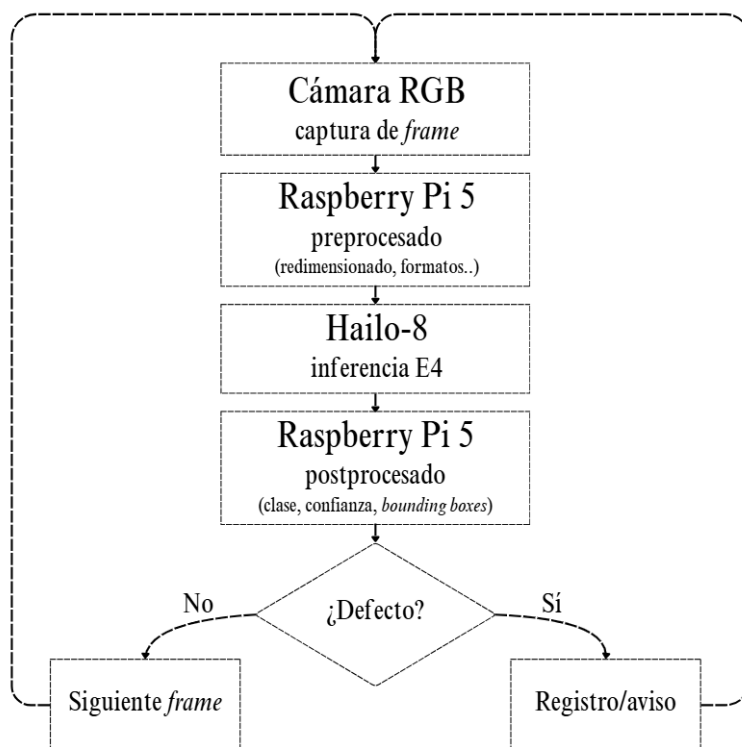


Figura 7.3. Flujo de procesamiento de imágenes durante la misión. Fuente: Elaboración propia.

7.2.2. Software embarcado y adquisición de imágenes

Para que el flujo anterior pueda llevarse a cabo, la Raspberry Pi 5 necesita ejecutar un programa que gestione de forma continua el procesamiento de las imágenes. Este software es el encargado de recibir cada *frame*, prepararlo para el modelo, enviarlo al Hailo-8 y recoger posteriormente el resultado de la inferencia.

En el sistema embarcado, la Raspberry actúa por tanto como punto de unión entre la cámara y el acelerador. El Hailo-8 se encarga de ejecutar el modelo E4, mientras que el programa que se ejecuta en la Raspberry controla la entrada de imágenes, interpreta las detecciones obtenidas y continúa el proceso con el siguiente *frame*.

En una implementación completa del UAS, estas imágenes procederían directamente de la cámara RGB embarcada durante el vuelo. Sin embargo, para las pruebas realizadas en este trabajo se utiliza inicialmente un vídeo (grabado con el DJI neo 2, para “simular” el “efecto dron”) que se almacena localmente en la Raspberry Pi, lo que permite trabajar con imágenes obtenidas desde una perspectiva aérea real antes de abordar la recepción de vídeo en tiempo real.

Para realizar esta primera validación se desarrolló el script ‘*detectar_video.py*’, ejecutándolo inicialmente en el equipo de desarrollo. Su objetivo es reproducir el flujo de inferencia descrito

anteriormente, procesando el vídeo *frame* a *frame* y generando una salida anotada con las detecciones obtenidas.

En esta primera versión, `'detectar_video.py'` carga el modelo E4 a partir de su archivo `'best.pt'` (donde guarda su mejor *epoch*) y define los principales parámetros de inferencia: mantiene una resolución de entrada de 640x640 píxeles y se establece un umbral de confianza de 0,5.

Además, se utiliza `'VID_STRIDE = 3'`, de forma que se procesa uno de cada tres *frames* del vídeo, reduciendo la carga de procesamiento durante estas primeras pruebas.

La función `'model.predict()'` recorre la secuencia de vídeo y obtiene las detecciones correspondientes a cada *frame* procesado. El programa representa sobre la imagen la clase identificada, su nivel de confianza y el *bounding box* asociado, y utiliza estos frames anotados para generar un nuevo archivo de vídeo de salida.

```
from ultralytics import YOLO

VIDEO_ENTRADA = 'video_dron.mp4'
VIDEO_SALIDA = 'output_dron.mp4'
VID_STRIDE = 3

model = YOLO('weights/best.pt')

for results in model.predict(
    source=VIDEO_ENTRADA,
    vid_stride=VID_STRIDE,
    imgsz=640,
    conf=0.5,
):
    annotated_frame = results.plot()
```

La llamada a `'model.predict()'` devuelve los resultados correspondientes a cada *frame* procesado. El bucle recorre estas salidas de forma secuencial, de manera que en cada iteración la variable `'results'` contiene la información generada por el detector para una de las imágenes analizadas.

A partir de esta información, `'results.plot()'` genera una versión anotada del *frame*, incorporando las detecciones obtenidas por el modelo.

Los *frames* anotados se almacenan de forma consecutiva para generar un nuevo archivo de vídeo de salida. De esta forma, al finalizar el procesamiento se obtiene una secuencia equivalente al vídeo original, pero incorporando visualmente las detecciones realizadas por E4. Esta salida permite comprobar el funcionamiento del detector sobre una secuencia completa y sirve como primera validación de la lógica que posteriormente se trasladará a la plataforma *Edge-AI*.

Una vez comprobado el funcionamiento de esta primera versión, el programa se trasladó a la Raspberry Pi 5 para ejecutar la inferencia sobre el acelerador Hailo-8. En este caso, la principal

diferencia (como se ha adelantado anteriormente) se encuentra en el modelo utilizado: ‘*best.pt*’ se sustituye por el archivo compilado ‘*E4.hef*’.

Para utilizar este archivo, el programa emplea HailoRT, que permite cargar el modelo en el acelerador y enviarle las imágenes que deben ser procesadas. La Raspberry continúa encargándose de la lectura y preparación de los *frames* mientras que la ejecución de E4 se realiza en el Hailo-8. Una vez finalizada cada inferencia, los resultados vuelven al programa para continuar con el procesamiento.

En el código utilizado para esta prueba, el cambio se observa de forma sencilla en dos operaciones principales: la carga del archivo compilado y la ejecución de la inferencia sobre el acelerador.

```
...  
  
WEIGHTS_PATH = '../solar_panels_yolo11n.hef'  
  
hef = HEF(WEIGHTS_PATH)  
  
raw_outputs = infer.pipeline.infer(  
    {input_name: input_tensor}  
)  
  
...
```

En primer lugar, ‘*WEIGHTS_PATH*’ indica la ubicación del archivo ‘*E4.hef*’⁴ generado durante la etapa de compilación. La instrucción ‘*HEF(WEIGHTS_PATH)*’ carga este modelo para poder configurarlo y ejecutarlo sobre el Hailo-8. Posteriormente, la llamada a ‘*infer()*’ envía al acelerador el *frame* preparado y devuelve las salidas generadas por el modelo.

Una vez cargado el modelo, el programa queda preparado para repetir esta operación sobre los sucesivos *frames* que recibe la Raspberry Pi 5. De esta forma, la Raspberry gestiona el flujo de imágenes y el tratamiento de los resultados, mientras que la ejecución del modelo E4 se delega en el Hailo-8.

Queda así completado el paso desde la prueba inicial en el equipo de desarrollo hasta la ejecución del modelo sobre la plataforma embarcada, manteniendo la misma lógica de procesamiento, pero utilizando ya el modelo compilado específicamente para Hailo-8.

7.2.3. Integración con el sistema de vuelo y supervisión en tierra

Como se definió en “4. DISEÑO E INTEGRACIÓN DEL HARDWARE”, la arquitectura propuesta para el UAS contempla una controladora Pixhawk encargada de la gestión del vuelo y una Raspberry Pi 5 actuando como “ordenador de a bordo” comunicadas mediante el protocolo *MAVLink*. Sobre esta arquitectura se incorpora ahora el sistema *Edge-AI* desarrollado, de manera que el procesamiento de las imágenes pueda coordinarse con el propio desarrollo de la misión.

Durante las pruebas realizadas, el programa de inferencia se inicia manualmente desde la Raspberry Pi 5. En una implementación embarcada, sin embargo, no sería necesario que el operador accediese al sistema para ejecutar el programa antes de cada vuelo. El software podría

⁴ En el código, ‘*solar_panels_yolo11n.hef*’ corresponde a ‘*E4.hef*’, no se ha modificado para evitar confusiones sobre el modelo final elegido (Capítulo 6).

configurarse para iniciarse automáticamente junto con el sistema operativo de la Raspberry y permanecer en espera mientras no se encuentre activa una inspección.

Durante ese estado de espera, el modelo y los recursos necesarios para el procesamiento quedarían preparados, pero no se realizaría el análisis continuo de imágenes. El comienzo y final de la inspección vendrían determinados por una señal procedente del sistema de vuelo, permitiendo separar el arranque del software de la activación efectiva del procesamiento.

A partir de esta arquitectura, el programa ejecutado en la Raspberry debe quedar preparado para iniciar y detener el procesamiento de imágenes en función de las necesidades de inspección. Su funcionamiento general puede resumirse mediante el siguiente flujo:

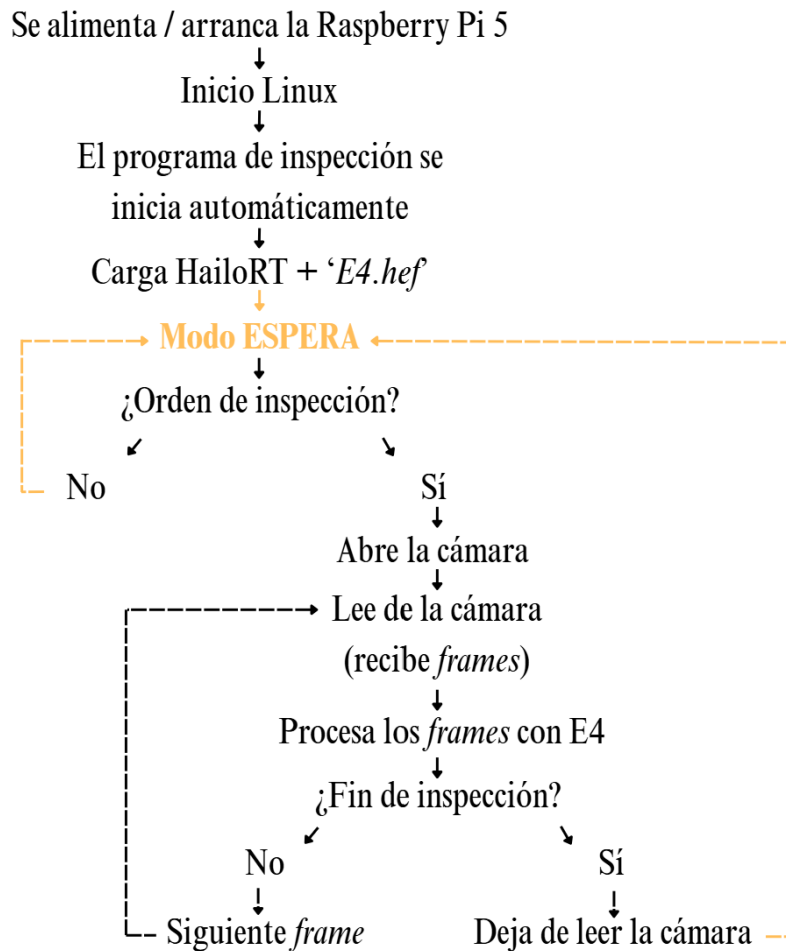


Figura 7.4. Flujo de funcionamiento del programa de inspección embarcado. Fuente: Elaboración propia.

“Figura 7.4” resume el funcionamiento previsto del software una vez embarcado en el UAS. Al encender el dron, la Raspberry Pi 5 recibe alimentación e inicia su sistema operativo. El programa de inspección puede configurarse de forma que arranque automáticamente, cargue HailoRT y el modelo ‘E4.hef’ y quede preparado en un modo de espera. En este estado el programa continúa ejecutándose, pero sin recibir ni procesar las imágenes de la cámara.

Activación de la inspección

El paso del modo de espera al procesamiento se produce cuando la Raspberry recibe una orden de inspección. Es en este punto donde interviene la controladora de vuelo (Pixhawk). Esta, recibe las órdenes asociadas al funcionamiento del UAS y puede comunicar a la Raspberry, mediante *MAVLink*, la información necesaria para indicar el comienzo o final de la inspección.

La orden de inspección puede definirse de distintas formas dentro de la arquitectura propuesta. Entre las posibilidades consideradas se encuentran:

- Estado de armado del UAS. El procesamiento podría comenzar automáticamente cuando la Pixhawk indique que el vehículo ha sido armado y detenerse al producirse el desarmado.
- Inicio de una misión de inspección programada. Cuando el vuelo se realiza siguiendo una ruta previamente definida, la Pixhawk conoce el avance de la misión y podría indicar a la Raspberry cuándo comienza y termina la zona destinada a inspeccionar los paneles, activando y desactivando así el procesamiento de imágenes automáticamente durante el recorrido.
- Orden manual desde el radiocontrol. Podría reservarse un botón o interruptor del radiocontrol para que el piloto active y desactive de forma manual el sistema de inspección durante el vuelo.

De las alternativas propuestas, se considera especialmente adecuada la activación manual mediante el radiocontrol, ya que permite al piloto decidir en qué momento del vuelo debe comenzar y finalizar la inspección.

Para llevarla a cabo, podría asignarse uno de los interruptores del radiocontrol a un canal auxiliar de la Pixhawk. No sería necesario desarrollar un programa independiente, sino que el propio software de inspección ejecutado en la Raspberry Pi 5 incorporaría una pequeña función encargada de consultar, mediante *MAVLink*, el estado de dicho canal. Cuando el interruptor se encontrase en la posición de activación (1), el programa saldría del modo de espera y comenzaría a leer y procesar las imágenes de la cámara; al volver a la posición inicial (0), detendría el procesamiento y regresaría al modo de espera.

La configuración concreta del canal y su implementación sobre el UAS físico se plantea como una posible ampliación del prototipo, ya que no resulta necesaria para comprobar el funcionamiento del sistema *Edge-AI* desarrollado.

Envío de avisos de detección

Una vez iniciada la inspección, la comunicación entre la Raspberry y el sistema de vuelo también puede utilizarse para informar al operador cuando se detecte una anomalía. Si el modelo identifica alguno de los defectos considerados, el programa ejecutado en la Raspberry podría generar un aviso con la información básica de la detección y enviarlo hacia la estación de control en tierra mediante *MAVLink*.

Cuando E4 detectase alguno de los defectos considerados ('*crack*', '*snow*' o '*soiling*') con una confianza suficiente, el programa podría generar un mensaje indicando el tipo de defecto identificado. Este aviso se enviaría mediante *MAVLink* a través de la Pixhawk para que pudiera mostrarse al operador en la estación de control en tierra. De este modo, el piloto tendría conocimiento de un posible defecto durante el propio desarrollo de la inspección sin necesidad de esperar al análisis posterior del vídeo.

Comunicación mediante *MAVLink*

Aunque *MAVLink* ya fue definido en “4. DISEÑO E INTEGRACIÓN DEL HARDWARE” como protocolo de comunicación entre la Pixhawk y la Raspberry Pi 5, conviene concretar brevemente qué papel desempeña dentro de las funciones planteadas para el sistema de inspección.

Tanto la activación del sistema de inspección como el envío de avisos hacia la estación de control se apoyan en *MAVLink*, que permite intercambiar diferentes tipos de mensajes sobre el enlace existente entre ambos dispositivos. Para utilizar esta información desde el programa de inspección sería necesario incorporar al propio *script* una librería compatible con *MAVLink* (como ‘*pymavlink*’ o ‘*MAVSDK*’), junto con las funciones necesarias para recibir los estados enviados por la Pixhawk y transmitir los avisos generados por el sistema de visión.

Por tanto, no sería necesario modificar el funcionamiento del modelo ni desarrollar un nuevo programa, sino ampliar el software ya realizado con pequeños bloques encargados de la comunicación con el sistema de vuelo.

Entorno de pruebas: *Mission Planner*

Para probar estas funciones sin disponer físicamente de la controladora Pixhawk y del UAS, se utilizó *Mission Planner* junto con ArduPilot SITL (*Software In The Loop*), *Mission Planner* actuó como estación de control en tierra, mientras que SITL permitió reproducir mediante software el comportamiento de la controladora de vuelo. De este modo fue posible trabajar con la comunicación *MAVLink* y comprobar la viabilidad del envío y recepción de las órdenes y avisos planteados para el sistema de inspección.

Cabe destacar que, para comprobar específicamente la lógica de activación de la inspección, se definió durante la simulación una condición temporal basada en la altura del UAS. Cuando el vehículo simulado superaba los 2 m de altura, se generaba la orden de inicio del procesamiento (lo que sería el estado 1 de la solución con el botón del mando).

Esta condición se utilizó únicamente como mecanismo de prueba, con el objetivo de verificar que el programa podía pasar correctamente del modo de espera al modo de inspección a partir de información recibida mediante *MAVLink*.

En una implementación real, el papel desempeñado por SITL sería sustituido por la Pixhawk embarcada en el UAS, mientras que *Mission Planner* podría mantenerse como herramienta de supervisión desde tierra. Esta configuración permitió probar parte de la integración prevista sin necesidad de disponer del prototipo completo.

Visualización remota del vídeo

Además de las órdenes de activación y de los avisos de detección, se probó también la visualización remota del vídeo procesado durante la inspección. A diferencia de las comunicaciones anteriores, este envío no se realizaría mediante *MAVLink*, ya que se trata de un flujo continuo de imágenes y requiere una conexión de red independiente.

Para esta prueba se utilizó *Cloudflare* *¡Error! No se encuentra el origen de la referencia.* como servicio intermedio para permitir el acceso remoto al flujo generado desde la Raspberry Pi 5. El propio programa de inspección puede enviar los *frames* ya procesados por E4 y enviarlos hacia este servicio, haciendo posible visualizar desde otro equipo el vídeo de la inspección con las detecciones representadas sobre la imagen.

El objetivo de esta prueba no fue desarrollar una infraestructura completa de retransmisión de vídeo, sino comprobar que la salida generada por el sistema *Edge-AI* podía utilizarse de forma remota durante la inspección.

En conjunto, las funciones planteadas pueden integrarse dentro del mismo programa de inspección ejecutado en la Raspberry Pi 5 mediante tres bloques diferenciados:

- Bloque encargado de recibir la orden de inicio y finalización de inspección.
- Bloque encargado de generar y enviar los avisos cuando E4 detecta un defecto,
- Bloque encargado de transmitir de forma remota el vídeo procesado al *stream* mediante las *urls* proporcionadas por *Cloudflare*.

Los dos primeros se apoyan en la comunicación *MAVLink* con la Pixhawk, mientras que el envío de vídeo utiliza una conexión de red independiente. De este modo, el software embarcado concentra tanto el procesamiento *Edge-AI* como las funciones básicas de control, aviso y supervisión.

La versión final organizada de este programa '*inspeccion_uas.py*', se incluirá en los anexos junto con el resto del código desarrollado.

Con esto, queda definido el flujo completo de información previsto para el sistema de inspección. Las imágenes procedentes de la cámara son recibidas y gestionadas por la Raspberry Pi 5, que utiliza el Hailo-8 para ejecutar el modelo entrenado E4 y obtener las detecciones. Al mismo tiempo, la comunicación *MAVLink* permite relacionar este procesamiento con el sistema de vuelo, tanto para recibir las órdenes de inicio y finalización de la inspección como para transmitir los avisos generados hacia la estación de control en tierra. De forma complementaria, el vídeo procesado puede enviarse mediante una conexión de red independiente para su visualización remota.

Es importante tener en cuenta el resto de las comunicaciones necesarias para el control, navegación y operación del UAS, que se mantienen de acuerdo con la arquitectura definida previamente en "*4. DISEÑO E INTEGRACIÓN DEL HARDWARE*".

7.3. Validación funcional con material real de UAS

Una vez descrito el proceso de despliegue y ejecución del modelo sobre la plataforma *Edge-AI*, se realizó una validación funcional utilizando material obtenido durante un vuelo real con UAS. Para ello se empleó una última secuencia de vídeo grabada con DJI Neo 2, con el objetivo de comprobar el comportamiento del sistema ante imágenes desde una perspectiva aérea y sometidas al movimiento del propio vuelo.

La prueba se realizó siguiendo el procedimiento de procesamiento de vídeo descrito en el apartado 7.2.2, utilizando en este caso la Raspberry Pi 5 y el Hailo-8 para ejecutar el modelo E4. Por tanto, no se vuelve a detallar aquí el funcionamiento interno del programa ni la preparación de los *frames*, este apartado se centra en recoger la validación realizada y analizar de forma visual el comportamiento del sistema sobre una secuencia obtenida desde un UAS real.

Para obtener una secuencia de prueba independiente de las imágenes utilizadas durante el desarrollo del modelo, se realizó una nueva grabación en una instalación fotovoltaica a la que tenía acceso uno de los colaboradores del proyecto.

Sobre algunos de los paneles se prepararon de forma controlada pequeñas muestras de suciedad, correspondientes a la clase *soiling*, ya que era la única de las anomalías consideradas que podía reproducirse sin alterar ni dañar físicamente la instalación.

La grabación se llevó a cabo en un día distinto y bajo condiciones de iluminación diferentes a las empleadas en las imágenes utilizadas durante el entrenamiento, siguiendo además una secuencia de vuelo independiente. Con ello se buscó evitar que la validación se realizase sobre escenas demasiado similares a las que ya vio el modelo y obtener una respuesta más representativa de su comportamiento ante material nuevo capturado con el UAS.

Por este motivo, la validación funcional se centró especialmente en la respuesta del sistema ante la presencia de *soiling*.

Para apoyar las pruebas de comunicación se utilizó *Mission Planner* como estación de control en tierra junto con ArduPilot SITL (*Software In The Loop*), que permitió simular por software el comportamiento de la controladora de vuelo. Este entorno se empleó para trabajar con mensajes *MAVLink* y comprobar de forma preliminar las funciones de control y aviso planteadas para el sistema.

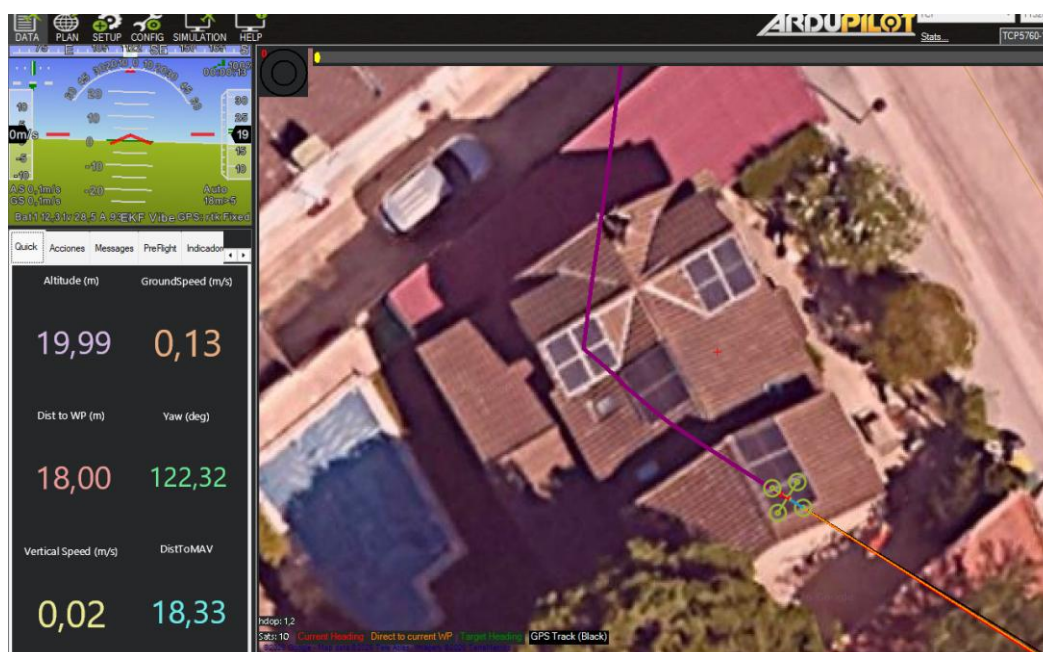


Figura 7.5. Simulación de una misión de inspección fotovoltaica mediante Mission Planner.

A continuación, se seleccionaron varios instantes representativos de la secuencia para observar el comportamiento del modelo sobre las imágenes obtenidas durante el vuelo. En estas capturas puede comprobarse cómo E4 analiza escenas diferentes a las utilizadas durante el entrenamiento y responde ante las muestras de suciedad colocadas sobre los paneles.

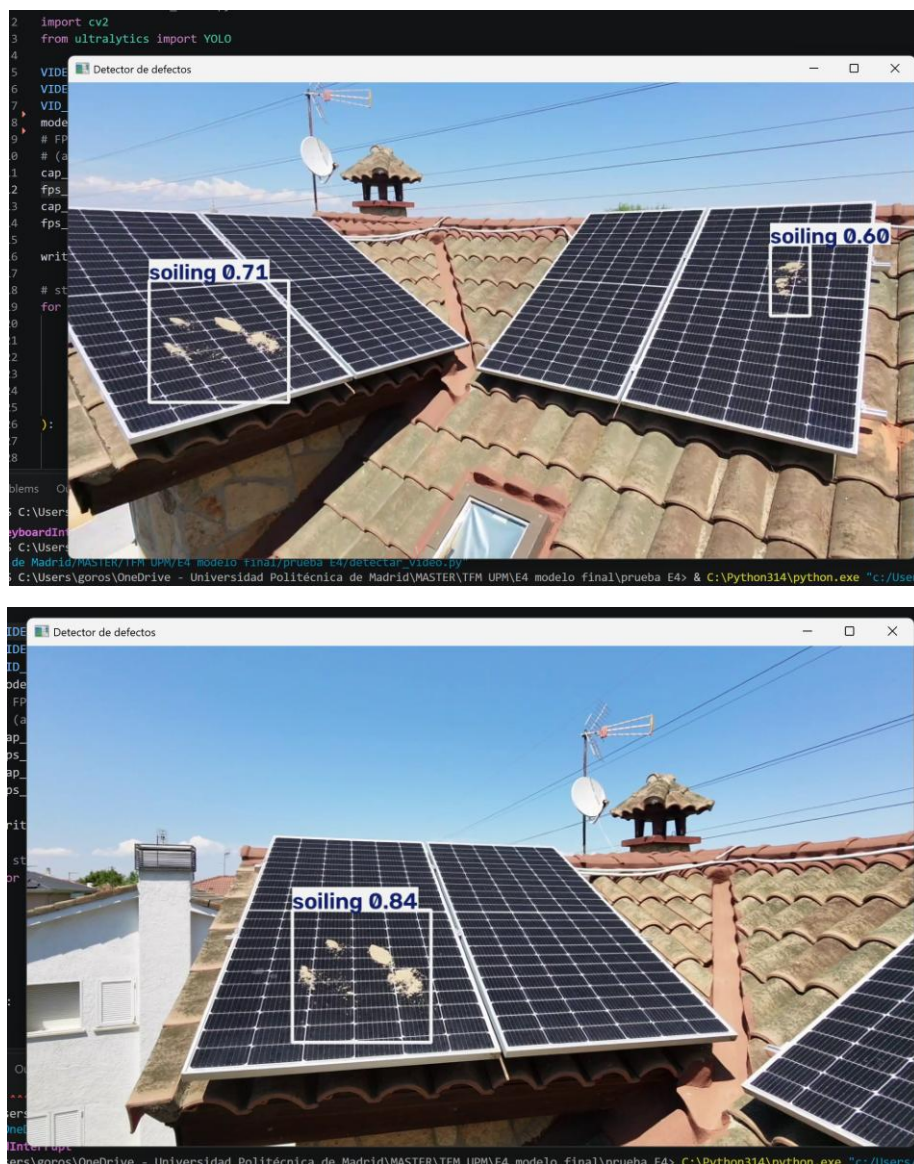


Ilustración 1. Ejemplos de detección de soiling durante la validación con vídeo real capturado mediante el DJI Neo 2.

En términos generales, el sistema fue capaz de identificar las muestras *soiling* presentes sobre los paneles a lo largo de la secuencia. Las detecciones aparecieron en diferentes momentos del vuelo y sobre las imágenes obtenidas bajo condiciones distintas a las de entrenamiento, lo que permite comprobar que el modelo es capaz de responder ante material nuevo.

No obstante, la detección no se mantuvo de forma constante durante toda la secuencia. En algunos *frames* el modelo no identificó la suciedad o dejó de detectarla temporalmente, comportamiento coherente con las limitaciones observadas durante la evaluación de E4 en el capítulo anterior. Por tanto, esta prueba no demuestra una detección perfecta de las anomalías, sino la capacidad del sistema completo para ejecutar el modelo sobre una secuencia aérea real.

En conjunto, la validación funcional permitió comprobar que el sistema desarrollado puede procesar de forma continuada una secuencia real aérea y generar las detecciones para las que ha sido entrenado. Una vez verificado este funcionamiento, el siguiente paso consiste en analizar el rendimiento de la plataforma embarcada y comprobar los tiempos de procesamiento y consumo de recursos.

7.4. Rendimiento y consideraciones de despliegue

Para analizar el rendimiento del sistema desplegado, se compararon las principales configuraciones utilizadas durante las pruebas sobre la Raspberry Pi 5.

Para facilitar la interpretación de los resultados, se consideran cuatro métricas principales:

- Tamaño del modelo, correspondiente al espacio ocupado por el archivo.
- Latencia, que indica el tiempo medio necesario para procesar un *frame*.
- FPS, que representan el número aproximado de imágenes que pueden procesarse por segundo.
- Uso de memoria RAM, asociado a los recursos consumidos durante la ejecución del programa.

Los valores obtenidos para cada configuración se recogen en la siguiente tabla:

Configuración	Tamaño [MB]	Latencia [ms/frame]	FPS	RAM [MB]
ONNX FP32 – Raspberry Pi 5	10,11	165,34	6,05	928,52
ONNX INT8 – Raspberry Pi 5	2,87	192,37	5,20	922,16
HEF – Raspberry Pi 5 – Hailo-8	6,87	9,29	107,59	213,10

Tabla 7.3. Comparación del rendimiento de las distintas configuraciones de despliegue del modelo en Raspberry Pi 5.

En primer lugar, se observa una diferencia importante en el tamaño de los archivos. El modelo ONNX en FP32 ocupa 10,11 MB mientras que su versión cuantizada a INT8 reduce este valor hasta 2,87 MB, lo que supone una disminución aproximada del 71,6%. Y aunque el archivo HEF presenta un tamaño mayor, su rendimiento debe valorarse también teniendo en cuenta la velocidad de inferencia sobre el hardware diferente.

En cuanto a la latencia, las dos versiones ONNX ejecutadas directamente sobre la CPU de la Raspberry Pi 5 presentan valores de 165,34 ms/frame para FP32 y 192,37 ms/frame para INT8. Por tanto, en las pruebas realizadas, la cuantización de INT8 no produjo una mejora en el tiempo de procesamiento. La diferencia más significativa aparece al utilizar el archivo HEF junto con Hailo-8, donde la latencia se reduce hasta 9,29 ms/frame, mostrando la ventaja de ejecutar el modelo sobre un acelerador diseñado específicamente para este tipo de operaciones.

Los resultados de FPS siguen la misma tendencia observada en la latencia. Las versiones ONNX ejecutadas sobre la CPU de la Raspberry Pi 5 alcanzan 6,05 FPS y 5,20 FPS para FP32 e INT8, respectivamente, mientras que la ejecución mediante HEF sobre el Hailo-8 alcanza 107,59 FPS. Esta diferencia vuelve a reflejar la mejora obtenida al utilizar el acelerador.

En cuanto al uso de memoria RAM, las dos versiones ONNX presentan valores muy similares en torno a 900 MB. En cambio, la ejecución mediante HEF sobre el Hailo-8 reduce este valor hasta 213,10 MB. Esta diferencia muestra que, al delegar la inferencia en el acelerador, el programa necesita menos memoria en la Raspberry Pi durante la ejecución.

En conclusión, los resultados obtenidos justifican la elección del formato HEF y del acelerador Hailo-8 para el despliegue final de E4. Aunque la cuantización del ONNX a INT8 permitió reducir considerablemente el tamaño del archivo, esta reducción no se tradujo en una mejora de su ejecución sobre la CPU de la Raspberry Pi.

En cambio, el uso de Hailo-8 permitió disminuir de forma notable la latencia y el uso de memoria, mientras aumentaba la velocidad de procesamiento. Estos resultados confirman la utilidad de incorporar un acelerador específico para la ejecución del modelo dentro de la plataforma embarcada y completan la validación del sistema *Edge-AI* propuesto.

8. RESULTADOS Y DISCUSIÓN

Los capítulos anteriores han permitido desarrollar de forma progresiva los distintos elementos que componen el sistema propuesto, desde el diseño de la plataforma UAS hasta la integración y validación del modelo de visión artificial sobre hardware *Edge-AI*. En este capítulo se recogen los principales resultados obtenidos y se analizan de forma conjunta, poniendo especial atención en las decisiones adoptadas, las mejoras conseguidas y las limitaciones observadas.

8.1. Resultado del diseño del UAS

El diseño desarrollado en el Capítulo “4. DISEÑO E INTEGRACIÓN DEL HARDWARE” permitió definir una plataforma UAS completa orientada específicamente a la inspección fotovoltaica. El resultado no se limita a la selección de los elementos de vuelo, sino que integra todos los sistemas de navegación y comunicación y, especialmente, la carga útil necesaria para ejecutar el procesamiento *Edge-AI*.

Resultado del diseño	Valor / Solución obtenida
Plataforma	Cuadricóptero sobre chasis F450
Masa estimada en orden de vuelo	1,74 kg
Empuje máximo total	3,84 kg
Relación empuje / peso	2,21
Alimentación	LiPo 4S – 5200 mAh
Consumo estimado en vuelo estacionario	≈ 22 A
Autonomía teórica	≈ 11 min
Control de vuelo	Pixhawk 6C
Procesamiento <i>Edge-AI</i>	Raspberry Pi 5 + Hailo-8
Adquisición de imágenes	Cámara RGB Sony IMX477

Tabla 8.1. Resumen de los principales resultados del diseño del UAS.

Los valores obtenidos muestran que el conjunto mantiene un margen adecuado de empuje para la masa estimada, al tiempo que permite incorporar la electrónica de procesamiento necesaria para ejecutar el sistema de visión artificial a bordo.

La autonomía próxima a 11 minutos constituye una de las principales limitaciones prácticas del diseño, especialmente en instalaciones de mayor extensión. Para reducir su impacto durante la operación, se contempla el uso de una segunda batería de iguales características, permitiendo realizar vuelos consecutivos mediante su sustitución en tierra y reduciendo así los tiempos de espera entre pasadas. Esta solución, aunque no aumenta la autonomía de cada vuelo, sí que mejora la continuidad de las tareas de inspección.

No obstante, al no haberse construido físicamente la plataforma completa, los resultados relativos al comportamiento del UAS corresponden al dimensionamiento teórico realizado y requerirían una validación experimental mediante el montaje definitivo del sistema.

Como cierre visual del diseño, se ha adaptado el esquema final de integración presentado en el Capítulo 4, sustituyendo los bloques funcionales por imágenes de los componentes seleccionados y manteniendo las mismas conexiones entre subsistemas.

Además, se ha incorporado un código de colores para diferenciar con mayor claridad las distintas líneas de alimentación y facilitar la identificación de las conexiones asociadas a la PDB, que resultan menos evidentes al emplear fotografías reales de los componentes.

La ilustración permite identificar de forma más intuitiva cómo quedaría configurado el UAS completo, desde la alimentación y el sistema de propulsión hasta el control de vuelo, las comunicaciones y la carga útil *Edge-AI*.

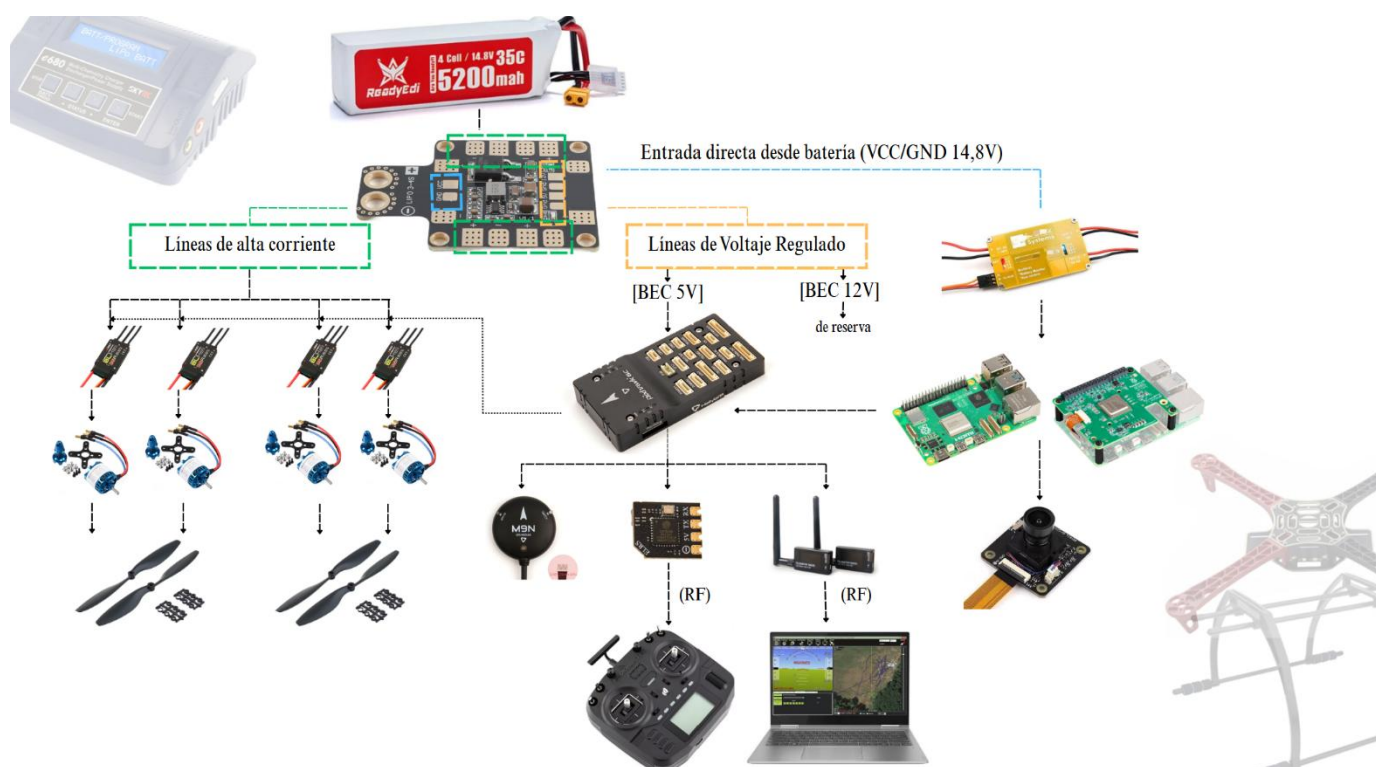


Figura 8.1. Esquema visual final de integración de los componentes del UAS propuesto. Fuente: Elaboración propia.

8.2. Resultados del sistema de visión artificial.

La construcción del conjunto de datos también evolucionó de forma progresiva. A partir de un dataset público inicial se realizó una depuración de clases y anotaciones, y posteriormente se amplió el material disponible mediante capturas propias, colaboraciones y recursos externos.

Para facilitar la obtención de nuevas imágenes se desarrolló además una página web específica para el TFM⁵, orientada a presentar el proyecto y canalizar distintas formas de colaboración.

⁵ Pag web desarrollada para la captación de colaboradores y recopilación de material para el TFM: <https://uas.luciagorostidi.com>

Por otro lado, las imágenes recopiladas y las sucesivas versiones del conjunto de datos se organizaron y gestionaron mediante un proyecto específico en Roboflow⁶.

En total, durante esta fase se reunieron 243 imágenes candidatas procedentes de las distintas fuentes consideradas. Tras su revisión individual se descartaron 78 imágenes por presentar problemas como falta de nitidez, escaso nivel de detalle, excesiva distancia al panel o elevada similitud con otras imágenes.

Finalmente, se seleccionaron 165 nuevas imágenes, que se incorporaron al conjunto de entrenamiento para ampliar la variedad de situaciones representadas.

La evolución del conjunto de datos a lo largo de las distintas etapas del desarrollo se resume en:

Etapas	Resultado
Dataset público inicial	845 imágenes
E1 – Dataset depurado	724 imágenes
Nuevo material recopilado	243 imágenes candidatas
Nuevas imágenes seleccionadas	165 imágenes
E2 – Dataset ampliado	889 imágenes
E3 – Revisión de clases	757 imágenes para ‘ <i>crack</i> ’ ‘ <i>snow</i> ’ y ‘ <i>soiling</i> ’
E4 – Aumento de datos	1903 imágenes totales, 1719 en <i>train</i>

Tabla 8.2. Evolución del conjunto de datos durante el desarrollo experimental.

A partir de estas sucesivas versiones del conjunto de datos se desarrollaron cuatro experimentos de entrenamiento, manteniendo una configuración común basada en YOLO11n para que los resultados fueran comparables.

La incorporación de nuevas imágenes reales en E2 produjo una mejora limitada, mientras que la revisión de las clases en E3 y la aplicación posterior de técnicas de aumento artificial de datos en E4 tuvieron un efecto más notable sobre el comportamiento del detector.

Experimento	Precision	Recall	mAP50	mAP50-95
E1 baseline	0,366	0,457	0,429	0,315
E2 extended	0,390	0,481	0,431	0,319
E3 revised	0,693	0,466	0,585	0,449
E4 augmented	0,615	0,631	0,558	0,417

Tabla 8.3. Comparación de los resultados obtenidos por los experimentos E1, E2, E3 y E4, sobre el conjunto de test.

⁶ Proyecto de Roboflow utilizado para la gestión, anotación y versionado del conjunto de datos: <https://app.roboflow.com/lucia-gorostidi-s-workspace/solar-panels-cyg5d/train>

Los resultados muestran una evolución clara entre los distintos experimentos. E2 apenas mejora las métricas con respecto a E1, mientras que E3 consigue el mayor valor de *Precision* y los mejores resultados de *mAP*. Por su parte, E4 alcanza el mayor *Recall*, con un valor de 0,631, y presenta además el mejor equilibrio global entre precisión y capacidad de detección.

Teniendo en cuenta que, en una tarea de inspección, resulta especialmente importante reducir el número de defectos que no han sido detectados, E4 fue el modelo seleccionado como modelo definitivo para su posterior integración sobre la plataforma *Edge-AI*.

Esta decisión queda reforzada al considerar el valor F1, que combina *Precision* y *Recall* en una única métrica. E3 alcanzó un F1 aproximado de 0,557, mientras que E4 obtuvo un valor de 0,623, confirmando un mejor equilibrio global entre ambas métricas.

El rendimiento del modelo final tampoco fue homogéneo entre las tres clases consideradas:

- ‘*snow*’ presentó el comportamiento más sólido, con un *mAP50* de 0,912
- ‘*soiling*’ alcanzó un *mAP50* de 0,571
- ‘*crack*’ fue, durante todos los experimentos, la clase más problemática, acabando en este último con un *mAP50* de 0,192 y un *Recall* de 0,429 (incluso después de aplicar *data augmentation*)

Este comportamiento puede estar relacionado, entre otros aspectos, con la menor disponibilidad de imágenes representativas de grietas en el conjunto de datos y con la dificultad visual asociada a ese tipo de defecto.

En conjunto, los resultados obtenidos muestran que el modelo final alcanza un comportamiento adecuado para las clases mejor representadas, aunque todavía existe margen de mejora. Una ampliación futura del conjunto de datos, incorporando un mayor número de imágenes reales y una mayor variedad de condiciones de captura, defectos y escenarios, permitiría reforzar la capacidad de generalización del detector (especialmente importante para la clase ‘*crack*’).

8.3. Integración, validación y rendimiento Edge-AI

En el Capítulo “7. INTEGRACIÓN DEL SISTEMA EDGE – AI” se abordó la integración del sistema de visión artificial dentro de la arquitectura propuesta, definiendo el flujo de funcionamiento desde la adquisición de las imágenes hasta su procesamiento a bordo y la comunicación con el sistema de vuelo y la estación de tierra.

Una vez seleccionado E4 como el modelo definitivo, este se adaptó para su ejecución sobre la plataforma *Edge-AI* formada por Raspberry Pi 5 y Hailo-8 (elementos de los que sí se disponía, por lo que fue posible realizar pruebas sobre estos), obteniéndose finalmente el modelo en formato ‘*.hef*’ compatible con el acelerador.

Además de la ejecución del detector, durante esta fase se abordaron tres aspectos relacionados con su utilización dentro de una misión de inspección:

- Se planteó la activación del proceso de inspección a partir de una orden recibida mediante *MAVLink*, de manera que el sistema pueda permanecer inicialmente a la espera y comenzar el procesamiento cuando se active la inspección.
- En segundo lugar, se trabajó sobre el envío de avisos hacia la estación de tierra (con *Mission Planner*) cuando se detecta una anomalía relevante.

- Por último, se comprobó la posibilidad de transmitir el vídeo procesado mediante *streaming*, permitiendo visualizar de forma remota el resultado de la inspección.

Estas pruebas permitieron verificar por separado los principales mecanismos necesarios para una futura integración completa con el sistema de vuelo. Se simularon con *Mission Planner* y *Ardupilot SITL*. Ante la ausencia de mando físico, se empleó de forma temporal para la prueba de activación del proceso de inspección, una condición de altura superior a 2 m de vuelo, verificando así el funcionamiento del mecanismo previsto con *MAVLink*.

El rendimiento de la solución desplegada se comparó con la ejecución del mismo detector sobre la CPU de la Raspberry Pi 5, utilizando tanto el modelo *‘.onnx’* en precisión FP32 como una versión del mismo cuantizada a INT8.

Configuración	Tamaño [MB]	Latencia [ms/frame]	FPS	RAM [MB]
ONNX FP32 – Raspberry Pi 5	10,11	165,34	6,05	928,52
ONNX INT8 – Raspberry Pi 5	2,87	192,37	5,20	922,16
HEF – Raspberry Pi 5 – Hailo-8	6,87	9,29	107,59	213,10

Tabla 8.4. Comparación del rendimiento del modelo en Raspberry Pi 5 mediante *.onnx* y aceleración Hailo-8.

Los resultados muestran una mejora muy significativa al ejecutar el modelo mediante acelerador. Frente a la ejecución del modelo *‘.onnx’* cuantizado sobre la CPU de la Raspberry Pi 5, el uso del formato *‘.hef’* junto con Hailo-8 permitió:

- Reducir la latencia aproximadamente un 94,4%.
- Aumentar la velocidad de procesamiento hasta 17,8 veces más imágenes por segundo (FPS).
- Reducir la memoria RAM utilizada por el proceso en torno a un 77,1%.

En términos absolutos, la latencia pasó de 165,34 a 9,29 ms/frame, la velocidad de 6,05 a 107,59 FPS y el uso de memoria de 928,52 a 213,10 MB.

Estos valores muestran la ventaja de utilizar un acelerador dedicado para la ejecución del detector a bordo, ya que permiten procesar el flujo de vídeo con un margen suficiente para una aplicación de inspección en tiempo real y además reducen la carga computacional soportada directamente por la Raspberry Pi 5.

La validación funcional se completó utilizando una secuencia de vídeo real capturada mediante el DJI Neo 2 sobre una instalación fotovoltaica. Para disponer de defectos visibles durante la prueba se colocaron muestras controladas de suciedad sobre algunos módulos, ya que no resultaba viable generar de forma intencionada situaciones de nieve o grietas.

La grabación empleada para esta validación se realizó, además, en condiciones y momentos diferentes a los del material utilizado durante el entrenamiento, evitando evaluar el sistema sobre las mismas imágenes.

Durante la secuencia, el sistema fue capaz de identificar las zonas de *soiling* en distintos momentos del recorrido, aunque también se observaron algunos momentos en los que la detección se perdía temporalmente. Este comportamiento es totalmente coherente con las limitaciones observadas previamente durante la evaluación del modelo y pone de manifiesto la influencia que pueden ejercer factores como el ángulo de captura, la distancia al panel, o las condiciones de iluminación.

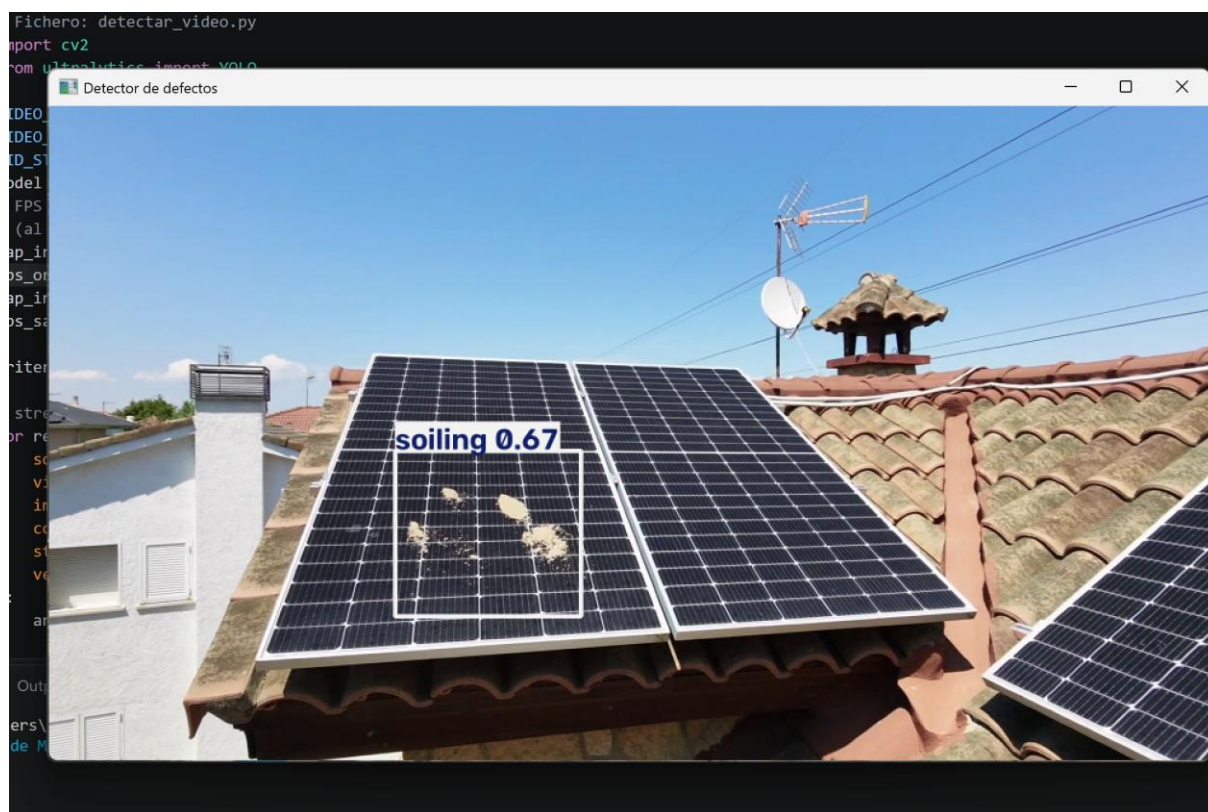


Figura 8.2. Ejemplo de detección de soiling durante la validación con video real mediante DJI Neo 2.

En conjunto, los resultados obtenidos demuestran la viabilidad de ejecutar el detector desarrollado sobre una plataforma *Edge-AI* compacta y potencialmente embarcable en el UAS propuesto.

La combinación de Raspberry Pi 5 y Hailo-8 permite alcanzar un rendimiento ampliamente superior al obtenido mediante ejecución exclusiva en CPU, mientras que las pruebas realizadas con material aéreo real y sobre los mecanismos de activación, avisos y *streaming* permiten avanzar hacia una integración completa del sistema de inspección. (Aunque esta última requeriría realmente de una plataforma UAS físicamente construida para validar conjuntamente todos los subsistemas durante una misión real).

9. MARCO NORMATIVO

El desarrollo de un sistema de inspección fotovoltaica mediante UAS debe considerar, además de los aspectos técnicos, la normativa que regula la operación de aeronaves no tripuladas y las condiciones en las que estas pueden utilizarse. En el caso del sistema propuesto, resultan especialmente relevantes las disposiciones relacionadas con la operación del UAS, la captación de imágenes, la protección de datos y los requisitos aplicables a las instalaciones fotovoltaicas.

9.1. Normativa UAS

En España, la utilización del UAS se encuentra actualmente dentro del marco normativo europeo, aplicable desde el 31 de diciembre de 2020, y de la normativa nacional que lo complementa. El Reglamento de Ejecución (UE) 2019/947 [5] establece las reglas y procedimientos aplicables a las operaciones con UAS y las clasifica, en función del riesgo, en tres categorías: abierta, específica y certificada. La diferencia entre categorías depende de las condiciones en las que vaya a utilizarse el UAS y del nivel de riesgo que pueda suponer para las personas y el entorno.

La categoría abierta corresponde a las operaciones de menor riesgo. Si se cumplen sus condiciones no es necesario solicitar una autorización operacional previa para cada vuelo.

La categoría específica se aplica cuando la operación que se quiere realizar ya no cumple alguna de las condiciones de la categoría abierta y, por tanto, presenta un riesgo mayor. En estos casos puede ser necesario demostrar previamente que la operación puede realizarse de forma segura, mediante una declaración para determinados escenarios o solicitando una autorización a AESA.

Finalmente, la categoría certificada está reservada para operaciones de riesgo mucho mayor, como determinados vuelos sobre concentraciones de personas, transporte de personas o transporte de mercancías peligrosas. En este tipo de operaciones pueden exigirse la certificación del UAS, la certificación del operador y, en algunos casos, una licencia específica de piloto.

De forma simplificada, puede entenderse que la categoría abierta corresponde a operaciones de bajo riesgo, la específica a aquellas que necesitan medidas o autorizaciones adicionales y la certificada a operaciones de riesgo elevado.

En el contexto de este trabajo, resulta especialmente relevante analizar con mayor detalle la categoría abierta, ya que es la primera que debe considerarse para una operación de bajo riesgo. Para permanecer dentro de ella deben cumplirse, entre otras condiciones:

- Masa máxima de despegue inferior a 25 kg.
- Mantenimiento del UAS dentro del alcance visual del piloto.
- Altura máxima de 120 m respecto al punto más próximo de la superficie.

Dentro de la categoría abierta se distinguen tres subcategorías, A1, A2 y A3. Cada una establece condiciones diferentes en función de las características del UAS y, sobre todo, de la proximidad a personas y zonas habitadas.

Sus principales diferencias se resumen en “Figura 9.1”.

Principales requisitos a partir 1/1/2024
Categoría abierta



UAS			Operación		Formación
Clase	DRI	MTOM	Subcat.	Restricciones operacionales	Requisitos a pilotos
Construcción privada	✗	< 250 g	A1	- No se recomienda volar por encima de personas - No está permitido el vuelo sobre reuniones de personas	Familiarización con el manual de usuario facilitado por el fabricante del UAS
Legacy < 250g	✗				
C0	✗				
C1	✓	< 900 g		- No volar por encima de ninguna persona no participante - No está permitido el vuelo sobre reuniones de personas	- Familiarización con el manual de usuario facilitado por el fabricante del UAS - Prueba de superación de formación de formación en línea
C2	✓	< 4 kg	A2	- No volar por encima de ninguna persona no participante 30 m de cualquier persona no participante 5 m de distancia si dispone de modo de baja velocidad	- Familiarización con el manual de usuario - Prueba de superación de formación en línea - Declaración de formación autopráctica - Certificado de Competencia de Piloto a Distancia
C3	✓	< 25 kg	A3	- No volar cerca de personas - Distancia de 150 m respecto de: - Zonas residenciales - Zonas comerciales - Zonas industriales - Zonas recreativas	- Familiarización con el manual de usuario facilitado por el fabricante del UAS - Prueba de superación de formación en línea
C4	✗				
Construcción privada	✗				
Legacy > 250g	✗				

Figura 9.1. Principales requisitos aplicables a operaciones UAS en categoría abierta. Fuente: AESA.

En el caso del UAS diseñado para este trabajo, la masa prevista es de aproximadamente 1,8 kg y se trata de una aeronave de construcción privada. De acuerdo con los requisitos mostrados en la tabla proporcionada por AESA, una operación realizada dentro de la categoría abierta quedaría encuadrada en la subcategoría A3, siempre que se cumpliesen las restricciones correspondientes.

Una de las principales limitaciones de la subcategoría A3 es la necesidad de mantener una distancia horizontal mínima de 150 m respecto a zonas residenciales, comerciales, industriales o recreativas, además de realizar la operación lejos de personas no participantes. Esta condición debe tenerse en cuenta en el sistema propuesto, ya que el objetivo del UAS es aproximarse a una instalación fotovoltaica para obtener imágenes de los paneles.

Por tanto, la categoría aplicable tendría que estudiarse para cada operación concreta. Una inspección realizada sobre una instalación aislada, y cumpliendo las distancias exigidas podría mantenerse dentro de A3.

Sin embargo, si la planta se encontrase en una zona industrial, residencial o en cualquier entorno en el que no fuese posible respetar las distancias, la operación dejaría de cumplir las condiciones de la categoría abierta y tendría que estudiarse su realización dentro de la categoría específica. En este caso podrían ser necesarios una declaración para determinados escenarios o una autorización operacional de AESA.

Además de determinar la categoría en la que puede realizarse una operación, es necesario comprobar las restricciones existentes en el lugar concreto donde se pretende realizar el vuelo. Una operación que, por sus características generales, pueda encuadrarse dentro de una

determinada categoría puede estar sometida a requisitos adicionales debido a la zona en la que se encuentre la instalación.

Para realizarse esta comprobación, como se adelantó anteriormente, puede utilizarse ENAIRE Drones, herramienta que muestra sobre un mapa la información aeronáutica relevante para las operaciones con UAS. A través de ella pueden consultarse, entre otras, zonas con prohibiciones o restricciones al vuelo, espacios aéreos controlados, proximidad a aeródromos y helipuertos y diferentes avisos que deben tenerse en cuenta antes de realizar la operación.

Antes de realizar cualquier inspección, es necesario comprobar el emplazamiento de la instalación y verificar si existen limitaciones adicionales al vuelo.

Dado que durante el desarrollo se ha utilizado un DJI Neo 2, pero el sistema final plantea un UAS de características diferentes, conviene distinguir los requisitos aplicables a cada uno de ellos:

Aspecto	DJI Neo 2	UAS diseñado
Masa aproximada	< 250 g	≈ 1,8 kg
Tipo de UAS	Comercial	Construcción privada
Categoría	A1	Abierta A3 / específica
Formación A1/A3	No obligatoria	Necesaria
Registro de operador	Aplicable (por cámara)	Necesario
Uso en el trabajo	Adquisición de imágenes y pruebas	Plataforma final propuesta para la inspección

Tabla 9.1. Comparación de los requisitos aplicables al DJI Neo 2 y al UAS diseñado.

Aunque la formación A1/A3 no era necesaria para realizar las pruebas con el DJI Neo 2, durante el desarrollo del trabajo se decidió completar el proceso al considerarse relevante para una futura utilización del UAS diseñado:

- Se realizó el registro como operador de UAS en AESA.
- Se completó la formación y el examen en línea correspondientes a A1/A3.

Este proceso permitió conocer de forma directa los requisitos básicos asociados a la operación de UAS, incluyendo aspectos relacionados con seguridad, limitaciones operacionales y responsabilidades del piloto. Permitiendo además conocer de forma práctica todo el proceso. (VER ANEXO II, ANEXO III)⁷

Otro aspecto que considerar es la necesidad de disponer de un seguro que cubra los posibles daños ocasionados durante la operación. De acuerdo con el Real Decreto 517/2024, las operaciones realizadas en categoría abierta A3 con UAS de masa inferior a 20 kg están exentas de esta obligación. Sin embargo, si las características de una inspección obligasen a realizarla dentro de la categoría específica, sería necesario disponer de la cobertura correspondiente.

⁷ Por confidencialidad, los archivos anexados quedan incompletos, ocultando con zonas grises los datos más sensibles.

9.2. Protección de datos y captación de imágenes

El sistema propuesto utiliza una cámara RGB para obtener las imágenes necesarias durante la inspección de los paneles fotovoltaicos. Aunque la finalidad de estas grabaciones no es captar información sobre personas, durante el vuelo podrían aparecer de forma accidental personas, vehículos, viviendas u otros elementos que permitan identificar a una persona física. En estos casos, las imágenes pasarían a estar sujetas a la normativa de protección de datos.

El tratamiento de este tipo de información se encuentra regulado principalmente por el Reglamento General de Protección de Datos, Reglamento (UE) 2016/679 (RGPD) [41], y por la Ley Orgánica 3/2018 de Protección de Datos Personales [42] y garantía de los derechos digitales.

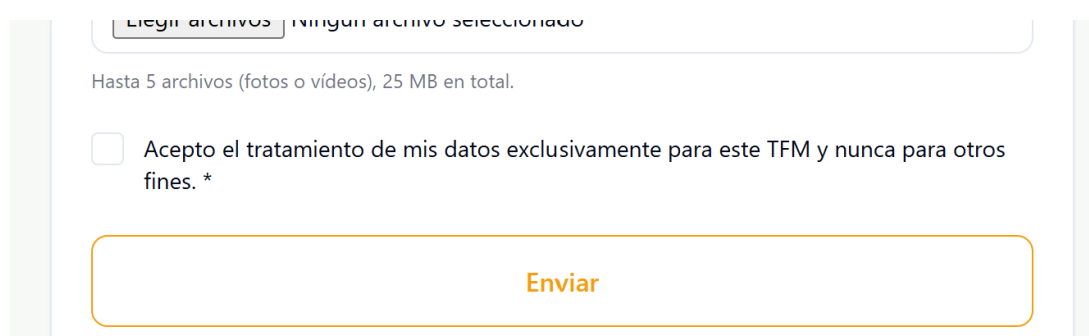
La Agencia Española de Protección de Datos (AEPD) dispone además de una guía específica sobre el uso de drones, en la que identifica las inspecciones de infraestructuras como operaciones cuya finalidad no implica inicialmente el tratamiento de datos personales, pero en las que estos pueden captarse de forma involuntaria.

En una futura utilización real del sistema, deberían adoptarse medidas para reducir al mínimo estas capturas. Entre ellas, por ejemplo:

- Planificar adecuadamente la trayectoria y orientación de la cámara (en Mission Planner se puede configurar la misión mediante distintos puntos a los que tiene que llegar el dron)
- Evitar la grabación innecesaria de personas o espacios privados
- Eliminar o anonimizar los datos personales que no sean necesarios
- Proteger el almacenamiento y la transmisión de las imágenes

En el caso desarrollado, las imágenes se utilizan exclusivamente para localizar posibles defectos sobre los módulos fotovoltaicos, por lo que cualquier información personal que pudiera aparecer en ellas sería ajena al objetivo de la inspección.

Como aplicación práctica de estos principios, el formulario utilizado para la aportación de imágenes al proyecto incorpora una aceptación expresa del tratamiento de los datos, indicando que estos se utilizarán exclusivamente para el desarrollo del TFM.



Elige archivos Ningún archivo seleccionado

Hasta 5 archivos (fotos o vídeos), 25 MB en total.

☐ Acepto el tratamiento de mis datos exclusivamente para este TFM y nunca para otros fines. *

Enviar

Figura 9. 2. Ejemplo de aceptación del tratamiento de datos incorporada al formulario de aportación de imágenes.

En conjunto, estas medidas permiten incorporar la protección de datos como un criterio adicional de diseño y operación del sistema de inspección.

9.3. Normativa y estándares relacionados con las instalaciones fotovoltaicas

El sistema desarrollado no interviene en el diseño ni en la puesta en servicio de la instalación fotovoltaica, sino que se plantea como una herramienta de apoyo a su inspección y mantenimiento una vez se encuentra en funcionamiento. Por este motivo, dentro de la normativa técnica aplicable resultan especialmente relevantes aquellas normas relacionadas con la inspección periódica y el mantenimiento de los sistemas fotovoltaicos.

Entre ellas destaca la serie UNE-EN IEC 62446, que establece requisitos relativos a los ensayos, documentación, inspección y mantenimiento de instalaciones fotovoltaicas. En particular interesan la UNE-EN 62446-1:2017 [44], que contempla la inspección de sistemas conectados a red, y la IEC 62446-2:2020 [45], que recoge aspectos asociados al mantenimiento preventivo y correctivo de este tipo de instalaciones.

El sistema de visión artificial propuesto se sitúa precisamente dentro de esta fase de mantenimiento, ya que busca localizar de forma automática los defectos más visibles. Su función no sería la de sustituir las inspecciones o procedimientos establecidos por estas normas, sino servir como herramienta complementaria que facilite la localización de zonas que puedan requerir una revisión posterior.

Cabe mencionar, también, la norma asociada a termografía infrarroja, IEC TS 62446-3:2017 [46], pero como finalmente queda fuera del alcance del trabajo, no se contempla.

9.4. Marco normativo relacionado con la IA

El uso de técnicas de inteligencia artificial dentro del sistema desarrollado hace necesario considerar también el Reglamento (UE) 2024/1689 [47][47], conocido como Reglamento Europeo de Inteligencia Artificial o *AI Act*. Esta normativa establece un marco común para el desarrollo y utilización de sistemas de IA dentro de la Unión Europea, siguiendo un enfoque basado en el nivel de riesgo asociado a cada aplicación. El Reglamento entró en vigor en 2024 y gran parte de sus disposiciones comenzaron a ser aplicables a partir del 2 de agosto de 2026.

Este Reglamento establece diferentes niveles de exigencia dependiendo del uso que se haga de la IA. Las obligaciones más estrictas se reservan para aquellos sistemas que puedan afectar de forma significativa a la seguridad, los derechos fundamentales o ámbitos especialmente sensibles.

El sistema desarrollado en este trabajo utiliza inteligencia artificial para analizar imágenes de módulos fotovoltaicos y localizar así los posibles defectos. Por su finalidad, no se identifica inicialmente con los usos considerados de alto riesgo por el Reglamento, ya que no toma decisiones sobre personas ni actúa directamente sobre elementos críticos de la instalación. Su resultado se plantea como información de apoyo para facilitar la inspección y el mantenimiento posterior.

No obstante, una futura comercialización o ampliación del sistema debería volver a analizarse atendiendo a su uso concreto y a las funciones que se incorporasen, especialmente si las detecciones llegasen a utilizarse para tomar decisiones automáticas sobre operaciones de la instalación.

10. CONCLUSIONES Y LÍNEAS FUTURAS

El desarrollo de este Trabajo de Fin de Máster ha permitido abordar de forma conjunta el diseño de una plataforma UAS, el desarrollo de un sistema de visión artificial y su posterior adaptación a una arquitectura *Edge-AI* capaz de ejecutar el modelo directamente a bordo del propio dron. Una de las principales conclusiones del trabajo es que el funcionamiento del sistema no depende únicamente de disponer de un buen modelo de inteligencia artificial, sino de que todos esos elementos sean compatibles entre sí y respondan a las necesidades reales de una operación de inspección.

En relación con la plataforma UAS, se ha obtenido un diseño completo que integra los sistemas de propulsión, alimentación, control de vuelo, comunicaciones, adquisición de imágenes y procesamiento. Sin embargo, la plataforma completa no llegó a construirse físicamente, principalmente por las limitaciones de recursos y coste del proyecto. Aun así, el proceso de diseño permitió conocer con bastante detalle las necesidades reales que tendría su construcción y detectar aspectos operativos que inicialmente no se habían considerado, como la conveniencia de disponer de baterías intercambiables para poder realizar varias pasadas de inspección con menores tiempos de espera.

Uno de los aprendizajes más importantes del trabajo está relacionado con el conjunto de datos. Durante el desarrollo se ha comprobado que entrenar un modelo de visión artificial no consiste únicamente en aumentar el número de imágenes, sino en disponer de ejemplos lo suficientemente variados, representativos y correctamente anotados. La ampliación realizada con nuevas imágenes reales produjo mejoras, pero también mostró que determinados problemas no se solucionaban simplemente incorporando más muestras. La revisión de las propias clases del problema fue necesaria para conseguir un detector más coherente con la información que realmente podía obtenerse mediante una cámara RGB.

En este sentido, se considera que la disponibilidad de datos continúa siendo una de las principales limitaciones del sistema desarrollado, y, al mismo tiempo, una de sus mayores oportunidades de mejora. El conjunto utilizado sigue siendo relativamente pequeño. Por ello, un dataset considerablemente más amplio y diverso podría permitir mejorar de forma importante la capacidad de generalización del detector. Los resultados obtenidos hacen pensar que existe todavía un margen considerable de mejora sin necesidad de modificar necesariamente la arquitectura del modelo, aunque esta hipótesis tendría que comprobarse experimentalmente.

El proceso seguido a través de los distintos experimentos también ha permitido entender mejor qué características son más importantes en un sistema destinado a inspección. El modelo finalmente seleccionado no fue simplemente el que alcanzó el mayor valor en una métrica aislada ni el que lo hizo en el máximo número de métricas posibles, sino el que ofrecía un mejor equilibrio y una mayor capacidad para localizar anomalías. En este tipo de aplicación, dejar un defecto real sin detectar puede resultar más perjudicial que generar una detección adicional que posteriormente pueda ser revisada. Esta decisión permitió orientar la selección del modelo hacia las necesidades reales del conjunto.

Otro de los resultados más relevantes ha sido comprobar que el despliegue del modelo sobre el hardware *Edge-AI* es una alternativa adecuada para este tipo de aplicación. La utilización de Hailo-8 redujo aproximadamente un 94% de la latencia respecto a otros modelos, reduciendo también considerablemente el uso de memoria. Más allá del valor obtenido, este resultado demuestra la ventaja de utilizar un acelerador cuando se pretende realizar procesamiento de IA a bordo de una plataforma con recursos más limitados. Esto evita que todo el sistema dependa de una conexión permanente con equipo externo.

La validación con vídeo aéreo real permitió dar un paso fuera del entorno utilizado durante el entrenamiento, evidenciando su capacidad de generalización. La detección de *soiling* fue posible en una secuencia capturada mediante dron, aunque con pérdidas puntuales de detección, por lo que el sistema debe considerarse todavía como una prueba de concepto.

Del mismo modo, las pruebas realizadas sobre la activación del proceso de inspección, el envío de avisos mediante *MAVLink* y la transmisión de vídeo mostraron que los principales mecanismos necesarios para la integración pueden funcionar de forma independiente. No obstante, será necesario disponer del UAS completo para validar todos estos elementos de forma conjunta durante una misión real.

Más allá de los resultados técnicos, el TFM ha supuesto también un aprendizaje práctico muy importante en el ámbito de los UAS. La posibilidad de asistir al club de aeromodelismo Alas de Galapagar permitió completar el trabajo realizado sobre documentación y diseño con la experiencia de personas habituadas al montaje y operación de aeronaves. Ver directamente diferentes montajes, comentar decisiones con personas con experiencia y observar cómo resuelven problemas cotidianos permitió entender aspectos que difícilmente se perciben trabajando únicamente sobre esquemas. Algunas decisiones incorporadas al proyecto surgieron posteriormente, precisamente gracias a este contacto con la práctica.

La experiencia en el club permitió adquirir mayor soltura en el manejo de drones y comprender mucho mejor su comportamiento durante el vuelo. Este aprendizaje ha sido especialmente valioso porque, al comienzo del trabajo, gran parte de los conocimientos estaban centrados en la parte de la visión artificial, mientras que el funcionamiento práctico de una plataforma aérea resultaba mucho menos familiar. El proyecto ha terminado siendo, por tanto, una oportunidad para aprender no solo sobre IA y sistemas embebidos, sino también sobre electrónica, montaje, operación de UAS y decisiones prácticas que aparecen cuando se intenta trasladar una solución desde el ordenador a un sistema físico real.

En conjunto, el trabajo realizado permite considerar viable el concepto propuesto, aunque todavía no pueda hablarse de un sistema de inspección completamente terminado, se ha conseguido:

- Desarrollar el modelo de detección
- Ejecutarlo sobre hardware *Edge-AI*
- Validarlo con material aéreo real
- Definir una arquitectura UAS capaz de integrarlo
- Explorar los mecanismos necesarios para su funcionamiento durante una misión

El siguiente paso natural sería unir todas estas partes en una plataforma física completa y someterla a pruebas de vuelo reales. El resultado obtenido constituye, por tanto, una base funcional sobre la que continuar desarrollando un sistema de inspección fotovoltaica cada vez más autónomo, robusto y aplicable a situaciones reales.

A partir de las limitaciones identificadas y de estos resultados, se plantean las siguientes líneas de continuación del trabajo:

- Construcción y validación física del UAS. El siguiente paso sería materializar la plataforma diseñada y comprobar en vuelo el comportamiento conjunto de todos sus subsistemas, incluyendo autonomía, estabilidad, comunicaciones y funcionamiento de la *payload*.
- Ampliación del conjunto de datos. Como se ha comentado en las conclusiones, sería especialmente conveniente incorporar un mayor (mucho mayor) número de imágenes reales procedentes de distintas instalaciones, condiciones de iluminación, alturas, ángulos de capturas y tipos de defectos.
- Validación con un mayor número de defectos. Las pruebas realizadas se centraron principalmente en *soiling*. Sería interesante ampliar la validación a casos reales de nieve, grietas y distintos tipos de suciedad.
- Integración completa durante una misión real. Una vez construido el UAS, podrían integrarse conjuntamente los mecanismos estudiados durante el proyecto: activación de la inspección, procesamiento a bordo, envío de avisos...
- Incorporación de información térmica. La utilización de una cámara térmica y el desarrollo de un modelo específico permitirían detectar defectos que no pueden identificarse de forma fiable únicamente con imágenes RGB. Sería muy interesante ampliar de esta forma las capacidades del sistema.
- Geolocalización y generación automática de informes. Como evolución del sistema, las detecciones podrían asociarse a la posición del dron y a módulos concretos de la instalación, permitiendo generar automáticamente un registro de anomalías que facilitase tareas de mantenimiento.

11. BIBLIOGRAFÍA

- [1] International Energy Agency Photovoltaic Power Systems Programme (IEA-PVPS), "Snapshot of Global PV Markets 2026," Report IEA-PVPS T1-47:2026, Abril 2026. [En línea]. Disponible en: <https://iea-pvps.org/snapshot-reports/snapshot-2026/>
- [2] *El Vuelo del Drone*, «Dron de ala fija Delair UX11: Características y especificaciones técnicas», 2024. [En línea]. Disponible en: <https://elvuelodeldrone.com/drones-profesionales/drones-industriales/drone-de-ala-fija-delair-ux11/>
- [3] *UAVforDrone*, «Multi-rotor drone (UAV): How it works & The working principle», 2024. [En línea]. Disponible en: <https://www.uavfordrone.com/es/multi-rotor-drone-uav-how-it-works-the-working-principle/>.
- [4] Agencia Estatal de Seguridad Aérea (AESA). *Guía sobre el marco regulatorio de UAS en categoría abierta y específica*. Ministerio de Transportes y Movilidad Sostenible, Gobierno de España, 2024. Disponible en: <https://www.seguridadaerea.gob.es/es/ambitos/drones>
- [5] Unión Europea. *Reglamento de Ejecución (UE) 2019/947 de la Comisión, de 24 de mayo de 2019, relativo a las normas y los procedimientos aplicables a la utilización de aeronaves no tripuladas*. Diario Oficial de la Unión Europea (DOUE), L 152, pp. 45–71, 2019. Disponible en: <https://www.easa.europa.eu/en/document-library/easy-access-rules/easy-access-rules-unmanned-aircraft-systems-regulations-eu>
- [6] AliExpress. "Frame Chasis Cuadricóptero F450 para dron". Disponible en: <https://es.aliexpress.com/item/1826445939.html>
- [7] KIMISS. "Juego de Tren de Aterrizaje Extendido para Multirrotor". [En línea]. Disponible en: <https://www.amazon.es/KIMISS-Aterrizaje-Extendido-Rendimiento-Controlados/dp/B0G8R18Y7C>
- [8] VSD Motor. "Is the drone motor clockwise or counterclockwise?". [En línea]. Disponible en: <https://es.vsdmotor.com/info/is-the-drone-motor-clockwise-or-counterclockwi-17461536531653632.html>
- [9] AliExpress. "Juego de motores sin escobillas Sunnysky X2212 III (980 KV) para multirrotor". [En línea]. Disponible en: <https://es.aliexpress.com/item/4000348341878.html>
- [10] AliExpress. "Juego de hélices de nylon 1045 (10x4,5") CW/CCW con adaptadores de eje para multirrotor". [En línea]. Disponible en: <https://es.aliexpress.com/item/1005005181944924.html>
- [11] AliExpress. "Juego de variadores electrónicos de velocidad (ESC 30A) con firmware BLHeli para multirrotor". [En línea]. Disponible en: <https://es.aliexpress.com/item/1005007219835308.html>
- [12] Amazon. "Ausi - Placa de distribución de potencia PDB Matek XT60 con doble regulador BEC (5V / 12V) para multirrotor". [En línea]. Disponible en: <https://www.amazon.es/dp/B0FMR53WTC>

- [13] RC Innovations. *"Controladora de vuelo Holybro Pixhawk 6C (Carcasa de plástico)"*. [En línea]. Disponible en: <https://rc-innovations.es/shop/11054-holybro-pixhawk-6c-carcasa-plastico-20014>
- [14] RC Innovations. *"Módulo GNSS secundario Holybro Micro M9N GPS (Conector JST-GHR)"*. [En línea]. Disponible en: <https://rc-innovations.es/shop/12029-holybro-m9n-gps-gnss-secundario-jst-ghr1-25mm-6pin-6499>
- [15] AliExpress. *"RadioMaster Boxer ELRS 2.4G"*. [En línea]. Disponible en: <https://es.aliexpress.com/item/1005010130183151.html>
- [16] Secretaría de Estado de Telecomunicaciones e Infraestructuras Digitales. *"Seguimiento de organismos de normalización técnica: ETSI (European Telecommunications Standards Institute)"*. Ministerio de Transformación Digital y de la Función Pública. [En línea]. Disponible en: <https://digital.gob.es/telecomunicaciones-infraestructuras-digitales/servicios/normalizacion-tecnica/seguimiento-organismos/seguimiento-etsi>
- [17] RadioMaster. *"RPI 2.4GHz ELRS Nano Receiver (LBT Specification)"*. [En línea]. Disponible en: <https://es.aliexpress.com/item/1005004720231554.html>
- [18] Holybro. *"SiK Telemetry Radio V3 433MHz Specification"*. [En línea]. Disponible en: <https://holybro.com/products/sik-telemetry-radio-v3>
- [19] Holybro, «Holybro Remote ID - Identificación remota de UAS,» RC-Innovations, 2026. [En línea]. Disponible en: <https://rc-innovations.es/shop/18089-holybro-remote-id-identificacion-remota-de-uas-6275>.
- [20] ReadyEDI. *"5200mAh 14.8V 35C UAV LiPo Battery Specifications"*. [En línea]. Disponible en: <https://readyedi.co.il/he-eu/products/readyedi-5200mah-14-8v-35c-uav-lipo-battery>
- [21] RC Innovations, "SkyRC e680 80W 8A AC/DC". [En línea]. Disponible en: <https://rc-innovations.es/shop/6930460005298-skyrc-e680-80w-8a-ac-dc-6152>
- [22] Matek Systems. *"UBEC Duo 4A/5A 5V & 12V Step-Down Converter Specifications"*. [En línea]. Disponible en: <https://www.amazon.es/Sistema-5V-12V-Quadcopter-Airplane-Multicopter/dp/B0CY2YHDZX>
- [23] Kubii, "Raspberry Pi 5 (8 GB RAM) – Ficha de producto y especificaciones técnicas," Kubii España, 2024. [En línea]. Disponible en: https://www.kubii.com/es/microordenadores/4106-1832-raspberry-pi-5-3272496315938.html#/ram-8_gb?src=raspberrypi.
- [24] Raspberry Pi Ltd., "Raspberry Pi Active Cooler – Página oficial de producto," Raspberry Pi Foundation, 2024. [En línea]. Disponible en: <https://www.raspberrypi.com/products/active-cooler/>.
- [25] Raspberry Pi, «Raspberry Pi AI Kit 13 TOPS Neural Network Accelerator (Hailo-8L)» Amazon España, 2026. [En línea]. Disponible en: <https://www.amazon.es/dp/B0DKTVTZ6L>.
- [26] AliExpress, "Cable de alimentación USB-C a 2 pines pelados (5V/3A, 24AWG) – Especificaciones de adquisición comercial," AliExpress España, 2024. [En línea]. Disponible en: <https://es.aliexpress.com/item/1005012297792906.html>.

- [27] Unmanned Tech. "Pixhawk UART Telemetry Cable - JST-GH 6-Pin to 3x Dupont (300mm)". Holybro / Unmanned Tech Shop. Disponible en: <https://www.unmannedtechshop.co.uk/products/pixhawk-uart-telemetry-cable-jst-gh-6-pin-to-3x-dupont-300mm>
- [28] AliExpress, "Módulo de cámara Raspberry Pi 5 de 12.3 MP con sensor IMX477 – Ficha de adquisición comercial," *AliExpress España*, 2024. [En línea]. Disponible en: <https://es.aliexpress.com/item/1005012297792906.html>.
- [29] ENAIRE, "ENAIRE Drones – Información geográfica para operaciones con UAS," ENAIRE. [En línea]. Disponible en: <https://drones.enaire.es/>.
- [30] Roboflow, "Roboflow: Computer Vision Platform," [En línea]. Disponible en: <https://roboflow.com/>.
- [31] Vercel, "Vercel," [En línea]. Disponible en: <https://vercel.com/>.
- [32] Cloudflare, "Cloudflare," [En línea]. Disponible en: <https://www.cloudflare.com/>.
- [33] Anthropic, "Claude Code," [En línea]. Disponible en: <https://docs.anthropic.com/en/docs/claude-code/getting-started>.
- [34] Alas de Galapagar, "Alas de Galapagar · Club de Aeromodelismo en Madrid," [En línea]. Disponible en: <https://alasdegalapagar.com/>.
- [35] Kaggle, "Kaggle: Your Machine Learning and Data Science Community," [En línea]. Disponible en: <https://www.kaggle.com/>.
- [36] Pexels, "Fotos y vídeos de stock gratuitos," [En línea]. Disponible en: <https://www.pexels.com/es-es/>.
- [37] Ultralytics, "Ultralytics YOLO Docs," [En línea]. Disponible en: <https://docs.ultralytics.com/>
- [38] Ultralytics, "Ultralytics YOLO11," [En línea]. Disponible en: <https://docs.ultralytics.com/models/yolo11/>
- [39] Google, "Google Colab," [En línea]. Disponible en: <https://colab.research.google.com/>.
- [40] ArduPilot, "Mission Planner," *ArduPilot Documentation*. [En línea]. Disponible en: <https://ardupilot.org/planner/>.
- [41] Parlamento Europeo y Consejo de la Unión Europea, "Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento general de protección de datos)," *Diario Oficial de la Unión Europea*, 2016. [En línea]. Disponible en: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/spa>.
- [42] Jefatura del Estado, "Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales," *Boletín Oficial del Estado*, núm. 294, 6 de diciembre de 2018. [En línea]. Disponible en: <https://www.boe.es/eli/es/lo/2018/12/05/3/con>.
- [43] Agencia Española de Protección de Datos (AEPD), "Drones y Protección de Datos," 2019. [En línea]. Disponible en: <https://www.aepd.es/documento/guia-drones.pdf>.

- [44] Asociación Española de Normalización (UNE), “UNE-EN 62446-1:2017. Sistemas fotovoltaicos (FV). Requisitos para ensayos, documentación y mantenimiento. Parte 1: Sistemas conectados a la red. Documentación, ensayos de puesta en marcha e inspección,” 2017. [En línea]. Disponible en: <https://tienda.aenor.com/es/p/norma-une-en-62446-1-2017-n0058027>.
- [45] International Electrotechnical Commission (IEC), “IEC 62446-2:2020. Photovoltaic (PV) systems – Requirements for testing, documentation and maintenance – Part 2: Grid connected systems – Maintenance of PV systems,” 2020. [En línea]. Disponible en: <https://webstore.iec.ch/en/publication/27382>
- [46] International Electrotechnical Commission (IEC), “IEC TS 62446-3:2017. Photovoltaic (PV) systems – Requirements for testing, documentation and maintenance – Part 3: Photovoltaic modules and plants – Outdoor infrared thermography,” 2017. [En línea]. Disponible en: <https://webstore.iec.ch/en/publication/28628>.
- [47] Parlamento Europeo y Consejo de la Unión Europea, “Reglamento (UE) 2024/1689, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial),” *Diario Oficial de la Unión Europea*, 2024. [En línea]. Disponible en: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/spa>.
- [48] Naciones Unidas, “Objetivos y metas de desarrollo sostenible – Agenda 2030,” *Naciones Unidas*. [En línea]. Disponible en: <https://www.un.org/sustainabledevelopment/es/sustainable-development-goals/>.

12. CONTRIBUCIÓN A LOS ODS

El sistema desarrollado en este trabajo presenta relación con varios de los Objetivos de Desarrollo Sostenible (ODS) definidos en la Agenda 2030 de Naciones Unidas [48]. Esta contribución se encuentra principalmente asociada al ámbito de las energías renovables, la aplicación de nuevas tecnologías al mantenimiento de infraestructuras y la mejora de herramientas destinadas a favorecer su funcionamiento.

ODS 7. Energía asequible y no contaminante.

El sistema propuesto está orientado a la inspección de instalaciones fotovoltaicas mediante la detección automática de posibles defectos. La identificación temprana de defectos o suciedad puede facilitar las tareas de mantenimiento y contribuir a mantener un funcionamiento adecuado de los módulos durante su vida útil.

ODS 9. Industria, innovación e infraestructura.

El proyecto combina tecnologías como UAS, visión artificial y el procesamiento *Edge-AI* para desarrollar una herramienta de apoyo a la inspección de infraestructuras energéticas.

Su utilización permitiría automatizar parcialmente una tarea que tradicionalmente requiere inspecciones manuales.

ODS 13. Acción por el clima.

El mantenimiento adecuado de las instalaciones fotovoltaicas favorece el aprovechamiento de fuentes de energía renovable. Aunque la contribución del sistema desarrollado es indirecta, su aplicación se enmarca en las tecnologías destinadas a facilitar la utilización y mantenimiento de sistemas de generación con bajas emisiones.



Figura 12. 1. Objetivos de Desarrollo Sostenible a los que contribuye principalmente el trabajo.

En conjunto, el sistema propuesto presenta impactos potencialmente positivos en términos de seguridad, eficiencia del mantenimiento y apoyo a la generación fotovoltaica.

De forma adicional, la aplicación del sistema podría contribuir indirectamente a otros objetivos dependiendo del uso de la instalación fotovoltaica. Por ejemplo, el mantenimiento de sistemas solares destinados al bombeo o abastecimiento de agua puede relacionarse con el **ODS 6. Agua limpia y saneamiento**. Mientras que su aplicación sobre instalaciones fotovoltaicas integradas en edificios e infraestructuras urbanas puede contribuir al **ODS 11. Ciudades y comunidades sostenibles**. No obstante, estas relaciones dependen del contexto concreto de utilización y no constituyen el objetivo principal del sistema desarrollado.



Figura 12. 2. Objetivos de Desarrollo Sostenible a los que contribuye indirectamente el trabajo.

13. ANÁLISIS DE IMPACTOS

La implantación de un sistema de inspección fotovoltaica mediante UAS y procesamiento *Edge-AI* puede producir efectos en distintos ámbitos. A continuación, se analizan los principales impactos asociados al sistema propuesto, considerando tanto su posible aplicación práctica como las características de la solución desarrollada.

Impacto social y sobre la seguridad:

La utilización de un UAS permite realizar parte de las tareas de inspección sin necesidad de que una persona acceda directamente a cubiertas, zonas elevadas o superficies de difícil acceso. Esto puede reducir la exposición del personal a determinadas situaciones de riesgo y facilitar la realización de inspecciones más frecuentes.

Impacto económico:

La automatización parcial de la instalación puede reducir el tiempo necesario para revisar instalaciones de cierta extensión y ayudar a localizar con mayor rapidez módulos que presentan posibles anomalías.

La detección temprana de defectos puede facilitar la planificación de las tareas de mantenimiento y evitar revisiones manuales innecesarias.

No obstante, en este trabajo no se realiza una estimación económica detallada del ahorro que supondría su implantación, ya que este dependería del tamaño de la instalación, la frecuencia de las inspecciones y los medios empleados actualmente.

Impacto medioambiental:

Al estar el sistema orientado al mantenimiento de instalaciones solares, su contribución medioambiental se produce de forma indirecta. La identificación de defectos puede favorecer un mejor aprovechamiento de la instalación durante su vida útil. Por otra parte, la solución requiere del uso de un UAS, baterías, dispositivos electrónicos como Raspberry Pi, Hailo... cuyo consumo energético y fabricación también presentan un impacto ambiental asociado.

Impacto tecnológico y operativo:

La incorporación de procesamiento *Edge-AI* permite realizar el análisis de las imágenes cerca del lugar en el que se generan, evitando depender completamente del envío de grandes cantidades de información a sistemas externos, pudiendo obtener detecciones durante la propia inspección. Al mismo tiempo, la incorporación de visión artificial, comunicaciones y hardware de procesamiento aumenta la complejidad respecto a una inspección convencional y requiere la correcta integración de los diferentes componentes.

En conclusión, el sistema propuesto presenta un impacto potencialmente positivo en términos generales, especialmente al facilitar la detección temprana de defectos y reducir la necesidad de realizar determinadas inspecciones de forma manual.

14. PLANIFICACIÓN TEMPORAL Y PRESUPUESTO

14.1. Planificación temporal

La realización del Trabajo Fin de Máster se desarrolló entre los meses de marzo y agosto del curso 2025-2026.

La planificación se organizó de forma progresiva, comenzando por el estudio del problema y el diseño de la plataforma UAS, para continuar posteriormente con la preparación del conjunto de datos, el desarrollo experimental del sistema de visión artificial, su integración sobre hardware y las pruebas de validación.

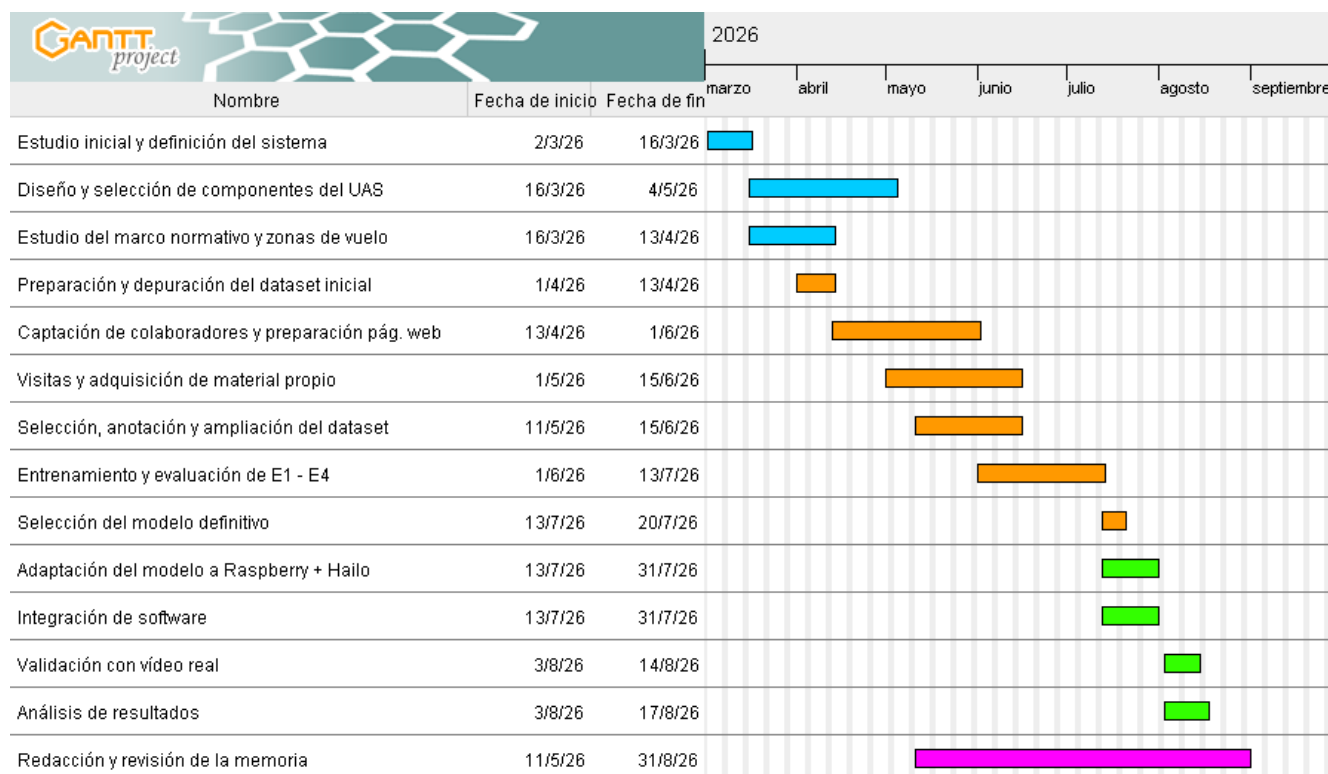


Figura 14.1. Diagrama de Gantt correspondiente a la planificación temporal del TFM. Fuente: Elaboración propia mediante GanttProject.

La planificación muestra un desarrollo parcialmente solapado entre las distintas fases, especialmente durante la preparación del dataset, el entrenamiento de los modelos y la integración *Edge-AI*. Este enfoque permitió ir adaptando las decisiones de diseño a medida que se obtenían nuevos resultados experimentales.

14.2. Presupuesto

El presupuesto del proyecto se ha elaborado diferenciando entre los componentes necesarios para materializar la plataforma UAS propuesta, los recursos materiales adicionales utilizados durante el desarrollo y las herramientas o servicios digitales empleados.

En el caso de la plataforma UAS, se contabiliza el valor de todos los elementos necesarios para su construcción, independientemente de que algunos de ellos estuvieran disponibles previamente durante la realización del TFM. De este modo, el presupuesto refleja el coste aproximado que tendría reproducir la solución diseñada.

Coste de personal

Se ha considerado conveniente realizar una estimación del coste que supondría el trabajo desarrollado en un entorno profesional, estimando una dedicación total de 400 horas por parte de la alumna, distribuidas entre las distintas partes del proyecto, y 30 horas correspondientes al seguimiento, revisión y orientación del tutor académico.

Considerando un coste de 20€/h para un perfil de ingeniería junior y de 40€/h para un perfil de ingeniería senior asociado a las tareas de tutorización.

Personal	Horas	Coste unitario	Coste Total
Ingeniero junior	400 h	20 €/h	8.000,00 €
Tutor académico	30 h	40 €/h	1.200,00 €
TOTAL	430 h	-	9.200,00 €

Tabla 14.1. Estimación del coste de recursos humanos asociados al proyecto.

La dedicación de la alumna se distribuye aproximadamente, atendiendo al diagrama de Gantt, entre las principales actividades desarrolladas durante el TFM:

Actividad	Horas estimadas
Estudio inicial y definición del sistema	30 h
Diseño del UAS y estudio normativo	70 h
Preparación, ampliación y gestión del dataset	90 h
Entrenamiento y evaluación de E1 – E4	80 h
Adaptación de integración <i>Edge-AI</i>	60 h
Validación y análisis de resultados	35 h
Redacción y revisión de la memoria	35 h
TOTAL	400 h

Tabla 14.2. Distribución aproximada de las horas dedicadas al desarrollo del TFM.

Componentes y equipamiento de la plataforma UAS

Para calcular el coste de materialización de la plataforma se han considerado todos los componentes necesarios para construir el UAS, independientemente de que algunos de ellos, como la Raspberry Pi 5 o el acelerador Hailo-8, estuvieran disponibles previamente.

De esta forma, el presupuesto representa el coste aproximado que supondría reproducir físicamente la solución diseñada.

Elemento	Ud.	Coste unitario	Coste Total
Chasis F450	x1	8,00 €	8,00 €
Tren de aterrizaje	x1	6,30 €	6,30 €
Motor SunnySky X2212 III 980 KV	x4	13,70 €	54,80 €
Hélice 1045	x4	2,13 €	4,26 €
ESC BLHeli 30A	x4	6,49 €	25,96 €
PDB XT60	x1	8,99 €	8,99 €
Pixhawk 6C Holybro	x1	192,00 €	192,00 €
Modulo Holybro M9N GPS	x1	70,00 €	70,00 €
RadioMaster Boxer LBT	x1	168,43 €	168,43 €
RadioMaster RP1 ELRS Nano Receiver	x1	20,99 €	20,99 €
Holybro Remote ID	x1	35,55 €	35,55 €
Módulo telemetría Holybro SiK 433 MHz	x1	52,00 €	52,00 €
Batería LiPo 4S 5200 mAh ReadyEDI	x2	98,63 €	197,26 €
Cargador SkyRc e680	x1	49,90 €	49,90 €
Matek UBEC 4A 5V	x1	60,29 €	60,29 €
Raspberry Pi 5	x1	204,00 €	204,00 €
Raspberry Pi 5 Active Cooler	x1	6,00 €	6,00 €
Hailo-8 HAT	x1	125,95 €	125,95 €
Raspberry Pi 5 IMX477 Camera	x1	106,00 €	106,00 €
Cableado, conectores, tornillería, estaño...	-		15,00 €
TOTAL	-		1411,68 €

Tabla 14.3. Coste de componentes y equipamiento del UAS

Amortización de equipos utilizados durante el desarrollo

Además de los componentes que formarían parte de la plataforma final, durante el desarrollo se utilizaron otros equipos que ya se encontraban disponibles previamente. Al no haber sido adquiridos específicamente para el presente trabajo, se ha considerado únicamente la parte de su coste correspondiente al periodo de utilización en el proyecto.

Para ello se adopta una vida útil estimada de cinco años y un periodo de uso imputable al TFM de seis meses (Ver Diagrama de Gantt). El coste correspondiente se calcula de acuerdo con la proporción entre el tiempo de utilización y la vida útil estimada del equipo.

Equipo	Coste de adquisición	Vida útil	Uso imputable al TFM	Coste imputable
HP Pavilion Plus	849,00 €	5 años	6 meses	84,90 €
DJI Neo 2	399,00 €	5 años	6 meses	39,90 €
TOTAL	--	--	--	124,80 €

Tabla 14.4. Amortización de los principales equipos utilizados durante el desarrollo.

Software, servicios y otros recursos

Una parte importante de las herramientas empleadas durante el proyecto pudo utilizarse sin coste gracias al uso de software libre, versiones gratuitas o recursos académicos. La única contratación digital realizada específicamente para el desarrollo fue la adquisición del dominio utilizado para la página web.

Recurso / servicio	Utilización	Coste imputado
Dominio (luciagorostidi.com)	Página web para captación de colaboradores	10,00 €
Roboflow	Gestión, anotación y versionado del dataset	0,00 €
Google Colab	Entrenamiento de los modelos	0,00 €
Mission Planner + ArduPilot SITL	Simulación y pruebas de integración	0,00 €
GanttProject	Elaboración de la planificación temporal	0,00 €
Cloudflare	Pruebas de acceso remoto y <i>streaming</i>	0,00 €
Python y librerías empleadas	Desarrollo del sistema de IA e integración	0,00 €
Club de aeromodelismo Alas de Galapagar	Aprendizaje práctico y apoyo durante el desarrollo	0,00 €
TOTAL	--	10,00 €

Tabla 14.5. Software, servicios y recursos adicionales utilizados durante el proyecto.

Resumen del presupuesto

A partir de las partidas anteriores, el presupuesto estimado del proyecto queda resumido de la siguiente forma:

Partida	Coste
Recursos humanos	9.200,00 €
Componentes y equipamiento del UAS	1.411,68 €
Amortización de equipos utilizados	124,80 €
Software y servicios con coste	10,00 €
PRESUPUESTO TOTAL ESTIMADO	10.746,48 €

Tabla 14.6. Presupuesto total estimado del proyecto.

ANEXO I. CÓDIGO DE ENTRENAMIENTOS EN GOOGLE COLAB⁸

⁸ Además del código para los entrenamientos de los modelos, en Google Colab, se aporta el acceso al GitHub con el resto del código para el test de las frutas en local y la inferencia del modelo en Edge-AI: https://github.com/luciagorostidi/solar_panels_defects

```
!nvidia-smi
```

[Mostrar salida oculta](#)

Insertar celda de código debajo (Ctrl+M B)

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch: 2.11.0+cu128
CUDA disponible: True
GPU: Tesla T4
```

```
!pip install -U ultralytics
```

[Mostrar salida oculta](#)

```
from ultralytics import YOLO
import ultralytics

print("Versión de Ultralytics:", ultralytics.__version__)
print("YOLO cargado correctamente.")
```

[Mostrar salida oculta](#)

```
!pip install -q roboflow
```

[Mostrar salida oculta](#)

```
from google.colab import userdata
import roboflow

baseline_version = 2

#para recuperar la API Key del baseline en roboflow que se ha guardado en Colab
api_key = userdata.get("ROBOFLOW_API_KEY")

#conectar con roboflow
rf = roboflow.Roboflow(api_key=api_key)

#acceder al proyecto
project = rf.workspace("luca-gorostidi-garca-s-workspace").project("solar-panels-cyg5d")

#seleccionar la versión del dataset "baseline"
version = project.version(baseline_version)

dataset = version.download("yolov11")

print("Dataset descargado correctamente")
print("Ubicación:", dataset.location)
```

[Mostrar salida oculta](#)

```
print("Data set descargado correctamente")
print("Ubicación:", dataset.location)
```

```
Data set descargado correctamente
Ubicación: /content/Solar-panels-2
```

```
import os
import yaml

dataset_path = dataset.location
yaml_path = os.path.join(dataset_path, "data.yaml")

#leer archivo data.yaml
with open(yaml_path, "r") as f:
    data_config = yaml.safe_load(f)

print("=== CONTENIDO DE data.yaml ===")
print(data_config)

#ver cuántas imágenes hay en train,valid y test
```



```
for split in ["train","valid","test"]:
    images_path = os.path.join(dataset_path,split,"images")

    if os.path.exists(images_path):
        n_images = len([
            f for f in os.listdir(images_path)
            if f.endswith((".jpg", ".jpeg", ".png", ".webp"))
        ])
        print(f"{split}: {n_images} imágenes")
    else:
        print(f"{split}: carpeta no encontrada")

=== CONTENIDO DE data.yaml ===
{'train': '../train/images', 'val': '../valid/images', 'test': '../test/images', 'nc': 6, 'names': ['brid_droppings', 'crack', 'dust', 'electro', 'snow', 'thermal_anomaly']}
train: 507 imágenes
valid: 145 imágenes
test: 72 imágenes
```

```
print("Número de clases:", data_config["nc"])
print("\nClases del dataset:")

for i, clase in enumerate(data_config["names"]):
    print(f"{i}: {clase}")

Número de clases: 6

Clases del dataset:
0: brid_droppings
1: crack
2: dust
3: electro
4: snow
5: thermal_anomaly
```

```
!pip show ultralytics

Mostrar salida oculta
```

```
!pip install -q -U ultralytics
```

```
import ultralytics
print("Versión:", ultralytics.__version__)

Versión: 8.4.129
```

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")

print("Modelo YOLO11n cargado correctamente.")

Modelo YOLO11n cargado correctamente.
```

```
!pip show ultralytics

Mostrar salida oculta
```

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

```
import os

ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"

print("Carpeta encontrada:", os.path.exists(ruta_experimentos))

Carpeta encontrada: True
```

```
results = model.train(
    data=yaml_path,
    epochs=100,
    patience=20,
    imgsz=640,
    batch=16,
    device=0,
    project="/content/drive/MyDrive/TFM/Experimentos",
```

```

name="E1_baseline_100ep_pat20_yolo11n",
seed=42,
deterministic=True,
plots=True
)

```

Insertar celda de código debajo (Ctrl+M B)

```

import pandas as pd

ruta_csv = "/content/drive/MyDrive/TFM/Experimentos/E1_baseline_100ep_pat20_yolo11n/results.csv"

df = pd.read_csv(ruta_csv)

print(df.head())
print(df.columns)

```

```

epoch      time  train/box_loss  train/cls_loss  train/df1_loss  \
0         1  39.0784         1.14434         3.82704         1.60420
1         2  48.6871         1.18216         3.43633         1.59203
2         3  59.1184         1.28760         3.12123         1.67081
3         4  67.5192         1.25607         2.93109         1.66193
4         5  78.4964         1.23056         2.89854         1.66146

metrics/precision(B)  metrics/recall(B)  metrics/mAP50(B)  \
0                0.00333                0.52339                0.06878
1                0.00167                0.58049                0.04689
2                0.38007                0.07997                0.07970
3                0.40891                0.19812                0.10103
4                0.30563                0.26864                0.11497

metrics/mAP50-95(B)  val/box_loss  val/cls_loss  val/df1_loss  lr/pg0  \
0                0.04056         1.37518         7.06844         1.80421  0.000323
1                0.01244         2.06196         7.35875         2.67684  0.000650
2                0.02313         2.17143         9.81495         2.89742  0.000970
3                0.03759         1.87998         6.70437         2.44036  0.000970
4                0.03653         2.39540         8.52561         2.78328  0.000960

lr/pg1  lr/pg2
0  0.000323  0.000323
1  0.000650  0.000650
2  0.000970  0.000970
3  0.000970  0.000970
4  0.000960  0.000960
Index(['epoch', 'time', 'train/box_loss', 'train/cls_loss', 'train/df1_loss',
      'metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)',
      'metrics/mAP50-95(B)', 'val/box_loss', 'val/cls_loss', 'val/df1_loss',
      'lr/pg0', 'lr/pg1', 'lr/pg2'],
      dtype='object')

```

```

# Mejor época según mAP50-95
idx_best = df["metrics/mAP50-95(B)"].idxmax()
best = df.loc[idx_best]

print("Épocas completadas:", len(df))
print("Mejor época:", int(best["epoch"]))
print("Precision:", round(best["metrics/precision(B)"], 4))
print("Recall:", round(best["metrics/recall(B)"], 4))
print("mAP50:", round(best["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(best["metrics/mAP50-95(B)"], 4))

print("\nMáximo mAP50 alcanzado:",
      round(df["metrics/mAP50(B)"].max(), 4),
      "en la época",
      int(df.loc[df["metrics/mAP50(B)"].idxmax(), "epoch"]))

```

```

Épocas completadas: 81
Mejor época: 61
Precision: 0.3728
Recall: 0.4514
mAP50: 0.3958
mAP50-95: 0.2363

```

```
Máximo mAP50 alcanzado: 0.3958 en la época 61
```

```

last = df.iloc[-1]

print("Última época:", int(last["epoch"]))
print("Precision:", round(last["metrics/precision(B)"], 4))
print("Recall:", round(last["metrics/recall(B)"], 4))
print("mAP50:", round(last["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(last["metrics/mAP50-95(B)"], 4))

```

Última época: 81
Precision: 0.35
Recall: 0.4105
mAP50: 0.3346
mAP50-95: 0.2258

Insertar celda de código debajo (Ctrl+M B)

```
from ultralytics import YOLO

best_model = YOLO(
    "/content/drive/MyDrive/TFM/Experimentos/E1_baseline_100ep_pat20_yolo11n/weights/best.pt"
)

test_results = best_model.val(
    data=yaml_path,
    split="test",
    device=0
)
```

Ultralytics 8.4.129  Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,322 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access  (ping: 0.0±0.0 ms, read: 1709.6±490.5 MB/s, size: 73.5 KB)
val: Scanning /content/Solar-panels-2/test/labels... 72 images, 22 backgrounds, 0 corrupt: 100%  72/72 2.4Kit/s
val: New cache created: /content/Solar-panels-2/test/labels.cache

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	
all	72	65	0.366	0.457	0.429	0.315	
brid_droppings	13	15	0.376	0.6	0.417	0.334	
crack	5	6	0.198	0.167	0.191	0.174	
dust	11	11	0.38	0.636	0.669	0.56	
electro	6	9	0.113	0.111	0.036	0.00839	
snow	12	12	0.728	0.894	0.875	0.68	
thermal_anomaly	6	12	0.403	0.333	0.385	0.135	

Speed: 10.2ms preprocess, 6.4ms inference, 0.0ms loss, 2.5ms postprocess per image
Results saved to /content/runs/detect/val

```
test_results = best_model.val(
    data=yaml_path,
    split="test",
    device=0,
    plots=True,
    project="/content/drive/MyDrive/TFM/Experimentos/E1_baseline_100ep_pat20_yolo11n",
    name="test"
)
```

Ultralytics 8.4.129  Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
val: Fast image access  (ping: 0.0±0.0 ms, read: 1860.9±564.8 MB/s, size: 62.5 KB)
val: Scanning /content/Solar-panels-2/test/labels.cache... 72 images, 22 backgrounds, 0 corrupt: 100%  72/72 20.
val: New cache created: /content/Solar-panels-2/test/labels.cache

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	
all	72	65	0.366	0.457	0.429	0.315	
brid_droppings	13	15	0.376	0.6	0.417	0.334	
crack	5	6	0.198	0.167	0.191	0.174	
dust	11	11	0.38	0.636	0.669	0.56	
electro	6	9	0.113	0.111	0.036	0.00839	
snow	12	12	0.728	0.894	0.875	0.68	
thermal_anomaly	6	12	0.403	0.333	0.385	0.135	

Speed: 7.3ms preprocess, 5.4ms inference, 0.0ms loss, 2.6ms postprocess per image
Results saved to /content/drive/MyDrive/TFM/Experimentos/E1_baseline_100ep_pat20_yolo11n/test

Una vez terminado el entrenamiento, cambio el entorno de ejecución a CPU para no gastar más sesión en GPU.

El problema es que, tras cambiarlo, me voy cuenta de que no he registrado el tiempo de entrenamiento. No pasa nada, con la propia CPU vuelvo a conectarme con drive y lo saco:

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd

ruta_csv = "/content/drive/MyDrive/TFM/Experimentos/E1_baseline_100ep_pat20_yolo11n/results.csv"
df = pd.read_csv(ruta_csv)

tiempo_segundos = df["time"].iloc[-1]

horas = int(tiempo_segundos // 3600)
minutos = int((tiempo_segundos % 3600) // 60)
segundos = tiempo_segundos % 60

print(f"Tiempo total: {horas} h {minutos} min {segundos:.1f} s")
```

Tiempo total: 0 h 14 min 21.9 s

Empieza el experimento 2. Se mantiene la misma configuración experimental utilizada en E1, solamente se modifica la versión del dataset empleada para el entrenamiento. Esto quiere decir que se pueden reutilizar muchas celdas de la sesión, no se copian de nuevo, Insertar celda de código debajo (Ctrl+M B) :ar.

Lo único que hay que modificar es la versión del dataset.

E1 → version(2) → baseline_clean E2 → version(3) → E2_expanded_dataset

```
from google.colab import userdata
import roboflow

e2_version = 3

# recuperar la misma API Key guardada en Colab
api_key = userdata.get("ROBOFLOW_API_KEY")

# conectar con Roboflow
rf = roboflow.Roboflow(api_key=api_key)

# acceder al mismo proyecto
project = rf.workspace("luca-gorostidi-garca-s-workspace").project("solar-panels-cyg5d")

# seleccionar la versión E2
version = project.version(e2_version)

dataset = version.download("yolov11")

print("Dataset E2 descargado correctamente")
print("Ubicación:", dataset.location)
```

```
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Solar-panels-3 to yolov11:: 100%|██████████| 66934/66934 [00:02<00:00, 24699.59it/s]

Extracting Dataset Version Zip to Solar-panels-3 in yolov11:: 100%|██████████| 1783/1783 [00:00<00:00, 5829.07it/s]Dataset E2
Ubicación: /content/Solar-panels-3
```

```
print("Data set descargado correctamente")
print("Ubicación:", dataset.location)
```

```
Data set descargado correctamente
Ubicación: /content/Solar-panels-3
```

```
import os
import yaml

dataset_path = dataset.location
yaml_path = os.path.join(dataset_path, "data.yaml")

#leer archivo data.yaml
with open(yaml_path, "r") as f:
    data_config = yaml.safe_load(f)

print("=== CONTENIDO DE data.yaml ===")
print(data_config)

#ver cuántas imágenes hay en train,valid y test
for split in ["train","valid","test"]:
    images_path = os.path.join(dataset_path,split,"images")

    if os.path.exists(images_path):
        n_images = len([
            f for f in os.listdir(images_path)
            if f.lower().endswith((".jpg",".jpeg",".png",".webp"))
        ])
        print(f"{split}: {n_images} imágenes")
    else:
        print(f"{split}: carpeta no encontrada")
```

```
=== CONTENIDO DE data.yaml ===
{'train': '../train/images', 'val': '../valid/images', 'test': '../test/images', 'nc': 6, 'names': ['brid_droppings', 'crack']}
train: 672 imágenes
valid: 145 imágenes
test: 72 imágenes
```

```
!pip show ultralytics
```

```
Name: ultralytics
Version: 8.4.129
Summary: Ultralytics YOLO 🚀 for SOTA object detection, multi-object tracking, instance segmentation, pose estimation, classification and video analysis
Home-page: https://ultralytics.com
Author-email: Glenn Jocher <glenn.jocher@ultralytics.com>, Jing Qiu <jing.qiu@ultralytics.com>
License: AGPL-3.0
Location: /usr/local/lib/python3.13/dist-packages
Requires: filelock, matplotlib, numpy, nvidia-ml-py, opencv-python, pillow, polars, psutil, pyyaml, requests, torch, torchvision
Required-by:
```

Insertar celda de código debajo (Ctrl+M B)

```
!pip install -q -U ultralytics
```

```
import ultralytics
print("Versión:", ultralytics.__version__)
```

```
Versión: 8.4.129
```

Para E2 se carga otro YOLO11N preentrenado desde cero respecto a E1.

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")

print("Modelo YOLO11n cargado correctamente.")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% — 5.4MB
Modelo YOLO11n cargado correctamente.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os

ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"

print("Carpeta encontrada:", os.path.exists(ruta_experimentos))
```

```
Carpeta encontrada: True
```

```
results = model.train(
    data=yaml_path,
    epochs=100,
    patience=20,
    imgsz=640,
    batch=16,
    device=0,
    project="/content/drive/MyDrive/TFM/Experimentos",
    name="E2_extended_100ep_pat20_yolo11n",
    seed=42,
    deterministic=True,
    plots=True
)
```

Mostrar salida oculta

```
import os

ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"

for carpeta in os.listdir(ruta_experimentos):
    print(carpeta)
```

```
E1_baseline_100ep_pat20_yolo11n
E2_extended_100ep_pat20_yolo11n
```

```
import pandas as pd

ruta_csv = "/content/drive/MyDrive/TFM/Experimentos/E2_extended_100ep_pat20_yolo11n/results.csv"

df_e2 = pd.read_csv(ruta_csv)

idx_best = df_e2["metrics/mAP50-95(B)"].idxmax()
best_e2 = df_e2.loc[idx_best]
last_e2 = df_e2.iloc[-1]
```

```
print("Épocas completadas:", len(df_e2))

print("\n--- MEJOR ÉPOCA ---")
print("Época:", int(best_e2["epoch"]))
print("Precision:", round(best_e2["metrics/precision(B)"], 4))
print("Recall:", round(best_e2["metrics/recall(B)"], 4))
print("mAP50:", round(best_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(best_e2["metrics/mAP50-95(B)"], 4))

print("\nMáximo mAP50:",
      round(df_e2["metrics/mAP50(B)"].max(), 4),
      "en época",
      int(df_e2.loc[df_e2["metrics/mAP50(B)"].idxmax(), "epoch"]))

print("\n--- ÚLTIMA ÉPOCA ---")
print("Época:", int(last_e2["epoch"]))
print("Precision:", round(last_e2["metrics/precision(B)"], 4))
print("Recall:", round(last_e2["metrics/recall(B)"], 4))
print("mAP50:", round(last_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(last_e2["metrics/mAP50-95(B)"], 4))

tiempo = df_e2["time"].iloc[-1]
print("\nTiempo total:",
      int(tiempo // 60), "min",
      round(tiempo % 60, 1), "s")
```

Épocas completadas: 99

--- MEJOR ÉPOCA ---
Época: 79
Precision: 0.3755
Recall: 0.4736
mAP50: 0.4147
mAP50-95: 0.2802

Máximo mAP50: 0.4147 en época 79

--- ÚLTIMA ÉPOCA ---
Época: 99
Precision: 0.3524
Recall: 0.4363
mAP50: 0.3515
mAP50-95: 0.2372

Tiempo total: 22 min 21.5 s

```
from ultralytics import YOLO

best_model_e2 = YOLO(
    "/content/drive/MyDrive/TFM/Experimentos/E2_extended_100ep_pat20_yolo11n/weights/best.pt"
)

test_results_e2 = best_model_e2.val(
    data=yaml_path,
    split="test",
    device=0,
    plots=True,
    project="/content/drive/MyDrive/TFM/Experimentos/E2_extended_100ep_pat20_yolo11n",
    name="test"
)
```

Ultralytics 8.4.129 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,322 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1466.1±405.1 MB/s, size: 63.0 KB)
val: Scanning /content/Solar-panels-3/test/labels... 72 images, 22 backgrounds, 0 corrupt: 100% ————— 72/72 2.1Kit/s
val: New cache created: /content/Solar-panels-3/test/labels.cache

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	72	65	0.39	0.481	0.431	0.319
brid_droppings	13	15	0.373	0.467	0.456	0.358
crack	5	6	0.272	0.167	0.138	0.103
dust	11	11	0.43	0.727	0.64	0.551
electro	6	9	0.154	0.111	0.108	0.0481
snow	12	12	0.702	0.917	0.915	0.743
thermal_anomaly	6	12	0.41	0.5	0.326	0.11

Speed: 17.5ms preprocess, 14.5ms inference, 0.2ms loss, 4.8ms postprocess per image
Results saved to /content/drive/MyDrive/TFM/Experimentos/E2_extended_100ep_pat20_yolo11n/test

Experimento E3 - Revised Classes

En este experimento se utiliza la versión E3_revised_classes, obtenida a partir de la revisión de los resultados de E1 y E2. El dataset queda reducido a tres clases: soiling, crack y snow, tras fusionar dust y bird_droppings y eliminar electro y thermal_anomaly.


```
from google.colab import userdata
import roboflow
```

```
e3_version = 4
```

Insertar celda de código debajo (Ctrl+M B)

```
rf = roboflow.Roboflow(api_key=api_key)
```

```
project = rf.workspace(
    "luca-gorostidi-garca-s-workspace"
).project("solar-panels-cyg5d")
```

```
version = project.version(e3_version)
```

```
dataset = version.download("yolov11")
```

```
print("Dataset E3 descargado correctamente")
print("Ubicación:", dataset.location)
```

```
loading Roboflow workspace...
loading Roboflow project...
Exporting format yolov11 in progress : 0.1%
Version export complete for yolov11 format
Downloading Dataset Version Zip in Solar-panels-4 to yolov11:: 100%|██████████| 60458/60458 [00:01<00:00, 54267.13it/s]
```

```
Extracting Dataset Version Zip to Solar-panels-4 in yolov11:: 100%|██████████| 1519/1519 [00:00<00:00, 5638.83it/s]Dataset E3
Ubicación: /content/Solar-panels-4
```

```
print("Data set descargado correctamente")
print("Ubicación:", dataset.location)
```

```
Data set descargado correctamente
Ubicación: /content/Solar-panels-4
```

```
import os
import yaml
```

```
dataset_path = dataset.location
yaml_path = os.path.join(dataset_path, "data.yaml")
```

```
#leer archivo data.yaml
with open(yaml_path, "r") as f:
    data_config = yaml.safe_load(f)
```

```
print("=== CONTENIDO DE data.yaml ===")
print(data_config)
```

```
#ver cuántas imágenes hay en train,valid y test
for split in ["train","valid","test"]:
    images_path = os.path.join(dataset_path,split,"images")
```

```
    if os.path.exists(images_path):
        n_images = len([
            f for f in os.listdir(images_path)
            if f.lower().endswith((".jpg", ".jpeg", ".png", ".webp"))
        ])
        print(f"{split}: {n_images} imágenes")
    else:
        print(f"{split}: carpeta no encontrada")
```

```
=== CONTENIDO DE data.yaml ===
{'train': '../train/images', 'val': '../valid/images', 'test': '../test/images', 'nc': 3, 'names': ['crack', 'snow', 'soilir
train: 573 imágenes
valid: 122 imágenes
test: 62 imágenes
```

```
!pip show ultralytics
```

Mostrar salida oculta

```
!pip install -q -U ultralytics
```

```
import ultralytics
print("Versión:", ultralytics.__version__)
```

```
Versión: 8.4.129
```

Se carga otra versión de YOLO11n preentrenado

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
```

Insertar celda de código debajo (Ctrl+M B)

```
print("Modelo YOLO11n cargado correctamente.")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt> to 'yolo11n.pt': 100% — 5.4
Modelo YOLO11n cargado correctamente.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os
```

```
ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"
```

```
print("Carpeta encontrada:", os.path.exists(ruta_experimentos))
```

Carpeta encontrada: True

```
results = model.train(
    data=yaml_path,
    epochs=100,
    patience=20,
    imgsz=640,
    batch=16,
    device=0,
    project="/content/drive/MyDrive/TFM/Experimentos",
    name="E3_revised_100ep_pat20_yolo11n",
    seed=42,
    deterministic=True,
    plots=True
)
```

Mostrar salida oculta

```
import os
```

```
ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"
```

```
for carpeta in os.listdir(ruta_experimentos):
    print(carpeta)
```

E1_baseline_100ep_pat20_yolo11n
E2_extended_100ep_pat20_yolo11n
E3_revised_100ep_pat20_yolo11n

```
import pandas as pd
```

```
ruta_csv = "/content/drive/MyDrive/TFM/Experimentos/E3_revised_100ep_pat20_yolo11n/results.csv"
```

```
df_e2 = pd.read_csv(ruta_csv)
```

```
idx_best = df_e2["metrics/mAP50-95(B)"].idxmax()
best_e2 = df_e2.loc[idx_best]
last_e2 = df_e2.iloc[-1]
```

```
print("Épocas completadas:", len(df_e2))
```

```
print("\n--- MEJOR ÉPOCA ---")
print("Época:", int(best_e2["epoch"]))
print("Precision:", round(best_e2["metrics/precision(B)"], 4))
print("Recall:", round(best_e2["metrics/recall(B)"], 4))
print("mAP50:", round(best_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(best_e2["metrics/mAP50-95(B)"], 4))
```

```
print("\nMáximo mAP50:",
      round(df_e2["metrics/mAP50(B)"].max(), 4),
      "en época",
      int(df_e2.loc[df_e2["metrics/mAP50(B)"].idxmax(), "epoch"])
```

```
print("\n--- ÚLTIMA ÉPOCA ---")
print("Época:", int(last_e2["epoch"]))
print("Precision:", round(last_e2["metrics/precision(B)"], 4))
print("Recall:", round(last_e2["metrics/recall(B)"], 4))
```

```
print("mAP50:", round(last_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(last_e2["metrics/mAP50-95(B)"], 4))
```

```
tiempo = df_e2["time"].iloc[-1]
print("\nTiempo total:",
      int(tiempo // 60), "min",
      int(tiempo % 60), "s")
```

Insertar celda de código debajo (Ctrl+M B)

Épocas completadas: 81

--- MEJOR ÉPOCA ---

Época: 61
Precision: 0.5148
Recall: 0.4298
mAP50: 0.4272
mAP50-95: 0.3137

Máximo mAP50: 0.4426 en época 69

--- ÚLTIMA ÉPOCA ---

Época: 81
Precision: 0.5032
Recall: 0.39
mAP50: 0.409
mAP50-95: 0.2774

Tiempo total: 15 min 31.4 s

```
from ultralytics import YOLO
```

```
best_model_e2 = YOLO(
    "/content/drive/MyDrive/TFM/Experimentos/E3_revised_100ep_pat20_yolo11n/weights/best.pt"
)
```

```
test_results_e2 = best_model_e2.val(
    data=yaml_path,
    split="test",
    device=0,
    plots=True,
    project="/content/drive/MyDrive/TFM/Experimentos/E3_revised_100ep_pat20_yolo11n",
    name="test"
)
```

Ultralytics 8.4.129 🚀 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,582,737 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1513.3±272.9 MB/s, size: 88.6 KB)
val: Scanning /content/Solar-panels-4/test/labels... 62 images, 22 backgrounds, 0 corrupt: 100% ————— 62/62 2.2Kit/s
val: New cache created: /content/Solar-panels-4/test/labels.cache

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	62	45	0.693	0.466	0.585	0.449
crack	5	7	0.411	0.143	0.244	0.165
snow	12	12	0.882	0.833	0.924	0.701
soiling	23	26	0.786	0.423	0.586	0.48

Speed: 11.5ms preprocess, 11.2ms inference, 0.0ms loss, 5.1ms postprocess per image
Results saved to /content/drive/MyDrive/TFM/Experimentos/E3_revised_100ep_pat20_yolo11n/test

Experimento E4 – Augmented Dataset

E4 parte de la misma estructura de clases definida en E3 y conserva sin cambios los conjuntos de validation y test. La diferencia se introduce exclusivamente en el conjunto de entrenamiento, que se amplía mediante técnicas de data augmentation aplicadas en Roboflow.

```
from google.colab import userdata
import roboflow

e4_version = 5

api_key = userdata.get("ROBOFLOW_API_KEY")

rf = roboflow.Roboflow(api_key=api_key)

project = rf.workspace(
    "luca-gorostidi-garca-s-workspace"
).project("solar-panels-cyg5d")

version = project.version(e4_version)

dataset = version.download("yolo11n")

print("Dataset E4 descargado correctamente")
print("Ubicación:", dataset.location)
```

```
loading Roboflow workspace...
loading Roboflow project...
Exporting format yolov11 in progress : 60.0%
Version export complete for yolov11 format
Downloading Dataset Version Zip in Solar-panels-5 to yolov11:: 100%|██████████| 113310/113310 [00:01<00:00, 65236.97it/s]
```

Insertar celda de código debajo (Ctrl+M B) | Zip to Solar-panels-5 in yolov11:: 100%|██████████| 3811/3811 [00:00<00:00, 5528.69it/s]

```
Dataset E4 descargado correctamente
Ubicación: /content/Solar-panels-5
```

```
import os
import yaml

dataset_path = dataset.location
yaml_path = os.path.join(dataset_path, "data.yaml")

#leer archivo data.yaml
with open(yaml_path, "r") as f:
    data_config = yaml.safe_load(f)

print("=== CONTENIDO DE data.yaml ===")
print(data_config)

#ver cuántas imágenes hay en train,valid y test
for split in ["train","valid","test"]:
    images_path = os.path.join(dataset_path,split,"images")

    if os.path.exists(images_path):
        n_images = len([
            f for f in os.listdir(images_path)
            if f.lower().endswith((".jpg",".jpeg",".png",".webp"))
        ])
        print(f"{split}: {n_images} imágenes")
    else:
        print(f"{split}: carpeta no encontrada")
```

```
DE data.yaml ===
train/images', 'val': '../valid/images', 'test': '../test/images', 'nc': 3, 'names': ['crack', 'snow', 'soiling'], 'roboflow'
ágenes
genes
tes
```

```
!pip show ultralytics
```

Mostrar salida oculta

```
!pip install -q -U ultralytics
```

```
import ultralytics
print("Versión:", ultralytics.__version__)
```

Creating new Ultralytics Settings v0.0.7 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>
Versión: 8.4.130

Se carga otra versión de YOLO11n preentrenado

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")

print("Modelo YOLO11n cargado correctamente.")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt> to 'yolo11n.pt': 100% |██████████| 5.4
Modelo YOLO11n cargado correctamente.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os

ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"

print("Carpeta encontrada:", os.path.exists(ruta_experimentos))
```

Carpeta encontrada: True

```
results = model.train(
    data=yaml_path,
    epochs=100
    imgs=640,
    batch=16,
    device=0,
    project="/content/drive/MyDrive/TFM/Experimentos",
    name="E4_augmentation_100ep_pat20_yolo11n",
    seed=42,
    deterministic=True,
    plots=True
)
```

Insertar celda de código debajo (Ctrl+M B)

Ultralytics 8.4.130 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7
Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% 755.1KB 2
Overriding model.yaml nc=80 with nc=3

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
13	-1	1	111296	ultralytics.nn.modules.block.C3k2	[384, 128, 1, False]
14	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
16	-1	1	32096	ultralytics.nn.modules.block.C3k2	[256, 64, 1, False]
17	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]
19	-1	1	86720	ultralytics.nn.modules.block.C3k2	[192, 128, 1, False]
20	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]
22	-1	1	378880	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True]
23	[16, 19, 22]	1	431257	ultralytics.nn.modules.head.Detect	[3, 16, None, [64, 128, 256]]

YOLO11n summary: 182 layers, 2,590,425 parameters, 2,590,409 gradients, 6.5 GFLOPs

Transferred 448/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: downloading yolo26n.pt for AMP checks (one-time, not used for training)...
Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'weights/yolo26n.pt': 100%
AMP: checks passed
train: Fast image access (ping: 0.0±0.0 ms, read: 2175.3±1231.6 MB/s, size: 72.3 KB)
train: Scanning /content/Solar-panels-5/train/labels... 1719 images, 524 backgrounds, 0 corrupt: 100% 1719/17
train: New cache created: /content/Solar-panels-5/train/labels.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_av
val: Fast image access (ping: 0.0±0.0 ms, read: 577.0±330.1 MB/s, size: 82.2 KB)
val: Scanning /content/Solar-panels-5/valid/labels... 122 images, 40 backgrounds, 0 corrupt: 100% 122/122 2.1
val: New cache created: /content/Solar-panels-5/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'm
optimizer: AdamW(lr=0.001429, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(d
Plotting labels to /content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/labels.jpg...
Using 1719 train, 122 val images for fraction=1.0 at imgs=640
Using 2 dataloader workers
Logging results to /content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n
Starting training for 100 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/100	2.38G	1.251	3.255	1.614	18	640: 100% 108/108 2.0it/s 54.0s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 4/4 2.6s/it 10
	all	122	103	0.692	0.109	0.12 0.0532

```
import os

ruta_experimentos = "/content/drive/MyDrive/TFM/Experimentos"

for carpeta in os.listdir(ruta_experimentos):
    print(carpeta)
```

E1_baseline_100ep_pat20_yolo11n
E2_extended_100ep_pat20_yolo11n
E3_revised_100ep_pat20_yolo11n
E4_augmentation_100ep_pat20_yolo11n

```
import pandas as pd

ruta_csv = "/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/results.csv"

df_e2 = pd.read_csv(ruta_csv)

Insertar celda de código debajo (Ctrl+M B)
idx_best = df_e2["metrics/mAP50-95(B)"].idxmax()
best_e2 = df_e2.loc[idx_best]
last_e2 = df_e2.iloc[-1]

print("Épocas completadas:", len(df_e2))

print("\n--- MEJOR ÉPOCA ---")
print("Época:", int(best_e2["epoch"]))
print("Precision:", round(best_e2["metrics/precision(B)"], 4))
print("Recall:", round(best_e2["metrics/recall(B)"], 4))
print("mAP50:", round(best_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(best_e2["metrics/mAP50-95(B)"], 4))

print("\nMáximo mAP50:",
      round(df_e2["metrics/mAP50(B)"].max(), 4),
      "en época",
      int(df_e2.loc[df_e2["metrics/mAP50(B)"].idxmax(), "epoch"]))

print("\n--- ÚLTIMA ÉPOCA ---")
print("Época:", int(last_e2["epoch"]))
print("Precision:", round(last_e2["metrics/precision(B)"], 4))
print("Recall:", round(last_e2["metrics/recall(B)"], 4))
print("mAP50:", round(last_e2["metrics/mAP50(B)"], 4))
print("mAP50-95:", round(last_e2["metrics/mAP50-95(B)"], 4))

tiempo = df_e2["time"].iloc[-1]
print("\nTiempo total:",
      int(tiempo // 60), "min",
      round(tiempo % 60, 1), "s")
```

Épocas completadas: 100

--- MEJOR ÉPOCA ---
Época: 83
Precision: 0.5857
Recall: 0.6011
mAP50: 0.5374
mAP50-95: 0.3601

Máximo mAP50: 0.5374 en época 83

--- ÚLTIMA ÉPOCA ---
Época: 100
Precision: 0.5509
Recall: 0.5297
mAP50: 0.4594
mAP50-95: 0.3356

Tiempo total: 57 min 24.4 s

```
from ultralytics import YOLO

best_model_e2 = YOLO(
    "/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/weights/best.pt"
)

test_results_e2 = best_model_e2.val(
    data=yaml_path,
    split="test",
    device=0,
    plots=True,
    project="/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n",
    name="test"
)
```

Ultralytics 8.4.130  Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,582,737 parameters, 0 gradients, 6.4 GFLOPs
test: Fast image access  (ping: 0.0±0.0 ms, read: 1557.6±258.0 MB/s, size: 96.2 KB)
test: Scanning /content/Solar-panels-5/test/labels.cache... 62 images, 23 backgrounds, 0 corrupt: 100%  62/62 11

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	62	44	0.615	0.631	0.558	0.417
crack	5	7	0.42	0.429	0.192	0.141
snow	12	12	0.794	0.917	0.912	0.627
soiling	22	25	0.632	0.549	0.571	0.485

Speed: 13.6ms preprocess, 7.5ms inference, 0.0ms loss, 8.0ms postprocess per image
Results saved to /content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/test

Descargo el archivo en .onnx

```
from google.colab import drive
drive.mount('/content/drive')
```

Insertar celda de código debajo (Ctrl+M B)

```
!pip install ultralytics onnx onnxruntime
```

[Mostrar salida oculta](#)

```
from ultralytics import YOLO
```

```
weights_path = '/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/weights/best.pt'
```

```
# Cargar y exportar a 640x640
model = YOLO(weights_path)
model.export(format='onnx', imgsz=640)
```

[Mostrar salida oculta](#)

```
from google.colab import files
```

```
onnx_path = '/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/weights/best.onnx'
files.download(onnx_path)
```

para cuantizar el .onnx a int8 y volver a probarlo cuantizado en la raspberry para ver si mejoran las métricas

```
# 1. Instalar dependencias necesarias tras reiniciar Colab
!pip install --quiet ultralytics onnx onnxruntime

import os
from google.colab import drive, files
from ultralytics import YOLO
import onnxruntime
from onnxruntime.quantization import quantize_dynamic, QuantType

# 2. Conectar con Google Drive
drive.mount('/content/drive')

# 3. Definir rutas (ajusta la carpeta en Drive si fuera necesario)
DIR_ORIGEN = '/content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/weights'
PT_PATH = os.path.join(DIR_ORIGEN, 'best.pt')
ONNX_PATH = os.path.join(DIR_ORIGEN, 'best.onnx')
INT8_ONNX_PATH = os.path.join(DIR_ORIGEN, 'best_int8.onnx')

# 4. Asegurar la exportación estricta a 640x640 si se parte de best.pt
# Si quieres forzar la regeneración en 640x640, pon regenerar_onnx = True
regenerar_onnx = False

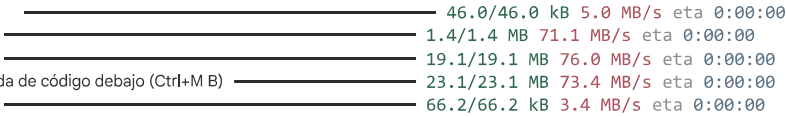
if not os.path.exists(ONNX_PATH) or regenerar_onnx:
    print("Exportando best.pt a ONNX FP32 con resolución 640x640...")
    if not os.path.exists(PT_PATH):
        raise FileNotFoundError(f"No se encontró el archivo: {PT_PATH}")
    model = YOLO(PT_PATH)
    # imgsz=(640, 640) fija explícitamente la dimensión espacial cuadrada
    model.export(format='onnx', imgsz=(640, 640), dynamic=False)

# 5. Aplicar cuantización dinámica a INT8 manteniendo la geometría
print(f"Cuantizando a INT8 el modelo (640x640): {ONNX_PATH} ...")
quantize_dynamic(
    model_input=ONNX_PATH,
    model_output=INT8_ONNX_PATH,
    weight_type=QuantType.QInt8
)

size_fp32 = os.path.getsize(ONNX_PATH) / (1024 * 1024)
size_int8 = os.path.getsize(INT8_ONNX_PATH) / (1024 * 1024)

print("\n" + "="*45)
print("    CUANTIZACIÓN INT8 (640x640) COMPLETADA")
print("="*45)
print(f"Modelo FP32 original:  {size_fp32:.2f} MB")
print(f"Modelo INT8 optimizado: {size_int8:.2f} MB")
print(f"Reducción de tamaño:  {((size_fp32 - size_int8) / size_fp32) * 100:.1f} %")
print("="*45 + "\n")
```

```
print("Iniciando descarga de best_int8.onnx a tu PC...")
files.download(INT8_ONNX_PATH)
```



Insertar celda de código debajo (Ctrl+M B)

```
Creating new Ultralytics Settings v0.0.8 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com
Mounted at /content/drive
WARNING:root:Please consider to run pre-processing before quantization. Refer to example: https://github.com/microsoft/onnxruntime
Cuantizando a INT8 el modelo (640x640): /content/drive/MyDrive/TFM/Experimentos/E4_augmentation_100ep_pat20_yolo11n/weights/

=====
          CUANTIZACIÓN INT8 (640x640) COMPLETADA
=====
Modelo FP32 original:  10.11 MB
Modelo INT8 optimizado: 2.87 MB
Reducción de tamaño:   71.6 %
=====

Iniciando descarga de best_int8.onnx a tu PC...
```

ANEXO II. REGISTRO DE OPERADOR DE UAS



ENTRADA
Registro General AESA
Número: 2026202704
Fecha: 13/07/2026 21:28

JUSTIFICANTE DE PRESENTACIÓN DE REGISTRO

1. DATOS DE LOS SOLICITANTES

LUCIA GOROSTIDI GARCIA

2. ASUNTO

NUEVO REGISTRO
INSCRIPCIÓN EN REGISTRO COMO OPERADOR DE UAS



REGISTRO DE OPERADOR DE SISTEMAS DE AERONAVES NO TRIPULADAS (UAS)

UNMANNED AIRCRAFT SYSTEM (UAS) OPERATOR REGISTRATION

Número de Operador de UAS /Registration number of the UAS operator		Fecha y hora de registro /Date and time of registration		13/07/2026 21:25:59
REGISTRO COMO OPERADOR DE UAS /REGISTRATION AS A UAS OPERATOR				
Tipo de registro /Registration type		☒ Inicial /Initial ☐ Modificación /Modification:		
OPERADOR DE UAS /UAS OPERATOR				
Nombre completo o razón social /Full name or corporate name		LUCIA GOROSTIDI GARCIA		
Nombre comercial /Trade name				
DNI, NIF, NIE o nº pasaporte /Identity card or passport number				
Nacionalidad /Nationality		España	Fecha de Nacimiento /Date of birth	
Teléfono /Telephone number			Correo electrónico /Email	
Dirección /Address				
Código postal /Zip-postal code			Municipio /City	
Provincia /State-province			País /Country	
Medio de notificación /Means of communication		☒ Notificación por comparecencia en sede electrónica /Preferably, notification will be made electronically ☐ Notificación en papel por correo postal /Notifications will be made on paper and sent by postal mail		
Número de póliza de seguro del UAS /Insurance policy number for UAS		No disponible /Not available		
¿Qué tipo de actividad o servicio va a realizar? /What type of activity or service are you going to perform?		☒ EASA ☐ NO EASA (excluidas del Reglamento (UE) 2018/1139) /Excluded from Regulation (EU) 2018/1139		
El operador se identifica como un club o asociación de aeromodelismo /The operator considers itself as a model aircraft club or association		☐ Sí / Yes ☒ No / No		

DECLARACIÓN RESPONSABLE/Declaration of responsibility

**ANEXO III. CERTIFICADO APTO FORMACIÓN PARA LA
REALIZACIÓN DE OPERACIONES CON UAS EN
CATEGORÍA ABIERTA**



MINISTERIO
DE TRANSPORTES
Y MOVILIDAD SOSTENIBLE



O F I C I O

LUCIA GOROSTIDI GARCIA - CALLE

S/REF:

N/REF:

2026/LPAE0/024348

ASUNTO:

RESOLUCIÓN POSITIVA

ORIGEN:

E04865601 - Agencia Estatal de Seguridad Aérea

DESTINATARIO:

- LUCIA GOROSTIDI GARCIA

**RESOLUCIÓN POR LA QUE SE OTORGA LA PRUEBA DE SUPERACIÓN DE FORMACIÓN EN LÍNEA PARA
REALIZAR OPERACIONES CON UAS EN CATEGORÍA ABIERTA, SUBCATEGORÍAS A1 Y A3**

En relación con la solicitud de emisión de la prueba de superación de formación en línea presentada en la Agencia Estatal de Seguridad Aérea con fecha **13 de julio de 2026** (registro de entrada **2026202686**), por D/Dª **GOROSTIDI GARCIA, LUCIA**, se comunica que, al haber superado la formación y examen exigible de acuerdo con los apartados UAS.OPEN.020 y UAS.OPEN.040 del Reglamento de Ejecución (UE) 2019/947 esta dirección

RESUELVE:

Conceder la emisión de la prueba de superación de formación en línea necesaria para realizar operaciones con UAS en categoría abierta, subcategorías A1 y A3, a favor de

LUCIA GOROSTIDI GARCIA, con DNI

EL JEFE DE LA DIVISIÓN DE SISTEMAS DE AERONAVES NO TRIPULADAS

(P.D.F. de la Directora de Seguridad de Aeronaves)

Fdo.: ROBERTO GÁNDARA OSSEL

(firmado digitalmente)

Utilice el siguiente enlace para descargar su certificado de piloto

Tipo de documento (Código CID)

PRUEBA DE SUPERACIÓN DE FORMACIÓN EN LÍNEA (AESASGEEPGE000OC24323RBKQ2HDA)

<https://sede.seguridadaaerea.gob.es/CID/servlet/Ficheros?id=AESASGEEPGE000OC24323RBKQ2HDA&origen=ws&firma=firmaBarra>

Contra la presente resolución, que no pone fin a la vía administrativa, los interesados pueden interponer, en los supuestos indicados en el artículo 112.1 y de conformidad con los artículos 121 y 122 de la Ley 39/2015, de 1 de octubre, del Procedimiento Administrativo Común de las Administraciones Públicas, recurso de alzada ante la Directora de la Agencia Estatal de Seguridad Aérea, en el plazo de un mes, a contar

desde el día siguiente al de su notificación, sin perjuicio de que puedan ejercitar cualquier otro que estimen oportuno.

CORREO ELECTRÓNICO:

drone.aesa@seguridadaaerea.es

CLASIFICACIÓN SENSIBLE

La clasificación de este documento indica el nivel de seguridad para su tratamiento interno en AESA.
Si el documento le ha llegado por los cauces legales, no tiene ningún efecto para usted

PASEO DE LA CASTELLANA 112

28046 MADRID

TEL.: +34 91 396 8000

SELLO ELECTRONICO AESA

AGENCIA ESTATAL DE SEGURIDAD AEREA

Código Seguro de Verificación (CSV/CID): AESASGEEPGE000OC2446AJVQ584DA

Este documento puede ser verificado en / This document may be verified at: <https://sede.seguridadaaerea.gob.es>





MINISTERIO
DE TRANSPORTES
Y MOVILIDAD SOSTENIBLE



Le recordamos que puede verificar la autenticidad de este documento desde la Sede Electrónica de AESA haciendo uso de la aplicación *Comprobación Documental*, cuya dirección de acceso es: <https://sede.seguridadeaerea.gob.es/CID/FrontController>, utilizando el siguiente código seguro de verificación (CSV/CID): **AESASGEEPGE0000C2446AJVQ584DA**.

CORREO ELECTRÓNICO:

drones.aesa@seguridadeaerea.es

CLASIFICACIÓN SENSIBLE

*La clasificación de este documento indica el nivel de seguridad para su tratamiento interno en AESA.
Si el documento le ha llegado por los cauces legales, no tiene ningún efecto para usted*

PASEO DE LA CASTELLANA 112

28046 MADRID

TEL.: +34 91 396 8000

SELLO ELECTRONICO AESA

AGENCIA ESTATAL DE SEGURIDAD AEREA

Código Seguro de Verificación (CSV/CID): AESASGEEPGE0000C2446AJVQ584DA

Este documento puede ser verificado en / This document may be verified at: <https://sede.seguridadeaerea.gob.es>

